```
//! rust_p2p_node — Harmony gateway, protocol v3.
//!
//! Identity
//!   * Every node owns an ML-DSA-87 identity key and an ML-KEM-1024 key, kept as
seeds in
//!     `<data>/<node-name or node-PORT>/` (files 0600, directory 0700 on Unix) and
published as one
//!     self-signed 8787-byte bundle at `GET /id`. A node's id is SHA-384(ctx ||
verifying key)[..32].
//! Handshake (`POST /hello`, one round trip, mutual auth, forward secret, identity
hiding)
//!   * initiator -> responder: an ML-KEM ciphertext to the responder's static key,
an ephemeral
//!     ML-KEM key, and the initiator's identity block (time, port, verifying key,
ML-DSA-87
//!     signature over the responder's id and both KEM values), AEAD-sealed under a
key only the
//!     responder can derive.
//!   * responder -> initiator: an ML-KEM ciphertext to the ephemeral key plus a key-
confirmation
//!     tag (only the holder of the static decapsulation key can produce it).
//!   * keys = HKDF-SHA-384(salt = transcript hash, ikm = ss_static || ss_ephemeral).
//! Frames (`POST /msg` and the UDP radio)
//!   * `session_id(8) | counter(8) | AES-256-GCM`, counter nonces, one key per
direction,
//!     64-wide replay window, payloads padded to 256 B / 1 / 4 / 16 KB.
//! Trust
//!   * `--trust-mode strict` talks to exactly the identities in `--trust` /
`HARMONY_TRUST`, inbound
//!     and outbound. `tofu` (the default, for first setup) pins every identity that
completes a
//!     handshake in `trusted_peers.txt`. `--revoke` beats both and cuts live
sessions.
//!   * `/id` and `/hello` answer only private, CGNAT, unique-local and loopback
addresses unless
//!     `--allow-public`.
//!   * Directives are accepted only from this node's own identity and `--cmd-allow`
ones; AI
//!     placement decisions only from this node, `--advisor` and `--cmd-allow`
identities.
//!   * Operational routes (`/health`, `/peers`, `/infer`, `/relay`, `/ui`, ...)
answer loopback
//!     only, refusing a Host or Origin that is not this machine (DNS rebinding,
cross-site pages),
//!     plus an optional `HARMONY_ADMIN_TOKEN` of 24 characters or more.
//! Piggyback propagation
```

```
//!   * Every gossip Ping/Pong carries the sender's vitals (temperature, thermal
limit, busy,
//!     tier). Offload and fan-out ask the coolest idle peers first and skip ones at
their limit.
//!   * Directives issued with `POST /relay` are signed by their origin (ML-DSA-87)
and ride on
//!     gossip pings, one per ping, acknowledged in the answering pong. Each hop
verifies the
//!     signature and its own `--cmd-allow` before forwarding and advising.
//! Signed updates
//!   * `--sign-update DIR` signs a manifest of a staged build (SHA-384 of every
file) with this
//!     node's identity; `--verify-update DIR --update-signer ID` checks one before
it is installed.
//! Inference
//!   * The model runs in llama.cpp's `llama-server`, started once and kept loaded
(`--llama-server`,
//!     unloaded after `--llama-idle-secs` without a job), so a job no longer pays
for loading it. A
//!     node without the server runs each job with `--llama-bin`, as before.
//!   * `GET /v1/models` and `POST /v1/chat/completions` (loopback/admin) speak the
OpenAI API, so
//!     existing chat apps and SDKs use the mesh unchanged. A conversation is placed
like an
//!     `/infer` job and goes to the coolest idle peer when this device is too hot or
busy.

use std::collections::{HashMap, HashSet};
use std::net::{IpAddr, Ipv4Addr, SocketAddr};
use std::path::{Path, PathBuf};
use std::process::Stdio;
use std::sync::atomic::{AtomicBool, AtomicU8, Ordering};
use std::sync::{Arc, Mutex, MutexGuard};
use std::time::{Duration, Instant, SystemTime, UNIX_EPOCH};

use axum::body::Bytes;
use axum::extract::{ConnectInfo, DefaultBodyLimit, State};
use axum::http::{header, HeaderMap, StatusCode};
use axum::response::{IntoResponse, Response};
use axum::routing::{get, post};
use axum::{Json, Router};
use clap::{Parser, ValueEnum};
use rand_core::{OsRng, RngCore};
use serde::{Deserialize, Serialize};
use tokio::net::{TcpListener, UdpSocket};
use tokio::sync::{broadcast, Semaphore};
use tracing::{debug, error, info, warn};
use zeroize::Zeroizing;

type BoxErr = Box<dyn std::error::Error + Send + Sync>;
```

```
// ──────────────────────────────────── constants ──────────────────────────────────────

const DEFAULT_PORT: u16 = 3005;
/// Reported in /health so scripts can tell which build is actually answering.
const BUILD_TAG: &str = "harmony-v19";
/// Life phases. Starting: listening, not yet bootstrapped. Ready: serving. Draining:
shutting
/// down — new jobs refused, peers told, in-flight work finishing.
const PHASE_STARTING: u8 = 0;
const PHASE_READY: u8 = 1;
const PHASE_DRAINING: u8 = 2;
/// Time allowed for saying goodbye to peers when shutting down.
const FAREWELL_BUDGET: Duration = Duration::from_secs(3);
/// Ports a node may rotate across above its requested port, and the window peers
sweep to find it.
const PORT_SWEEP: u16 = 10;
/// Consecutive gossip failures after which a peer is looked for on nearby ports (it
may have
/// rotated) — twice, before MAX_PEER_FAILS drops it.
const REDISCOVER_AT_FAILS: [u32; 2] = [2, 4];
/// With no peers at all, retry the seeds (sweeping for them) every this many gossip
rounds.
const RESEED_EVERY_ROUNDS: u32 = 3;
const DEFAULT_UDP_PORT: u16 = 4000;

/// Protocol 3 encrypts the initiator's identity. It is wire-incompatible with 2 by
design:
/// a node that still speaks 2 would leak the very thing 3 exists to hide, so there
is no
/// downgrade path and every node in a mesh must move together.
const PROTO_VER: u8 = 3;
// ML-KEM-1024 (FIPS 203)
const KEM_EK_LEN: usize = 1568;
const KEM_CT_LEN: usize = 1568;
// ML-DSA-87 (FIPS 204): verifying key 2592 B, signature 4627 B.
const SIG_PK_LEN: usize = 2592;
const SIG_LEN: usize = 4627;
const TAG_LEN: usize = 16;

// identity bundle: sig_pk | kem_ek | signature(over both)
const BUNDLE_LEN: usize = SIG_PK_LEN + KEM_EK_LEN + SIG_LEN;

// hello1 (v3): ver(1) | kem_ct_static | kem_ek_ephemeral | AEAD(identity block)
//
// Nothing before the ciphertext identifies the initiator. The two KEM blobs are
// indistinguishable from random to anyone without the responder's static
decapsulation key,
// so an observer learns only that *somebody* is handshaking with this address.
const H1_CT_OFF: usize = 1;
```

```
const H1_EK_OFF: usize = H1_CT_OFF + KEM_CT_LEN;
const H1_CIPHER_OFF: usize = H1_EK_OFF + KEM_EK_LEN;
// encrypted identity block: ts(8) | listen_port(2) | init_sig_pk | signature
const INNER_PORT_OFF: usize = 8;
const INNER_VK_OFF: usize = INNER_PORT_OFF + 2;
const INNER_SIG_OFF: usize = INNER_VK_OFF + SIG_PK_LEN;
const INNER_LEN: usize = INNER_SIG_OFF + SIG_LEN;
const HELLO1_LEN: usize = H1_CIPHER_OFF + INNER_LEN + TAG_LEN;
// hello2: kem_ct_ephemeral | key-confirmation tag
const HELLO2_LEN: usize = KEM_CT_LEN + TAG_LEN;
const FRAME_HDR_LEN: usize = 16;

/// Frames are padded up to the next bucket so a ping, a peer list and an inference
request
/// are the same size on the wire. Without this, message *type* is readable from
length alone
/// even though the contents are not.
const PAD_BUCKETS: [usize; 4] = [256, 1024, 4096, 16384];
/// Two bytes of real length live inside the padded plaintext.
const PAD_PREFIX: usize = 2;

const MAX_HTTP_BODY: usize = 16 * 1024;
/// The largest a padded frame can legitimately get. Derived from the ladder so
adding a
/// bucket cannot silently exceed the body limit and make big messages undeliverable.
const MAX_FRAME_LEN: usize = PAD_BUCKETS[PAD_BUCKETS.len() - 1] + FRAME_HDR_LEN +
TAG_LEN;
const MAX_UDP_FRAME: usize = 1400;
const MAX_CLOCK_SKEW_SECS: u64 = 120;
const MAX_FRAMES_PER_SESSION: u64 = 1_000_000;
const CLIENT_SESSION_MAX_AGE: Duration = Duration::from_secs(15 * 60);
const SERVER_SESSION_MAX_AGE: Duration = Duration::from_secs(30 * 60);
const SERVER_SESSION_IDLE: Duration = Duration::from_secs(10 * 60);
const MAX_SERVER_SESSIONS: usize = 256;
const MAX_CLIENT_SESSIONS: usize = 256;
const MAX_PEERS: usize = 256;
const MAX_PEER_FAILS: u32 = 6;
const MAX_TRUSTED: usize = 1024;
const MAX_PEERS_PER_REPLY: usize = 32;
const HELLO_CACHE_MAX: usize = 4096;
const HELLO_PER_IP: u32 = 10;
const HELLO_WINDOW: Duration = Duration::from_secs(10);
/// `/msg` is cheap (symmetric only) but still worth bounding per source address.
const MSG_PER_IP: u32 = 240;
const MSG_WINDOW: Duration = Duration::from_secs(10);
const MAX_PROMPT_BYTES: usize = 1024;
/// Cap on an advisory prompt. The node builds it (it is not a client's prompt): six
devices and
/// the longest note a directive can carry (MAX_PROMPT_BYTES) fit, about 600 tokens —
far inside
```

```
/// the 4K context.
const MAX_ADVISORY_PROMPT_BYTES: usize = 2048;
/// Peers queried in one distributed inference, and the hard ceiling on that.
const DEFAULT_FANOUT: usize = 3;
const MAX_FANOUT: usize = 8;
/// A remote peer may not pin our executor open longer than this, whatever it asks
for.
/// Ceiling on work a *peer* can make this node do. Bounded tightly: an inference
request is
/// the most expensive thing an authenticated peer can ask for, and a generous cap
here is a
/// resource-exhaustion primitive handed to anyone in the trust store.
const MAX_REMOTE_INFER: Duration = Duration::from_secs(180);
/// Ceiling on what the *operator* can ask their own node for, over loopback or with
an admin
/// token. Much higher, because this is not an attack surface — it is someone waiting
at a
/// terminal — and an 8B model on a phone CPU legitimately needs minutes for a long
answer.
const MAX_LOCAL_INFER: Duration = Duration::from_secs(900);
const MAX_LLM_OUTPUT_CHARS: usize = 4000;
const LLM_COOLDOWN: Duration = Duration::from_secs(2);
/// /v1/chat/completions (v19): a conversation is cut, oldest turns first, to this
many bytes (its
/// text plus a few bytes a turn for the template), about 2K tokens, so
CHAT_MAX_TOKENS of answer
/// still fits LLM_CTX. A peer refuses a longer one.
const CHAT_MAX_PROMPT_BYTES: usize = 8 * 1024;
/// The body a chat client may send. Clients resend the whole history every time;
what is over
/// CHAT_MAX_PROMPT_BYTES is cut, not refused.
const CHAT_MAX_BODY: usize = 256 * 1024;
/// Answer budget of a chat request without max_tokens, and the ceiling when it has
one.
const CHAT_DEFAULT_TOKENS: u32 = 256;
const CHAT_MAX_TOKENS: u32 = 1024;
/// A chat answer may run longer than an /infer one; it still has to fit one frame
back from a peer.
const CHAT_MAX_OUTPUT_CHARS: usize = 6000;
/// How long the operator's chat request waits for this device's model while another
job holds it,
/// before it goes to a peer. /infer and peers' requests never wait: a busy device
refuses at once.
const CHAT_LOCAL_WAIT: Duration = Duration::from_secs(20);
/// The model id /v1/models lists. A request may name any model: each device answers
with its own.
const OPENAI_MODEL_ID: &str = "harmony";
const RADIO_MAX_INFLIGHT: usize = 64;
/// Gossip intervals are randomised by up to this fraction either way. A transmitter
that
```

```
/// emits on an exact period is trivially fingerprinted and direction-found.
const JITTER_FRACTION: f64 = 0.5;
/// Consecutive failures to a peer before we treat the link as degraded rather than
unlucky.
const DEGRADED_AFTER_FAILS: u32 = 3;
/// Concurrent handshakes allowed. Each costs an ML-KEM decapsulation plus an ML-DSA-
87
/// verification, so without a ceiling a modest number of sources can saturate the
CPU.
const MAX_CONCURRENT_HANDSHAKES: usize = 4;
/// Remembered hello1 digests, so a captured hello cannot be replayed for free work
inside
/// the clock-skew window. Sized well above what any honest mesh produces in that
window.
const HELLO_SEEN_MAX: usize = 8192;
/// How many table entries any single peer may be responsible for introducing. Caps
the
/// damage a trusted-but-misbehaving node can do by flooding peer exchanges.
const MAX_PEERS_PER_SOURCE: usize = 32;

// These keep their v2 labels on purpose. CTX_FINGERPRINT defines what a NodeId *is*,
and
// CTX_BUNDLE is covered by every identity signature already written to disk —
renaming either
// would invalidate every existing identity, pin and revocation on every node. The
protocol
// version is carried explicitly in the signed hello input instead, which is what
actually
// separates a v2 signature from a v3 one.
const CTX_BUNDLE: &[u8] = b"harmony-v2/id";
const CTX_HELLO: &[u8] = b"harmony-v2/hello1";
const CTX_TRANSCRIPT: &[u8] = b"harmony-v2/th";
const CTX_KEYS: &[u8] = b"harmony-v2/keys";
const CTX_FINGERPRINT: &[u8] = b"harmony-v2/fpr";
/// Binds the pre-identity key to everything an observer can already see, so the
provisional
/// key cannot be reused across different handshakes.
const CTX_H0: &[u8] = b"harmony-v3/h0";
const CTX_K0: &[u8] = b"harmony-v3/k0";

/// A node is identified publicly by its full ML-DSA-87 verifying key, but internally
we key
/// maps and the trust store by a 32-byte fingerprint = SHA-384(ctx || vk)[..32]. The
hex of
/// this fingerprint is what --trust / --cmd-allow / trusted_peers.txt contain.
type NodeId = [u8; 32];

// ──────────────────────────────────── utilities ────────────────────────────────────

fn now_unix() -> u64 {
```

```
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_secs())
        .unwrap_or(0)
}

/// Lock a mutex, ignoring poisoning (a panicked task must not take the whole node
down).
fn lock<T>(m: &Mutex<T>) -> MutexGuard<'_, T> {
    m.lock().unwrap_or_else(|e| e.into_inner())
}

fn hex(bytes: &[u8]) -> String {
    use std::fmt::Write;
    let mut s = String::with_capacity(bytes.len() * 2);
    for b in bytes {
        let _ = write!(s, "{:02x}", b);
    }
    s
}

fn unhex32(s: &str) -> Option<NodeId> {
    let s = s.trim();
    if s.len() != 64 || !s.bytes().all(|b| b.is_ascii_hexdigit()) {
        return None;
    }
    let mut out = [0u8; 32];
    for (i, o) in out.iter_mut().enumerate() {
        *o = u8::from_str_radix(s.get(2 * i..2 * i + 2)?, 16).ok()?;
    }
    Some(out)
}

/// First 12 characters of an already-hex node id, for logs.
fn short_hex(h: &str) -> String {
    h.chars().take(12).collect()
}

fn short_id(id: &NodeId) -> String {
    hex(&id[..6])
}

/// Strip control characters from text that came from the model or a peer.
///
/// This text lands in logs and JSON. Raw control bytes and ANSI escapes can corrupt
a
/// terminal reading those logs and can forge log lines, so only newline and tab
survive.
fn sanitize(s: &str) -> String {
    s.chars()
```

```
        .filter(|c| !c.is_control() || *c == '\n' || *c == '\t')
        .collect()
}

fn truncate(s: &str, n: usize) -> String {
    s.chars().take(n).collect()
}

/// Non-cryptographic uniform-ish pick in `0..n` (peer selection, loss simulation).
fn rand_below(n: usize) -> usize {
    if n <= 1 {
        return 0;
    }
    (OsRng.next_u64() % (n as u64)) as usize
}

/// Randomise a period by ±`JITTER_FRACTION` so this node's transmissions do not fall
on a
/// fixed cadence.
///
/// A regular 5-second beacon is a fingerprint: it identifies the software, survives
/// encryption, and gives a direction-finder a predictable window to listen in.
Jitter costs
/// nothing and removes the pattern. It does not hide *that* the node transmits.
fn jittered(base: Duration) -> Duration {
    let ms = base.as_millis().min(u64::MAX as u128) as u64;
    if ms == 0 {
        return base;
    }
    let span = ((ms as f64) * JITTER_FRACTION) as u64;
    if span == 0 {
        return base;
    }
    // Uniform in [ms - span, ms + span].
    let offset = (OsRng.next_u64() % (2 * span + 1)) as i64 - span as i64;
    Duration::from_millis((ms as i64 + offset).max(1) as u64)
}

fn ct_eq(a: &[u8], b: &[u8]) -> bool {
    use subtle::ConstantTimeEq;
    a.ct_eq(b).into()
}

fn write_secret(path: &Path, data: &[u8]) -> std::io::Result<()> {
    use std::io::Write;
    let mut opts = std::fs::OpenOptions::new();
    opts.write(true).create_new(true);
    #[cfg(unix)]
    {
        use std::os::unix::fs::OpenOptionsExt;
```

```
        opts.mode(0o600);
    }
    let mut f = opts.open(path)?;
    f.write_all(data)?;
    f.sync_all()
}

/// Replace an existing secret atomically: write a 0600 temp file in the same
directory, fsync,
/// then rename over the target. A crash mid-rotation leaves either the old key or
the new one,
/// never a truncated file.
fn replace_secret(path: &Path, data: &[u8]) -> std::io::Result<()> {
    let dir = path.parent().unwrap_or_else(|| Path::new("."));
    let tmp = dir.join(format!(
        ".{}.tmp",
        path.file_name().and_then(|s| s.to_str()).unwrap_or("secret")
    ));
    let _ = std::fs::remove_file(&tmp);
    write_secret(&tmp, data)?;
    std::fs::rename(&tmp, path)?;
    // Best effort: fsync the directory so the rename itself is durable.
    if let Ok(d) = std::fs::File::open(dir) {
        let _ = d.sync_all();
    }
    Ok(())
}

/// Pick the identity directory name. This is what ties a set of keys to a node, so
it must
/// never depend on anything that can change at runtime — in particular not on the
port the
/// listener actually ends up bound to. With `--node-name`, identity is fully
independent of
/// the port; without it we fall back to the *requested* port, which is still stable
across
/// rotation because rotation only changes where we listen, not what we asked for.
fn identity_dir_name(node_name: &Option<String>, requested_port: u16) -> String {
    match node_name {
        Some(n) => {
            let cleaned: String = n
                .chars()
                .map(|c| if c.is_ascii_alphanumeric() || c == '-' || c == '_' { c }
else { '_' })
                .take(64)
                .collect();
            if cleaned.is_empty() {
                format!("node-{requested_port}")
            } else {
                cleaned
```

```
            }
        }
        None => format!("node-{requested_port}"),
    }
}

/// The device label: `--label` if given (cleaned, at most 24 characters), else a
guess from
/// the platform that a directive written by a person is likely to use.
fn device_label(given: Option<&str>) -> String {
    if let Some(l) = given {
        let l: String = sanitize(l).trim().chars().take(24).collect();
        if !l.is_empty() {
            return l;
        }
    }
    if std::env::var_os("TERMUX_VERSION").is_some() ||
Path::new("/system/build.prop").exists() {
        return "phone".into();
    }
    let battery = std::fs::read_dir("/sys/class/power_supply")
        .map(|d| d.flatten().any(|e|
e.file_name().to_string_lossy().starts_with("BAT")))
        .unwrap_or(false);
    if battery { "laptop" } else { "desktop" }.into()
}

/// `<home>/.harmony`: $HOME, else %USERPROFILE% (Windows, where HOME is usually
unset).
fn default_data_base() -> PathBuf {
    let home = ["HOME", "USERPROFILE"]
        .into_iter()
        .filter_map(std::env::var_os)
        .find(|h| !h.is_empty());
    match home {
        Some(h) => PathBuf::from(h).join(".harmony"),
        None => PathBuf::from(".harmony"),
    }
}

fn parse_peer(s: &str, default_port: u16) -> Option<SocketAddr> {
    let s = s.trim();
    if let Ok(a) = s.parse::<SocketAddr>() {
        return Some(a);
    }
    s.parse::<IpAddr>().ok().map(|ip| SocketAddr::new(ip, default_port))
}

fn is_cgnat(ip: Ipv4Addr) -> bool {
    let o = ip.octets();
```

```
    o[0] == 100 && (o[1] & 0xc0) == 64
}

/// Which addresses we are willing to dial (gossip candidates, heartbeat addresses).
/// Default: private LAN ranges only, so a forged peer entry cannot aim us at the
internet,
/// at localhost services or at link-local metadata endpoints.
fn peer_ok(a: &SocketAddr, allow_public: bool, allow_loopback: bool) -> bool {
    let ip = a.ip().to_canonical();
    if a.port() == 0 || ip.is_unspecified() || ip.is_multicast() {
        return false;
    }
    if ip.is_loopback() {
        return allow_loopback;
    }
    match ip {
        IpAddr::V4(v4) => {
            if v4.is_broadcast() || v4.is_link_local() {
                return false;
            }
            v4.is_private() || is_cgnat(v4) || allow_public
        }
        IpAddr::V6(v6) => {
            let seg = v6.segments();
            if (seg[0] & 0xffc0) == 0xfe80 {
                return false; // link-local
            }
            (seg[0] & 0xfe00) == 0xfc00 || allow_public // unique-local
        }
    }
}

/// Who may reach the mesh endpoints (`/id`, `/hello`) at all: this machine and the
addresses the
/// node itself dials (private, CGNAT, unique-local), unless --allow-public. Bound to
0.0.0.0 on a
/// network with public addresses, a node would otherwise take handshakes from the
whole internet.
fn inbound_ok(ip: IpAddr, allow_public: bool) -> bool {
    let ip = ip.to_canonical();
    ip.is_loopback() || peer_ok(&SocketAddr::new(ip, 1), allow_public, false)
}

// ──────────────────────────────────── CLI / config ────────────────────────────────────

/// Which model this node runs, chosen from what the device can actually carry.
///
/// This is a capability tier, not a quality ranking: a Light node is a full
participant that
/// happens to answer with a smaller model, and its replies are labelled so a caller
```

```
fanning
/// out across the mesh can weigh them accordingly.
#[derive(Clone, Copy, Debug, PartialEq, Eq, ValueEnum)]
enum ModelTier {
    /// Decide from RAM and core count at startup
    Auto,
    /// ~1B parameters — phones, low-memory boards
    Light,
    /// ~3B parameters — 6 GB RAM and 6+ cores
    Standard,
    /// ~6B parameters — opt-in only.
    ///
    /// The realistic choice at this size is Yi-1.5-6B-Chat; 6B is an unusual size
and the
    /// field is thin. At Q4_K_M it is roughly 3.8 GB resident against 4.9 GB for an
8B, so
    /// on a 10 GiB device it buys headroom and perhaps 25% more tokens per second —
both
    /// are memory-bandwidth bound on a phone, so neither is fast. It uses ChatML, so
pair
    /// it with `--prompt-format chatml`.
    Mid,
    /// ~8B parameters — opt-in only.
    ///
    /// `Auto` never selects this. At Q4_K_M an 8B model is roughly 5 GB resident,
which on a
    /// phone competes with the OS and the low-memory killer, and CPU-only generation
runs at
    /// a few tokens per second under sustained thermal load. It is a deliberate
choice, not a
    /// default, so pick it knowingly and keep the thermal gate and timeout generous.
    Heavy,
}

impl ModelTier {
    fn label(self) -> &'static str {
        match self {
            ModelTier::Auto => "auto",
            ModelTier::Light => "light-1b",
            ModelTier::Standard => "standard-3b",
            ModelTier::Mid => "mid-6b",
            ModelTier::Heavy => "heavy-8b",
        }
    }

    /// How long a generation on this tier is allowed to take by default.
    ///
    /// These are deliberately generous. Model load alone is several seconds for a 5
GB file
    /// off flash, and CPU generation on a phone runs at single-digit tokens per
```

```
second under
    /// thermal load — so a timeout tuned for a 1B model reads as "the model is
broken" when
    /// it is really "the clock was too short".
    fn default_timeout_secs(self) -> u64 {
        match self {
            ModelTier::Light => 60,
            ModelTier::Standard => 120,
            ModelTier::Mid => 300,
            ModelTier::Heavy => 420,
            // Unreachable after resolve(), but a sane value beats a panic.
            ModelTier::Auto => 120,
        }
    }

    /// Resolve `Auto` against the hardware. Deliberately conservative: a 3B model
that pushes
    /// a phone into swap or thermal throttling is worse than a 1B model that
answers.
    fn resolve(self) -> ModelTier {
        if self != ModelTier::Auto {
            return self;
        }
        let mem = total_memory_bytes().unwrap_or(0);
        let cores = std::thread::available_parallelism().map(|n|
n.get()).unwrap_or(1);
        // Auto deliberately tops out at Standard. Mid and Heavy are viable on a
device like
        // this one but trade against memory pressure and sustained heat, so they
stay
        // explicit choices rather than something hardware detection talks you into.
        let tier = if mem >= 6 * 1024 * 1024 * 1024 && cores >= 6 {
            ModelTier::Standard
        } else {
            ModelTier::Light
        };
        info!(
            "🧮 hardware: {:.1} GiB RAM, {cores} cores → {} model",
            mem as f64 / (1024.0 * 1024.0 * 1024.0),
            tier.label()
        );
        tier
    }
}

/// Total physical memory, from /proc/meminfo. `None` where that is unavailable.
fn total_memory_bytes() -> Option<u64> {
    let text = std::fs::read_to_string("/proc/meminfo").ok()?;
    for line in text.lines() {
        if let Some(rest) = line.strip_prefix("MemTotal:") {
```

```
            let kb: u64 = rest.split_whitespace().next()?.parse().ok()?;
            return kb.checked_mul(1024);
        }
    }
    None
}

/// Prompt wrapping applied before the model sees the text.
#[derive(Clone, Copy, Debug, PartialEq, Eq, ValueEnum)]
enum PromptFormat {
    /// Llama 3.x instruct chat template (Llama 3.0/3.1/3.2 all share it)
    Llama3,
    /// ChatML — Qwen, Yi, and others
    Chatml,
    /// Hand the prompt through untouched (base/completion models)
    Raw,
}

impl PromptFormat {
    fn wrap(self, prompt: &str) -> String {
        match self {
            PromptFormat::Raw => prompt.to_string(),
            PromptFormat::Llama3 => format!(
                "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\n\
                 {prompt}<|eot_id|><|start_header_id|>assistant<|end_header_id|>\n\n"
            ),
            PromptFormat::Chatml => format!(
                "<|im_start|>user\n{prompt}<|im_end|>\n<|im_start|>assistant\n"
            ),
        }
    }

    /// A conversation in this template, ending where the assistant's reply begins.
One user turn
    /// gives exactly what `wrap` gives.
    fn wrap_chat(self, turns: &[ChatTurn]) -> String {
        match self {
            PromptFormat::Raw => {
                if let [only] = turns {
                    return only.content.clone();
                }
                let mut out = String::new();
                for t in turns {
                    let who = match t.role.as_str() {
                        "system" => "System",
                        "assistant" => "Assistant",
                        _ => "User",
                    };
                    out.push_str(&format!("{who}: {}\n\n", t.content));
                }
```

```
                out.push_str("Assistant:");
                out
            }
            PromptFormat::Llama3 => {
                let mut out = String::from("<|begin_of_text|>");
                for t in turns {
                    out.push_str(&format!(
                        "<|start_header_id|>{}<|end_header_id|>\n\n{}<|eot_id|>",
                        t.role, t.content
                    ));
                }
                out.push_str("<|start_header_id|>assistant<|end_header_id|>\n\n");
                out
            }
            PromptFormat::Chatml => {
                let mut out = String::new();
                for t in turns {
                    out.push_str(&format!("<|im_start|>{}\n{}<|im_end|>\n", t.role,
t.content));
                }
                out.push_str("<|im_start|>assistant\n");
                out
            }
        }
    }
}

/// Who may spend this node's inference budget.
///
/// Reaching the inference path at all already requires a completed ML-KEM/ML-DSA
handshake,
/// admission by the trust store, and surviving the revocation check — so `Mesh`
means "any
/// trusted mesh member", not "anyone". The protections that actually bound cost
(thermal gate,
/// cooldown, one-inference-at-a-time, prompt cap, deadline) apply identically in
every mode.
#[derive(Clone, Copy, Debug, PartialEq, Eq, ValueEnum)]
enum InferPolicy {
    /// Any trusted, non-revoked peer on the mesh (default)
    Mesh,
    /// Only identities given to --infer-allow (or --cmd-allow when that is empty)
    Allowlist,
    /// Serve no remote inference at all
    None,
}

#[derive(Clone, Copy, Debug, PartialEq, Eq, ValueEnum)]
enum TrustMode {
    /// Pin identities on first contact (default)
```

```
    Tofu,
    /// Accept only identities from --trust or trusted_peers.txt
    Strict,
}

#[derive(Parser, Debug)]
#[command(
    name = "rust_p2p_node",
    version,
    about = "Harmony gateway node — CNSA 2.0 mesh with identity-hiding handshake
(proto 3)"
)]
struct Cli {
    /// TCP port of the mesh API. The identity directory is keyed by this port: one
port per node.
    #[arg(long, env = "HARMONY_PORT", default_value_t = DEFAULT_PORT)]
    port: u16,
    /// Address the mesh API binds to
    #[arg(long, env = "HARMONY_BIND", default_value = "0.0.0.0")]
    bind: IpAddr,
    /// UDP port of the radio simulator (walks up to 16 ports if busy)
    #[arg(long, env = "HARMONY_UDP_PORT", default_value_t = DEFAULT_UDP_PORT)]
    udp_port: u16,
    /// Address the radio simulator binds to (loopback unless you really need
otherwise)
    #[arg(long, env = "HARMONY_UDP_BIND", default_value = "127.0.0.1")]
    udp_bind: IpAddr,
    /// Seed peer as ip:port (repeatable or comma-separated)
    #[arg(long = "peer", env = "HARMONY_PEERS", value_delimiter = ',')]
    peers: Vec<String>,
    /// Base data directory (a per-node subdirectory is created inside it)
    #[arg(long, env = "HARMONY_DATA_DIR")]
    data_dir: Option<PathBuf>,
    /// Name of this node's identity directory inside the data dir. Defaults to
    /// `node-<port>`. Set it explicitly to decouple identity from the listening
port, so a
    /// node keeps the same keys (and the same trust from its peers) wherever it
binds.
    #[arg(long, env = "HARMONY_NODE_NAME")]
    node_name: Option<String>,
    /// On AddrInUse, try up to this many ports above --port. Identity stays with --
node-name (or
    /// the requested port), and a lock stops a second copy of the same identity from
rotating
    /// onto the mesh. Peers learn the new port from the signed handshake, and nodes
sweep nearby
    /// ports to find a peer that moved. 0 fails immediately instead.
    #[arg(long, env = "HARMONY_PORT_ROTATE", default_value_t = PORT_SWEEP)]
    port_rotate: u16,
    /// Seconds to let in-flight work finish during shutdown before abandoning it
```

```
    #[arg(long, env = "HARMONY_SHUTDOWN_GRACE", default_value_t = 10)]
    shutdown_grace: u64,
    /// How unknown peer identities are handled
    #[arg(long, env = "HARMONY_TRUST_MODE", value_enum, default_value_t =
TrustMode::Tofu)]
    trust_mode: TrustMode,
    /// Trusted peer identity (64 hex chars, see --print-id); repeatable or comma-
separated. Under
    /// `--trust-mode strict` these are the only identities this node talks to.
    #[arg(long = "trust", env = "HARMONY_TRUST", value_delimiter = ',')]
    trust: Vec<String>,
    /// Permanently refuse this identity (64 hex chars). Overrides any pin, is
appended to
    /// revoked_peers.txt, and takes effect immediately for live sessions.
Repeatable.
    #[arg(long = "revoke", value_delimiter = ',')]
    revoke: Vec<String>,
    /// Identity allowed to send radio directives (this node's own identity is always
allowed)
    #[arg(long = "cmd-allow", env = "HARMONY_CMD_ALLOW", value_delimiter = ',')]
    cmd_allow: Vec<String>,
    /// Identity whose AI placement decisions this node adopts (64 hex, repeatable or
    /// comma-separated), besides its own and --cmd-allow ones. A decision steers
where this node's
    /// prompts go for up to ten minutes, so nobody else's is adopted.
    #[arg(long = "advisor", env = "HARMONY_ADVISOR", value_delimiter = ',')]
    advisor: Vec<String>,
    /// Also gossip with public (non-private, non-CGNAT, non-ULA) addresses
    #[arg(long, env = "HARMONY_ALLOW_PUBLIC")]
    allow_public: bool,
    /// Allow loopback peers (several nodes on one device)
    #[arg(long, env = "HARMONY_DEV_LOOPBACK")]
    dev_allow_loopback: bool,
    /// Refuse LLM advisory runs at or above this temperature (°C)
    #[arg(long, env = "HARMONY_MAX_TEMP_C", default_value_t = 65.0)]
    max_temp: f32,
    /// Seconds between gossip rounds. The actual interval is randomised around this
by
    /// ±50%, so the node does not emit on a predictable clock.
    #[arg(long, env = "HARMONY_GOSSIP_SECS", default_value_t = 5)]
    gossip_secs: u64,
    /// Start in emission control: answer what arrives, originate nothing. Toggle at
runtime
    /// with `POST /emcon` (loopback/admin).
    #[arg(long, env = "HARMONY_EMCON", default_value_t = false)]
    emcon: bool,
    /// Fuzz a parser instead of starting the node. Build with `--features fuzz`.
    #[cfg(feature = "fuzz")]
    #[arg(long, value_enum)]
    fuzz: Option<fuzz::Target>,
```

```
    /// Cases per target.
    #[cfg(feature = "fuzz")]
    #[arg(long, default_value_t = 20_000)]
    fuzz_iters: u64,
    /// Seed for the mutation RNG. The same seed replays the same run exactly.
    #[cfg(feature = "fuzz")]
    #[arg(long, default_value_t = 1)]
    fuzz_seed: u64,
    /// Where to write inputs that panic, forge, or break an invariant.
    #[cfg(feature = "fuzz")]
    #[arg(long, default_value = "fuzz-findings")]
    fuzz_out: PathBuf,
    /// Stop serving `GET /id`. Identity-hiding in the handshake is undone if this
node hands
    /// its own bundle to anyone who asks, so a deployment that cares about the
participant
    /// graph provisions bundles out of band (drop them in `<data-dir>/bundles/`) and
sets
    /// this. It breaks trust-on-first-use: peers without a provisioned bundle cannot
dial in.
    #[arg(long, env = "HARMONY_NO_ID_ENDPOINT", default_value_t = false)]
    no_id_endpoint: bool,
    /// Kill the LLM process after this many seconds. Unset, it follows the model
tier:
    /// 60s for 1B, 120s for 3B, 300s for 6B, 420s for 8B. A flat default was a real
bug —
    /// 60 seconds kills almost every 8B generation on a phone CPU before it
finishes, and
    /// the failure looks like a broken model rather than a clock.
    #[arg(long, env = "HARMONY_LLM_TIMEOUT_SECS")]
    llm_timeout_secs: Option<u64>,
    /// Which model tier to run. `auto` picks from RAM and core count at startup.
    #[arg(long, env = "HARMONY_MODEL_TIER", value_enum, default_value_t =
ModelTier::Auto)]
    model_tier: ModelTier,
    /// GGUF used by the light (~1B) tier
    #[arg(long, env = "HARMONY_MODEL_LIGHT", default_value = "llama-3.2-1b-
instruct.gguf")]
    model_light: String,
    /// GGUF used by the standard (~3B) tier
    #[arg(long, env = "HARMONY_MODEL_STANDARD", default_value = "llama-3.2-3b-
instruct.gguf")]
    model_standard: String,
    /// GGUF used by the mid (~6B) tier. Yi-1.5-6B-Chat is the realistic option at
this size;
    /// it expects ChatML, so pair it with `--prompt-format chatml`.
    #[arg(long, env = "HARMONY_MODEL_MID", default_value = "yi-1.5-6b-chat-
q4_k_m.gguf")]
    model_mid: String,
    /// GGUF used by the heavy (~8B) tier. Use a Q4_K_M build on memory-constrained
```

```
devices.
    #[arg(long, env = "HARMONY_MODEL_HEAVY", default_value = "llama-3.1-8b-instruct-
q4_k_m.gguf")]
    model_heavy: String,
    /// Deprecated alias for --model-light; kept so existing invocations keep working
    #[arg(long, env = "HARMONY_MODEL_PATH")]
    model_path: Option<String>,
    /// Who may request inference from this node. Defaults to every trusted mesh
peer, which
    /// is what makes the mesh a shared compute pool; tighten to `allowlist` or
`none` if this
    /// device should not answer for others.
    #[arg(long, env = "HARMONY_INFER_POLICY", value_enum, default_value_t =
InferPolicy::Mesh)]
    infer_policy: InferPolicy,
    /// Identities allowed to request inference when --infer-policy=allowlist (64 hex
each,
    /// repeatable). Falls back to --cmd-allow when empty.
    #[arg(long = "infer-allow", env = "HARMONY_INFER_ALLOW", value_delimiter = ',')]
    infer_allow: Vec<String>,
    /// How many peers a distributed inference queries by default
    #[arg(long, env = "HARMONY_FANOUT", default_value_t = DEFAULT_FANOUT)]
    fanout: usize,
    /// llama.cpp binary
    #[arg(long, env = "HARMONY_LLAMA_BIN", default_value = "llama-cli")]
    llama_bin: String,
    /// Extra llama-cli flags, whitespace separated.
    ///
    /// `--repeat-penalty` matters: llama.cpp defaults it to 1.0, which *disables*
it, and an
    /// unpenalised model readily falls into a loop repeating one token forever. `-
no-cnv`
    /// keeps `-p` a plain completion prompt rather than entering interactive chat.
    #[arg(
        long,
        env = "HARMONY_LLAMA_FLAGS",
        default_value = "-no-cnv --repeat-penalty 1.15 --temp 0.7 --top-p 0.9",
        allow_hyphen_values = true
    )]
    llama_flags: String,
    /// How to wrap the prompt before handing it to the model.
    ///
    /// Instruct models expect their chat template; without it they see an
unstructured blob
    /// and degenerate. `-no-cnv` turns off llama.cpp's own template handling, so we
apply it
    /// here instead — which also keeps behaviour identical across llama.cpp
versions.
    #[arg(long, env = "HARMONY_PROMPT_FORMAT", value_enum, default_value_t =
PromptFormat::Llama3)]
```

```
    prompt_format: PromptFormat,
    /// llama.cpp's `llama-server`, which keeps the model loaded between jobs (v19).
`auto` looks
    /// beside --llama-bin, then on PATH; a path names it; `off` runs every job with
--llama-bin.
    /// Without one, or when it will not start, jobs run with --llama-bin as before.
    #[arg(long, env = "HARMONY_LLAMA_SERVER", default_value = "auto")]
    llama_server: String,
    /// Unload the kept-loaded model after this many seconds without a job, giving
its memory back;
    /// the next job loads it again. 0 keeps it loaded.
    #[arg(long, env = "HARMONY_LLAMA_IDLE_SECS", default_value_t = 900)]
    llama_idle_secs: u64,
    /// Simulated radio packet loss in percent
    #[arg(long, env = "RF_LOSS_RATE", default_value_t = 0)]
    rf_loss_rate: u32,
    /// Simulated radio latency in milliseconds
    #[arg(long, env = "RF_LATENCY_MS", default_value_t = 800)]
    rf_latency_ms: u64,
    /// Optional token for the loopback admin routes (Authorization: Bearer <token>)
    #[arg(long, env = "HARMONY_ADMIN_TOKEN", hide_env_values = true)]
    admin_token: Option<String>,
    /// What this device is, as the advisory model and the dashboard name it
("phone",
    /// "laptop", "jetson-2"...). Gossiped with the vitals, so a directive can name a
device.
    /// Default: "phone" under Termux/Android, else "laptop" if there is a battery,
else
    /// "desktop".
    #[arg(long, env = "HARMONY_LABEL")]
    label: Option<String>,
    /// Print this node's identity (hex) and exit
    #[arg(long)]
    print_id: bool,
    /// Replace this node's ML-KEM key and exit. The ML-DSA identity (and therefore
the
    /// fingerprint peers trust) is unchanged, and the new encapsulation key is
published in a
    /// freshly signed bundle, so peers pick it up on their next handshake with no
re-pinning.
    #[arg(long)]
    rotate_kem: bool,
    /// Sign the staged build in DIR with this node's identity (writes MANIFEST and
MANIFEST.sig
    /// there) and exit. mesh_upgrade.sh runs it on the phone for the Windows
notebook's build.
    #[arg(long, value_name = "DIR")]
    sign_update: Option<PathBuf>,
    /// Check that the staged build in DIR is exactly what --update-signer signed;
print
```

```
    /// "OK build=... issued=..." or "FAIL <reason>" and exit 0 or 1. node.ps1 runs
it with the node
    /// it already has, before it installs a new one.
    #[arg(long, value_name = "DIR")]
    verify_update: Option<PathBuf>,
    /// The identity (64 hex) whose signature --verify-update requires
    #[arg(long, value_name = "ID")]
    update_signer: Option<String>,
}

struct Config {
    bind: IpAddr,
    port: u16,
    udp_bind: IpAddr,
    udp_port: u16,
    seeds: Vec<SocketAddr>,
    data_dir: PathBuf,
    port_rotate: u16,
    shutdown_grace: Duration,
    trust_mode: TrustMode,
    trusted: Vec<NodeId>,
    revoked: Vec<NodeId>,
    cmd_allow: HashSet<NodeId>,
    /// Identities whose AI placement decisions this node adopts (besides its own and
cmd_allow).
    advisors: HashSet<NodeId>,
    allow_public: bool,
    allow_loopback: bool,
    max_temp: f32,
    gossip_secs: u64,
    emcon: bool,
    no_id_endpoint: bool,
    llm_timeout: Duration,
    tier: ModelTier,
    model_path: String,
    infer_policy: InferPolicy,
    infer_allow: HashSet<NodeId>,
    fanout: usize,
    llama_bin: String,
    llama_flags: Vec<String>,
    prompt_format: PromptFormat,
    /// The kept-loaded model's server as given ("auto" or a path); None runs every
job with
    /// llama_bin.
    llama_server: Option<String>,
    /// Unload the kept-loaded model after this long without a job; None keeps it
loaded.
    llama_idle: Option<Duration>,
    loss_rate: u32,
    latency_ms: u64,
```

```
    admin_token: Option<String>,
    label: String,
}

impl Config {
    fn from_cli(cli: Cli) -> Result<Self, BoxErr> {
        if cli.rf_loss_rate > 100 {
            return Err("RF_LOSS_RATE / --rf-loss-rate must be between 0 and
100".into());
        }
        // An empty entry (HARMONY_TRUST= in an env file) is no entry, not a
malformed id.
        let mut trusted = Vec::new();
        for t in cli.trust.iter().filter(|t| !t.trim().is_empty()) {
            trusted.push(
                unhex32(t).ok_or_else(|| format!("--trust expects 64 hex characters,
got {t:?}"))?,
            );
        }
        let mut revoked = Vec::new();
        for t in &cli.revoke {
            revoked.push(
                unhex32(t)
                    .ok_or_else(|| format!("--revoke expects 64 hex characters, got
{t:?}"))?,
            );
        }
        let mut cmd_allow = HashSet::new();
        for t in &cli.cmd_allow {
            cmd_allow.insert(
                unhex32(t)
                    .ok_or_else(|| format!("--cmd-allow expects 64 hex characters,
got {t:?}"))?,
            );
        }
        let mut advisors = HashSet::new();
        for t in cli.advisor.iter().filter(|t| !t.trim().is_empty()) {
            advisors.insert(
                unhex32(t)
                    .ok_or_else(|| format!("--advisor expects 64 hex characters, got
{t:?}"))?,
            );
        }
        // The token is the only thing guarding the admin routes off loopback, and it
travels in a
        // plain HTTP header: a short one is a guess away.
        if cli.admin_token.as_deref().is_some_and(|t| !t.is_empty() && t.len() < 24)
{
            return Err("HARMONY_ADMIN_TOKEN must be at least 24 characters (or
unset)".into());
```

```
        }
        let mut infer_allow = HashSet::new();
        for t in &cli.infer_allow {
            infer_allow.insert(
                unhex32(t)
                    .ok_or_else(|| format!("--infer-allow expects 64 hex characters,
got {t:?}"))?,
            );
        }
        // The allowlist only bites under --infer-policy=allowlist; inherit the
command list
        // there so operators do not have to repeat themselves.
        if infer_allow.is_empty() {
            infer_allow = cmd_allow.clone();
        }

        let tier = cli.model_tier.resolve();
        // --model-path is the old single-model flag; honour it as the light model so
existing
        // deployments keep the file they already have.
        let model_path = match (&cli.model_path, tier) {
            (Some(p), _) => p.clone(),
            (None, ModelTier::Heavy) => cli.model_heavy.clone(),
            (None, ModelTier::Mid) => cli.model_mid.clone(),
            (None, ModelTier::Standard) => cli.model_standard.clone(),
            (None, _) => cli.model_light.clone(),
        };
        match tier {
            ModelTier::Heavy => warn!(
                "⚠ heavy (8B) tier: ~5 GB resident, single-digit tokens/sec on CPU.
Timeout \
                 defaults to {}s for this tier. On a big.LITTLE phone try \
                 --llama-flags '-t 4 -n 256': llama.cpp defaults to every core, and
the \
                 little cores drag the whole batch down to their own speed.",
                tier.default_timeout_secs()
            ),
            ModelTier::Mid => warn!(
                "⚠ mid (6B) tier selected: expect ~3.8 GB resident. Yi-1.5-6B-Chat
wants \
                 --prompt-format chatml; the wrong template costs more quality than
the \
                 extra parameters buy."
            ),
            _ => {}
        }
        let mut seeds = Vec::new();
        for s in &cli.peers {
            match parse_peer(s, DEFAULT_PORT) {
                Some(a)
```

```
                    if a.port() != 0 && !a.ip().is_unspecified() &&
!a.ip().is_multicast() =>
                {
                    seeds.push(a)
                }
                _ => warn!("ignoring invalid --peer {s:?} (expected ip:port)"),
            }
        }
        let base = cli.data_dir.clone().unwrap_or_else(default_data_base);
        Ok(Config {
            data_dir: base.join(identity_dir_name(&cli.node_name, cli.port)),
            port_rotate: cli.port_rotate,
            shutdown_grace: Duration::from_secs(cli.shutdown_grace.max(1)),
            bind: cli.bind,
            port: cli.port,
            udp_bind: cli.udp_bind,
            udp_port: cli.udp_port,
            seeds,
            trust_mode: cli.trust_mode,
            trusted,
            revoked,
            cmd_allow,
            advisors,
            allow_public: cli.allow_public,
            allow_loopback: cli.dev_allow_loopback,
            max_temp: cli.max_temp,
            gossip_secs: cli.gossip_secs.max(1),
            emcon: cli.emcon,
            no_id_endpoint: cli.no_id_endpoint,
            llm_timeout: Duration::from_secs(
                cli.llm_timeout_secs.map(|s| s.max(1)).unwrap_or_else(||
tier.default_timeout_secs()),
            ),
            tier,
            model_path,
            infer_policy: cli.infer_policy,
            infer_allow,
            fanout: cli.fanout.clamp(1, MAX_FANOUT),
            llama_bin: prefer_completion_bin(cli.llama_bin),
            llama_flags:
cli.llama_flags.split_whitespace().map(String::from).collect(),
            prompt_format: cli.prompt_format,
            llama_server: server_choice(&cli.llama_server),
            llama_idle: (cli.llama_idle_secs > 0).then(||
Duration::from_secs(cli.llama_idle_secs)),
            loss_rate: cli.rf_loss_rate,
            latency_ms: cli.rf_latency_ms,
            admin_token: cli.admin_token.filter(|t| !t.is_empty()),
            label: device_label(cli.label.as_deref()),
        })
```

```
    }
}

/// `--llama-server` as given; None when it is off.
fn server_choice(s: &str) -> Option<String> {
    match s.trim() {
        "" | "off" | "none" | "no" | "false" | "0" => None,
        s => Some(s.to_string()),
    }
}

// ──────────────────────────────────── crypto backend
────────────────────────────────────
// Everything that touches a third-party crypto API lives in this module, so a crate
// upgrade only ever means editing this block.

mod crypto {
    use aes_gcm::aead::{Aead, KeyInit as AeadKeyInit, Payload};
    use aes_gcm::{Aes256Gcm, Nonce};
    use hkdf::Hkdf;
    use ml_dsa::{
        Generate as DsaGenerate, KeyExport as DsaKeyExport, KeyInit as DsaKeyInit,
Keypair,
        MlDsa87, Signature as DsaSignature, SignatureEncoding, Signer, SigningKey,
Verifier,
        VerifyingKey,
    };
    // NOTE: ml-kem and ml-dsa re-export the same `Generate` / `KeyExport` traits
from
    // crypto-common, so importing them once (above, from ml_dsa) covers both crates'
call
    // sites. Importing them twice is what produced the "unused import" warnings.
    use ml_kem::{
        Decapsulate, DecapsulationKey1024, Encapsulate, EncapsulationKey1024, Seed as
KemSeed,
        TryKeyInit,
    };
    use sha2::{Digest, Sha384};
    use zeroize::Zeroizing;

    use super::{
        BUNDLE_LEN, CTX_BUNDLE, CTX_FINGERPRINT, CTX_K0, CTX_KEYS, KEM_CT_LEN,
KEM_EK_LEN,
        NodeId, SIG_LEN, SIG_PK_LEN,
    };

    // ── ML-KEM-1024 (FIPS 203), ml-kem 0.3.x ──

    pub struct KemKeypair {
        dk: DecapsulationKey1024,
```

```
        ek_bytes: [u8; KEM_EK_LEN],
    }

    impl KemKeypair {
        pub fn generate() -> Self {
            Self::from_dk(DecapsulationKey1024::generate())
        }

        pub fn from_seed(seed: &[u8; 64]) -> Self {
            Self::from_dk(DecapsulationKey1024::from_seed(KemSeed::from(*seed)))
        }

        fn from_dk(dk: DecapsulationKey1024) -> Self {
            let ek = dk.encapsulation_key().to_bytes();
            let mut ek_bytes = [0u8; KEM_EK_LEN];
            ek_bytes.copy_from_slice(ek.as_slice());
            Self { dk, ek_bytes }
        }

        /// The 64-byte seed is the canonical private-key serialization.
        pub fn seed(&self) -> Option<[u8; 64]> {
            self.dk.to_seed().map(|s| {
                let mut out = [0u8; 64];
                out.copy_from_slice(s.as_slice());
                out
            })
        }

        pub fn ek_bytes(&self) -> &[u8; KEM_EK_LEN] {
            &self.ek_bytes
        }

        /// ML-KEM uses implicit rejection: a forged ciphertext still yields *a*
secret, it
        /// just will not match the peer's, so the key-confirmation tag fails later.
        pub fn decapsulate(&self, ct: &[u8]) -> Option<Zeroizing<[u8; 32]>> {
            let ss = self.dk.decapsulate_slice(ct).ok()?;
            let mut out = Zeroizing::new([0u8; 32]);
            out.copy_from_slice(ss.as_slice());
            Some(out)
        }
    }

    /// Encapsulate to a peer's encapsulation key (validates the key encoding).
    pub fn kem_encapsulate(ek_bytes: &[u8]) -> Option<([u8; KEM_CT_LEN],
Zeroizing<[u8; 32]>)> {
        let ek = EncapsulationKey1024::new_from_slice(ek_bytes).ok()?;
        let (ct, ss) = ek.encapsulate();
        let mut ct_out = [0u8; KEM_CT_LEN];
        ct_out.copy_from_slice(ct.as_slice());
```

```
        let mut ss_out = Zeroizing::new([0u8; 32]);
        ss_out.copy_from_slice(ss.as_slice());
        Some((ct_out, ss_out))
    }

    // ── ML-DSA-87 (FIPS 204) identity ──
    //
    // A node is named publicly by its full 2592-byte verifying key. Everywhere
outside this
    // module it is referred to by its 32-byte fingerprint (see `fingerprint`), which
is what
    // maps, the trust store and the CLI flags use.

    /// SHA-384(ctx || vk)[..32] — a stable, collision-resistant short name for a
verifying key.
    pub fn fingerprint(vk_bytes: &[u8]) -> NodeId {
        let full = sha384_parts(&[CTX_FINGERPRINT, vk_bytes]);
        let mut out = [0u8; 32];
        out.copy_from_slice(&full[..32]);
        out
    }

    pub struct IdKey {
        sk: SigningKey<MlDsa87>,
        vk_bytes: Vec<u8>,
        pub fpr: NodeId,
    }

    impl IdKey {
        pub fn from_seed(seed: &[u8; 32]) -> Self {
            // ml-dsa Seed is a non-generic 32-byte array type (B32); build it from
the bytes.
            let xi = ml_dsa::Seed::from(*seed);
            Self::from_sk(SigningKey::<MlDsa87>::from_seed(&xi))
        }

        pub fn generate() -> Self {
            Self::from_sk(SigningKey::<MlDsa87>::generate())
        }

        fn from_sk(sk: SigningKey<MlDsa87>) -> Self {
            let vk_bytes = sk.verifying_key().to_bytes().as_slice().to_vec();
            let fpr = fingerprint(&vk_bytes);
            Self { sk, vk_bytes, fpr }
        }

        pub fn vk_bytes(&self) -> &[u8] {
            &self.vk_bytes
        }
```

```
        pub fn seed(&self) -> [u8; 32] {
            // ml-dsa Seed is `#[repr(transparent)] struct Seed(pub [u8; 32])`.
            self.sk.to_seed().0
        }

        pub fn sign(&self, msg: &[u8]) -> Vec<u8> {
            // `to_vec` (alloc feature) yields the fixed-size encoded signature as
bytes.
            self.sk.sign(msg).to_vec()
        }
    }

    /// Verify `sig` over `msg` against a full ML-DSA-87 verifying key encoding.
    pub fn verify(vk_bytes: &[u8], msg: &[u8], sig: &[u8]) -> bool {
        if vk_bytes.len() != SIG_PK_LEN || sig.len() != SIG_LEN {
            return false;
        }
        let Ok(vk) = VerifyingKey::<MlDsa87>::new_from_slice(vk_bytes) else {
            return false;
        };
        let Ok(sig) = DsaSignature::<MlDsa87>::try_from(sig) else {
            return false;
        };
        vk.verify(msg, &sig).is_ok()
    }

    // ── AES-256-GCM with counter nonces (a key is never used with the same counter
twice) ──

    fn nonce_bytes(counter: u64) -> [u8; 12] {
        let mut n = [0u8; 12];
        n[4..].copy_from_slice(&counter.to_be_bytes());
        n
    }

    pub fn aead_seal(key: &[u8; 32], counter: u64, aad: &[u8], pt: &[u8]) ->
Option<Vec<u8>> {
        let cipher = Aes256Gcm::new_from_slice(key).ok()?;
        let n = nonce_bytes(counter);
        cipher
            .encrypt(Nonce::from_slice(&n), Payload { msg: pt, aad })
            .ok()
    }

    /// Decrypted plaintext is wrapped so it is wiped when dropped rather than left
in freed
    /// heap for whatever allocates there next.
    pub fn aead_open(
        key: &[u8; 32],
        counter: u64,
```

```
        aad: &[u8],
        ct: &[u8],
    ) -> Option<Zeroizing<Vec<u8>>> {
        let cipher = Aes256Gcm::new_from_slice(key).ok()?;
        let n = nonce_bytes(counter);
        cipher
            .decrypt(Nonce::from_slice(&n), Payload { msg: ct, aad })
            .ok()
            .map(Zeroizing::new)
    }

    // ── hashing / key schedule ──

    /// Full 48-byte SHA-384 digest of the concatenated parts.
    pub fn sha384_parts(parts: &[&[u8]]) -> [u8; 48] {
        let mut h = Sha384::new();
        for p in parts {
            h.update(*p);
        }
        let d = h.finalize();
        let mut out = [0u8; 48];
        out.copy_from_slice(&d);
        out
    }

    /// SHA-384 of a file, read in 64 KiB pieces, and its size.
    pub fn sha384_file(path: &std::path::Path) -> std::io::Result<(u64, [u8; 48])> {
        use std::io::Read;
        let mut f = std::fs::File::open(path)?;
        let mut h = Sha384::new();
        let mut buf = vec![0u8; 64 * 1024];
        let mut size = 0u64;
        loop {
            let n = f.read(&mut buf)?;
            if n == 0 {
                break;
            }
            h.update(&buf[..n]);
            size += n as u64;
        }
        let d = h.finalize();
        let mut out = [0u8; 48];
        out.copy_from_slice(&d);
        Ok((size, out))
    }

    /// Transcript hash, truncated to 32 bytes for use as HKDF salt and AEAD AAD.
    pub fn transcript32(parts: &[&[u8]]) -> [u8; 32] {
        let full = sha384_parts(parts);
        let mut out = [0u8; 32];
```

```
        out.copy_from_slice(&full[..32]);
        out
    }

    pub struct SessionKeys {
        pub k_i2r: Zeroizing<[u8; 32]>,
        pub k_r2i: Zeroizing<[u8; 32]>,
        pub k_confirm: Zeroizing<[u8; 32]>,
        pub session_id: [u8; 8],
    }

    pub fn derive_session_keys(
        th: &[u8; 32],
        ss_static: &[u8; 32],
        ss_ephemeral: &[u8; 32],
    ) -> Option<SessionKeys> {
        let mut ikm = Zeroizing::new([0u8; 64]);
        ikm[..32].copy_from_slice(ss_static);
        ikm[32..].copy_from_slice(ss_ephemeral);
        let hk = Hkdf::<Sha384>::new(Some(&th[..]), &ikm[..]);
        let mut okm = Zeroizing::new([0u8; 104]);
        hk.expand(CTX_KEYS, &mut okm[..]).ok()?;

        let mut k_i2r = Zeroizing::new([0u8; 32]);
        let mut k_r2i = Zeroizing::new([0u8; 32]);
        let mut k_confirm = Zeroizing::new([0u8; 32]);
        let mut session_id = [0u8; 8];
        k_i2r.copy_from_slice(&okm[0..32]);
        k_r2i.copy_from_slice(&okm[32..64]);
        k_confirm.copy_from_slice(&okm[64..96]);
        session_id.copy_from_slice(&okm[96..104]);
        Some(SessionKeys {
            k_i2r,
            k_r2i,
            k_confirm,
            session_id,
        })
    }

    /// Derive the pre-identity key from the static KEM secret alone.
    ///
    /// This is what lets the initiator encrypt *who it is* in its very first
message: only
    /// the holder of the responder's static decapsulation key can recover
`ss_static`, so
    /// only the intended responder can read the identity block.
    ///
    /// Limitation worth stating plainly: this key depends on a long-term secret, so
an
    /// adversary who records a handshake and later compromises the responder's
```

```
static key
    /// can recover the initiator's identity retroactively. Session *contents* are
unaffected
    /// — those also require the ephemeral secret, which is discarded.
    pub fn derive_pre_identity_key(
        h0: &[u8; 32],
        ss_static: &[u8; 32],
    ) -> Option<Zeroizing<[u8; 32]>> {
        let hk = Hkdf::<Sha384>::new(Some(&h0[..]), &ss_static[..]);
        let mut k0 = Zeroizing::new([0u8; 32]);
        hk.expand(CTX_K0, &mut k0[..]).ok()?;
        Some(k0)
    }

    /// Key-confirmation tag: AES-GCM over an empty message with a single-use key.
    pub fn confirm_tag(k_confirm: &[u8; 32], th: &[u8; 32]) -> Option<Vec<u8>> {
        aead_seal(k_confirm, 0, th, &[])
    }

    // ── identity bundle: sig_vk(2592) | kem_ek(1568) | ML-DSA signature(4627) over
vk||ek ──

    fn bundle_sig_input(vk: &[u8], ek: &[u8]) -> Vec<u8> {
        let mut v = Vec::with_capacity(CTX_BUNDLE.len() + vk.len() + ek.len());
        v.extend_from_slice(CTX_BUNDLE);
        v.extend_from_slice(vk);
        v.extend_from_slice(ek);
        v
    }

    pub fn build_bundle(id: &IdKey, kem: &KemKeypair) -> Vec<u8> {
        let mut v = Vec::with_capacity(BUNDLE_LEN);
        v.extend_from_slice(id.vk_bytes());
        v.extend_from_slice(kem.ek_bytes());
        let sig = id.sign(&bundle_sig_input(id.vk_bytes(), kem.ek_bytes()));
        v.extend_from_slice(&sig);
        debug_assert_eq!(v.len(), BUNDLE_LEN);
        v
    }

    #[derive(Clone)]
    pub struct PeerBundle {
        pub fpr: NodeId,
        /// Full verifying key. Retained for callers that need to re-verify or re-
publish an
        /// identity; the handshake itself works from `fpr`.
        #[allow(dead_code)]
        pub vk: Vec<u8>,
        pub ek: Vec<u8>,
    }
```

```
    pub fn parse_bundle(b: &[u8]) -> Option<PeerBundle> {
        if b.len() != BUNDLE_LEN {
            return None;
        }
        let vk = &b[..SIG_PK_LEN];
        let ek = &b[SIG_PK_LEN..SIG_PK_LEN + KEM_EK_LEN];
        let sig = &b[SIG_PK_LEN + KEM_EK_LEN..];
        if !verify(vk, &bundle_sig_input(vk, ek), sig) {
            return None;
        }
        // Validate the encapsulation key encoding now, so a later failure is
attributable.
        if EncapsulationKey1024::new_from_slice(ek).is_err() {
            return None;
        }
        Some(PeerBundle {
            fpr: fingerprint(vk),
            vk: vk.to_vec(),
            ek: ek.to_vec(),
        })
    }
}

// ──────────────────────────────────── identity ────────────────────────────────────

struct Identity {
    id: crypto::IdKey,
    kem: crypto::KemKeypair,
    bundle: Vec<u8>,
}

impl Identity {
    fn from_parts(id: crypto::IdKey, kem: crypto::KemKeypair) -> Self {
        let bundle = crypto::build_bundle(&id, &kem);
        Self { id, kem, bundle }
    }

    fn fpr(&self) -> NodeId {
        self.id.fpr
    }

    fn vk_bytes(&self) -> &[u8] {
        self.id.vk_bytes()
    }

    fn kem_path(dir: &Path) -> PathBuf {
        dir.join("kem.mlkem1024.seed")
    }
```

```
    /// Replace only the ML-KEM key, keeping the ML-DSA identity. Because the bundle
is signed
    /// by the unchanged identity key, peers verify the new encapsulation key with
the
    /// fingerprint they already trust — rotation needs no re-pinning and no
coordination.
    ///
    /// Sessions already established stay valid (their symmetric keys are derived and
cached);
    /// the new key is used from the next handshake onward.
    fn rotate_kem(dir: &Path) -> Result<Self, BoxErr> {
        let me = Self::load_or_create(dir)?;
        let fresh = crypto::KemKeypair::generate();
        let seed = Zeroizing::new(fresh.seed().ok_or("freshly generated ML-KEM key
has no seed")?);
        replace_secret(&Self::kem_path(dir), &seed[..])?;
        info!("🔄 rotated ML-KEM key; identity {} unchanged", hex(&me.fpr()));
        Self::load_or_create(dir)
    }

    fn load_or_create(dir: &Path) -> Result<Self, BoxErr> {
        std::fs::create_dir_all(dir)?;
        #[cfg(unix)]
        {
            use std::os::unix::fs::PermissionsExt;
            std::fs::set_permissions(dir, std::fs::Permissions::from_mode(0o700))?;
        }

        let sig_path = dir.join("identity.mldsa87.seed");
        let id = match std::fs::read(&sig_path) {
            Ok(b) => {
                let seed = Zeroizing::new(<[u8;
32]>::try_from(b.as_slice()).map_err(|_| {
                    format!("{} must be exactly 32 bytes", sig_path.display())
                })?);
                crypto::IdKey::from_seed(&seed)
            }
            Err(e) if e.kind() == std::io::ErrorKind::NotFound => {
                let k = crypto::IdKey::generate();
                let seed = Zeroizing::new(k.seed());
                write_secret(&sig_path, &seed[..])?;
                info!("🔑 generated a new node identity in {}", dir.display());
                k
            }
            Err(e) => return Err(e.into()),
        };

        let kem_path = Self::kem_path(dir);
        let kem = match std::fs::read(&kem_path) {
            Ok(b) => {
```

```
                let seed = Zeroizing::new(<[u8;
64]>::try_from(b.as_slice()).map_err(|_| {
                    format!("{} must be exactly 64 bytes", kem_path.display())
                })?);
                crypto::KemKeypair::from_seed(&seed)
            }
            Err(e) if e.kind() == std::io::ErrorKind::NotFound => {
                let k = crypto::KemKeypair::generate();
                let seed = Zeroizing::new(k.seed().ok_or("ML-KEM key has no seed")?);
                write_secret(&kem_path, &seed[..])?;
                k
            }
            Err(e) => return Err(e.into()),
        };
        Ok(Self::from_parts(id, kem))
    }
}

#[cfg(any(test, feature = "fuzz"))]
impl Identity {
    fn ephemeral() -> Self {
        Self::from_parts(crypto::IdKey::generate(), crypto::KemKeypair::generate())
    }
}

// ──────────────────────────────────── handshake (pure functions)
────────────────────────────────────

/// Everything an observer can see before the identity block, hashed.
///
/// Binding the provisional key to this means it is unique per handshake and cannot
be lifted
/// from one exchange and applied to another.
fn hello_h0(resp_fpr: &NodeId, ver: u8, ct_static: &[u8], ek_eph: &[u8]) -> [u8; 32]
{
    crypto::transcript32(&[CTX_H0, &resp_fpr[..], &[ver][..], ct_static, ek_eph])
}

/// What the initiator signs.
///
/// It covers the responder's fingerprint (so a hello for A cannot be replayed at B),
both
/// KEM values (so neither can be swapped), and the timestamp and port. The signature
lives
/// *inside* the encrypted block, so it is never visible on the wire.
fn hello_sig_input(
    resp_fpr: &NodeId,
    ver: u8,
    ct_static: &[u8],
    ek_eph: &[u8],
```

```
    ts: u64,
    port: u16,
) -> Vec<u8> {
    let mut v = Vec::with_capacity(CTX_HELLO.len() + 32 + 1 + ct_static.len() +
ek_eph.len() + 10);
    v.extend_from_slice(CTX_HELLO);
    v.extend_from_slice(resp_fpr);
    v.push(ver);
    v.extend_from_slice(ct_static);
    v.extend_from_slice(ek_eph);
    v.extend_from_slice(&ts.to_be_bytes());
    v.extend_from_slice(&port.to_be_bytes());
    v
}

fn transcript_hash(h0: &[u8; 32], ct_e: &[u8]) -> [u8; 32] {
    crypto::transcript32(&[CTX_TRANSCRIPT, &h0[..], ct_e])
}

struct ClientHello {
    hello1: Vec<u8>,
    eph: crypto::KemKeypair,
    ss_s: Zeroizing<[u8; 32]>,
    h0: [u8; 32],
}

/// Initiator step 1 — identity-hiding hello.
///
/// Layout: `ver(1) | kem_ct_static | kem_ek_ephemeral | AEAD_k0(ts | port | vk |
sig)`.
/// The two KEM values look random without the responder's static key, and the
identity block
/// is encrypted under a key derived from that key, so nothing on the wire names the
sender.
fn client_build_hello(
    me: &Identity,
    peer: &crypto::PeerBundle,
    listen_port: u16,
    now: u64,
) -> Option<ClientHello> {
    let (ct_s, ss_s) = crypto::kem_encapsulate(&peer.ek)?;
    let eph = crypto::KemKeypair::generate();

    let h0 = hello_h0(&peer.fpr, PROTO_VER, &ct_s, eph.ek_bytes());
    let k0 = crypto::derive_pre_identity_key(&h0, &ss_s)?;

    let sig = me.id.sign(&hello_sig_input(
        &peer.fpr,
        PROTO_VER,
        &ct_s,
```

```
        eph.ek_bytes(),
        now,
        listen_port,
    ));
    if sig.len() != SIG_LEN {
        return None;
    }

    let mut inner = Zeroizing::new(Vec::with_capacity(INNER_LEN));
    inner.extend_from_slice(&now.to_be_bytes());
    inner.extend_from_slice(&listen_port.to_be_bytes());
    inner.extend_from_slice(me.vk_bytes());
    inner.extend_from_slice(&sig);
    if inner.len() != INNER_LEN {
        return None;
    }

    // The visible header is authenticated as associated data, so an attacker cannot
alter
    // the KEM values and have the ciphertext still open.
    let mut hello1 = Vec::with_capacity(HELLO1_LEN);
    hello1.push(PROTO_VER);
    hello1.extend_from_slice(&ct_s);
    hello1.extend_from_slice(eph.ek_bytes());
    let aad = hello1.clone();
    let sealed = crypto::aead_seal(&k0, 0, &aad, &inner)?;
    hello1.extend_from_slice(&sealed);
    if hello1.len() != HELLO1_LEN {
        return None;
    }

    Some(ClientHello {
        hello1,
        eph,
        ss_s,
        h0,
    })
}

#[derive(Debug, PartialEq, Eq)]
enum HelloError {
    Malformed,
    Stale,
    BadSignature,
    /// The identity block would not open — wrong responder, tampering, or garbage.
    Undecryptable,
}

/// What the responder learns once it has opened and verified a hello1.
struct VerifiedHello {
```

```
    init_fpr: NodeId,
    port: u16,
    /// Carried forward so the expensive KEM step does not have to redo the cheap
work.
    h0: [u8; 32],
    ss_s: Zeroizing<[u8; 32]>,
}

/// Responder step 1: decapsulate the static ciphertext, open the identity block,
verify.
///
/// Unlike v2 this *must* do the static decapsulation up front — the identity is
encrypted
/// under a key derived from it, so there is no cheaper ordering available. The
replay cache
/// and the handshake semaphore in `post_hello` are what keep that affordable.
fn server_verify_hello(
    me: &Identity,
    body: &[u8],
    now: u64,
) -> Result<VerifiedHello, HelloError> {
    if body.len() != HELLO1_LEN || body[0] != PROTO_VER {
        return Err(HelloError::Malformed);
    }
    let ct_s = &body[H1_CT_OFF..H1_CT_OFF + KEM_CT_LEN];
    let ek_e = &body[H1_EK_OFF..H1_EK_OFF + KEM_EK_LEN];
    let sealed = &body[H1_CIPHER_OFF..];

    let ss_s = me.kem.decapsulate(ct_s).ok_or(HelloError::Undecryptable)?;
    let h0 = hello_h0(&me.fpr(), PROTO_VER, ct_s, ek_e);
    let k0 = crypto::derive_pre_identity_key(&h0,
&ss_s).ok_or(HelloError::Undecryptable)?;

    // ML-KEM uses implicit rejection, so a bogus ciphertext yields *a* secret rather
than an
    // error — the forgery surfaces here, as a tag that will not verify.
    let aad = &body[..H1_CIPHER_OFF];
    let inner = crypto::aead_open(&k0, 0, aad,
sealed).ok_or(HelloError::Undecryptable)?;
    if inner.len() != INNER_LEN {
        return Err(HelloError::Malformed);
    }

    let mut ts_b = [0u8; 8];
    ts_b.copy_from_slice(&inner[..8]);
    let ts = u64::from_be_bytes(ts_b);
    let port = u16::from_be_bytes([inner[INNER_PORT_OFF], inner[INNER_PORT_OFF +
1]]);
    let init_vk = &inner[INNER_VK_OFF..INNER_VK_OFF + SIG_PK_LEN];
    let sig = &inner[INNER_SIG_OFF..INNER_LEN];
```

```
    if now.abs_diff(ts) > MAX_CLOCK_SKEW_SECS {
        return Err(HelloError::Stale);
    }

    let signed = hello_sig_input(&me.fpr(), PROTO_VER, ct_s, ek_e, ts, port);
    if !crypto::verify(init_vk, &signed, sig) {
        return Err(HelloError::BadSignature);
    }

    Ok(VerifiedHello {
        init_fpr: crypto::fingerprint(init_vk),
        port,
        h0,
        ss_s,
    })
}

struct ServerHandshake {
    hello2: Vec<u8>,
    keys: crypto::SessionKeys,
}

/// Responder step 2: ephemeral encapsulation, key schedule, confirmation tag.
fn server_finish_hello(body: &[u8], verified: &VerifiedHello) ->
Option<ServerHandshake> {
    if body.len() != HELLO1_LEN {
        return None;
    }
    let ek_e = &body[H1_EK_OFF..H1_EK_OFF + KEM_EK_LEN];
    let (ct_e, ss_e) = crypto::kem_encapsulate(ek_e)?;
    let th = transcript_hash(&verified.h0, &ct_e);
    let keys = crypto::derive_session_keys(&th, &verified.ss_s, &ss_e)?;
    let tag = crypto::confirm_tag(&keys.k_confirm, &th)?;

    let mut hello2 = Vec::with_capacity(HELLO2_LEN);
    hello2.extend_from_slice(&ct_e);
    hello2.extend_from_slice(&tag);
    if hello2.len() != HELLO2_LEN {
        return None;
    }
    Some(ServerHandshake { hello2, keys })
}

/// Initiator step 2: the confirmation tag is what authenticates the responder — only
a node
/// holding the static decapsulation key can produce it.
fn client_finish_hello(ch: ClientHello, resp: &[u8]) -> Option<crypto::SessionKeys> {
    if resp.len() != HELLO2_LEN {
        return None;
```

```
    }
    let (ct_e, tag) = resp.split_at(KEM_CT_LEN);
    let ss_e = ch.eph.decapsulate(ct_e)?;
    let th = transcript_hash(&ch.h0, ct_e);
    let keys = crypto::derive_session_keys(&th, &ch.ss_s, &ss_e)?;
    let expect = crypto::confirm_tag(&keys.k_confirm, &th)?;
    if !ct_eq(&expect, tag) {
        return None;
    }
    Some(keys)
}

// ──────────────────────────────────── frames & sessions
────────────────────────────────────


/// Round a payload up to a fixed bucket so the ciphertext length stops describing
the
/// plaintext.
///
/// Without this, every message type has a recognisable size: a peer-gossip reply, a
directive
/// and an inference answer are each distinguishable on sight, and an observer can
follow the
/// shape of a conversation it cannot read — which request went where, which node
answered,
/// how long the answer was. Four buckets is a compromise: it hides the difference
between
/// message *kinds*, not every byte of a payload's length.
///
/// The cost is real bandwidth. On a radio mesh a 40-byte heartbeat becomes 256
bytes. That is
/// the price of not broadcasting a traffic profile.
fn pad_payload(pt: &[u8], ladder: &[usize]) -> Option<Vec<u8>> {
    if pt.len() > u16::MAX as usize {
        return None;
    }
    let needed = PAD_PREFIX + pt.len();
    let largest = *ladder.last()?;
    // Past the ladder, step in whole multiples of the largest bucket rather than
failing: a
    // long inference answer should still send, just without a size guarantee finer
than that.
    let bucket = ladder
        .iter()
        .copied()
        .find(|b| *b >= needed)
        .unwrap_or_else(|| needed.div_ceil(largest) * largest);
    let mut out = vec![0u8; bucket];
    out[..PAD_PREFIX].copy_from_slice(&(pt.len() as u16).to_be_bytes());
    out[PAD_PREFIX..needed].copy_from_slice(pt);
```

```
    Some(out)
}

/// Recover the payload. The length prefix is inside the AEAD, so a tampered length
cannot
/// reach here — by this point the tag has already verified.
fn unpad_payload(padded: &[u8]) -> Option<&[u8]> {
    if padded.len() < PAD_PREFIX {
        return None;
    }
    let len = u16::from_be_bytes([padded[0], padded[1]]) as usize;
    padded.get(PAD_PREFIX..PAD_PREFIX + len)
}

fn seal_msg(key: &[u8; 32], sid: &[u8; 8], ctr: u64, msg: &Msg) -> Option<Vec<u8>> {
    let pt = Zeroizing::new(serde_json::to_vec(msg).ok()?);
    let padded = Zeroizing::new(pad_payload(&pt, &PAD_BUCKETS)?);
    seal_frame(key, sid, ctr, &padded)
}

fn open_msg(pt: &[u8]) -> Option<Msg> {
    serde_json::from_slice(unpad_payload(pt)?).ok()
}

fn seal_frame(key: &[u8; 32], sid: &[u8; 8], ctr: u64, pt: &[u8]) -> Option<Vec<u8>>
{
    let mut hdr = [0u8; FRAME_HDR_LEN];
    hdr[..8].copy_from_slice(sid);
    hdr[8..].copy_from_slice(&ctr.to_be_bytes());
    let ct = crypto::aead_seal(key, ctr, &hdr, pt)?;
    let mut out = Vec::with_capacity(FRAME_HDR_LEN + ct.len());
    out.extend_from_slice(&hdr);
    out.extend_from_slice(&ct);
    Some(out)
}

/// Returns (session id, counter, header bytes used as AAD, ciphertext).
fn parse_frame(frame: &[u8]) -> Option<([u8; 8], u64, &[u8], &[u8])> {
    if frame.len() < FRAME_HDR_LEN + TAG_LEN || frame.len() > MAX_FRAME_LEN {
        return None;
    }
    let mut sid = [0u8; 8];
    sid.copy_from_slice(&frame[..8]);
    let mut c = [0u8; 8];
    c.copy_from_slice(&frame[8..16]);
    Some((
        sid,
        u64::from_be_bytes(c),
        &frame[..FRAME_HDR_LEN],
        &frame[FRAME_HDR_LEN..],
```

```
    ))
}

/// 64-wide sliding replay window. `check` first, `update` only after the tag
verified.
#[derive(Default)]
struct ReplayWindow {
    top: u64,
    bitmap: u64,
    started: bool,
}

impl ReplayWindow {
    fn check(&self, ctr: u64) -> bool {
        if !self.started || ctr > self.top {
            return true;
        }
        let off = self.top - ctr;
        if off >= 64 {
            return false;
        }
        (self.bitmap & (1u64 << off)) == 0
    }

    fn update(&mut self, ctr: u64) {
        if !self.started {
            self.started = true;
            self.top = ctr;
            self.bitmap = 1;
            return;
        }
        if ctr > self.top {
            let shift = ctr - self.top;
            self.bitmap = if shift >= 64 { 0 } else { self.bitmap << shift };
            self.bitmap |= 1;
            self.top = ctr;
        } else {
            let off = self.top - ctr;
            if off < 64 {
                self.bitmap |= 1u64 << off;
            }
        }
    }
}

struct ClientSession {
    peer_id: NodeId,
    sid: [u8; 8],
    k_send: Zeroizing<[u8; 32]>,
    k_recv: Zeroizing<[u8; 32]>,
```

```
    send_ctr: u64,
    recv_win: ReplayWindow,
    created: Instant,
}

impl ClientSession {
    fn from_keys(peer_id: NodeId, keys: crypto::SessionKeys) -> Self {
        Self {
            peer_id,
            sid: keys.session_id,
            k_send: keys.k_i2r,
            k_recv: keys.k_r2i,
            send_ctr: 0,
            recv_win: ReplayWindow::default(),
            created: Instant::now(),
        }
    }

    fn expired(&self) -> bool {
        self.created.elapsed() > CLIENT_SESSION_MAX_AGE || self.send_ctr >=
MAX_FRAMES_PER_SESSION
    }

    fn seal(&mut self, msg: &Msg) -> Option<Vec<u8>> {
        let ctr = self.send_ctr;
        self.send_ctr = ctr.checked_add(1)?;
        seal_msg(&self.k_send, &self.sid, ctr, msg)
    }

    fn open(&mut self, frame: &[u8]) -> Option<Msg> {
        let (sid, ctr, hdr, ct) = parse_frame(frame)?;
        if sid != self.sid || !self.recv_win.check(ctr) {
            return None;
        }
        let pt = crypto::aead_open(&self.k_recv, ctr, hdr, ct)?;
        self.recv_win.update(ctr);
        open_msg(&pt)
    }
}

struct ServerSession {
    sid: [u8; 8],
    peer_id: NodeId,
    peer_ip: IpAddr,
    peer_port: u16,
    k_recv: Zeroizing<[u8; 32]>,
    k_send: Zeroizing<[u8; 32]>,
    recv_win: ReplayWindow,
    send_ctr: u64,
    created: Instant,
```

```
    last_used: Instant,
}

impl ServerSession {
    fn expired(&self) -> bool {
        self.created.elapsed() > SERVER_SESSION_MAX_AGE
            || self.last_used.elapsed() > SERVER_SESSION_IDLE
    }
}

// ──────────────────────────────────── trust store ─────────────────────────────────────

struct TrustStore {
    mode: TrustMode,
    pinned: HashSet<NodeId>,
    revoked: HashSet<NodeId>,
    path: PathBuf,
    revoked_path: PathBuf,
}

/// Read a file of one hex fingerprint per line, ignoring blanks and `#` comments.
fn read_fingerprint_file(path: &Path, into: &mut HashSet<NodeId>) {
    if let Ok(text) = std::fs::read_to_string(path) {
        for line in text.lines() {
            let line = line.split('#').next().unwrap_or("").trim();
            if let Some(id) = unhex32(line) {
                into.insert(id);
            }
        }
    }
}

impl TrustStore {
    fn load(mode: TrustMode, dir: &Path, seeded: &[NodeId]) -> Self {
        let path = dir.join("trusted_peers.txt");
        let revoked_path = dir.join("revoked_peers.txt");

        let mut pinned: HashSet<NodeId> = seeded.iter().copied().collect();
        // Strict trusts exactly the configured list (--trust / HARMONY_TRUST).
trusted_peers.txt is
        // TOFU's record of first contacts; reading it here would carry every
identity ever pinned
        // (a test node, a stranger that joined while TOFU was on) into strict mode
unlisted.
        if mode == TrustMode::Tofu {
            read_fingerprint_file(&path, &mut pinned);
        }

        let mut revoked: HashSet<NodeId> = HashSet::new();
        read_fingerprint_file(&revoked_path, &mut revoked);
```

```
        Self {
            mode,
            pinned,
            revoked,
            path,
            revoked_path,
        }
    }

    /// Apply `--revoke` identities: record them in memory and append any that are
new to
    /// revoked_peers.txt (so the file does not grow on every restart).
    fn apply_revocations(&mut self, ids: &[NodeId]) {
        for id in ids {
            if self.revoked.insert(*id) {
                if let Err(e) = append_fingerprint(&self.revoked_path, id) {
                    warn!("could not persist revocation for {}: {e}", short_id(id));
                }
                warn!("⛔ revoked identity {}", short_id(id));
            }
        }
    }

    fn is_revoked(&self, id: &NodeId) -> bool {
        self.revoked.contains(id)
    }

    /// Whether policy would let us talk to this identity, without pinning it:
dialling asks this
    /// before it sends a hello, which carries this node's identity.
    fn allows(&self, id: &NodeId) -> bool {
        !self.revoked.contains(id) && (self.pinned.contains(id) || self.mode ==
TrustMode::Tofu)
    }

    /// Does policy let us talk to this identity, and should we persist it now?
    fn admit(&mut self, id: &NodeId) -> bool {
        // Revocation is absolute: it beats an existing pin and beats TOFU.
        if self.revoked.contains(id) {
            return false;
        }
        if self.pinned.contains(id) {
            return true;
        }
        match self.mode {
            TrustMode::Strict => false,
            TrustMode::Tofu => {
                if self.pinned.len() >= MAX_TRUSTED {
                    warn!("trust store full; refusing new identity {}",
```

```
short_id(id));
                    return false;
                }
                self.pinned.insert(*id);
                if let Err(e) = append_fingerprint(&self.path, id) {
                    warn!("could not persist trusted peer: {e}");
                }
                info!("📌 pinned new peer {} (TOFU)", short_id(id));
                true
            }
        }
    }
}

fn append_fingerprint(path: &Path, id: &NodeId) -> std::io::Result<()> {
    use std::io::Write;
    let mut f = std::fs::OpenOptions::new()
        .create(true)
        .append(true)
        .open(path)?;
    writeln!(f, "{}", hex(id))
}

// ─────────────────────────────────── shared state ──────────────────────────────────

#[derive(Clone)]
struct Peer {
    addr: SocketAddr,
    fails: u32,
    /// Which peer told us about this one. `None` means a seed or a direct inbound
handshake,
    /// both of which we observed ourselves rather than took on another node's word.
    learned_from: Option<NodeId>,
}

/// What the node believes about its own connectivity.
///
/// This is a *link* assessment, not a jamming detector. Software cannot tell a
jammer from
/// peers that went to sleep, a dead Wi-Fi AP, or a mesh that simply moved out of
range — that
/// needs RF-layer evidence (noise floor, error-vector magnitude) this node has no
access to.
/// What it can say is useful anyway: whether contact was lost from every peer at
once after
/// each was individually healthy, which looks different from losing them one by one.
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
enum LinkState {
    Healthy,
    /// At least one contact attempt has failed, but not enough in a row to call it a
```

```
    /// blackout. Ordinary attrition and congestion look like this.
    Degraded,
    /// Known peers, none reachable. Suspicious when it happens abruptly.
    Isolated,
}

impl LinkState {
    fn label(self) -> &'static str {
        match self {
            LinkState::Healthy => "healthy",
            LinkState::Degraded => "degraded",
            LinkState::Isolated => "isolated",
        }
    }
}

/// Copied out from under the lock so `/health` does not serialise while holding it.
struct LinkSnapshot {
    rounds: u64,
    ok: u64,
    failed: u64,
    blackouts: u32,
    last_contact: Option<Instant>,
}

#[derive(Default)]
struct LinkHealth {
    rounds: u64,
    ok: u64,
    failed: u64,
    /// Consecutive rounds in which a contact was attempted and none succeeded.
    blackouts: u32,
    last_contact: Option<Instant>,
    /// Set once when a blackout crosses the threshold, so the warning is logged a
single time
    /// rather than every round for as long as the condition lasts.
    warned: bool,
}

impl LinkHealth {
    fn record(&mut self, attempted: bool, success: bool) {
        self.rounds += 1;
        if !attempted {
            return;
        }
        if success {
            self.ok += 1;
            self.blackouts = 0;
            self.warned = false;
            self.last_contact = Some(Instant::now());
```

```
        } else {
            self.failed += 1;
            self.blackouts = self.blackouts.saturating_add(1);
        }
    }

    fn state(&self, known_peers: usize) -> LinkState {
        if known_peers == 0 {
            return LinkState::Healthy; // nothing to be cut off from yet
        }
        if self.blackouts >= DEGRADED_AFTER_FAILS {
            LinkState::Isolated
        } else if self.blackouts > 0 {
            LinkState::Degraded
        } else {
            LinkState::Healthy
        }
    }
}

struct AppState {
    cfg: Config,
    /// The port the mesh API is actually listening on. May differ from `cfg.port` when
    /// `--port-rotate` moved it, and it — not the requested port — is what we advertise to
    /// peers in the handshake so they can reach us back.
    bound_port: u16,
    me: Identity,
    trust: Mutex<TrustStore>,
    peers: Mutex<HashMap<NodeId, Peer>>,
    server_sessions: Mutex<HashMap<[u8; 8], ServerSession>>,
    /// Outbound sessions we keep alive so gossip does not redo a full post-quantum handshake
    /// every round. Taken out while in use, put back when the exchange succeeds.
    client_sessions: Mutex<HashMap<NodeId, ClientSession>>,
    hello_rate: Mutex<HashMap<IpAddr, (u32, Instant)>>,
    msg_rate: Mutex<HashMap<IpAddr, (u32, Instant)>>,
    /// Digests of hello1 messages already processed, with the time they were seen.
    seen_hellos: Mutex<HashMap<[u8; 32], Instant>>,
    /// Bounds how much asymmetric work unauthenticated callers can demand at once.
    handshake_gate: Semaphore,
    llm_gate: Semaphore,
    llm_last: Mutex<Instant>,
    thermal: ThermalProvider,
    /// Emission control. While set, the node originates nothing: no gossip dials, no seed
    /// bootstrap, no outbound inference fan-out. It still answers what reaches it, because a
    /// node that stops responding is as distinctive as one that transmits.
```

```
    ///
    /// The point is to stop *being a transmitter*. A direction-finder needs
emissions to work
    /// with; a receive-only node gives it nothing. This cannot hide the fact that
the host
    /// radio is powered on, and it does not stop replies — EMCON here is about not
initiating.
    emcon: AtomicBool,
    link: Mutex<LinkHealth>,
    /// Verified peer bundles, so `GET /id` is fetched once per peer instead of once
per dial.
    ///
    /// This was the worst emission in the node: every gossip round re-fetched the
responder's
    /// full 8787-byte identity in cleartext, which told any observer exactly who was
talking
    /// to whom — far more than the old `hello1` ever leaked. Backed by `<data-
dir>/bundles/`,
    /// which is also where out-of-band provisioned bundles go.
    bundles: Mutex<HashMap<NodeId, crypto::PeerBundle>>,
    /// Peers' piggybacked vitals and when they arrived.
    peer_vitals: Mutex<HashMap<NodeId, (Vitals, Instant)>>,
    /// Directives in flight through this node.
    relay: Mutex<RelayState>,
    /// The AI placement decision in force here, if any.
    placement: Mutex<Option<Placement>>,
    /// Recent events, for the dashboard.
    events: Mutex<EventLog>,
    /// PHASE_STARTING, PHASE_READY or PHASE_DRAINING.
    phase: AtomicU8,
    /// A shutdown asked for over the operator API (POST /shutdown) rather than by a
signal.
    stop: tokio::sync::Notify,
    shutdown: broadcast::Sender<()>,
    /// The kept-loaded model (v19). Held while it starts; /health reads the view.
    engine: tokio::sync::Mutex<Engine>,
    engine_view: Mutex<EngineView>,
    /// When the last model run ended, for the idle unload.
    engine_used: Mutex<Instant>,
    /// For the kept-loaded model only: loopback, never through a proxy.
    engine_client: reqwest::Client,
}

type Shared = Arc<AppState>;

/// Fixed-window counter shared by the /hello and /msg limiters.
fn rate_ok(
    map: &Mutex<HashMap<IpAddr, (u32, Instant)>>,
    ip: IpAddr,
    limit: u32,
```

```
    window: Duration,
) -> bool {
    let now = Instant::now();
    let mut m = lock(map);
    if m.len() > HELLO_CACHE_MAX {
        m.retain(|_, (_, t)| now.duration_since(*t) < window);
    }
    let e = m.entry(ip).or_insert((0, now));
    if now.duration_since(e.1) >= window {
        *e = (0, now);
    }
    e.0 += 1;
    e.0 <= limit
}

impl AppState {
    /// Admission check shared by /hello and /msg: self-exclusion + trust policy.
    fn admit_peer(&self, id: &NodeId) -> bool {
        if id == &self.me.fpr() {
            return false; // never talk to ourselves
        }
        lock(&self.trust).admit(id)
    }

    /// `admit_peer` without pinning: may this identity be dialled, or taken from a
peer list?
    fn trust_allows(&self, id: &NodeId) -> bool {
        id != &self.me.fpr() && lock(&self.trust).allows(id)
    }

    /// Revocation is checked on every authenticated message, not just at handshake,
so
    /// revoking an identity cuts off sessions that are already established.
    fn is_revoked(&self, id: &NodeId) -> bool {
        lock(&self.trust).is_revoked(id)
    }

    /// Drop every cached session and peer entry belonging to a revoked identity.
    fn purge_revoked(&self) {
        let revoked: Vec<NodeId> = {
            let t = lock(&self.trust);
            t.revoked.iter().copied().collect()
        };
        if revoked.is_empty() {
            return;
        }
        lock(&self.peers).retain(|id, _| !revoked.contains(id));
        lock(&self.client_sessions).retain(|id, _| !revoked.contains(id));
        lock(&self.server_sessions).retain(|_, s| !revoked.contains(&s.peer_id));
    }
```

```
    /// Take a live outbound session for this peer, if one is cached and still
usable.
    fn take_client_session(&self, id: &NodeId) -> Option<ClientSession> {
        let mut map = lock(&self.client_sessions);
        map.retain(|_, s| !s.expired());
        map.remove(id).filter(|s| !s.expired())
    }

    /// Return a session to the cache after a successful exchange.
    fn put_client_session(&self, sess: ClientSession) {
        if sess.expired() {
            return;
        }
        let mut map = lock(&self.client_sessions);
        map.retain(|_, s| !s.expired());
        if map.len() >= MAX_CLIENT_SESSIONS && !map.contains_key(&sess.peer_id) {
            return;
        }
        map.insert(sess.peer_id, sess);
    }

    /// Record a peer we observed directly — a seed we dialled, or one that handshook
with us.
    fn remember_peer(&self, id: NodeId, addr: SocketAddr) {
        self.remember_peer_from(id, addr, None)
    }

    /// Record a peer, tracking who vouched for it.
    ///
    /// Entries introduced by another node are capped per introducer. A trusted peer
that
    /// turns hostile can otherwise fill the table with addresses it controls and
crowd out
    /// every real peer — authentication says who it is, not that it is behaving.
    fn remember_peer_from(&self, id: NodeId, addr: SocketAddr, source:
Option<NodeId>) {
        if id == self.me.fpr()
            || !peer_ok(&addr, self.cfg.allow_public, self.cfg.allow_loopback)
        {
            return;
        }
        let mut peers = lock(&self.peers);

        if !peers.contains_key(&id) {
            if peers.len() >= MAX_PEERS {
                return;
            }
            if let Some(src) = source {
                let introduced = peers
```

```
                    .values()
                    .filter(|p| p.learned_from == Some(src))
                    .count();
                if introduced >= MAX_PEERS_PER_SOURCE {
                    debug!(
                        "ignoring peer from {}: already introduced {introduced}",
                        short_id(&src)
                    );
                    return;
                }
            }
        }

        peers
            .entry(id)
            .and_modify(|p| update_known_peer(p, addr, source))
            .or_insert(Peer {
                addr,
                fails: 0,
                learned_from: source,
            });
    }

    fn note_peer_result(&self, id: &NodeId, ok: bool) {
        let mut peers = lock(&self.peers);
        if ok {
            if let Some(p) = peers.get_mut(id) {
                p.fails = 0;
            }
        } else if let Some(p) = peers.get_mut(id) {
            p.fails += 1;
            if p.fails >= MAX_PEER_FAILS {
                peers.remove(id);
            }
        }
    }

    fn bundle_dir(&self) -> PathBuf {
        self.cfg.data_dir.join("bundles")
    }

    /// Which known identity currently sits at `addr`, if any.
    ///
    /// Only ever used to *skip* a bundle fetch. The handshake still proves who
answered, so a
    /// wrong guess here costs a failed handshake, never a wrong trust decision.
    fn peers_by_addr(&self, addr: SocketAddr) -> Option<NodeId> {
        lock(&self.peers)
            .iter()
            .find(|(_, p)| p.addr == addr)
```

```
            .map(|(id, _)| *id)
    }

    fn cached_bundle(&self, id: &NodeId) -> Option<crypto::PeerBundle> {
        lock(&self.bundles).get(id).cloned()
    }

    /// Cache a bundle we just verified, and persist it so a restart does not re-
fetch.
    ///
    /// Writing is best-effort: a cache that fails to persist costs an extra fetch,
which is
    /// not worth failing a handshake over.
    fn cache_bundle(&self, raw: &[u8], parsed: &crypto::PeerBundle) {
        let fresh = lock(&self.bundles)
            .insert(parsed.fpr, parsed.clone())
            .is_none();
        if !fresh {
            return;
        }
        let dir = self.bundle_dir();
        if let Err(e) = std::fs::create_dir_all(&dir) {
            debug!("could not create {}: {e}", dir.display());
            return;
        }
        let path = dir.join(format!("{}.bundle", hex(&parsed.fpr)));
        if let Err(e) = std::fs::write(&path, raw) {
            debug!("could not cache bundle for {}: {e}", short_id(&parsed.fpr));
        }
    }

    /// Drop a cached bundle, on disk as well as in memory.
    ///
    /// Needed because `--rotate-kem` changes a peer's encapsulation key while
keeping its
    /// identity: a cached bundle then holds a key nobody can decapsulate to, and
without this
    /// the node would keep failing against it forever.
    fn forget_bundle(&self, id: &NodeId) {
        lock(&self.bundles).remove(id);
        let path = self.bundle_dir().join(format!("{}.bundle", hex(id)));
        if path.exists() {
            if let Err(e) = std::fs::remove_file(&path) {
                debug!("could not remove {}: {e}", path.display());
            }
        }
    }

    fn emcon(&self) -> bool {
        self.emcon.load(Ordering::Relaxed)
```

```
    }

    fn set_emcon(&self, on: bool) {
        let was = self.emcon.swap(on, Ordering::Relaxed);
        if was != on {
            if on {
                warn!("🔇 EMCON on — this node will originate nothing until it is
lifted");
            } else {
                info!("🔊 EMCON off — resuming gossip");
            }
        }
    }

    /// Fold one gossip round into the link assessment and surface a transition once.
    fn note_link(&self, attempted: bool, success: bool) {
        let peers = lock(&self.peers).len();
        let mut link = lock(&self.link);
        link.record(attempted, success);
        if link.state(peers) == LinkState::Isolated && !link.warned {
            link.warned = true;
            warn!(
                "📡 link isolated: {} consecutive failed contacts with {peers} known
peer(s).                   Could be jamming, could be peers that went away — this node
cannot tell the                   difference from software alone.",
                link.blackouts
            );
        }
    }

    fn link_state(&self) -> LinkState {
        let peers = lock(&self.peers).len();
        lock(&self.link).state(peers)
    }

    fn hello_rate_ok(&self, ip: IpAddr) -> bool {
        rate_ok(&self.hello_rate, ip, HELLO_PER_IP, HELLO_WINDOW)
    }

    fn msg_rate_ok(&self, ip: IpAddr) -> bool {
        rate_ok(&self.msg_rate, ip, MSG_PER_IP, MSG_WINDOW)
    }

    /// Record a hello1 and report whether it is fresh.
    ///
    /// A captured hello1 stays signature-valid for the whole clock-skew window, so
without
    /// this an attacker can resend one cheaply and make us pay for an ML-KEM
decapsulation
    /// every time. Hashing the entire message means any tampering changes the digest
```

```
*and*
    /// breaks the signature, so there is nothing to gain by mutating a replay.
    fn hello_is_fresh(&self, body: &[u8]) -> bool {
        let digest = crypto::transcript32(&[b"harmony-v2/seen", body]);
        let now = Instant::now();
        let window = Duration::from_secs(MAX_CLOCK_SKEW_SECS * 2);

        let mut seen = lock(&self.seen_hellos);
        seen.retain(|_, t| now.duration_since(*t) < window);
        if seen.len() >= HELLO_SEEN_MAX {
            // Full table: drop the oldest rather than stop tracking, since silently
            // forgetting would reopen the replay window under load.
            if let Some(oldest) = seen.iter().min_by_key(|(_, t)| **t).map(|(k, _)|
*k) {
                seen.remove(&oldest);
            }
        }
        seen.insert(digest, now).is_none()
    }

    fn store_server_session(&self, s: ServerSession) {
        let mut map = lock(&self.server_sessions);
        map.retain(|_, v| !v.expired());
        if map.len() >= MAX_SERVER_SESSIONS {
            if let Some(oldest) = map.iter().min_by_key(|(_, v)| v.last_used).map(|
(k, _)| *k) {
                map.remove(&oldest);
            }
        }
        map.insert(s.sid, s);
    }

    /// Decrypt an inbound frame against its session, enforcing replay protection,
    /// and return the request plus the keys needed to answer.
    fn open_server_frame(&self, frame: &[u8]) -> Option<(Msg, NodeId, [u8; 8],
IpAddr, u16)> {
        let (sid, ctr, hdr, ct) = parse_frame(frame)?;
        let mut map = lock(&self.server_sessions);
        let s = map.get_mut(&sid)?;
        if s.expired() || !s.recv_win.check(ctr) {
            return None;
        }
        let pt = crypto::aead_open(&s.k_recv, ctr, hdr, ct)?;
        s.recv_win.update(ctr);
        s.last_used = Instant::now();
        let msg = open_msg(&pt)?;
        Some((msg, s.peer_id, sid, s.peer_ip, s.peer_port))
    }

    /// Seal a reply on the responder->initiator key of an existing session.
```

```
    fn seal_server_reply(&self, sid: &[u8; 8], msg: &Msg) -> Option<Vec<u8>> {
        let mut map = lock(&self.server_sessions);
        let s = map.get_mut(sid)?;
        let ctr = s.send_ctr;
        s.send_ctr = ctr.checked_add(1)?;
        seal_msg(&s.k_send, sid, ctr, msg)
    }
}

// ──────────────────────────────────── application messages
─────────────────────────────────
// The authenticated payloads carried inside frames. Everything here has already been
// through the KEM handshake, so a sender is cryptographically identified.

#[derive(Serialize, Deserialize, Debug)]
#[serde(tag = "t", rename_all = "snake_case")]
enum Msg {
    /// Liveness probe (replaces the forgeable plaintext heartbeat), and the carrier
for
    /// piggyback propagation: the sender's vitals and at most one relayed directive
ride along.
    /// Empty fields are omitted, so a plain ping is byte-identical to the old format
and older
    /// nodes simply ignore the extra fields.
    Ping {
        nonce: u64,
        #[serde(default, skip_serializing_if = "Option::is_none")]
        vitals: Option<Vitals>,
        #[serde(default, skip_serializing_if = "Option::is_none")]
        carry: Option<Relayed>,
        #[serde(default, skip_serializing_if = "Vec::is_empty")]
        acks: Vec<String>,
    },
    Pong {
        nonce: u64,
        #[serde(default, skip_serializing_if = "Option::is_none")]
        vitals: Option<Vitals>,
        /// Ids of directives received in the ping this answers: the piggybacked
acknowledgement.
        #[serde(default, skip_serializing_if = "Vec::is_empty")]
        acks: Vec<String>,
    },
    /// Peer-exchange: share known reachable peers as id:addr pairs.
    GetPeers,
    Peers { peers: Vec<PeerEntry> },
    /// A routing directive for the local node (what the radio used to carry in the
clear).
    Directive { body: String },
    DirectiveAck { accepted: bool, detail: String },
    /// Ask this peer to run a prompt on its local model and return the text.
```

```
    ///
    /// This is task-level distribution: each node answers independently with its
whole model.
    /// Splitting one model across nodes is not viable here — per-token activation
exchange
    /// over a lossy, high-latency radio link costs far more than the compute it
saves.
    InferRequest {
        id: u64,
        prompt: String,
        /// Caller's budget in ms; the executor clamps this to its own limits.
        deadline_ms: u64,
        /// A conversation (v19). `prompt` then holds its last user message: that is
what a v18 node
        /// answers, since it ignores fields it does not know.
        #[serde(default, skip_serializing_if = "Option::is_none")]
        chat: Option<Vec<ChatTurn>>,
        /// The conversation's answer budget and sampling (v19); the executor clamps
them.
        #[serde(default, skip_serializing_if = "Option::is_none")]
        max_tokens: Option<u32>,
        #[serde(default, skip_serializing_if = "Option::is_none")]
        temperature: Option<f32>,
        #[serde(default, skip_serializing_if = "Option::is_none")]
        top_p: Option<f32>,
    },
    /// The sender is shutting down gracefully: drop it now rather than after failed
rounds.
    /// Nodes that predate it cannot parse it and ignore it.
    Bye,
    InferReply {
        id: u64,
        ok: bool,
        /// Answer text, or the reason there isn't one.
        text: String,
        /// Which tier produced it, so a caller can weigh a 1B answer against a 3B
one.
        tier: String,
        /// Responder fingerprint, for attribution in a fan-out.
        node: String,
        elapsed_ms: u64,
    },
}

#[derive(Serialize, Deserialize, Debug)]
struct PeerEntry {
    id: String,   // hex Ed25519 identity
    addr: String, // ip:port
}
```

```
/// One turn of a conversation, as /v1/chat/completions takes it and the mesh carries
it (v19).
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
struct ChatTurn {
    /// "system", "user" or "assistant"
    role: String,
    content: String,
}

// ──────────────────────────────────── piggyback propagation
──────────────────────────────────
//
// Two things ride on gossip messages that are sent anyway, so neither costs a
transmission of
// its own — which matters on a radio link and under emission discipline:
//
//   * Vitals. Every Ping and Pong carries the sender's temperature, thermal limit,
busy flag
//     and tier, so each node knows its peers' thermal headroom. Offload and fan-out
ask the
//     coolest idle peers first and skip peers already at their limit, which would
refuse.
//   * Directives. A directive issued at one node (`POST /relay`) is signed there
with ML-DSA-87
//     and spreads hop by hop, at most one per Ping. The receiver acknowledges it in
the Pong
//     that answers that very Ping, so the sender stops carrying it to that peer.
Every hop
//     verifies the origin's signature and checks the origin against its own `--cmd-
allow`
//     before it forwards the directive or runs it through the local Llama as a non-
actuating
//     advisory. A hot node still relays (relaying is free) but defers its advisory
until it
//     cools, then retries.

/// How long a directive may circulate after it was signed.
const RELAY_MAX_AGE: Duration = Duration::from_secs(10 * 60);
/// Links a directive may travel. Bounds propagation even if a relay tampers with
`hops`.
const RELAY_MAX_HOPS: u8 = 8;
/// Directives tracked at once (oldest evicted first).
const RELAY_MAX_ITEMS: usize = 64;
/// Live directives accepted per origin, so one authorized origin cannot crowd out
the rest.
const RELAY_PER_ORIGIN: usize = 8;
/// Advisory attempts per directive (the first plus retries after thermal or busy
deferrals).
const RELAY_ADVISORY_ATTEMPTS: u8 = 3;
/// Acks processed per message.
```

```
const RELAY_MAX_ACKS: usize = 16;
const CTX_RELAY: &[u8] = b"harmony-v3/relay";

/// A node's thermal and capacity state, piggybacked on every gossip Ping and Pong.
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
struct Vitals {
    thermal_c: f32,
    max_temp: f32,
    /// The local model is generating right now.
    busy: bool,
    tier: String,
    /// What the device is ("phone", "laptop", or its --label), so the advisory model
can match
    /// a directive that names a device. Omitted when empty; older nodes neither send
nor read it.
    #[serde(default, skip_serializing_if = "String::is_empty")]
    label: String,
}

impl Vitals {
    fn headroom(&self) -> f32 {
        self.max_temp - self.thermal_c
    }

    /// At or above its own limit, so it would refuse inference for heat.
    fn hot(&self) -> bool {
        self.thermal_c >= self.max_temp
    }

    /// Reject values no real sensor produces, so a buggy peer cannot top the
ranking.
    fn plausible(&self) -> bool {
        self.thermal_c.is_finite()
            && self.max_temp.is_finite()
            && (-50.0..=150.0).contains(&self.thermal_c)
            && (0.0..=150.0).contains(&self.max_temp)
    }
}

/// A directive in transit. Everything except `hops` is covered by the origin's
signature.
#[derive(Serialize, Deserialize, Debug, Clone)]
struct Relayed {
    /// 16 random bytes, hex. Names the directive for dedup and acks.
    id: String,
    /// Origin fingerprint, hex. Must equal `fingerprint(vk)`.
    origin: String,
    /// Unix seconds at signing; bounds how long the directive may circulate.
    issued: u64,
    /// Links travelled so far. Unsigned: a relay could change it, but only within
```

```
    /// RELAY_MAX_HOPS and RELAY_MAX_AGE, which bound propagation regardless.
    hops: u8,
    body: String,
    /// The origin's ML-DSA-87 verifying key, base64, so any hop can verify with no
directory.
    vk: String,
    /// ML-DSA-87 signature over `relay_sig_input`, base64.
    sig: String,
}

/// Length-prefixed so no two different directives share a signing input.
fn relay_sig_input(id: &str, origin: &str, issued: u64, body: &str) -> Vec<u8> {
    let mut m = Vec::with_capacity(CTX_RELAY.len() + id.len() + origin.len() +
body.len() + 16);
    m.extend_from_slice(CTX_RELAY);
    for part in [id.as_bytes(), origin.as_bytes()] {
        m.extend_from_slice(&(part.len() as u16).to_be_bytes());
        m.extend_from_slice(part);
    }
    m.extend_from_slice(&issued.to_be_bytes());
    m.extend_from_slice(&(body.len() as u32).to_be_bytes());
    m.extend_from_slice(body.as_bytes());
    m
}

fn sign_relayed(me: &Identity, body: &str, issued: u64) -> Relayed {
    let mut raw = [0u8; 16];
    OsRng.fill_bytes(&mut raw);
    let id = hex(&raw);
    let origin = hex(&me.fpr());
    let sig = me.id.sign(&relay_sig_input(&id, &origin, issued, body));
    Relayed {
        id,
        origin,
        issued,
        hops: 0,
        body: body.to_string(),
        vk: b64::encode(me.vk_bytes()),
        sig: b64::encode(&sig),
    }
}

fn is_relay_id(id: &str) -> bool {
    id.len() == 32 && id.bytes().all(|c| c.is_ascii_hexdigit())
}

/// Check a relayed directive end to end: shape, size, age, hop bound, that the key
is the
/// origin's (its fingerprint), and the origin's signature. Returns the origin on
success.
```

```
fn verify_relayed(r: &Relayed, now: u64) -> Result<NodeId, &'static str> {
    if !is_relay_id(&r.id) {
        return Err("bad id");
    }
    let origin = unhex32(&r.origin).ok_or("bad origin")?;
    if r.body.trim().is_empty() || r.body.len() > MAX_PROMPT_BYTES {
        return Err("bad body size");
    }
    if r.body.chars().any(|c| c.is_control() && c != '\n' && c != '\t') {
        return Err("control characters");
    }
    if r.hops > RELAY_MAX_HOPS {
        return Err("too many hops");
    }
    if r.issued > now.saturating_add(MAX_CLOCK_SKEW_SECS) {
        return Err("issued in the future");
    }
    if now.saturating_sub(r.issued) > RELAY_MAX_AGE.as_secs() {
        return Err("expired");
    }
    let vk = b64::decode(&r.vk).ok_or("bad key encoding")?;
    if crypto::fingerprint(&vk) != origin {
        return Err("key does not match origin");
    }
    let sig = b64::decode(&r.sig).ok_or("bad signature encoding")?;
    if !crypto::verify(&vk, &relay_sig_input(&r.id, &r.origin, r.issued, &r.body),
&sig) {
        return Err("bad signature");
    }
    Ok(origin)
}

/// Order candidate peers for offload and fan-out: idle peers with fresh vitals by
thermal
/// headroom, then busy ones, then peers with no fresh vitals; peers at or over their
own limit
/// are dropped, since they would refuse for heat anyway. Pure, so it is tested
directly.
fn rank_targets(
    mut cands: Vec<(NodeId, SocketAddr, Option<Vitals>)>,
    n: usize,
) -> Vec<(NodeId, SocketAddr)> {
    cands.retain(|(_, _, v)| !v.as_ref().is_some_and(|v| v.hot()));
    let key = |v: &Option<Vitals>| match v {
        Some(v) if !v.busy => (0u8, v.headroom()),
        Some(v) => (1, v.headroom()),
        None => (2, 0.0),
    };
    cands.sort_by(|a, b| {
        let (ca, ha) = key(&a.2);
```

```
        let (cb, hb) = key(&b.2);
        ca.cmp(&cb)
            .then(hb.partial_cmp(&ha).unwrap_or(std::cmp::Ordering::Equal))
            .then(a.0.cmp(&b.0))
    });
    cands.into_iter().take(n).map(|(id, addr, _)| (id, addr)).collect()
}

/// Minimal standard base64. A signed directive carries a 2592-byte key and a 4627-
byte
/// signature; hex would push the largest one past a single 16 KiB frame, base64
keeps it in.
mod b64 {
    const ALPHA: &[u8; 64] =
b"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

    pub fn encode(data: &[u8]) -> String {
        let mut out = String::with_capacity(data.len().div_ceil(3) * 4);
        for chunk in data.chunks(3) {
            let b1 = chunk.get(1).copied().unwrap_or(0);
            let b2 = chunk.get(2).copied().unwrap_or(0);
            let n = (u32::from(chunk[0]) << 16) | (u32::from(b1) << 8) |
u32::from(b2);
            out.push(ALPHA[(n >> 18) as usize & 63] as char);
            out.push(ALPHA[(n >> 12) as usize & 63] as char);
            out.push(if chunk.len() > 1 { ALPHA[(n >> 6) as usize & 63] as char }
else { '=' });
            out.push(if chunk.len() > 2 { ALPHA[n as usize & 63] as char } else { '='
});
        }
        out
    }

    fn val(c: u8) -> Option<u32> {
        match c {
            b'A'..=b'Z' => Some(u32::from(c - b'A')),
            b'a'..=b'z' => Some(u32::from(c - b'a') + 26),
            b'0'..=b'9' => Some(u32::from(c - b'0') + 52),
            b'+' => Some(62),
            b'/' => Some(63),
            _ => None,
        }
    }

    pub fn decode(s: &str) -> Option<Vec<u8>> {
        let s = s.as_bytes();
        if s.len() % 4 != 0 {
            return None;
        }
        let quads = s.len() / 4;
```

```
        let mut out = Vec::with_capacity(quads * 3);
        for (i, q) in s.chunks(4).enumerate() {
            let pad = q.iter().rev().take_while(|&&c| c == b'=').count();
            if pad > 2 || (pad > 0 && i + 1 != quads) {
                return None;
            }
            let mut n = 0u32;
            for (j, &c) in q.iter().enumerate() {
                let v = if j >= 4 - pad { 0 } else { val(c)? };
                n = (n << 6) | v;
            }
            out.push((n >> 16) as u8);
            if pad < 2 {
                out.push((n >> 8) as u8);
            }
            if pad < 1 {
                out.push(n as u8);
            }
        }
        Some(out)
    }
}

#[derive(Clone, Debug)]
enum Advisory {
    /// Issued here; the origin does not advise on its own directive.
    Origin,
    Running,
    Done(String),
    Deferred(String),
    /// A placement decision, not a directive: it is applied, never advised on.
    Placement(String),
}

impl Advisory {
    fn label(&self) -> String {
        match self {
            Advisory::Origin => "origin (issued here)".into(),
            Advisory::Placement(s) => format!("AI placement decision: {s}"),
            Advisory::Running => "running".into(),
            Advisory::Done(s) => s.clone(),
            Advisory::Deferred(s) => format!("deferred: {s}"),
        }
    }
}

struct RelayItem {
    msg: Relayed,
    origin: NodeId,
    /// The neighbour it arrived from; `None` if issued here.
```

```
    from: Option<NodeId>,
    first_seen: Instant,
    /// Peers known to hold it (acked, sent it to us, or its origin). Never carried
to them.
    delivered: HashSet<NodeId>,
    advisory: Advisory,
    attempts: u8,
}

#[derive(Default)]
struct RelayState {
    items: Vec<RelayItem>,
    /// Ids whose signature already verified (accepted or refused for authority), for
dedup.
    /// Only verified ids go in, so a forged copy cannot pre-empt the genuine
directive.
    seen: HashMap<String, Instant>,
}

impl RelayState {
    fn prune(&mut self) {
        self.items.retain(|i| i.first_seen.elapsed() < RELAY_MAX_AGE);
        self.seen.retain(|_, t| t.elapsed() < RELAY_MAX_AGE * 2);
    }

    fn push(&mut self, item: RelayItem) {
        if self.items.len() >= RELAY_MAX_ITEMS {
            self.items.remove(0);
        }
        self.items.push(item);
    }
}

impl AppState {
    /// This node's vitals, as piggybacked on gossip.
    fn vitals(&self) -> Vitals {
        Vitals {
            thermal_c: self.thermal.temperature(),
            max_temp: self.cfg.max_temp,
            busy: self.llm_gate.available_permits() == 0 || self.draining(),
            tier: self.cfg.tier.label().to_string(),
            label: self.cfg.label.clone(),
        }
    }

    /// How long a peer's piggybacked vitals are trusted. Gossip reaches a given peer
only every
    /// few rounds in a larger mesh, so this spans several rounds, floored at a
minute.
    fn vitals_ttl(&self) -> Duration {
```

```
        Duration::from_secs(self.cfg.gossip_secs.saturating_mul(6).max(60))
    }

    fn note_vitals(&self, peer: &NodeId, v: Option<Vitals>) {
        let Some(mut v) = v else { return };
        if !v.plausible() {
            return;
        }
        v.tier = truncate(&sanitize(&v.tier), 32);
        v.label = truncate(sanitize(&v.label).trim(), 24);
        let mut m = lock(&self.peer_vitals);
        if m.len() >= MAX_PEERS && !m.contains_key(peer) {
            return;
        }
        m.insert(*peer, (v, Instant::now()));
    }

    fn fresh_vitals(&self, peer: &NodeId) -> Option<(Vitals, Duration)> {
        let ttl = self.vitals_ttl();
        lock(&self.peer_vitals)
            .get(peer)
            .map(|(v, at)| (v.clone(), at.elapsed()))
            .filter(|(_, age)| *age <= ttl)
    }

    /// Peers to ask, coolest idle first (see `rank_targets`).
    fn ranked_targets(&self, n: usize) -> Vec<(NodeId, SocketAddr)> {
        let peers: Vec<(NodeId, SocketAddr)> =
            lock(&self.peers).iter().map(|(id, p)| (*id, p.addr)).collect();
        let cands = peers
            .into_iter()
            .map(|(id, addr)| {
                let v = self.fresh_vitals(&id).map(|(v, _)| v);
                (id, addr, v)
            })
            .collect();
        rank_targets(cands, n)
    }

    /// The offload order with the vitals it was based on, for logs and the `/infer`
response.
    fn offload_order(&self) -> Vec<(NodeId, Option<Vitals>)> {
        self.ranked_targets(MAX_FANOUT)
            .into_iter()
            .map(|(id, _)| (id, self.fresh_vitals(&id).map(|(v, _)| v)))
            .collect()
    }

    /// The next directive this peer does not hold yet, with its hop count advanced.
    fn next_carry(&self, peer: &NodeId) -> Option<Relayed> {
```

```
        let mut rs = lock(&self.relay);
        rs.prune();
        rs.items
            .iter()
            .find(|i| {
                i.origin != *peer
                    && i.from != Some(*peer)
                    && !i.delivered.contains(peer)
                    && i.msg.hops < RELAY_MAX_HOPS
            })
            .map(|i| {
                let mut m = i.msg.clone();
                m.hops = i.msg.hops + 1;
                m
            })
    }

    /// The gossip ping for `peer`: our vitals plus, if one is due, a directive it
lacks.
    fn gossip_ping(&self, peer: &NodeId, nonce: u64) -> Msg {
        let carry = self.next_carry(peer);
        if let Some(r) = &carry {
            debug!("📦 carrying directive {} to {}", short_hex(&r.id),
short_id(peer));
            let what = if r.body.starts_with(PLACEMENT_PREFIX) { "AI decision" } else
{ "directive" };
            self.event(
                "carry",
                &format!("{what} {} piggybacked on gossip to {}", short_hex(&r.id),
self.name_of(peer)),
                Some(&self.me.fpr()),
                Some(peer),
            );
        }
        Msg::Ping {
            nonce,
            vitals: Some(self.vitals()),
            carry,
            acks: Vec::new(),
        }
    }

    fn note_acks(&self, peer: &NodeId, acks: &[String]) {
        if acks.is_empty() {
            return;
        }
        let mut newly = Vec::new();
        {
            let mut rs = lock(&self.relay);
            for id in acks.iter().take(RELAY_MAX_ACKS) {
```

```
                if let Some(item) = rs.items.iter_mut().find(|i| &i.msg.id == id) {
                    if item.delivered.insert(*peer) {
                        info!("📦 {} acknowledged directive {}", short_id(peer),
short_hex(id));
                        newly.push((id.clone(),
item.msg.body.starts_with(PLACEMENT_PREFIX)));
                    }
                }
            }
        }
        for (id, decision) in newly {
            let what = if decision { "AI decision" } else { "directive" };
            self.event(
                "ack",
                &format!("{} acknowledged {what} {} on its pong", self.name_of(peer),
short_hex(&id)),
                Some(peer),
                Some(&self.me.fpr()),
            );
        }
    }

    fn note_pong(&self, peer: &NodeId, vitals: Option<Vitals>, acks: &[String]) {
        self.note_vitals(peer, vitals);
        self.note_acks(peer, acks);
    }
}

/// Take in a directive that rode in on a ping from `from`. Returns the id to
acknowledge, or
/// `None` for a copy that failed verification (not acked, so it is not mistaken for
delivery).
fn accept_relayed(st: &Shared, from: NodeId, r: Relayed) -> Option<String> {
    if !is_relay_id(&r.id) {
        return None;
    }
    let id = r.id.clone();
    {
        let mut rs = lock(&st.relay);
        rs.prune();
        if rs.seen.contains_key(&id) {
            if let Some(item) = rs.items.iter_mut().find(|i| i.msg.id == id) {
                item.delivered.insert(from);
            }
            return Some(id);
        }
    }

    let origin = match verify_relayed(&r, now_unix()) {
        Ok(o) => o,
```

```
        Err(why) => {
            warn!("📦 rejected directive {} via {}: {why}", short_hex(&id),
short_id(&from));
            return None;
        }
    };
    if let Some(pb) = parse_placement(&r.body) {
        return accept_placement(st, from, origin, r, pb);
    }

    let authorized = origin == st.me.fpr() || st.cfg.cmd_allow.contains(&origin);
    let revoked = st.is_revoked(&origin);
    let mut rs = lock(&st.relay);
    // Re-check under the lock: two pings may have carried the same directive
concurrently.
    if rs.seen.contains_key(&id) {
        return Some(id);
    }
    rs.seen.insert(id.clone(), Instant::now());
    if revoked || !authorized {
        drop(rs);
        let why = if revoked { "origin revoked" } else { "origin not in --cmd-allow"
};
        warn!(
            "🚫 directive {} from {} not accepted ({why}); not forwarded or advised",
            short_hex(&id),
            short_id(&origin),
        );
        st.event("reject", &format!("directive {} not accepted: {why}",
short_hex(&id)), Some(&origin), Some(&st.me.fpr()));
        return Some(id);
    }
    if rs.items.iter().filter(|i| i.origin == origin).count() >= RELAY_PER_ORIGIN {
        warn!("📦 directive {} dropped: origin {} at its live limit", short_hex(&id),
short_id(&origin));
        return Some(id);
    }

    let hops = r.hops;
    let body = r.body.clone();
    let mut delivered = HashSet::new();
    delivered.insert(from);
    delivered.insert(origin);
    rs.push(RelayItem {
        msg: r,
        origin,
        from: Some(from),
        first_seen: Instant::now(),
        delivered,
        advisory: Advisory::Running,
```

```
        attempts: 1,
    });
    drop(rs);

    info!(
        "📦 directive {} from {} arrived via {} after {hops} hop(s); verified,
advising",
        short_hex(&id),
        short_id(&origin),
        short_id(&from)
    );
    st.event(
        "arrive",
        &format!(
            "directive {} from {} arrived ({hops} hop(s)); ML-DSA-87 signature
verified; asking Llama",
            short_hex(&id),
            st.name_of(&origin)
        ),
        Some(&from),
        Some(&st.me.fpr()),
    );
    spawn_advisory(st, id.clone(), body);
    Some(id)
}

/// Run the local Llama advisory for a directive in the background and record the
outcome.
fn spawn_advisory(st: &Shared, id: String, body: String) {
    let st = Arc::clone(st);
    tokio::spawn(async move {
        let outcome = advise(&st, &body).await;
        let detail = outcome.detail;
        {
            let mut rs = lock(&st.relay);
            if let Some(item) = rs.items.iter_mut().find(|i| i.msg.id == id) {
                item.advisory = match detail.strip_prefix("deferred: ") {
                    Some(why) => Advisory::Deferred(why.to_string()),
                    None => Advisory::Done(truncate(&detail, 400)),
                };
            }
        }
        let me = st.me.fpr();
        match detail.strip_prefix("deferred: ") {
            Some(why) => st.event("deferred", &format!("Llama advisory deferred:
{why}"), Some(&me), None),
            None => st.event("advisory", detail.trim_start_matches("advisory: "),
Some(&me), None),
        }
        if let Some((target, name, reason)) = outcome.pick {
```

```
            issue_placement(&st, &id, target, &name, &reason, outcome.rules);
        }
    });
}

/// May a deferred advisory be retried now? Only below the temperature limit, with
Llama free and
/// past its cooldown: a retry while another generation runs fails at once ("already
in progress")
/// and spends one of the directive's few attempts for nothing.
fn advisory_retry_due(temp: f32, max_temp: f32, llm_free: bool, since_last_run:
Duration) -> bool {
    temp < max_temp && llm_free && since_last_run >= LLM_COOLDOWN
}

/// Once per gossip round: a directive whose advisory was deferred for heat or load
gets
/// another attempt once this device is below its limit and its Llama is free.
fn retry_deferred_advisories(st: &Shared) {
    let free = st.llm_gate.available_permits() > 0;
    let since = lock(&st.llm_last).elapsed();
    if !advisory_retry_due(st.thermal.temperature(), st.cfg.max_temp, free, since) {
        return;
    }
    let job = {
        let mut rs = lock(&st.relay);
        rs.items
            .iter_mut()
            .find(|i| {
                matches!(&i.advisory, Advisory::Deferred(why) if
is_capacity_failure(why))
                    && i.attempts < RELAY_ADVISORY_ATTEMPTS
            })
            .map(|i| {
                i.advisory = Advisory::Running;
                i.attempts += 1;
                (i.msg.id.clone(), i.msg.body.clone())
            })
    };
    if let Some((id, body)) = job {
        info!("📦 retrying deferred advisory {}: below the limit and Llama is free",
short_hex(&id));
        spawn_advisory(st, id, body);
    }
}

// ──────────────────────────────────── AI placement ────────────────────────────────────
//
// The Llama advisory is applied, not just logged. The model reads the operator's
note (the
```

```
// directive) — which devices it spares, which it asks for — so an intent the thermal rules
// cannot read ("the phone's battery is low", "the laptop is in a call") can move work where the
// rules alone would not. The node turns that reading into a device with the rules' ranking.
// The node that ran the model signs the pick as a placement decision (ML-DSA-87, like any
// directive) and it rides back across the mesh on gossip. Each node then places single-answer jobs on the AI's pick while the thermal
// rules allow it: the device must be below its temperature limit, idle, and reporting fresh
// vitals. When the rules veto the pick, the job goes where the rules put it and the veto is
// recorded. The model chooses; the rules keep it safe.
//
// A decision is adopted only as the answer to a directive this node holds, and only from this
// node, an --advisor or a --cmd-allow identity. (v17 took one from any signer when this node had
// asked, so a stranger that saw the directive could answer it and steer where prompts went.) A
// decision lasts at most PLACEMENT_TTL_SECS from its issue time or this node's clock, whichever
// is earlier, and the rules' veto still applies.

/// Body prefix that marks a relayed message as a placement decision rather than a directive.
const PLACEMENT_PREFIX: &str = "harmony-placement/1 ";
/// How long a decision stays in force unless a newer one replaces it.
const PLACEMENT_TTL_SECS: u64 = 600;

#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
struct PlacementBody {
    /// Id of the directive this decision answers.
    answers: String,
    /// The chosen device, hex.
    target: String,
    #[serde(default)]
    label: String,
    reason: String,
    /// What the thermal rules chose when the decision was made, hex, for comparison.
    #[serde(default, skip_serializing_if = "Option::is_none")]
    rules: Option<String>,
    ttl_s: u64,
}

fn placement_body(p: &PlacementBody) -> String {
    format!("{PLACEMENT_PREFIX}{}", serde_json::to_string(p).unwrap_or_default())
}
```

```
/// A placement decision carried in a relayed body, if the body is one and it is well
formed.
fn parse_placement(body: &str) -> Option<PlacementBody> {
    let p: PlacementBody =
serde_json::from_str(body.strip_prefix(PLACEMENT_PREFIX)?).ok()?;
    let ok = is_relay_id(&p.answers)
        && unhex32(&p.target).is_some()
        && p.rules.as_deref().is_none_or(|r| unhex32(r).is_some())
        && p.reason.chars().count() <= 300
        && p.label.chars().count() <= 32
        && (60..=3600).contains(&p.ttl_s);
    ok.then_some(p)
}

/// A decision in force at this node.
#[derive(Clone, Debug)]
struct Placement {
    decision: String,
    answers: String,
    target: NodeId,
    label: String,
    reason: String,
    advisor: NodeId,
    rules: Option<NodeId>,
    issued: u64,
    expires: u64,
}

impl Placement {
    fn from_body(decision: &str, advisor: NodeId, issued: u64, b: &PlacementBody) ->
Option<Self> {
        Some(Placement {
            decision: decision.to_string(),
            answers: b.answers.clone(),
            target: unhex32(&b.target)?,
            label: truncate(sanitize(&b.label).trim(), 24),
            reason: truncate(&sanitize(&b.reason), 200),
            advisor,
            rules: b.rules.as_deref().and_then(unhex32),
            issued,
            expires: issued.saturating_add(b.ttl_s.min(PLACEMENT_TTL_SECS)),
        })
    }
}

/// Why the thermal rules refuse a device as a placement right now, or `None` if they
allow it.
fn placement_veto(v: Option<&Vitals>) -> Option<String> {
    match v {
```

```
        None => Some("no fresh vitals from it".into()),
        Some(v) if v.hot() => Some(format!(
            "{:.0} °C, at or over its {:.0} °C limit",
            v.thermal_c, v.max_temp
        )),
        Some(v) if v.busy => Some("busy with another job".into()),
        Some(_) => None,
    }
}

/// Recent events for the dashboard, newest last. Capped; sequence numbers let a
poller ask
/// only for what it has not seen.
const EVENT_LOG_MAX: usize = 300;

#[derive(Serialize, Clone, Debug)]
struct Event {
    seq: u64,
    t_ms: u64,
    kind: &'static str,
    text: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    from: Option<String>,
    #[serde(skip_serializing_if = "Option::is_none")]
    to: Option<String>,
}

#[derive(Default)]
struct EventLog {
    next: u64,
    items: std::collections::VecDeque<Event>,
}

impl EventLog {
    fn push(&mut self, kind: &'static str, text: &str, from: Option<&NodeId>, to:
Option<&NodeId>) {
        self.next += 1;
        let t_ms = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .map(|d| d.as_millis() as u64)
            .unwrap_or(0);
        if self.items.len() >= EVENT_LOG_MAX {
            self.items.pop_front();
        }
        self.items.push_back(Event {
            seq: self.next,
            t_ms,
            kind,
            text: truncate(&sanitize(text), 300),
            from: from.map(|i| hex(i)),
```

```
            to: to.map(|i| hex(i)),
        });
    }

    fn since(&self, seq: u64) -> Vec<Event> {
        self.items.iter().filter(|e| e.seq > seq).cloned().collect()
    }
}

impl AppState {
    fn draining(&self) -> bool {
        self.phase.load(Ordering::SeqCst) == PHASE_DRAINING
    }

    fn phase_label(&self) -> &'static str {
        match self.phase.load(Ordering::SeqCst) {
            PHASE_STARTING => "starting",
            PHASE_READY => "ready",
            _ => "draining",
        }
    }

    /// Starting → ready, once. Never moves a draining node back to ready.
    fn set_ready(&self) -> bool {
        self.phase
            .compare_exchange(PHASE_STARTING, PHASE_READY, Ordering::SeqCst,
Ordering::SeqCst)
            .is_ok()
    }

    /// Drop a peer that said goodbye: no more gossip, offload or placement to it
until it
    /// handshakes again.
    fn forget_peer(&self, id: &NodeId) {
        lock(&self.peers).remove(id);
        lock(&self.peer_vitals).remove(id);
        lock(&self.client_sessions).remove(id);
    }

    fn event(&self, kind: &'static str, text: &str, from: Option<&NodeId>, to:
Option<&NodeId>) {
        lock(&self.events).push(kind, text, from, to);
    }

    /// A device's display name: its label if known, else its short id.
    fn name_of(&self, id: &NodeId) -> String {
        if *id == self.me.fpr() {
            return self.cfg.label.clone();
        }
        self.fresh_vitals(id)
```

```
            .map(|(v, _)| v.label)
            .filter(|l| !l.is_empty())
            .unwrap_or_else(|| short_id(id))
    }

    fn active_placement(&self) -> Option<Placement> {
        let now = now_unix();
        lock(&self.placement).clone().filter(|p| p.expires > now)
    }

    /// Adopt a decision unless a newer one is already in force.
    fn adopt_placement(&self, p: Placement) -> bool {
        let now = now_unix();
        let mut cur = lock(&self.placement);
        if cur.as_ref().is_some_and(|c| c.issued > p.issued && c.expires > now) {
            return false;
        }
        *cur = Some(p);
        true
    }

    /// The rules' objection to running a job on `target` right now, if any.
    fn placement_check(&self, target: &NodeId) -> Option<String> {
        if *target == self.me.fpr() {
            return placement_veto(Some(&self.vitals()));
        }
        if !lock(&self.peers).contains_key(target) {
            return Some("not a current peer".into());
        }
        placement_veto(self.fresh_vitals(target).map(|(v, _)| v).as_ref())
    }

    /// Where the thermal rules alone would run a job started here.
    fn rules_target(&self) -> Option<NodeId> {
        if placement_veto(Some(&self.vitals())).is_none() {
            return Some(self.me.fpr());
        }
        self.ranked_targets(1).first().map(|(id, _)| *id)
    }

    /// The chosen device's name: its live label, else the label in the decision,
else short id.
    fn placement_name(&self, p: &Placement) -> String {
        let live = self.name_of(&p.target);
        if live == short_id(&p.target) && !p.label.is_empty() {
            p.label.clone()
        } else {
            live
        }
    }
```

```
    /// The decision in force, with the rules' current view of it, for /health and
the dashboard.
    fn placement_json(&self) -> serde_json::Value {
        let Some(p) = self.active_placement() else {
            return serde_json::Value::Null;
        };
        let rules_now = self.rules_target();
        serde_json::json!({
            "decision": p.decision,
            "answers": p.answers,
            "target": hex(&p.target),
            "target_name": self.placement_name(&p),
            "reason": p.reason,
            "advisor": hex(&p.advisor),
            "advisor_name": self.name_of(&p.advisor),
            "rules_at_decision": p.rules.map(|r| hex(&r)),
            "rules_now": rules_now.map(|r| hex(&r)),
            "veto_now": self.placement_check(&p.target),
            "expires_in_s": p.expires.saturating_sub(now_unix()),
        })
    }
}

/// Sign the model's pick as a placement decision, adopt it here, and start it
spreading.
fn issue_placement(st: &Shared, answers: &str, target: NodeId, name: &str, reason:
&str, rules: Option<NodeId>) {
    let b = PlacementBody {
        answers: answers.to_string(),
        target: hex(&target),
        label: truncate(name, 24),
        reason: truncate(&sanitize(reason), 200),
        rules: rules.map(|r| hex(&r)),
        ttl_s: PLACEMENT_TTL_SECS,
    };
    let issued = now_unix();
    let r = sign_relayed(&st.me, &placement_body(&b), issued);
    let me = st.me.fpr();
    let Some(p) = Placement::from_body(&r.id, me, issued, &b) else {
        return;
    };
    let shown = st.name_of(&target);
    {
        let mut rs = lock(&st.relay);
        rs.prune();
        rs.seen.insert(r.id.clone(), Instant::now());
        rs.push(RelayItem {
            msg: r,
            origin: me,
```

```
            from: None,
            first_seen: Instant::now(),
            delivered: HashSet::new(),
            advisory: Advisory::Placement(format!("jobs → {shown}")),
            attempts: 0,
        });
    }
    info!("🧭 AI placement: jobs → {shown} ({}); signed, spreading on gossip",
p.reason);
    st.event("decision", &format!("Llama decided: jobs → {shown}. {}", p.reason),
Some(&me), Some(&target));
    st.adopt_placement(p);
}

/// Who may place this node's jobs with an AI decision: this node, an `--advisor`, or
a
/// `--cmd-allow` identity. Not "whoever answers": a decision decides where prompts
go for up to
/// its TTL, so it needs an explicit grant, as a directive does.
fn placement_signer_ok(
    signer: &NodeId,
    me: &NodeId,
    advisors: &HashSet<NodeId>,
    cmd_allow: &HashSet<NodeId>,
) -> bool {
    signer == me || advisors.contains(signer) || cmd_allow.contains(signer)
}

/// Take in a placement decision that rode in on a ping. Same dedup and ack rules as
a directive.
fn accept_placement(st: &Shared, from: NodeId, signer: NodeId, r: Relayed, pb:
PlacementBody) -> Option<String> {
    let id = r.id.clone();
    let me = st.me.fpr();
    let revoked = st.is_revoked(&signer);
    let mut rs = lock(&st.relay);
    if rs.seen.contains_key(&id) {
        return Some(id);
    }
    rs.seen.insert(id.clone(), Instant::now());
    let asked_by = rs.items.iter().find(|i| i.msg.id == pb.answers).map(|i|
i.origin);
    let signer_ok = placement_signer_ok(&signer, &me, &st.cfg.advisors,
&st.cfg.cmd_allow);
    let authorized = !revoked && asked_by.is_some() && signer_ok;
    if !authorized {
        drop(rs);
        let why = if revoked {
            "signer revoked"
        } else if asked_by.is_none() {
```

```
            "it answers no directive held here"
        } else {
            "signer is not an --advisor or --cmd-allow identity here"
        };
        warn!("🧭 placement decision {} from {} not adopted ({why})", short_hex(&id),
short_id(&signer));
        st.event("reject", &format!("AI decision {} not adopted: {why}",
short_hex(&id)), Some(&signer), Some(&me));
        return Some(id);
    }
    if rs.items.iter().filter(|i| i.origin == signer).count() >= RELAY_PER_ORIGIN {
        return Some(id);
    }
    // Dated ahead of this node's clock (within the allowed skew), a decision would
outrank newer
    // ones and outlive its TTL; it counts from now at the latest.
    let issued = r.issued.min(now_unix());
    let Some(p) = Placement::from_body(&id, signer, issued, &pb) else {
        return Some(id);
    };
    let mut delivered = HashSet::new();
    delivered.insert(from);
    delivered.insert(signer);
    let target = p.target;
    rs.push(RelayItem {
        msg: r,
        origin: signer,
        from: Some(from),
        first_seen: Instant::now(),
        delivered,
        advisory: Advisory::Placement(format!("jobs → {}", short_id(&target))),
        attempts: 0,
    });
    drop(rs);
    let (shown, by, reason) = (st.name_of(&target), st.name_of(&signer),
p.reason.clone());
    if st.adopt_placement(p) {
        info!("🧭 adopted AI placement from {by}: jobs → {shown} ({reason})");
        st.event("decision", &format!("adopted {by}'s AI decision: jobs → {shown}.
{reason}"), Some(&signer), Some(&target));
    }
    Some(id)
}

// ──────────────────────────────────── thermal provider
──────────────────────────────────────

#[derive(Clone)]
struct ThermalProvider;
```

```
impl ThermalProvider {
    /// Best-effort CPU temperature in °C. `HARMONY_MOCK_TEMP` overrides for testing;
    /// a missing sensor reads as a safe-but-unknown 40 °C.
    fn temperature(&self) -> f32 {
        if let Ok(v) = std::env::var("HARMONY_MOCK_TEMP") {
            if let Ok(t) = v.trim().parse::<f32>() {
                return t;
            }
        }
        sensor_celsius().unwrap_or(40.0)
    }
}

/// Linux and Android: the first thermal zone, in millidegrees.
#[cfg(not(windows))]
fn sensor_celsius() -> Option<f32> {
    let s = std::fs::read_to_string("/sys/class/thermal/thermal_zone0/temp").ok()?;
    s.trim().parse::<f32>().ok().map(|m| m / 1000.0)
}

/// Windows: the hottest ACPI thermal zone, from a background sampler.
#[cfg(windows)]
fn sensor_celsius() -> Option<f32> {
    windows_thermal::latest()
}

/// One line from the Windows sampler: °C as text ("none", or anything implausible,
is no reading).
#[cfg_attr(not(windows), allow(dead_code))]
fn thermal_sample(line: &str) -> Option<f32> {
    line.trim()
        .parse::<f32>()
        .ok()
        .filter(|t| (-50.0..=150.0).contains(t))
}

/// Windows has no /sys thermal zones. Its ACPI thermal zones are published as the
"Thermal Zone
/// Information" performance counters (Kelvin, readable without admin rights). One
hidden
/// PowerShell process samples them every 5 s for as long as this node runs, and a
reader thread
/// keeps the latest value. Firmware that exposes no zone gives no reading, and the
node keeps
/// its safe-but-unknown default.
#[cfg(windows)]
mod windows_thermal {
    use std::io::{BufRead, BufReader};
    use std::os::windows::process::CommandExt;
    use std::process::{Command, Stdio};
```

```
    use std::sync::atomic::{AtomicU32, Ordering};
    use std::sync::Once;
    use std::time::Duration;

    /// The latest reading in °C, as f32 bits; NaN while there is none.
    static LATEST: AtomicU32 = AtomicU32::new(0x7fc0_0000);
    static START: Once = Once::new();
    /// CreateProcess flag: no console window for the sampler.
    const CREATE_NO_WINDOW: u32 = 0x0800_0000;

    pub fn latest() -> Option<f32> {
        START.call_once(|| {
            std::thread::spawn(sample_forever);
        });
        let v = f32::from_bits(LATEST.load(Ordering::Relaxed));
        v.is_finite().then_some(v)
    }

    /// The sampler loop; it ends by itself once this node's process is gone.
    fn script() -> String {
        format!(
            "$ErrorActionPreference = 'SilentlyContinue'; \
             while (Get-Process -Id {pid}) {{ \
               $k = (Get-CimInstance -ClassName
Win32_PerfFormattedData_Counters_ThermalZoneInformation | \
                     Measure-Object -Property Temperature -Maximum).Maximum; \
               if ($k -gt 0) {{ [Console]::Out.WriteLine(($k -
273.15).ToString([Globalization.CultureInfo]::InvariantCulture)) }} \
               else {{ [Console]::Out.WriteLine('none') }}; \
               [Console]::Out.Flush(); Start-Sleep -Seconds 5 }}",
            pid = std::process::id()
        )
    }

    fn sample_forever() {
        let mut announced = false;
        loop {
            let child = Command::new("powershell")
                .args(["-NoProfile", "-NonInteractive", "-ExecutionPolicy", "Bypass",
"-Command"])
                .arg(script())
                .stdin(Stdio::null())
                .stdout(Stdio::piped())
                .stderr(Stdio::null())
                .creation_flags(CREATE_NO_WINDOW)
                .spawn();
            match child {
                Ok(mut c) => {
                    if let Some(out) = c.stdout.take() {
                        for line in BufReader::new(out).lines().map_while(Result::ok)
```

```
{
                            let v = super::thermal_sample(&line);
                            if !announced {
                                announced = true;
                                match v {
                                    Some(t) => tracing::info!("🌡 Windows thermal
zones: {t:.1} °C"),
                                    None => tracing::warn!(
                                        "🌡 no Windows thermal zone reports a
temperature; using a safe-but-unknown 40 °C"
                                    ),
                                }
                            }
                            LATEST.store(v.unwrap_or(f32::NAN).to_bits(),
Ordering::Relaxed);
                        }
                    }
                    let _ = c.wait();
                }
                Err(e) => {
                    if !announced {
                        announced = true;
                        tracing::warn!(
                            "🌡 cannot start the PowerShell thermal sampler ({e});
using a safe-but-unknown 40 °C"
                        );
                    }
                }
            }
            std::thread::sleep(Duration::from_secs(30));
        }
    }
}

// ──────────────────────────────────── HTTP responses
────────────────────────────────────


fn bytes_response(body: Vec<u8>) -> Response {
    (
        [(header::CONTENT_TYPE, "application/octet-stream")],
        body,
    )
        .into_response()
}

/// True if the request originates from loopback (or carries the configured admin
token).
/// Used to fence the operational routes that no longer have a shared bearer secret.
fn admin_authorized(state: &AppState, who: SocketAddr, headers: &HeaderMap) -> bool {
    if who.ip().to_canonical().is_loopback() {
```

```
        // Local tools are trusted, but a web page open in this device's browser also
connects
        // from loopback. Refuse what only a foreign page would send: a Host that is
not a
        // loopback name (DNS rebinding) or an Origin that is not this machine
(cross-site).
        return loopback_request_ok(headers);
    }
    if let Some(expected) = &state.cfg.admin_token {
        if let Some(got) = headers
            .get(header::AUTHORIZATION)
            .and_then(|v| v.to_str().ok())
            .and_then(|s| s.strip_prefix("Bearer "))
        {
            return ct_eq(got.as_bytes(), expected.as_bytes());
        }
    }
    false
}

/// Host and Origin of a loopback request name this machine. Any port is accepted, so
an SSH
/// tunnel (`ssh -L 8080:127.0.0.1:3005`) still works; a missing header is fine (curl
sends no
/// Origin), but `Origin: null` from a sandboxed frame is not.
fn loopback_request_ok(headers: &HeaderMap) -> bool {
    fn local_host(h: &str) -> bool {
        let h = h.trim().to_ascii_lowercase();
        let name = match h.strip_prefix('[') {
            // "[::1]" or "[::1]:3005"; anything else after the bracket is not a
loopback name
            Some(rest) => match rest.split_once(']') {
                Some((addr, tail)) if tail.is_empty() || tail.starts_with(':') =>
format!("[{addr}]"),
                _ => return false,
            },
            None => h.split(':').next().unwrap_or("").to_string(),
        };
        matches!(name.as_str(), "127.0.0.1" | "localhost" | "[::1]")
    }
    let host_ok = headers
        .get(header::HOST)
        .is_none_or(|v| v.to_str().is_ok_and(local_host));
    let origin_ok = headers.get(header::ORIGIN).is_none_or(|v| {
        v.to_str()
            .ok()
            .and_then(|o| o.strip_prefix("http://"))
            .is_some_and(local_host)
    });
    host_ok && origin_ok
```

```
}

/// A JSON body was declared. Browsers cannot send this cross-site without a CORS
preflight,
/// which this node never grants, so a form on a foreign page cannot reach the route.
fn json_content(headers: &HeaderMap) -> bool {
    headers
        .get(header::CONTENT_TYPE)
        .and_then(|v| v.to_str().ok())
        .is_some_and(|c|
c.trim_start().to_ascii_lowercase().starts_with("application/json"))
}

// ──────────────────────────────────── handlers ────────────────────────────────────

/// Public: our signed identity bundle (verifying key + KEM encapsulation key +
signature).
/// This node's signed identity bundle.
///
/// Serving this publicly is what makes trust-on-first-use work, and it is also the
largest
/// remaining metadata leak: an observer who watches a bundle fetch learns the
identity that
/// `hello1` goes to the trouble of hiding. `--no-id-endpoint` closes it for
deployments that
/// provision bundles out of band.
async fn get_id(State(st): State<Shared>, ConnectInfo(who): ConnectInfo<SocketAddr>)
-> Response {
    if st.cfg.no_id_endpoint || !inbound_ok(who.ip(), st.cfg.allow_public) {
        return StatusCode::NOT_FOUND.into_response();
    }
    bytes_response(st.me.bundle.clone())
}

/// Loopback/admin: liveness + thermal, with no secrets in the body.
async fn get_health(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    let peers = lock(&st.peers).len();
    let sessions = lock(&st.server_sessions).len();
    let (model_present, bin_present) = model_ready(&st.cfg);
    let relay_tracked = lock(&st.relay).items.len();
    let (trust_mode, trusted) = {
        let t = lock(&st.trust);
        (t.mode, t.pinned.len())
```

```
    };
    let mut advisors: Vec<String> = st.cfg.advisors.iter().map(|a| hex(a)).collect();
    advisors.sort();
    let link = {
        let l = lock(&st.link);
        LinkSnapshot {
            rounds: l.rounds,
            ok: l.ok,
            failed: l.failed,
            blackouts: l.blackouts,
            last_contact: l.last_contact,
        }
    };
    Json(serde_json::json!({
        "status": "ok",
        "state": st.phase_label(),
        "build": BUILD_TAG,
        "pid": std::process::id(),
        "node": hex(&st.me.fpr()),
        "proto": PROTO_VER,
        "port": st.bound_port,
        "requested_port": st.cfg.port,
        "tier": st.cfg.tier.label(),
        "label": st.cfg.label,
        "busy": st.llm_gate.available_permits() == 0,
        "placement": st.placement_json(),
        "model": st.cfg.model_path,
        "infer_policy": format!("{:?}", st.cfg.infer_policy).to_lowercase(),
        "trust": {
            "mode": format!("{trust_mode:?}").to_lowercase(),
            "trusted": trusted,
        },
        "advisors": advisors,
        "suite": "ML-KEM-1024 + ML-DSA-87 + AES-256-GCM + SHA-384",
        "peers": peers,
        "sessions": sessions,
        "thermal_c": st.thermal.temperature(),
        "max_temp": st.cfg.max_temp,
        "relay_tracked": relay_tracked,
        "emcon": st.emcon(),
        "model_file_present": model_present,
        "llama_bin_found": bin_present,
        "engine": st.engine_json(),
        "openai": "/v1",
        "link": {
            "state": st.link_state().label(),
            "rounds": link.rounds,
            "contacts_ok": link.ok,
            "contacts_failed": link.failed,
            "consecutive_blackouts": link.blackouts,
```

```
            "secs_since_contact": link.last_contact.map(|t| t.elapsed().as_secs()),
        },
    }))
    .into_response()
}

/// Loopback/admin: turn emission control on or off.
///
/// `POST /emcon` with `{"on":true}`. Deliberately not reachable from the mesh: a
peer that
/// could silence this node has a denial-of-service primitive, and an attacker who
could set
/// it on every node could shut down the whole mesh without breaking any crypto.
/// Takes `Bytes` rather than `Json<_>` so the authorization check runs before the
body is
/// parsed. With an extractor, a malformed body gets 400 and a good one gets 403,
which tells
/// an unauthorized prober that the endpoint exists. Everything off-loopback now gets
403.
async fn post_emcon(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
    body: Bytes,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if !json_content(&headers) {
        return (StatusCode::UNSUPPORTED_MEDIA_TYPE, "send content-type:
application/json").into_response();
    }
    let Ok(req) = serde_json::from_slice::<EmconReq>(&body) else {
        return (StatusCode::BAD_REQUEST, "expected
{\"on\":true|false}").into_response();
    };
    st.set_emcon(req.on);
    Json(serde_json::json!({ "emcon": st.emcon() })).into_response()
}

#[derive(Deserialize)]
struct EmconReq {
    on: bool,
}

/// Loopback/admin: shut down gracefully, exactly as SIGTERM or Ctrl+C would — new
jobs refused,
/// peers told, in-flight work finished within the grace period. It is the portable
way to stop
/// a node running in the background where it cannot be sent a signal (Windows). A
```

```
second
/// request exits at once, like a second signal. JSON only, so a web page cannot send
it.
async fn post_shutdown(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if !json_content(&headers) {
        return (StatusCode::UNSUPPORTED_MEDIA_TYPE, "send content-type:
application/json").into_response();
    }
    st.stop.notify_one();
    Json(serde_json::json!({ "stopping": true, "pid": std::process::id()
})).into_response()
}

/// Loopback/admin: the peer table (identities + addresses). Not exposed to the mesh
in the
/// clear — mesh peers learn peers only over an authenticated session via
`Msg::GetPeers`.
async fn get_peers(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    let peers: Vec<(NodeId, SocketAddr)> =
        lock(&st.peers).iter().map(|(id, p)| (*id, p.addr)).collect();
    let list: Vec<serde_json::Value> = peers
        .iter()
        .map(|(id, addr)| {
            let v = st.fresh_vitals(id);
            serde_json::json!({
                "id": hex(id),
                "addr": addr.to_string(),
                "thermal_c": v.as_ref().map(|(v, _)| v.thermal_c),
                "max_temp": v.as_ref().map(|(v, _)| v.max_temp),
                "headroom_c": v.as_ref().map(|(v, _)| v.headroom()),
                "busy": v.as_ref().map(|(v, _)| v.busy),
                "tier": v.as_ref().map(|(v, _)| v.tier.clone()),
                "label": v.as_ref().map(|(v, _)| v.label.clone()).filter(|l|
!l.is_empty()),
                "vitals_age_s": v.as_ref().map(|(_, age)| age.as_secs()),
            })
```

```
        })
        .collect();
    Json(list).into_response()
}

#[derive(Deserialize)]
struct RelayReq {
    body: String,
}

/// Loopback/admin: issue a directive to the mesh. It is signed here and spreads by
riding on
/// gossip pings — nothing extra is transmitted, and nothing at all while EMCON is
on.
async fn post_relay(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
    body: Bytes,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if !json_content(&headers) {
        return (StatusCode::UNSUPPORTED_MEDIA_TYPE, "send content-type:
application/json").into_response();
    }
    if st.draining() {
        return (StatusCode::SERVICE_UNAVAILABLE, "this node is shutting
down").into_response();
    }
    let Ok(req) = serde_json::from_slice::<RelayReq>(&body) else {
        return (StatusCode::BAD_REQUEST, "expected
{\"body\":\"...\"}").into_response();
    };
    if let Some(problem) = directive_problem(&req.body) {
        return problem.into_response();
    }
    let r = sign_relayed(&st.me, &req.body, now_unix());
    let id = r.id.clone();
    {
        let mut rs = lock(&st.relay);
        rs.prune();
        rs.seen.insert(id.clone(), Instant::now());
        rs.push(RelayItem {
            msg: r,
            origin: st.me.fpr(),
            from: None,
            first_seen: Instant::now(),
            delivered: HashSet::new(),
```

```
            advisory: Advisory::Origin,
            attempts: 0,
        });
    }
    info!("📦 issued directive {} ({} bytes); spreading on gossip", short_hex(&id),
req.body.len());
    st.event(
        "directive",
        &format!("signed directive {} (ML-DSA-87): {}", short_hex(&id),
truncate(&req.body, 140)),
        Some(&st.me.fpr()),
        None,
    );
    let peers = lock(&st.peers).len();
    Json(serde_json::json!({
        "id": id,
        "origin": hex(&st.me.fpr()),
        "peers": peers,
        "emcon": st.emcon(),
    }))
    .into_response()
}

/// Why a directive body would be refused, if it would be: the same rules for
`/relay` and
/// `/advise`.
fn directive_problem(body: &str) -> Option<(StatusCode, String)> {
    if body.trim().is_empty() {
        return Some((StatusCode::BAD_REQUEST, "empty directive".into()));
    }
    if body.len() > MAX_PROMPT_BYTES {
        return Some((StatusCode::PAYLOAD_TOO_LARGE, format!("directive exceeds
{MAX_PROMPT_BYTES} bytes")));
    }
    if sanitize(body) != body {
        return Some((StatusCode::BAD_REQUEST, "control characters are not
allowed".into()));
    }
    None
}

/// Loopback/admin: a dry run of the advisory. This device's Llama reads `{"body":
note}` exactly
/// as it reads a directive — same prompt, grammar and greedy decoding — and the node
says where
/// that reading would send jobs; nothing is signed, issued or relayed. For trying
notes before
/// sending one to the mesh (`test_harmony.sh reading`). Answers when the run ends
(tens of
/// seconds on a laptop CPU); 503 with the reason when Llama is busy, cooling down or
```

```
hot.
async fn post_advise(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
    body: Bytes,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if !json_content(&headers) {
        return (StatusCode::UNSUPPORTED_MEDIA_TYPE, "send content-type:
application/json").into_response();
    }
    if st.draining() {
        return (StatusCode::SERVICE_UNAVAILABLE, "this node is shutting
down").into_response();
    }
    let Ok(req) = serde_json::from_slice::<RelayReq>(&body) else {
        return (StatusCode::BAD_REQUEST, "expected
{\"body\":\"...\"}").into_response();
    };
    if let Some(problem) = directive_problem(&req.body) {
        return problem.into_response();
    }
    let started = Instant::now();
    let nodes = st.advisory_nodes();
    let named = mentioned_names(&nodes, &req.body);
    let evidence: Vec<serde_json::Value> = device_evidence(&nodes, &req.body)
        .into_iter()
        .map(|(n, e)| serde_json::json!({ "name": n, "says": e }))
        .collect();
    let out = advise_among(&st, &nodes, &req.body).await;
    if let Some(why) = out.detail.strip_prefix("deferred: ") {
        let busy = serde_json::json!({ "deferred": why });
        return (StatusCode::SERVICE_UNAVAILABLE, Json(busy)).into_response();
    }
    let devices: Vec<serde_json::Value> = nodes
        .iter()
        .map(|n| {
            serde_json::json!({
                "id": hex(&n.id),
                "name": note_name(&n.name),
                "me": n.me,
                "thermal_c": n.vitals.as_ref().map(|v| v.thermal_c),
                "max_temp": n.vitals.as_ref().map(|v| v.max_temp),
                "busy": n.vitals.as_ref().map(|v| v.busy),
            })
        })
        .collect();
```

```
    let reading = out.reading.as_ref().map(|r| {
        serde_json::json!({
            "avoid": r.avoid,
            "use": r.prefer,
            "reason": r.reason,
        })
    });
    Json(serde_json::json!({
        "advisory": out.detail.strip_prefix("advisory: ").unwrap_or(&out.detail),
        "named": named,
        "evidence": evidence,
        "reading": reading,
        "target": out.pick.as_ref().map(|p| hex(&p.0)),
        "target_name": out.pick.as_ref().map(|p| p.1.clone()),
        "reason": out.pick.as_ref().map(|p| p.2.clone()),
        "rules": out.rules.map(|id| hex(&id)),
        "devices": devices,
        "elapsed_ms": started.elapsed().as_millis() as u64,
    }))
    .into_response()
}

/// Loopback/admin: directives in flight through this node, who holds each, and the
advisory.
async fn get_relay(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    let mut rs = lock(&st.relay);
    rs.prune();
    let list: Vec<serde_json::Value> = rs
        .items
        .iter()
        .map(|i| {
            let mut holders: Vec<String> = i.delivered.iter().map(|p|
hex(p)).collect();
            holders.sort();
            serde_json::json!({
                "id": i.msg.id,
                "origin": hex(&i.origin),
                "from": i.from.map(|f| hex(&f)),
                "hops": i.msg.hops,
                "age_s": i.first_seen.elapsed().as_secs(),
                "body": truncate(&i.msg.body, 160),
                "held_by": holders,
                "advisory": i.advisory.label(),
```

```
            })
        })
        .collect();
    drop(rs);
    Json(list).into_response()
}

#[derive(Deserialize)]
struct InferBody {
    prompt: String,
    #[serde(default)]
    fanout: Option<usize>,
    #[serde(default)]
    local: Option<bool>,
    #[serde(default)]
    timeout_secs: Option<u64>,
    /// When this node cannot serve (too hot, busy, rate limited), hand the work to a
cooler
    /// peer instead of just failing. Default true — this is the point of a mesh.
    #[serde(default)]
    offload: Option<bool>,
    /// "ai" (default): a single-answer job goes to the device of the AI placement
decision in
    /// force, unless the thermal rules veto it. "rules": ignore the decision.
    #[serde(default)]
    placement: Option<String>,
}

/// Did a local attempt fail for a reason a different node could succeed at?
///
/// Thermal, busy and rate-limit failures are about *this* device's capacity right
now, so a
/// peer is worth asking. A malformed prompt would fail identically everywhere, so it
is not.
fn is_capacity_failure(reason: &str) -> bool {
    let r = reason.to_lowercase();
    r.contains("thermal")
        || r.contains("already in progress")
        || r.contains("rate limited")
        || r.contains("shutting down")
        || r.contains("still loading")
}

/// Loopback/admin: run a prompt across the mesh and report every answer plus the
consensus.
///
/// Admin-gated rather than mesh-exposed on purpose — this spends other nodes'
batteries, and
/// the decision to do that belongs to the operator, not to an arbitrary peer.
async fn post_infer(
```

```
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
    Json(body): Json<InferBody>,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if body.prompt.trim().is_empty() {
        return (StatusCode::BAD_REQUEST, "empty prompt").into_response();
    }
    if body.prompt.len() > MAX_PROMPT_BYTES {
        return (
            StatusCode::PAYLOAD_TOO_LARGE,
            format!("prompt exceeds {MAX_PROMPT_BYTES} bytes"),
        )
            .into_response();
    }

    // Under EMCON the fan-out is clamped to zero rather than refused: a local answer
is
    // still useful, and silently dialling peers would defeat the point of the mode.
    let emcon = st.emcon();
    let fanout = if emcon {
        0
    } else {
        body.fanout.unwrap_or(st.cfg.fanout).clamp(0, MAX_FANOUT)
    };
    let include_local = body.local.unwrap_or(true) || emcon;
    // Two ceilings, because the two paths have different threat models. The local
budget is
    // what this node will spend on its own generation; the remote budget is what it
will ask
    // a peer to spend, and that one stays tight.
    let asked = body.timeout_secs.map(Duration::from_secs);
    let local_budget = asked.unwrap_or(st.cfg.llm_timeout).min(MAX_LOCAL_INFER);
    let remote_budget = asked.unwrap_or(st.cfg.llm_timeout).min(MAX_REMOTE_INFER);

    let client = match build_client() {
        Ok(c) => c,
        Err(e) => {
            error!("infer client: {e}");
            return StatusCode::INTERNAL_SERVER_ERROR.into_response();
        }
    };

    let started = Instant::now();
    let me_id = st.me.fpr();

    // AI placement for a single-answer job, unless the caller pinned where it runs.
```

```
    let ai_eligible = fanout == 0
        && body.local != Some(false)
        && body.offload != Some(false)
        && body.placement.as_deref() != Some("rules");
    let mut placement_json = serde_json::json!({ "mode": "rules" });
    let mut ai_target: Option<String> = None;
    let mut placed: Vec<InferOutcome> = Vec::new();
    if let Some(p) = ai_eligible.then(|| st.active_placement()).flatten() {
        let rules = st.rules_target();
        let veto = st.placement_check(&p.target);
        let target_name = st.placement_name(&p);
        let rules_name = rules.map(|r| st.name_of(&r)).unwrap_or_else(|| "no
device".into());
        match &veto {
            Some(why) => {
                warn!("⚖ rules veto the AI placement on {target_name}: {why}");
                st.event(
                    "veto",
                    &format!("rules veto Llama's pick {target_name} ({why}); the job
goes where the rules put it"),
                    Some(&me_id),
                    Some(&p.target),
                );
            }
            None if p.target == me_id => {
                info!("🧭 AI placement: job runs here (rules: {rules_name})");
                st.event(
                    "place",
                    &format!("AI placement: job runs here on {target_name} (rules
alone: {rules_name})"),
                    Some(&me_id),
                    Some(&me_id),
                );
            }
            None => {
                info!("🧭 AI placement: job → {target_name} (rules alone:
{rules_name})");
                st.event(
                    "place",
                    &format!("AI placement: job → {target_name} (rules alone:
{rules_name})"),
                    Some(&me_id),
                    Some(&p.target),
                );
                let addr = lock(&st.peers).get(&p.target).map(|q| q.addr);
                if let Some(addr) = addr {
                    let t0 = Instant::now();
                    let o = match infer_from_peer(&st, &client, p.target, addr,
&body.prompt, remote_budget).await {
                        Ok(o) => o,
```

```
                        Err(e) => InferOutcome {
                            node: hex(&p.target),
                            tier: "unknown".into(),
                            ok: false,
                            text: e,
                            elapsed_ms: t0.elapsed().as_millis() as u64,
                        },
                    };
                    if !o.ok {
                        st.event(
                            "veto",
                            &format!("job failed on {target_name} ({}); falling back
to the rules", truncate(&o.text, 80)),
                            Some(&p.target),
                            Some(&me_id),
                        );
                    }
                    placed.push(o);
                }
            }
        }
        ai_target = Some(hex(&p.target));
        placement_json = serde_json::json!({
            "mode": "ai",
            "decision": p.decision,
            "advisor": hex(&p.advisor),
            "target": hex(&p.target),
            "target_name": target_name,
            "reason": p.reason,
            "rules_target": rules.map(|r| hex(&r)),
            "rules_name": rules_name,
            "veto": veto,
            "expires_in_s": p.expires.saturating_sub(now_unix()),
        });
    }

    let mut outcomes = if placed.iter().any(|o| o.ok) {
        placed
    } else {
        let mut v = placed;
        v.extend(
            infer_distributed(
                &st,
                &client,
                &body.prompt,
                fanout,
                include_local,
                local_budget,
                remote_budget,
            )
```

```
            .await,
        );
        v
    };

    // Thermal offload: if this device could not serve and the failure was about its
own
    // capacity rather than the request, ask peers instead of returning nothing. Only
needed
    // when no peers were queried already — a fan-out has effectively done this.
    let mut offloaded_to = None;
    let mut offload_order: Vec<serde_json::Value> = Vec::new();
    let offload = body.offload.unwrap_or(true);
    let local_blocked = outcomes
        .iter()
        .any(|o| !o.ok && o.node == hex(&st.me.fpr()) &&
is_capacity_failure(&o.text));
    if offload && local_blocked && !outcomes.iter().any(|o| o.ok) {
        let reason = outcomes
            .iter()
            .find(|o| !o.ok && o.node == hex(&st.me.fpr()))
            .map(|o| o.text.clone())
            .unwrap_or_default();
        let order = st.offload_order();
        let summary = order
            .iter()
            .map(|(id, v)| match v {
                Some(v) => format!("{}@{:.1}C", short_id(id), v.thermal_c),
                None => format!("{}@?", short_id(id)),
            })
            .collect::<Vec<_>>()
            .join(" ");
        info!("🌡 local node cannot serve ({reason}); offloading, coolest first:
{summary}");
        st.event("thermal", &format!("this device refused the job: {reason};
offloading by the rules"), Some(&me_id), None);
        offload_order = order
            .iter()
            .map(|(id, v)| {
                serde_json::json!({
                    "node": hex(id),
                    "thermal_c": v.as_ref().map(|v| v.thermal_c),
                    "headroom_c": v.as_ref().map(|v| v.headroom()),
                    "busy": v.as_ref().map(|v| v.busy),
                })
            })
            .collect();
        let relayed = infer_distributed(
            &st,
            &client,
```

```
            &body.prompt,
            st.cfg.fanout,
            false,
            local_budget,
            remote_budget,
        )
        .await;
        if let Some(ok) = relayed.iter().find(|o| o.ok) {
            offloaded_to = Some(ok.node.clone());
            info!("🌡️ offloaded to {}", short_hex(&ok.node));
            if let Some(to) = unhex32(&ok.node) {
                st.event("offload", &format!("job offloaded to {} (coolest idle
peer)", st.name_of(&to)), Some(&me_id), Some(&to));
            }
        }
        outcomes.extend(relayed);
    }

    let (agree, answered, text) = consensus(&outcomes);
    if let Some(o) = outcomes.iter().find(|o| o.ok) {
        if let Some(by) = unhex32(&o.node) {
            st.event(
                "job",
                &format!("answered by {} ({}) in {} ms: {}", st.name_of(&by), o.tier,
o.elapsed_ms, truncate(&o.text, 80)),
                Some(&by),
                Some(&me_id),
            );
        }
    } else if let Some(o) = outcomes.last() {
        st.event("job", &format!("no device answered: {}", truncate(&o.text, 100)),
None, Some(&me_id));
    }
    if let (Some(t), Some(obj)) = (&ai_target, placement_json.as_object_mut()) {
        obj.insert("applied".into(), serde_json::json!(outcomes.iter().any(|o| o.ok
&& &o.node == t)));
    }

    Json(serde_json::json!({
        "prompt": body.prompt,
        "answered": answered,
        "queried": outcomes.len(),
        "agreement": agree,
        // Agreement across independently-run models is evidence, not proof:
identical models
        // can share an error, so a unanimous answer is not automatically a correct
one.
        "consensus": if agree >= 2 { Some(text) } else { None },
        "elapsed_ms": started.elapsed().as_millis() as u64,
        "offloaded_to": offloaded_to,
```

```
        "offload_order": offload_order,
        "thermal_c": st.thermal.temperature(),
        "emcon": emcon,
        "placement": placement_json,
        "answers": outcomes,
    }))
    .into_response()
}

// ──────────────────────────────────── OpenAI-compatible API (v19)
────────────────────────────────────
//
// Chat apps, editor plugins and the official SDKs all speak OpenAI's chat-
completions API. Serving
// it here makes the mesh a drop-in local backend for them: base URL
http://127.0.0.1:<port>/v1, any
// API key (off loopback, HARMONY_ADMIN_TOKEN). The fence is /infer's: loopback with
a local Host and
// Origin, or the admin token. No CORS: a page in a browser cannot call it; a program
can.

/// A chat-completions request: the fields this mesh uses. Everything else a client
sends (tools,
/// penalties, response formats...) is ignored.
#[derive(Deserialize)]
struct ChatRequest {
    #[serde(default)]
    model: Option<String>,
    messages: Vec<ChatMessageIn>,
    #[serde(default)]
    max_tokens: Option<i64>,
    #[serde(default)]
    max_completion_tokens: Option<i64>,
    #[serde(default)]
    temperature: Option<f32>,
    #[serde(default)]
    top_p: Option<f32>,
    #[serde(default)]
    stream: Option<bool>,
    #[serde(default)]
    n: Option<u32>,
    /// Harmony's own options; other servers' clients never send them.
    #[serde(default)]
    harmony: Option<ChatOpts>,
}

#[derive(Deserialize)]
struct ChatMessageIn {
    role: String,
    #[serde(default)]
```

```
    content: Option<serde_json::Value>,
}

/// Where a conversation may run, as /infer's options.
#[derive(Deserialize, Default, Clone, Debug)]
struct ChatOpts {
    /// "ai" (default): the AI placement decision in force, unless the rules veto it.
"rules": ignore it.
    #[serde(default)]
    placement: Option<String>,
    /// false: this device or nothing.
    #[serde(default)]
    offload: Option<bool>,
    #[serde(default)]
    timeout_secs: Option<u64>,
}

/// The text of a message's `content`: a string, or the text parts of an array.
Images and other
/// parts are not something these models read.
fn content_text(v: &serde_json::Value) -> String {
    match v {
        serde_json::Value::String(s) => s.clone(),
        serde_json::Value::Array(parts) => parts
            .iter()
            .filter_map(|p| match p {
                serde_json::Value::String(s) => Some(s.as_str()),
                serde_json::Value::Object(o)
                    if o.get("type").and_then(|t| t.as_str()).is_none_or(|t| t ==
"text") =>
                {
                    o.get("text").and_then(|t| t.as_str())
                }
                _ => None,
            })
            .collect::<Vec<_>>()
            .join("\n"),
        _ => String::new(),
    }
}

/// The conversation in a chat request, checked and cut to fit (see
`fit_conversation`).
fn chat_job(req: ChatRequest) -> Result<ChatJob, String> {
    if req.n.is_some_and(|n| n > 1) {
        return Err("n > 1 is not supported: the mesh gives one answer".into());
    }
    let mut turns = Vec::new();
    for m in req.messages {
        let role = match m.role.as_str() {
```

```
            "system" | "developer" => "system",
            "user" => "user",
            "assistant" => "assistant",
            // No tool calling here; a tool's output without the call that asked for
it means nothing.
            "tool" | "function" => continue,
            other => return Err(format!("unknown message role {:?}", truncate(other,
32))),
        };
        let content = m.content.as_ref().map(content_text).unwrap_or_default();
        if content.trim().is_empty() && role != "user" {
            continue;
        }
        turns.push(ChatTurn { role: role.to_string(), content });
    }
    if !turns.iter().any(|t| t.role == "user" && !t.content.trim().is_empty()) {
        return Err("messages must include a user message with text".into());
    }
    let (turns, _) = fit_conversation(turns, CHAT_MAX_PROMPT_BYTES)?;
    let max_tokens = req
        .max_completion_tokens
        .or(req.max_tokens)
        .filter(|n| *n > 0)
        .map(|n| n.min(i64::from(CHAT_MAX_TOKENS)) as u32);
    Ok(ChatJob {
        turns,
        max_tokens,
        temperature: req.temperature.filter(|t| t.is_finite()).map(|t| t.clamp(0.0,
2.0)),
        top_p: req.top_p.filter(|p| p.is_finite()).map(|p| p.clamp(0.0, 1.0)),
    })
}

/// An error in OpenAI's shape, which every client knows how to show.
fn openai_error(status: StatusCode, kind: &str, message: &str) -> Response {
    (
        status,
        Json(serde_json::json!({
            "error": { "message": message, "type": kind, "param": null, "code": null
}
        })),
    )
        .into_response()
}

/// Tokens in a text, roughly (about four characters a token in English), for `usage`
when the
/// runner did not count them.
fn approx_tokens(text: &str) -> u64 {
    (text.chars().count() as u64).div_ceil(4)
```

```
}

/// The chat.completion object for an answer.
fn chat_completion_json(
    id: &str,
    created: u64,
    model: &str,
    a: &ChatAnswer,
    harmony: serde_json::Value,
) -> serde_json::Value {
    let prompt = a.prompt_tokens.unwrap_or(0);
    let completion = a.completion_tokens.unwrap_or_else(|| approx_tokens(&a.text));
    serde_json::json!({
        "id": id,
        "object": "chat.completion",
        "created": created,
        "model": model,
        "choices": [{
            "index": 0,
            "message": { "role": "assistant", "content": a.text },
            "finish_reason": a.finish,
        }],
        "usage": {
            "prompt_tokens": prompt,
            "completion_tokens": completion,
            "total_tokens": prompt + completion,
        },
        "harmony": harmony,
    })
}

/// The same answer as server-sent events: OpenAI's chunks (role, content, finish)
and [DONE]. The
/// whole answer is one content chunk, sent once it is complete.
fn chat_stream_body(id: &str, created: u64, model: &str, a: &ChatAnswer) -> String {
    let chunk = |delta: serde_json::Value, finish: Option<&str>| {
        serde_json::json!({
            "id": id,
            "object": "chat.completion.chunk",
            "created": created,
            "model": model,
            "choices": [{ "index": 0, "delta": delta, "finish_reason": finish }],
        })
    };
    let mut out = String::new();
    for c in [
        chunk(serde_json::json!({ "role": "assistant", "content": "" }), None),
        chunk(serde_json::json!({ "content": a.text }), None),
        chunk(serde_json::json!({}), Some(a.finish)),
    ] {
```

```
        out.push_str("data: ");
        out.push_str(&c.to_string());
        out.push_str("\n\n");
    }
    out.push_str("data: [DONE]\n\n");
    out
}

/// A conversation's answer, and how the mesh got it.
struct ChatRun {
    answer: Option<(InferOutcome, ChatAnswer)>,
    /// Devices asked that did not answer, in order.
    tried: Vec<InferOutcome>,
    /// "ai" when the AI placement decision chose the first device, else "rules".
    placement: &'static str,
    /// The request itself was refused (too long, malformed): another device would
refuse it too.
    bad_request: bool,
}

/// Run a conversation where the mesh's rules put it: the AI placement decision in
force unless the
/// rules veto it, else this device; then, when the device asked cannot take it, the
coolest idle
/// peers. Unlike an /infer offload, peers are asked one at a time, so one answer
costs one device.
async fn chat_dispatch(
    st: &Shared,
    client: &reqwest::Client,
    job: &ChatJob,
    opts: &ChatOpts,
) -> ChatRun {
    let me = st.me.fpr();
    let me_hex = hex(&me);
    let offload = opts.offload.unwrap_or(true) && !st.emcon();
    let asked = opts.timeout_secs.map(Duration::from_secs);
    let local_budget = asked.unwrap_or(st.cfg.llm_timeout).min(MAX_LOCAL_INFER);
    let remote_budget = asked.unwrap_or(st.cfg.llm_timeout).min(MAX_REMOTE_INFER);
    let mut run = ChatRun {
        answer: None,
        tried: Vec::new(),
        placement: "rules",
        bad_request: false,
    };

    let mut order: Vec<NodeId> = Vec::new();
    if offload && opts.placement.as_deref() != Some("rules") {
        if let Some(p) = st.active_placement() {
            let name = st.placement_name(&p);
            match st.placement_check(&p.target) {
```

```
                Some(why) => {
                    warn!("⚖ rules veto the AI placement on {name}: {why}");
                    st.event(
                        "veto",
                        &format!("rules veto Llama's pick {name} ({why}); the chat
goes where the rules put it"),
                        Some(&me),
                        Some(&p.target),
                    );
                }
                None => {
                    run.placement = "ai";
                    info!("🧭 AI placement: chat → {name}");
                    st.event("place", &format!("AI placement: chat → {name}"),
Some(&me), Some(&p.target));
                    order.push(p.target);
                }
            }
        }
    }
    if !order.contains(&me) {
        order.push(me);
    }
    if offload {
        for (id, _) in st.ranked_targets(st.cfg.fanout) {
            if !order.contains(&id) {
                order.push(id);
            }
        }
    }

    for target in order {
        let t0 = Instant::now();
        if target == me {
            match infer_chat_locally(st, job, local_budget, CHAT_LOCAL_WAIT).await {
                Ok(a) => {
                    let o = InferOutcome {
                        node: me_hex.clone(),
                        tier: st.cfg.tier.label().to_string(),
                        ok: true,
                        text: a.text.clone(),
                        elapsed_ms: t0.elapsed().as_millis() as u64,
                    };
                    st.event(
                        "job",
                        &format!("chat answered here ({}) in {} ms: {}", o.tier,
o.elapsed_ms, truncate(&o.text, 80)),
                        Some(&me),
                        Some(&me),
                    );
```

```
                    run.answer = Some((o, a));
                    return run;
                }
                Err(e) => {
                    let fault = request_fault(&e);
                    if fault {
                        run.bad_request = true;
                    } else if offload {
                        info!("🌡️ this device cannot take the chat ({e}); asking the
coolest idle peers");
                        st.event(
                            "thermal",
                            &format!("this device refused the chat: {e}; offloading
by the rules"),
                            Some(&me),
                            None,
                        );
                    }
                    run.tried.push(InferOutcome {
                        node: me_hex.clone(),
                        tier: st.cfg.tier.label().to_string(),
                        ok: false,
                        text: e,
                        elapsed_ms: t0.elapsed().as_millis() as u64,
                    });
                    if fault {
                        return run;
                    }
                }
            }
            continue;
        }
        let failed = |text: String| InferOutcome {
            node: hex(&target),
            tier: "unknown".into(),
            ok: false,
            text,
            elapsed_ms: t0.elapsed().as_millis() as u64,
        };
        let addr = lock(&st.peers).get(&target).map(|p| p.addr);
        let outcome = match (addr, chat_request(job, remote_budget)) {
            (Some(addr), Some(req)) => {
                match infer_msg_from_peer(st, client, target, addr, &req,
remote_budget).await {
                    Ok(o) => o,
                    Err(e) => failed(e),
                }
            }
            (None, _) => failed("not a current peer".into()),
            (Some(_), None) => failed("the conversation is too long for one mesh
```

```
frame".into()),
        };
        if outcome.ok {
            let name = st.name_of(&target);
            if run.tried.iter().any(|o| o.node == me_hex) {
                info!("🌡️ chat offloaded to {name}");
                st.event(
                    "offload",
                    &format!("chat offloaded to {name} (coolest idle peer)"),
                    Some(&me),
                    Some(&target),
                );
            }
            st.event(
                "job",
                &format!(
                    "chat answered by {name} ({}) in {} ms: {}",
                    outcome.tier,
                    outcome.elapsed_ms,
                    truncate(&outcome.text, 80)
                ),
                Some(&target),
                Some(&me),
            );
            let a = ChatAnswer {
                text: outcome.text.clone(),
                prompt_tokens: None,
                completion_tokens: None,
                finish: "stop",
            };
            run.answer = Some((outcome, a));
            return run;
        }
        run.tried.push(outcome);
    }
    if let Some(o) = run.tried.last() {
        st.event(
            "job",
            &format!("no device answered the chat: {}", truncate(&o.text, 100)),
            None,
            Some(&me),
        );
    }
    run
}

/// Loopback/admin: the OpenAI model list. One model, the mesh: each device answers
with its own.
async fn get_models(
    State(st): State<Shared>,
```

```
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    Json(serde_json::json!({
        "object": "list",
        "data": [{
            "id": OPENAI_MODEL_ID,
            "object": "model",
            "created": now_unix(),
            "owned_by": "harmony",
            "harmony": { "this_device": st.cfg.model_path, "tier":
st.cfg.tier.label() },
        }],
    }))
    .into_response()
}

/// Loopback/admin: OpenAI-compatible chat completions (v19). Takes `Bytes`, so the
fence is checked
/// before the body is parsed (as /emcon does). `stream: true` gets server-sent
events, in one piece
/// once the answer is complete; `harmony: {placement, offload, timeout_secs}` steers
it like /infer.
async fn post_chat_completions(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
    body: Bytes,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if !json_content(&headers) {
        return openai_error(
            StatusCode::UNSUPPORTED_MEDIA_TYPE,
            "invalid_request_error",
            "send the body as Content-Type: application/json",
        );
    }
    let req: ChatRequest = match serde_json::from_slice(&body) {
        Ok(r) => r,
        Err(e) => {
            return openai_error(StatusCode::BAD_REQUEST, "invalid_request_error",
&format!("invalid request: {e}"))
        }
    };
    let stream = req.stream.unwrap_or(false);
```

```
    let model = req
        .model
        .as_deref()
        .map(|m| truncate(sanitize(m).trim(), 64))
        .filter(|m| !m.is_empty())
        .unwrap_or_else(|| OPENAI_MODEL_ID.to_string());
    let opts = req.harmony.clone().unwrap_or_default();
    let job = match chat_job(req) {
        Ok(j) => j,
        Err(e) => return openai_error(StatusCode::BAD_REQUEST,
"invalid_request_error", &e),
    };
    let client = match build_client() {
        Ok(c) => c,
        Err(e) => {
            error!("chat client: {e}");
            return openai_error(StatusCode::INTERNAL_SERVER_ERROR, "server_error",
"could not build the mesh client");
        }
    };
    let started = Instant::now();
    let run = chat_dispatch(&st, &client, &job, &opts).await;
    let Some((by, answer)) = run.answer else {
        let reason = run
            .tried
            .last()
            .map(|o| truncate(&o.text, 300))
            .unwrap_or_else(|| "no device could take it".into());
        return if run.bad_request {
            openai_error(StatusCode::BAD_REQUEST, "invalid_request_error", &reason)
        } else {
            openai_error(StatusCode::SERVICE_UNAVAILABLE, "server_error", &format!
("no device answered: {reason}"))
        };
    };
    let device = unhex32(&by.node)
        .map(|id| st.name_of(&id))
        .unwrap_or_else(|| short_hex(&by.node));
    let id = format!("chatcmpl-{:016x}", OsRng.next_u64());
    let created = now_unix();
    let mut resp = if stream {
        (
            StatusCode::OK,
            [(header::CONTENT_TYPE, "text/event-stream"), (header::CACHE_CONTROL,
"no-cache")],
            chat_stream_body(&id, created, &model, &answer),
        )
            .into_response()
    } else {
        let harmony = serde_json::json!({
```

```
            "node": by.node,
            "device": device,
            "tier": by.tier,
            "elapsed_ms": started.elapsed().as_millis() as u64,
            "placement": run.placement,
            "answered_here": by.node == hex(&st.me.fpr()),
            "tried": run.tried.len() + 1,
        });
        Json(chat_completion_json(&id, created, &model, &answer,
harmony)).into_response()
    };
    if let Ok(v) = axum::http::HeaderValue::from_str(&device) {
        resp.headers_mut().insert("x-harmony-device", v);
    }
    if let Ok(v) = axum::http::HeaderValue::from_str(&by.node) {
        resp.headers_mut().insert("x-harmony-node", v);
    }
    resp
}

/// Loopback/admin: recent events (`?since=<seq>` for only the new ones), for the
dashboard.
async fn get_events(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    headers: HeaderMap,
    uri: axum::http::Uri,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    let since = uri
        .query()
        .and_then(|q| q.split('&').find_map(|kv| kv.strip_prefix("since=")))
        .and_then(|v| v.parse::<u64>().ok())
        .unwrap_or(0);
    let (next, events) = {
        let log = lock(&st.events);
        (log.next, log.since(since))
    };
    Json(serde_json::json!({ "next": next, "events": events })).into_response()
}

/// Loopback/admin: the live dashboard. One self-contained page (no CDN: the mesh may
be
/// offline) that polls /health, /peers, /events and /relay, and can issue directives
and jobs.
async fn get_ui(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
```

```
    headers: HeaderMap,
) -> Response {
    if !admin_authorized(&st, who, &headers) {
        return StatusCode::FORBIDDEN.into_response();
    }
    (
        [
            (header::CONTENT_TYPE, "text/html; charset=utf-8"),
            (header::CACHE_CONTROL, "no-store"),
            (header::X_CONTENT_TYPE_OPTIONS, "nosniff"),
            (header::REFERRER_POLICY, "no-referrer"),
            (
                header::CONTENT_SECURITY_POLICY,
                "default-src 'none'; script-src 'unsafe-inline'; style-src 'unsafe-
inline'; \
                 connect-src 'self'; img-src data:; frame-ancestors 'none'; base-uri
'none'; \
                 form-action 'none'",
            ),
        ],
        UI_HTML,
    )
        .into_response()
}

/// The dashboard page served by `get_ui`.
const UI_HTML: &str = r##"<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Harmony Mesh</title>
<style>
:root{
  --bg:#0b0f14; --panel:#121821; --panel2:#18202b; --line:#243041; --text:#e6edf3; --
muted:#8b98a8; --dim:#5d6b7c;
  --cool:#3fb950; --warm:#d29922; --hot:#f85149; --ai:#a371f7; --rules:#2dd4bf; --
carry:#58a6ff; --ack:#3fb950; --job:#e3b341;
  --radius:12px; --mono:ui-monospace,SFMono-Regular,Menlo,Consolas,monospace;
}
*{box-sizing:border-box}
html,body{margin:0;background:var(--bg);color:var(--text);font:14px/1.45 system-ui,-
apple-system,"Segoe UI",Roboto,sans-serif}
button,input,textarea{font:inherit;color:inherit}
.wrap{max-width:1280px;margin:0 auto;padding:16px}
header{display:flex;flex-wrap:wrap;align-items:center;gap:10px 18px;margin-
bottom:14px}
header h1{font-size:18px;margin:0;letter-spacing:.2px}
header h1 span{color:var(--muted);font-weight:500}
.pill{display:inline-flex;align-items:center;gap:6px;padding:3px 10px;border:1px
```

```
solid var(--line);border-radius:999px;color:var(--muted);font-size:12px;white-space:nowrap}
.dot{width:8px;height:8px;border-radius:50%;background:var(--dim)}
.dot.live{background:var(--cool);box-shadow:0 0 0 0 rgba(63,185,80,.6);animation:beat 2s infinite}
.dot.down{background:var(--hot)}
@keyframes beat{0%{box-shadow:0 0 0 0 rgba(63,185,80,.55)}70%{box-shadow:0 0 0 8px rgba(63,185,80,0)}100%{box-shadow:0 0 0 0 rgba(63,185,80,0)}}
.grid{display:grid;gap:14px}
.top{grid-template-columns:minmax(0,1.35fr) minmax(0,1fr)}
.mid{grid-template-columns:minmax(0,1fr) minmax(0,1.1fr)}
@media (max-width:900px){.top,.mid{grid-template-columns:minmax(0,1fr)}}
.panel{background:var(--panel);border:1px solid var(--line);border-radius:var(--radius);padding:14px;min-width:0}
.panel h2{font-size:12px;text-transform:uppercase;letter-spacing:.08em;color:var(--muted);margin:0 0 10px;font-weight:600;display:flex;justify-content:space-between;align-items:center;gap:8px}
.panel h2 small{text-transform:none;letter-spacing:0;font-weight:400;color:var(--dim)}
#map{width:100%;height:auto;display:block}
.devices{display:grid;grid-template-columns:repeat(auto-fill,minmax(250px,1fr));gap:12px;margin:14px 0}
.dev{background:var(--panel);border:1px solid var(--line);border-radius:var(--radius);padding:14px;position:relative}
.dev.ai{border-color:var(--ai);box-shadow:0 0 0 1px var(--ai) inset}
.dev .name{font-size:18px;font-weight:650;display:flex;align-items:center;gap:8px;flex-wrap:wrap}
.dev .id{font:12px var(--mono);color:var(--dim)}
.chips{display:flex;gap:6px;flex-wrap:wrap;margin-top:6px;min-height:22px}
.chip{font-size:11px;padding:2px 8px;border-radius:999px;border:1px solid var(--line);color:var(--muted)}
.chip.me{border-color:#3b4b60;color:var(--text)}
.chip.ai{border-color:var(--ai);color:var(--ai)}
.chip.rules{border-color:var(--rules);color:var(--rules)}
.chip.veto{border-color:var(--hot);color:var(--hot)}
.chip.busy{border-color:var(--warm);color:var(--warm)}
.temp{display:flex;align-items:baseline;gap:8px;margin:12px 0 6px}
.temp b{font:600 34px/1 var(--mono);letter-spacing:-1px}
.temp span{color:var(--muted)}
.bar{position:relative;height:8px;border-radius:6px;background:var(--panel2);overflow:hidden}
.bar i{position:absolute;left:0;top:0;bottom:0;border-radius:6px;transition:width .6s ease,background .6s}
.facts{display:grid;grid-template-columns:auto 1fr;gap:3px 12px;margin-top:10px;font-size:12.5px}
.facts dt{color:var(--dim)}
.facts dd{margin:0;font-family:var(--mono);overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.place{display:flex;flex-direction:column;gap:10px}
.pick{font-size:13px;color:var(--muted)}
```

```
.pick b{display:block;font-size:26px;color:var(--ai);letter-spacing:.3px;margin-top:2px}
.reason{border-left:3px solid var(--ai);padding:6px 10px;background:rgba(163,113,247,.08);border-radius:0 8px 8px 0;font-style:italic}
.cmp{display:grid;grid-template-columns:1fr 1fr;gap:10px}
.cmp div{background:var(--panel2);border-radius:10px;padding:10px}
.cmp small{color:var(--dim);display:block;font-size:11px;text-transform:uppercase;letter-spacing:.06em}
.cmp strong{font-size:16px}
.verdict{border-radius:10px;padding:10px 12px;font-weight:550}
.verdict.ok{background:rgba(63,185,80,.12);color:#7ee787;border:1px solid rgba(63,185,80,.35)}
.verdict.veto{background:rgba(248,81,73,.12);color:#ffa198;border:1px solid rgba(248,81,73,.4)}
.verdict.none{background:var(--panel2);color:var(--muted);border:1px solid var(--line);font-weight:400}
.meta{color:var(--dim);font-size:12px}
label.lbl{display:block;color:var(--muted);font-size:12px;margin:0 0 6px}
textarea,input[type=text]{width:100%;background:var(--panel2);border:1px solid var(--line);border-radius:10px;padding:10px 12px;outline:none}
textarea{min-height:64px;resize:vertical}
textarea:focus,input[type=text]:focus{border-color:#3b82f6}
.row{display:flex;gap:8px;flex-wrap:wrap;align-items:center;margin-top:8px}
.btn{background:#1f6feb;border:0;border-radius:10px;padding:9px 14px;font-weight:600;cursor:pointer}
.btn:hover{background:#2f81f7}
.btn:disabled{opacity:.5;cursor:default}
.btn.ghost{background:transparent;border:1px solid var(--line);color:var(--muted);font-weight:500;padding:6px 10px;font-size:12.5px}
.btn.ghost:hover{border-color:#3b4b60;color:var(--text)}
.seg{display:inline-flex;border:1px solid var(--line);border-radius:10px;overflow:hidden}
.seg button{background:transparent;border:0;padding:7px 12px;color:var(--muted);cursor:pointer}
.seg button.on{background:var(--panel2);color:var(--text)}
.result{margin-top:12px;background:var(--panel2);border-radius:10px;padding:12px;display:none}
.result.show{display:block}
.result .ans{font-size:16px;margin:4px 0 8px}
.sep{height:1px;background:var(--line);margin:14px 0}
.events{list-style:none;margin:0;padding:0;max-height:430px;overflow:auto}
.events li{display:grid;grid-template-columns:62px 74px 1fr;gap:8px;padding:7px 2px;border-bottom:1px solid #1b2430;align-items:start}
.events li.new{animation:flash 1.6s ease}
@keyframes flash{from{background:rgba(88,166,255,.14)}to{background:transparent}}
.events time{font:12px var(--mono);color:var(--dim);padding-top:2px}
.kind{font-size:10.5px;text-transform:uppercase;letter-spacing:.06em;border-radius:6px;padding:2px 6px;text-align:center;align-self:start;border:1px solid}
.ev-text{min-width:0;overflow-wrap:anywhere}
.ev-route{display:block;color:var(--dim);font-size:11.5px}
```

```
table{width:100%;border-collapse:collapse;font-size:12.5px}
th{color:var(--dim);font-weight:500;text-align:left;padding:6px 8px;border-bottom:1px
solid var(--line)}
td{padding:7px 8px;border-bottom:1px solid #1b2430;vertical-align:top}
td.mono{font-family:var(--mono)}
.scroll{overflow-x:auto}
.banner{display:none;margin-bottom:12px;padding:10px 12px;border-
radius:10px;background:rgba(248,81,73,.12);border:1px solid
rgba(248,81,73,.4);color:#ffa198}
.banner.show{display:block}
.empty{color:var(--dim);font-size:13px;padding:6px 0}
.spin{display:inline-block;width:12px;height:12px;border:2px solid var(--
line);border-top-color:var(--text);border-radius:50%;animation:rot .8s linear
infinite;vertical-align:-2px;margin-right:6px}
@keyframes rot{to{transform:rotate(360deg)}}
</style>
</head>
<body>
<div class="wrap">
  <header>
    <h1>Harmony mesh <span id="who">·</span></h1>
    <span class="pill"><span class="dot" id="live"></span><span
id="link">connecting</span></span>
    <span class="pill" title="Session suite">ML-KEM-1024 · ML-DSA-87 · AES-256-
GCM</span>
    <span class="pill" id="peerCount">0 peers</span>
    <span class="pill" id="phase">–</span>
  </header>
  <div class="banner" id="down">This node is not answering. Is rust_p2p_node running?
(bash ~/node.sh status, or node.cmd status on Windows)</div>
  <div class="banner" id="draining">This node is shutting down: new jobs are refused
and its peers have been told.</div>

  <div class="grid top">
    <section class="panel">
      <h2>Mesh <small>pulses: gossip, acks, decisions, jobs</small></h2>
      <svg id="map" viewBox="0 0 600 270" role="img" aria-label="Mesh map">
        <defs>
          <radialGradient id="glowAi"><stop offset="0" stop-color="#a371f7" stop-
opacity=".35"/><stop offset="1" stop-color="#a371f7" stop-opacity="0"/>
</radialGradient>
        </defs>
        <g id="mapLayer"></g>
        <g id="pulseLayer"></g>
        <circle id="pulseTpl" r="6" fill="#58a6ff" opacity="0"/>
      </svg>
    </section>
    <section class="panel place" id="placement">
      <h2>AI placement <small id="ttl"></small></h2>
      <div id="placeBody" class="empty">Loading…</div>
```

```
    </section>
  </div>

  <div class="devices" id="devices"></div>

  <div class="grid mid">
    <section class="panel">
      <h2>Directive <small>signed here, piggybacked on gossip, answered by
Llama</small></h2>
      <label class="lbl" for="dir">Tell the mesh where AI work should run</label>
      <textarea id="dir" maxlength="900">Which device should take the next AI job?
</textarea>
      <div class="row" id="presets"></div>
      <div class="row"><button class="btn" id="send">Sign &amp;amp; send
directive</button><span class="meta" id="sendMsg" aria-live="polite"></span></div>
      <div class="sep"></div>
      <h2>Run a job <small>one answer; placed by the AI decision or the rules</small>
</h2>
      <label class="lbl" for="prompt">Prompt</label>
      <input type="text" id="prompt" maxlength="900" value="What is the capital of
France? Answer in one word.">
      <div class="row">
        <div class="seg" role="group" aria-label="Placement mode"><button id="modeAi"
class="on" type="button">AI placement</button><button id="modeRules"
type="button">Rules only</button></div>
        <button class="btn" id="run">Run job</button>
      </div>
      <div class="result" id="result" aria-live="polite"></div>
    </section>
    <section class="panel">
      <h2>Events <small id="evCount"></small></h2>
      <ul class="events" id="events"></ul>
    </section>
  </div>

  <section class="panel" style="margin-top:14px">
    <h2>Directives &amp;amp; decisions in flight <small>signature verified at every
hop</small></h2>
    <div class="scroll"><table><thead><tr><th>Id</th><th>Kind</th><th>Origin</th>
<th>Hops</th><th>Held by</th><th>Status</th></tr></thead><tbody id="relay"></tbody>
</table></div>
  </section>
</div>

<script>
"use strict";
const $ = (id) => document.getElementById(id);
const esc = (s) => String(s == null ? "" : s).replace(/[&<>"']/g, (c) =>
({"&":"&amp;","<":"&lt;",">":"&gt;","\"":"&quot;","'":"&#39;"}[c]));
const short = (h) => (h || "").slice(0, 12);
```

```
const KIND = {
  directive:["directive","#58a6ff"], carry:["gossip","#58a6ff"], ack:
["ack","#3fb950"], arrive:["arrived","#58a6ff"],
  advisory:["llama","#a371f7"], decision:["decision","#a371f7"], place:
["placed","#e3b341"], veto:["veto","#f85149"],
  thermal:["thermal","#ff7b72"], offload:["offload","#e3b341"], job:
["job","#3fb950"], reject:["rejected","#f85149"], deferred:["deferred","#8b98a8"],
  ready:["ready","#3fb950"], leaving:["leaving","#d29922"], left:["left","#d29922"]
};
const PRESETS = [
  ["Which device?", "Which device should take the next AI job?"],
  ["Phone battery low", "The phone's battery is low. Run AI jobs on the laptop for
the next 10 minutes."],
  ["Laptop in a call", "The laptop is needed for a video call. Keep AI jobs off the
laptop unless nothing else is safe."],
  ["Heavy job", "A heavy inference job is next. Pick the device with the most thermal
headroom."]
];

const S = { me:null, label:"", health:null, peers:[], names:{}, pos:{}, since:0,
first:true, events:[], mode:"ai", busy:false, relayAt:0 };

function nameOf(id) {
  if (!id) return "?";
  if (S.names[id]) return S.names[id];
  return short(id);
}
function tempColor(t, max) {
  if (t == null || max == null) return "#5d6b7c";
  const head = max - t;
  return head <= 0 ? "#f85149" : head < 12 ? "#d29922" : "#3fb950";
}
function ago(sec) {
  if (sec == null) return "no data";
  return sec < 2 ? "just now" : sec < 60 ? sec + " s ago" : Math.round(sec / 60) + "
min ago";
}
function clock(ms) {
  const d = new Date(ms);
  return d.toTimeString().slice(0, 8);
}
async function getJSON(path) {
  const r = await fetch(path, { cache: "no-store" });
  if (!r.ok) throw new Error(path + " " + r.status);
  return r.json();
}
async function postJSON(path, body) {
  const r = await fetch(path, { method: "POST", headers: { "Content-Type":
"application/json" }, body: JSON.stringify(body) });
  const text = await r.text();
```

```
  let data = null;
  try { data = JSON.parse(text); } catch (e) { /* plain-text error */ }
  if (!r.ok) throw new Error(data && data.error ? data.error : (text || ("HTTP " +
r.status)));
  return data;
}

/* ---------------- devices ---------------- */
function deviceList() {
  const h = S.health;
  if (!h) return [];
  const list = [{ id: h.node, label: h.label, me: true, t: h.thermal_c, max:
h.max_temp, busy: h.busy, tier: h.tier, age: 0, addr: "this device", engine: h.engine
}];
  for (const p of S.peers) {
    list.push({ id: p.id, label: p.label, me: false, t: p.thermal_c, max: p.max_temp,
busy: p.busy, tier: p.tier, age: p.vitals_age_s, addr: p.addr });
  }
  return list;
}
function renderDevices() {
  const pl = S.health && S.health.placement;
  const html = deviceList().map((d) => {
    const isAi = pl && pl.target === d.id;
    const isRules = pl ? pl.rules_now === d.id : false;
    const vetoed = isAi && pl.veto_now;
    const pct = d.t != null && d.max ? Math.max(3, Math.min(100, (d.t / d.max) *
100)) : 0;
    const head = d.t != null && d.max != null ? (d.max - d.t) : null;
    const chips = [
      d.me ? '<span class="chip me">this device</span>' : "",
      isAi ? '<span class="chip ai">Llama's pick</span>' : "",
      isRules && !isAi ? '<span class="chip rules">rules' pick</span>' : "",
      vetoed ? '<span class="chip veto">vetoed</span>' : "",
      d.busy ? '<span class="chip busy">busy</span>' : ""
    ].join("");
    return '<article class="dev' + (isAi && !vetoed ? " ai" : "") + '">' +
      '<div class="name">' + esc(d.label || short(d.id)) + '</div>' +
      '<div class="id">' + esc(short(d.id)) + '</div>' +
      '<div class="chips">' + chips + '</div>' +
      '<div class="temp"><b style="color:' + tempColor(d.t, d.max) + '">' + (d.t !=
null ? d.t.toFixed(1) : "–") + '</b><span>°C' + (d.max != null ? " / limit " +
d.max.toFixed(0) : "") + '</span></div>' +
      '<div class="bar"><i style="width:' + pct.toFixed(1) + '%;background:' +
tempColor(d.t, d.max) + '"></i></div>' +
      '<dl class="facts">' +
        '<dt>headroom</dt><dd>' + (head != null ? head.toFixed(1) + " °C" : "–") +
'</dd>' +
        '<dt>model</dt><dd>' + esc(d.tier || "–") + (d.engine ? " · " +
esc(runnerText(d.engine)) : "") + '</dd>' +
```

```
        '<dt>state</dt><dd>' + (d.busy == null ? "–" : d.busy ? "generating" :
"idle") + '</dd>' +
        '<dt>vitals</dt><dd>' + (d.me ? "local sensor" : esc(ago(d.age)) + " ·
piggybacked") + '</dd>' +
        '<dt>address</dt><dd>' + esc(d.addr) + '</dd>' +
      '</dl></article>';
  }).join("");
  $("devices").innerHTML = html || '<div class="empty">No devices yet.</div>';
}

/* how this device runs its model (v19) */
function runnerText(e) {
  if (e.runner !== "llama-server") return "loaded per job";
  if (e.state === "ready") return "kept loaded" + (e.load_ms ? " (loaded in " +
(e.load_ms / 1000).toFixed(1) + " s)" : "");
  if (e.state === "loading") return "loading…";
  if (e.state === "unloaded") return "unloaded while idle";
  return "loaded per job";
}

/* ---------------- map ---------------- */
function layout(devs, narrow) {
  const pos = {};
  if (!devs.length) return pos;
  const me = devs[0];
  const peers = devs.slice(1);
  const n = peers.length;
  if (narrow) {
    // phone screens: this device on top, peers in a row below, so labels stay
readable
    pos[me.id] = { x: 170, y: n ? 95 : 190 };
    peers.forEach((p, i) => { pos[p.id] = { x: n === 1 ? 170 : 55 + (230 * i) / (n -
1), y: 300 }; });
    return pos;
  }
  if (!n) { pos[me.id] = { x: 300, y: 125 }; return pos; }
  pos[me.id] = { x: 130, y: 125 };
  peers.forEach((p, i) => {
    const y = n === 1 ? 125 : 45 + (160 * i) / (n - 1);
    pos[p.id] = { x: 470, y: y };
  });
  return pos;
}
function renderMap() {
  const devs = deviceList();
  const narrow = $("map").clientWidth < 520;
  $("map").setAttribute("viewBox", narrow ? "0 0 340 400" : "0 0 600 270");
  S.pos = layout(devs, narrow);
  const pl = S.health && S.health.placement;
  let g = "";
```

```
  const me = devs[0];
  for (const p of devs.slice(1)) {
    const a = S.pos[me.id], b = S.pos[p.id];
    g += '<line x1="' + a.x + '" y1="' + a.y + '" x2="' + b.x + '" y2="' + b.y + '"
stroke="#243041" stroke-width="2" stroke-dasharray="' + (p.age == null ? "4 6" :
"none") + '"/>';
  }
  for (const d of devs) {
    const p = S.pos[d.id];
    const col = tempColor(d.t, d.max);
    const isAi = pl && pl.target === d.id;
    const isRules = pl && pl.rules_now === d.id;
    const r = d.me ? 40 : 34;
    const frac = d.t != null && d.max ? Math.max(0, Math.min(1, d.t / d.max)) : 0;
    const circ = 2 * Math.PI * (r + 6);
    if (isAi) g += '<circle cx="' + p.x + '" cy="' + p.y + '" r="' + (r + 26) + '"
fill="url(#glowAi)"/>';
    if (isRules && !isAi) g += '<circle cx="' + p.x + '" cy="' + p.y + '" r="' + (r +
12) + '" fill="none" stroke="#2dd4bf" stroke-width="1.5" stroke-dasharray="3 5"/>';
    g += '<circle cx="' + p.x + '" cy="' + p.y + '" r="' + (r + 6) + '" fill="none"
stroke="#18202b" stroke-width="5"/>';
    g += '<circle cx="' + p.x + '" cy="' + p.y + '" r="' + (r + 6) + '" fill="none"
stroke="' + col + '" stroke-width="5" stroke-linecap="round" stroke-dasharray="' +
(circ * frac).toFixed(1) + ' ' + circ.toFixed(1) + '" transform="rotate(-90 ' + p.x +
' ' + p.y + ')"/>';
    g += '<circle cx="' + p.x + '" cy="' + p.y + '" r="' + r + '" fill="#121821"
stroke="' + (isAi ? "#a371f7" : "#2b3a4d") + '" stroke-width="' + (isAi ? 2.5 : 1.5)
+ '"/>';
    g += '<text x="' + p.x + '" y="' + (p.y + 6) + '" text-anchor="middle" fill="' +
col + '" font-size="17" font-weight="700" font-family="ui-
monospace,Menlo,monospace">' + (d.t != null ? d.t.toFixed(0) + "°" : "?") +
'</text>';
    g += '<text x="' + p.x + '" y="' + (p.y + r + 26) + '" text-anchor="middle"
fill="#e6edf3" font-size="14" font-weight="600">' + esc(d.label || short(d.id)) +
(d.me ? " (here)" : "") + '</text>';
    if (isAi) g += '<text x="' + p.x + '" y="' + (p.y - r - 16) + '" text-
anchor="middle" fill="#a371f7" font-size="12" font-weight="600">' + (pl.veto_now ?
"Llama’s pick · vetoed" : "Llama’s pick") + '</text>';
    else if (isRules) g += '<text x="' + p.x + '" y="' + (p.y - r - 16) + '" text-
anchor="middle" fill="#2dd4bf" font-size="12">rules’ pick</text>';
  }
  $("mapLayer").innerHTML = g;
}
let pulses = 0;
function pulse(from, to, color) {
  const a = S.pos[from], b = S.pos[to] || S.pos[from];
  if (!a || pulses > 8) return;
  const c = $("pulseTpl").cloneNode(false);
  c.removeAttribute("id");
  c.setAttribute("fill", color);
```

```
  $("pulseLayer").appendChild(c);
  pulses++;
  const ring = from === to || !S.pos[to];
  const t0 = performance.now(), dur = ring ? 900 : 1100;
  function step(now) {
    const k = Math.min(1, (now - t0) / dur);
    if (ring) {
      c.setAttribute("cx", a.x); c.setAttribute("cy", a.y);
      c.setAttribute("r", 40 + 30 * k); c.setAttribute("fill", "none");
      c.setAttribute("stroke", color); c.setAttribute("stroke-width", 3 * (1 - k) +
0.5); c.setAttribute("opacity", 1 - k);
    } else {
      const e = k < .5 ? 2 * k * k : 1 - Math.pow(-2 * k + 2, 2) / 2;
      c.setAttribute("cx", a.x + (b.x - a.x) * e); c.setAttribute("cy", a.y + (b.y -
a.y) * e);
      c.setAttribute("opacity", k < .1 ? k * 10 : k > .9 ? (1 - k) * 10 : 1);
    }
    if (k < 1) requestAnimationFrame(step); else { c.remove(); pulses--; }
  }
  requestAnimationFrame(step);
}

/* ---------------- placement ---------------- */
function fmtTTL(s) {
  const m = Math.floor(s / 60), r = s % 60;
  return m + ":" + String(r).padStart(2, "0");
}
function renderPlacement() {
  const h = S.health;
  if (!h) return;
  const pl = h.placement;
  if (!pl) {
    $("ttl").textContent = "";
    $("placeBody").className = "";
    $("placeBody").innerHTML =
      '<div class="verdict none">No AI decision in force. Jobs follow the thermal
rules: run here while below the limit, otherwise on the coolest idle peer.</div>' +
      '<p class="meta">Send a directive: it is signed here, rides on gossip to a
peer, the peer’s Llama picks a device, and that signed decision rides back. Jobs then
go to Llama’s pick unless the rules veto it.</p>';
    return;
  }
  $("ttl").textContent = "expires in " + fmtTTL(pl.expires_in_s);
  const rulesNow = pl.rules_now ? nameOf(pl.rules_now) : "no device";
  const agree = pl.rules_now === pl.target;
  const v = pl.veto_now
    ? '<div class="verdict veto">Rules veto: ' + esc(pl.target_name) + " is " +
esc(pl.veto_now) + ". Jobs go to " + esc(rulesNow) + " until it is safe again.</div>"
    : '<div class="verdict ok">In force: single-answer jobs go to ' +
esc(pl.target_name) + (agree ? " (the rules agree)." : ", overriding the rules’ pick
```

```
(" + esc(rulesNow) + ").") + "</div>";
  $("placeBody").className = "";
  $("placeBody").innerHTML =
    '<div class="pick">Llama’s pick<b>' + esc(pl.target_name) + '</b></div>' +
    (pl.reason ? '<div class="reason">“' + esc(pl.reason) + '”</div>' : "") +
    '<div class="cmp"><div><small>Decided by</small><strong>' + esc(pl.advisor_name)
+ ''s Llama</strong></div><div><small>Rules alone now</small><strong>' +
esc(rulesNow) + '</strong></div></div>' +
    v +
    '<div class="meta">Decision ' + esc(short(pl.decision)) + ' answers directive ' +
esc(short(pl.answers)) + ' · signed ML-DSA-87 by ' + esc(pl.advisor_name) + '</div>';
}

/* ---------------- events ---------------- */
function renderEvents(fresh) {
  const list = S.events.slice(-80).reverse();
  $("evCount").textContent = S.events.length ? S.events.length + " recent" : "";
  if (!list.length) { $("events").innerHTML = '<li class="empty"
style="display:block">Nothing yet. Send a directive or run a job.</li>'; return; }
  $("events").innerHTML = list.map((e) => {
    const k = KIND[e.kind] || [e.kind, "#8b98a8"];
    const route = e.from && e.to && e.from !== e.to ? esc(nameOf(e.from)) + " → " +
esc(nameOf(e.to)) : e.from ? esc(nameOf(e.from)) : "";
    return '<li' + (fresh.has(e.seq) ? ' class="new"' : "") + '><time>' +
clock(e.t_ms) + '</time>' +
      '<span class="kind" style="color:' + k[1] + ';border-color:' + k[1] + '55">' +
esc(k[0]) + '</span>' +
      '<span class="ev-text">' + esc(e.text) + (route ? '<span class="ev-route">' +
route + "</span>" : "") + "</span></li>";
  }).join("");
}

/* ---------------- relay table ---------------- */
function renderRelay(items) {
  if (!items.length) { $("relay").innerHTML = '<tr><td colspan="6" class="empty">None
in flight.</td></tr>'; return; }
  $("relay").innerHTML = items.slice().reverse().map((r) => {
    const decision = (r.advisory || "").indexOf("AI placement decision") === 0;
    return "<tr><td class=\"mono\">" + esc(short(r.id)) + "</td><td>" + (decision ?
'<span style="color:#a371f7">AI decision</span>' : "directive") + "</td><td>" +
esc(nameOf(r.origin)) +
      "</td><td>" + esc(r.hops) + "</td><td>" + esc((r.held_by ||
[]).map(nameOf).join(", ") || "–") + "</td><td>" + esc(decision ?
r.advisory.replace("AI placement decision: ", "") : (r.advisory || "") + (r.body ? "
— " + r.body : "")) + "</td></tr>";
  }).join("");
}

/* ---------------- polling ---------------- */
async function tick() {
```

```
  try {
    const [h, peers, ev] = await Promise.all([getJSON("/health"), getJSON("/peers"),
getJSON("/events?since=" + S.since)]);
    S.health = h; S.peers = peers || []; S.me = h.node;
    S.names = {}; S.names[h.node] = h.label || short(h.node);
    for (const p of S.peers) S.names[p.id] = p.label || short(p.id);
    $("who").textContent = "· " + (h.label || "") + " " + short(h.node);
    $("link").textContent = "link " + (h.link && h.link.state ? h.link.state : "?");
    $("live").className = "dot live";
    $("down").className = "banner";
    $("peerCount").textContent = S.peers.length + (S.peers.length === 1 ? " peer" : "
peers");
    $("phase").textContent = (h.state || "ready") + (h.build ? " · " +
h.build.replace("harmony-", "") : "");
    $("draining").className = h.state === "draining" ? "banner show" : "banner";
    renderDevices(); renderMap(); renderPlacement();
    const fresh = new Set();
    if (ev && ev.events && ev.events.length) {
      for (const e of ev.events) {
        S.events.push(e); fresh.add(e.seq);
        if (!S.first) {
          const k = KIND[e.kind];
          if (e.from) pulse(e.from, e.to || e.from, k ? k[1] : "#58a6ff");
        }
      }
      if (S.events.length > 300) S.events = S.events.slice(-300);
      S.since = ev.next;
    } else if (ev && ev.next < S.since) {
      S.since = ev.next;  // node restarted
    }
    if (fresh.size || S.first) renderEvents(S.first ? new Set() : fresh);
    S.first = false;
    if (Date.now() - S.relayAt > 2500) {
      S.relayAt = Date.now();
      renderRelay(await getJSON("/relay"));
    }
  } catch (e) {
    $("live").className = "dot down";
    $("link").textContent = "unreachable";
    $("down").className = "banner show";
  }
}
let timer = null;
function loop() { clearTimeout(timer); timer = setTimeout(async () => { if
(!document.hidden) await tick(); loop(); }, 1000); }

/* ---------------- controls ---------------- */
PRESETS.forEach(([label, text]) => {
  const b = document.createElement("button");
  b.className = "btn ghost"; b.type = "button"; b.textContent = label;
```

```
  b.addEventListener("click", () => { $("dir").value = text; $("dir").focus(); });
  $("presets").appendChild(b);
});
$("send").addEventListener("click", async () => {
  const body = $("dir").value.trim();
  if (!body) return;
  $("send").disabled = true; $("sendMsg").textContent = "signing…";
  try {
    const r = await postJSON("/relay", { body });
    $("sendMsg").textContent = "Signed directive " + short(r.id) + ". It rides on the
next gossip round; watch the events.";
  } catch (e) {
    $("sendMsg").textContent = "Not sent: " + e.message;
  } finally {
    $("send").disabled = false; tick();
  }
});
function setMode(m) {
  S.mode = m;
  $("modeAi").className = m === "ai" ? "on" : "";
  $("modeRules").className = m === "rules" ? "on" : "";
}
$("modeAi").addEventListener("click", () => setMode("ai"));
$("modeRules").addEventListener("click", () => setMode("rules"));
$("run").addEventListener("click", async () => {
  const prompt = $("prompt").value.trim();
  if (!prompt || S.busy) return;
  S.busy = true; $("run").disabled = true;
  const box = $("result");
  const t0 = Date.now();
  box.className = "result show";
  box.innerHTML = '<span class="spin"></span>Running… <span id="el">0</span> s';
  const iv = setInterval(() => { const el = $("el"); if (el) el.textContent =
Math.round((Date.now() - t0) / 1000); }, 500);
  try {
    const r = await postJSON("/infer", { prompt, fanout: 0, timeout_secs: 180,
placement: S.mode });
    const ok = (r.answers || []).find((a) => a.ok);
    const pl = r.placement || {};
    let path;
    if (pl.mode === "ai" && pl.veto) path = "Rules vetoed Llama’s pick (" +
esc(pl.target_name) + ": " + esc(pl.veto) + ")" + (r.offloaded_to ? "; offloaded to "
+ esc(nameOf(r.offloaded_to)) : "") + ".";
    else if (pl.mode === "ai" && pl.applied) path = "Placed by Llama’s decision on "
+ esc(pl.target_name) + (pl.rules_target && pl.rules_target !== pl.target ? " (rules
alone would have used " + esc(pl.rules_name) + ")" : " (the rules agree)") + ".";
    else if (r.offloaded_to) path = "This device refused for heat; the rules
offloaded it to " + esc(nameOf(r.offloaded_to)) + ".";
    else if (pl.mode === "ai") path = "Llama’s pick could not answer; the rules
placed it.";
```

```
    else path = S.mode === "rules" ? "Rules only: ran where the thermal rules put
it." : "No AI decision in force: the thermal rules placed it.";
    box.innerHTML = ok
      ? '<div class="meta">Answer</div><div class="ans">' + esc(ok.text) + '</div>
<div>Answered by <b>' + esc(nameOf(ok.node)) + "</b> (" + esc(ok.tier) + ") in " +
(ok.elapsed_ms / 1000).toFixed(1) + " s</div><div class=\"meta\" style=\"margin-
top:6px\">" + path + "</div>"
      : '<div class="ans" style="color:#ffa198">No device answered.</div><div
class="meta">' + esc(((r.answers || []).slice(-1)[0] || {}).text || "") + '</div><div
class="meta">' + path + "</div>";
  } catch (e) {
    box.innerHTML = '<div class="ans" style="color:#ffa198">Request failed</div><div
class="meta">' + esc(e.message) + "</div>";
  } finally {
    clearInterval(iv); S.busy = false; $("run").disabled = false; tick();
  }
});
document.addEventListener("visibilitychange", () => { if (!document.hidden) tick();
});
tick().then(loop);
</script>
</body>
</html>
"##;

/// Mesh: handshake endpoint. Body is hello1; a success returns hello2. On success
the server
/// stores a session keyed by the derived session id.
async fn post_hello(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    body: Bytes,
) -> Response {
    if !inbound_ok(who.ip(), st.cfg.allow_public) {
        return StatusCode::FORBIDDEN.into_response();
    }
    if !st.hello_rate_ok(who.ip()) {
        return StatusCode::TOO_MANY_REQUESTS.into_response();
    }
    if body.len() != HELLO1_LEN {
        return StatusCode::BAD_REQUEST.into_response();
    }

    // Replay check before any asymmetric work: it is a hash and a lookup, where
signature
    // verification and decapsulation are not.
    if !st.hello_is_fresh(&body) {
        debug!("🔁 replayed hello1 from {who}");
        return StatusCode::BAD_REQUEST.into_response();
    }
```

```
    // Cap how much public-key work unauthenticated callers can have in flight at
once.
    let _handshake_permit = match st.handshake_gate.try_acquire() {
        Ok(p) => p,
        Err(_) => {
            debug!("handshake capacity reached; shedding hello from {who}");
            return StatusCode::SERVICE_UNAVAILABLE.into_response();
        }
    };

    let now = now_unix();

    // One response for every rejection reason. Distinguishing "stale" from "bad
signature"
    // told an attacker whether their clock guess landed, which is free information.
In v3 the
    // set of reasons also includes "the identity block would not open", which is
what a
    // scanner probing for Harmony nodes sees — indistinguishable from a bad
signature.
    let verified = match server_verify_hello(&st.me, &body, now) {
        Ok(v) => v,
        Err(e) => {
            debug!("rejected hello from {who}: {e:?}");
            return StatusCode::BAD_REQUEST.into_response();
        }
    };

    // Trust policy before the *ephemeral* KEM step. The static decapsulation has
already run
    // — it has to, since the identity is encrypted under it — so this no longer
saves the
    // first KEM operation, only the second. The replay cache and the handshake
semaphore above
    // are what bound that cost.
    if !st.admit_peer(&verified.init_fpr) {
        warn!(
            "🚫 rejected hello from untrusted {} ({})",
            short_id(&verified.init_fpr),
            who
        );
        return StatusCode::FORBIDDEN.into_response();
    }

    let hs = match server_finish_hello(&body, &verified) {
        Some(hs) => hs,
        None => return StatusCode::BAD_REQUEST.into_response(),
    };
```

```
    let peer_addr = SocketAddr::new(who.ip(), verified.port);
    st.store_server_session(ServerSession {
        peer_id: verified.init_fpr,
        peer_ip: who.ip(),
        peer_port: verified.port,
        k_recv: hs.keys.k_i2r.clone(),
        k_send: hs.keys.k_r2i.clone(),
        recv_win: ReplayWindow::default(),
        send_ctr: 0,
        sid: hs.keys.session_id,
        created: Instant::now(),
        last_used: Instant::now(),
    });
    st.remember_peer(verified.init_fpr, peer_addr);
    info!(
        "🤝 session up with {} via {}",
        short_id(&verified.init_fpr),
        who
    );
    bytes_response(hs.hello2)
}

/// Mesh: an encrypted application frame on an established session. Returns an
encrypted reply
/// frame (or 401 if the session is unknown / the frame fails to authenticate).
async fn post_msg(
    State(st): State<Shared>,
    ConnectInfo(who): ConnectInfo<SocketAddr>,
    body: Bytes,
) -> Response {
    if !st.msg_rate_ok(who.ip()) {
        return StatusCode::TOO_MANY_REQUESTS.into_response();
    }
    let (msg, peer_id, sid, peer_ip, peer_port) = match st.open_server_frame(&body) {
        Some(v) => v,
        None => return StatusCode::UNAUTHORIZED.into_response(),
    };
    // A session established before revocation must stop working the moment we
revoke.
    if st.is_revoked(&peer_id) {
        warn!("⛔ dropping frame from revoked peer {}", short_id(&peer_id));
        lock(&st.server_sessions).remove(&sid);
        return StatusCode::FORBIDDEN.into_response();
    }

    let reply = handle_msg(&st, peer_id, SocketAddr::new(peer_ip, peer_port),
msg).await;

    match reply.and_then(|m| st.seal_server_reply(&sid, &m)) {
        Some(frame) => bytes_response(frame),
```

```
        None => StatusCode::NO_CONTENT.into_response(),
    }
}

/// Dispatch an authenticated application message. The sender is cryptographically
identified
/// by `peer_id`, so authorization decisions here can trust it.
async fn handle_msg(
    st: &Shared,
    peer_id: NodeId,
    peer_addr: SocketAddr,
    msg: Msg,
) -> Option<Msg> {
    match msg {
        Msg::Ping {
            nonce,
            vitals,
            carry,
            acks,
        } => {
            st.note_vitals(&peer_id, vitals);
            st.note_acks(&peer_id, &acks);
            let acked: Vec<String> = carry
                .and_then(|r| accept_relayed(st, peer_id, r))
                .into_iter()
                .collect();
            Some(Msg::Pong {
                nonce,
                vitals: Some(st.vitals()),
                acks: acked,
            })
        }
        Msg::Pong { .. } => None,
        Msg::GetPeers => {
            let mut entries: Vec<PeerEntry> = lock(&st.peers)
                .iter()
                .filter(|(id, _)| *id != &peer_id)
                .map(|(id, p)| PeerEntry {
                    id: hex(id),
                    addr: p.addr.to_string(),
                })
                .collect();
            entries.truncate(MAX_PEERS_PER_REPLY);
            Some(Msg::Peers { peers: entries })
        }
        Msg::Peers { peers } => {
            ingest_peer_entries(st, peer_id, &peers);
            None
        }
        Msg::Directive { body } => {
```

```
            // Directives actuate the local node, so they require command
authorization on top
            // of a valid session: our own identity, or an explicit --cmd-allow
entry.
            let allowed = peer_id == st.me.fpr() ||
st.cfg.cmd_allow.contains(&peer_id);
            if !allowed {
                warn!("🚫 directive from non-command peer {}", short_id(&peer_id));
                return Some(Msg::DirectiveAck {
                    accepted: false,
                    detail: "not authorized to issue directives".into(),
                });
            }
            let _ = peer_addr;
            let detail = run_directive(st, &body).await;
            Some(Msg::DirectiveAck {
                accepted: true,
                detail,
            })
        }
        Msg::DirectiveAck { .. } => None,
        Msg::InferRequest {
            id,
            prompt,
            deadline_ms,
            chat,
            max_tokens,
            temperature,
            top_p,
        } => {
            let started = Instant::now();
            let me = hex(&st.me.fpr());
            let tier = st.cfg.tier.label().to_string();

            // Reaching this point already means: authenticated handshake, admitted
by the
            // trust store, not revoked. Policy decides whether that is enough.
            let allowed = peer_id == st.me.fpr()
                || match st.cfg.infer_policy {
                    InferPolicy::Mesh => true,
                    InferPolicy::Allowlist => st.cfg.infer_allow.contains(&peer_id),
                    InferPolicy::None => false,
                };
            if !allowed {
                warn!(
                    "🚫 inference refused for {} (policy {:?})",
                    short_id(&peer_id),
                    st.cfg.infer_policy
                );
                return Some(Msg::InferReply {
```

```
                    id,
                    ok: false,
                    text: "this node does not serve inference to you".into(),
                    tier,
                    node: me,
                    elapsed_ms: 0,
                });
            }

            // The caller's deadline only ever shortens ours; it can never extend it.
            let budget = Duration::from_millis(deadline_ms)
                .min(MAX_REMOTE_INFER)
                .min(st.cfg.llm_timeout);

            // A conversation (v19) runs as one; `prompt` is then only its last user
message.
            let run = match chat {
                Some(turns) => {
                    let job = ChatJob { turns, max_tokens, temperature, top_p };
                    infer_chat_locally(st, &job, budget,
Duration::ZERO).await.map(|a| a.text)
                }
                None => infer_locally(st, &prompt, budget).await,
            };
            let (ok, text) = match run {
                Ok(out) => (true, out),
                Err(e) => (false, e),
            };
            let mut reply = Msg::InferReply {
                id,
                ok,
                text,
                tier,
                node: me,
                elapsed_ms: started.elapsed().as_millis() as u64,
            };
            fit_reply(&mut reply);
            Some(reply)
        }
        Msg::Bye => {
            let name = st.name_of(&peer_id);
            st.forget_peer(&peer_id);
            info!("👋 {name} ({}) is shutting down; dropped from the peer table",
short_id(&peer_id));
            st.event("left", &format!("{name} shut down gracefully and left the
mesh"), Some(&peer_id), Some(&st.me.fpr()));
            None
        }
        Msg::InferReply { .. } => None,
    }
```

```
}

/// What a local model run is for.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
enum LlmRun {
    /// An answer for a client or a peer: the operator's sampling settings, the
client prompt cap.
    Job,
    /// A placement advisory: greedy decoding, so the same mesh state and note always
give the
    /// same pick, and the larger cap of a prompt this node built itself.
    Advisory,
}

impl LlmRun {
    fn prompt_cap(self) -> usize {
        match self {
            LlmRun::Job => MAX_PROMPT_BYTES,
            LlmRun::Advisory => MAX_ADVISORY_PROMPT_BYTES,
        }
    }

    fn greedy(self) -> bool {
        self == LlmRun::Advisory
    }
}

/// Run the local model under every guard that protects this device: prompt cap,
thermal gate,
/// cooldown, and a single-inference-at-a-time semaphore. Shared by local and remote
callers so
/// a remote peer can never reach the model on an easier path than a local one.
async fn infer_locally(st: &Shared, prompt: &str, budget: Duration) -> Result<String,
String> {
    infer_locally_with(st, prompt, budget, None, LlmRun::Job).await
}

/// A constrained run that fails this fast failed on the grammar itself (an older
llama.cpp, or
/// one built without grammar support), not on generation, so it is retried
unconstrained.
const GRAMMAR_FALLBACK_WINDOW: Duration = Duration::from_secs(30);

/// Did llama.cpp refuse the grammar itself (an old build without `--grammar`, or one
whose
/// parser rejects it), as opposed to failing for an unrelated reason such as memory?
fn grammar_rejected(err: &str) -> bool {
    let e = err.to_lowercase();
    !e.contains("sigkill")
        && (e.contains("grammar") || e.contains("rejected a flag") ||
```

```
e.contains("sampler"))
}

/// `infer_locally`, optionally constraining the output with a GBNF grammar.
async fn infer_locally_with(
    st: &Shared,
    prompt: &str,
    budget: Duration,
    grammar: Option<&str>,
    kind: LlmRun,
) -> Result<String, String> {
    if st.draining() {
        return Err("shutting down".into());
    }
    let cap = kind.prompt_cap();
    if prompt.len() > cap {
        return Err(format!("prompt exceeds {cap} bytes"));
    }
    let temp = st.thermal.temperature();
    if temp >= st.cfg.max_temp {
        return Err(format!(
            "thermal {temp:.1}C at or above limit {:.1}C",
            st.cfg.max_temp
        ));
    }
    let _permit = llm_slot(st, Duration::ZERO).await?;

    let started = Instant::now();
    let out = match run_llm_with(st, prompt, budget, grammar, kind.greedy()).await {
        Err(e)
            if grammar.is_some()
                && started.elapsed() < GRAMMAR_FALLBACK_WINDOW
                && grammar_rejected(&e) =>
        {
            warn!("🧠 constrained run failed fast ({e}); retrying without the
grammar");
            run_llm_with(
                st,
                prompt,
                budget.saturating_sub(started.elapsed()),
                None,
                kind.greedy(),
            )
            .await
        }
        r => r,
    };
    out.map(|out| truncate(&sanitize(out.trim()), MAX_LLM_OUTPUT_CHARS))
}
```

```
/// The cooldown and the one-run-at-a-time slot, in front of every model run. With
`wait` zero
/// (/infer, peers, advisories) a busy or just-used model refuses at once, which is
what lets the
/// caller offload; the operator's own chat request waits up to `wait` for it
instead.
async fn llm_slot(st: &Shared, wait: Duration) ->
Result<tokio::sync::SemaphorePermit<'_>, String> {
    let deadline = Instant::now() + wait;
    loop {
        let refused = {
            let mut last = lock(&st.llm_last);
            if last.elapsed() < LLM_COOLDOWN {
                "rate limited"
            } else {
                *last = Instant::now();
                match st.llm_gate.try_acquire() {
                    Ok(permit) => return Ok(permit),
                    Err(_) => "inference already in progress",
                }
            }
        };
        if Instant::now() >= deadline || st.draining() {
            return Err(refused.to_string());
        }
        tokio::time::sleep(Duration::from_millis(250)).await;
    }
}

/// A conversation to answer (v19), from /v1/chat/completions or a peer.
#[derive(Clone, Debug)]
struct ChatJob {
    turns: Vec<ChatTurn>,
    max_tokens: Option<u32>,
    temperature: Option<f32>,
    top_p: Option<f32>,
}

/// A conversation's answer, with token counts when the kept-loaded model reported
them.
#[derive(Clone, Debug)]
struct ChatAnswer {
    text: String,
    prompt_tokens: Option<u64>,
    completion_tokens: Option<u64>,
    /// OpenAI's finish_reason: "stop", or "length" when the answer hit its token
budget.
    finish: &'static str,
}
```

```
/// What a turn costs against CHAT_MAX_PROMPT_BYTES: its text and its share of the
template, so a
/// flood of empty turns cannot grow the prompt past the cap either.
fn turn_cost(t: &ChatTurn) -> usize {
    t.content.len() + 16
}

fn chat_bytes(turns: &[ChatTurn]) -> usize {
    turns.iter().map(turn_cost).sum()
}

/// Fit a conversation into `cap`: drop the oldest turns first, never a system turn
or the last
/// turn, then any assistant turn left at the front, so it still opens with the user.
Returns the
/// turns and how many were dropped; Err when the turns that must stay are over the
cap alone.
fn fit_conversation(turns: Vec<ChatTurn>, cap: usize) -> Result<(Vec<ChatTurn>,
usize), String> {
    let n = turns.len();
    let mut keep = vec![true; n];
    let mut size = chat_bytes(&turns);
    let mut dropped = 0;
    let mut i = 0;
    while size > cap && i + 1 < n {
        if turns[i].role != "system" {
            keep[i] = false;
            size -= turn_cost(&turns[i]);
            dropped += 1;
        }
        i += 1;
    }
    if dropped > 0 {
        for j in 0..n.saturating_sub(1) {
            if !keep[j] || turns[j].role == "system" {
                continue;
            }
            if turns[j].role != "assistant" {
                break;
            }
            keep[j] = false;
            size -= turn_cost(&turns[j]);
            dropped += 1;
        }
    }
    if size > cap {
        return Err(format!(
            "the last message and the system prompt exceed {cap} bytes on their own"
        ));
    }
```

```
    let kept = turns.into_iter().zip(keep).filter_map(|(t, k)|
k.then_some(t)).collect();
    Ok((kept, dropped))
}

/// `s` cut to at most `n` bytes, on a character boundary.
fn truncate_bytes(s: &str, n: usize) -> String {
    if s.len() <= n {
        return s.to_string();
    }
    let mut end = n;
    while !s.is_char_boundary(end) {
        end -= 1;
    }
    s[..end].to_string()
}

/// A chat request's answer budget in tokens.
fn chat_tokens(asked: Option<u32>) -> u32 {
    asked.unwrap_or(CHAT_DEFAULT_TOKENS).clamp(1, CHAT_MAX_TOKENS)
}

/// The roles a conversation may hold here: the ones every template knows.
fn chat_role_ok(role: &str) -> bool {
    matches!(role, "system" | "user" | "assistant")
}

/// The request itself is at fault (too long, malformed), so another device would
refuse it too.
fn request_fault(reason: &str) -> bool {
    let r = reason.to_lowercase();
    r.contains("exceed") || r.contains("no user message") || r.contains("role other
than")
}

/// Answer a conversation on this device (v19): /v1/chat/completions, or a peer's
chat request.
/// The same guards as `infer_locally` (draining, size, thermal gate, cooldown, one
run at a time)
/// with the chat caps; `wait` lets the operator's own request queue briefly for a
busy model.
async fn infer_chat_locally(
    st: &Shared,
    job: &ChatJob,
    budget: Duration,
    wait: Duration,
) -> Result<ChatAnswer, String> {
    if st.draining() {
        return Err("shutting down".into());
    }
```

```
    if job.turns.iter().any(|t| !chat_role_ok(&t.role)) {
        return Err("conversation has a role other than system, user or
assistant".into());
    }
    if !job.turns.iter().any(|t| t.role == "user") {
        return Err("conversation has no user message".into());
    }
    if chat_bytes(&job.turns) > CHAT_MAX_PROMPT_BYTES {
        return Err(format!("conversation exceeds {CHAT_MAX_PROMPT_BYTES} bytes"));
    }
    let hot = |st: &Shared| {
        let temp = st.thermal.temperature();
        (temp >= st.cfg.max_temp)
            .then(|| format!("thermal {temp:.1}C at or above limit {:.1}C",
st.cfg.max_temp))
    };
    if let Some(why) = hot(st) {
        return Err(why);
    }
    let started = Instant::now();
    let _permit = llm_slot(st, wait).await?;
    // It may have warmed up while this request waited for the model.
    if let Some(why) = hot(st) {
        return Err(why);
    }
    let call = ModelCall {
        formatted: st.cfg.prompt_format.wrap_chat(&job.turns),
        grammar: None,
        greedy: false,
        n_predict: Some(chat_tokens(job.max_tokens)),
        temperature: job.temperature,
        top_p: job.top_p,
    };
    let a = run_model(st, &call, budget.saturating_sub(started.elapsed())).await?;
    Ok(ChatAnswer {
        text: truncate(&sanitize(a.text.trim()), CHAT_MAX_OUTPUT_CHARS),
        prompt_tokens: a.prompt_tokens,
        completion_tokens: a.completion_tokens,
        finish: if a.hit_limit { "length" } else { "stop" },
    })
}

/// Does this message, serialized and padded, fit the largest frame, which is all a
peer opens?
fn fits_one_frame(msg: &Msg) -> bool {
    serde_json::to_vec(msg).is_ok_and(|v| PAD_PREFIX + v.len() <=
PAD_BUCKETS[PAD_BUCKETS.len() - 1])
}

/// Shorten an inference reply until it fits one frame. An /infer answer always does;
```

```
a long chat
/// answer full of multi-byte characters or quotes might not.
fn fit_reply(reply: &mut Msg) {
    while !fits_one_frame(reply) {
        let Msg::InferReply { text, .. } = reply else { return };
        if text.is_empty() {
            return;
        }
        let keep = text.chars().count() * 3 / 4;
        *text = text.chars().take(keep).collect();
    }
}

/// A conversation as a mesh request. `prompt` carries its last user message (cut to
the old cap)
/// for nodes that predate conversations, and the conversation is shortened further
when the request
/// would not fit one frame. None when even its last user message does not.
fn chat_request(job: &ChatJob, budget: Duration) -> Option<Msg> {
    let last_user = job
        .turns
        .iter()
        .rev()
        .find(|t| t.role == "user")
        .map(|t| truncate_bytes(&t.content, MAX_PROMPT_BYTES))
        .unwrap_or_default();
    let mut cap = CHAT_MAX_PROMPT_BYTES;
    loop {
        let (turns, _) = fit_conversation(job.turns.clone(), cap).ok()?;
        let msg = Msg::InferRequest {
            id: OsRng.next_u64(),
            prompt: last_user.clone(),
            deadline_ms: budget.as_millis() as u64,
            chat: Some(turns),
            max_tokens: job.max_tokens,
            temperature: job.temperature,
            top_p: job.top_p,
        };
        if fits_one_frame(&msg) {
            return Some(msg);
        }
        if cap <= 1024 {
            return None;
        }
        cap = cap * 3 / 4;
    }
}

/// Merge peer entries learned over an authenticated session, applying address policy
and caps.
```

```
/// Merge peers learned from `source`, which vouched for them but which we have not
yet
/// spoken to ourselves. Attribution is what makes the per-introducer cap
enforceable.
fn ingest_peer_entries(st: &Shared, source: NodeId, entries: &[PeerEntry]) {
    for e in entries.iter().take(MAX_PEERS_PER_REPLY) {
        let Some(id) = unhex32(&e.id) else { continue };
        let Some(addr) = parse_peer(&e.addr, st.cfg.port) else {
            continue;
        };
        // revoked, or outside a strict trust list: not worth a dial
        if !st.trust_allows(&id) {
            continue;
        }
        st.remember_peer_from(id, addr, Some(source));
    }
}

/// Run a routing directive through the local LLM as an advisory.
///
/// The node finds the devices the operator's note names and what it says about each;
the model
/// labels each AVOID (the note spares it), USE (the note asks for it) or ANY. The
node turns that
/// reading into a
/// target with the thermal rules' ranking (`reading_choice`); the advisor signs it
as a placement
/// decision (`issue_placement`), and the rules can still veto it at job time.
/// Guard rails: thermal gate, length cap, process timeout, and a concurrency limit
so a burst of
/// directives cannot fork-bomb the device.
async fn run_directive(st: &Shared, body: &str) -> String {
    advise(st, body).await.detail
}

/// An advisory's result: the line recorded for the directive, what the model read in
the note,
/// the device that reading sends jobs to (with its name and the reason), and what
the rules
/// alone would pick.
struct AdvisoryOutcome {
    detail: String,
    reading: Option<NoteReading>,
    pick: Option<(NodeId, String, String)>,
    rules: Option<NodeId>,
}

async fn advise(st: &Shared, body: &str) -> AdvisoryOutcome {
    let nodes = st.advisory_nodes();
    advise_among(st, &nodes, body).await
```

```
}

/// The advisory over a given list of devices (`/advise` returns the list with the
result).
async fn advise_among(st: &Shared, nodes: &[AdvisoryNode], body: &str) ->
AdvisoryOutcome {
    let rules = rules_choice(nodes);
    let evidence = device_evidence(nodes, body);
    let named: Vec<String> = evidence.iter().map(|(n, _)| n.clone()).collect();
    let Some(grammar) = advisory_grammar(&evidence) else {
        // Nothing for the model to read: the rules' pick stands, and the decision
says why.
        let none = NoteReading { avoid: Vec::new(), prefer: Vec::new(), reason:
String::new() };
        let target = reading_choice(nodes, &none);
        let names = note_names(nodes);
        let left = if names.is_empty() {
            "no device here has a name a note could use (see --label); Llama not
asked".to_string()
        } else {
            format!("the note names none of {}; Llama not asked", names.join(", "))
        };
        let right = match target {
            Some(t) => format!("jobs → {}; thermal rules agree", device_tag(nodes,
&t)),
            None => "no device is below its thermal limit".into(),
        };
        info!("🧠 advisory: {left} | {right}");
        let pick = target.and_then(|t| nodes.iter().find(|n| n.id == t)).map(|n|
(n.id, n.name.clone(), decision_reason(&none)));
        return AdvisoryOutcome { detail: format!("advisory: {left} | {right}"),
reading: Some(none), pick, rules };
    };
    let prompt = build_advisory_prompt(body);
    let kind = LlmRun::Advisory;
    match infer_locally_with(st, &prompt, st.cfg.llm_timeout, Some(&grammar),
kind).await {
        Ok(out) => {
            let mut reading = parse_reading(&out);
            // one line per device in the answer; one line in the log and /relay
            let out = out.lines().map(str::trim).filter(|l| !l.is_empty()).collect::
<Vec<_>>().join("; ");
            // Only devices the note names can be spared or asked for. The grammar
already holds
            // the model to them; this also covers a run without the grammar (older
llama.cpp).
            if let Some(r) = reading.as_mut() {
                r.avoid.retain(|n| named.contains(n));
                r.prefer.retain(|n| named.contains(n));
            }
```

```
            let target = reading.as_ref().and_then(|r| reading_choice(nodes, r));
            let verdict = reading_verdict(reading.as_ref(), target, nodes);
            info!("🧠 advisory: {} | {verdict}", truncate(&out, 120));
            let pick = match (&reading, target) {
                (Some(r), Some(id)) => nodes
                    .iter()
                    .find(|n| n.id == id)
                    .map(|n| (n.id, n.name.clone(), decision_reason(r))),
                _ => None,
            };
            AdvisoryOutcome {
                detail: format!("advisory: {} | {verdict}", truncate(&out, 300)),
                reading,
                pick,
                rules,
            }
        }
        Err(e) => {
            warn!("llm unavailable: {e}");
            AdvisoryOutcome { detail: format!("deferred: {e}"), reading: None, pick:
None, rules }
        }
    }
}

/// One device as the advisory sees it.
#[derive(Clone, Debug)]
struct AdvisoryNode {
    id: NodeId,
    vitals: Option<Vitals>,
    me: bool,
    /// The device's own label ("phone", "laptop"...), empty if unknown.
    name: String,
}

/// Devices shown to the model at most (this one plus peers), to keep the prompt
small.
const ADVISORY_MAX_NODES: usize = 6;

impl AppState {
    /// This device first, then known peers in a stable order, with their piggybacked
vitals.
    fn advisory_nodes(&self) -> Vec<AdvisoryNode> {
        let mut nodes = vec![AdvisoryNode {
            id: self.me.fpr(),
            vitals: Some(self.vitals()),
            me: true,
            name: self.cfg.label.clone(),
        }];
        let mut peers: Vec<NodeId> = lock(&self.peers).keys().copied().collect();
```

```
        peers.sort();
        for id in peers.into_iter().take(ADVISORY_MAX_NODES - 1) {
            let vitals = self.fresh_vitals(&id).map(|(v, _)| v);
            let name = vitals.as_ref().map(|v| v.label.clone()).unwrap_or_default();
            nodes.push(AdvisoryNode {
                id,
                vitals,
                me: false,
                name,
            });
        }
        nodes
    }
}

/// A device's name as a note and the advisory grammar use it: its label, trimmed,
lowercased,
/// spaces as '-', if that is then one plain word (ASCII letters, digits, '-' or '_',
at most 24).
/// Only such names go into the grammar, where they are literal strings; a device
without one is
/// left out of the reading, and the rules still rank it.
fn note_name(label: &str) -> Option<String> {
    let n = label.split_whitespace().collect::<Vec<_>>().join("-
").to_ascii_lowercase();
    let plain = (1..=24).contains(&n.len())
        && n.starts_with(|c: char| c.is_ascii_alphanumeric())
        && n.chars().all(|c| c.is_ascii_alphanumeric() || c == '-' || c == '_');
    plain.then_some(n)
}

/// The names a note can use, each once, in the order the devices are listed (this
one first).
fn note_names(nodes: &[AdvisoryNode]) -> Vec<String> {
    let mut names: Vec<String> = Vec::new();
    for n in nodes {
        if let Some(name) = note_name(&n.name) {
            if !names.contains(&name) {
                names.push(name);
            }
        }
    }
    names
}

/// The devices a note names, in the order it first names them: a device's name as a
whole word,
/// any case, also plural or possessive ("phones", "phone's"), with '-' or '_' read
as a space
/// ("living room" for living-room). Found by the node, so the model is asked only
```

```
about devices
/// the note really names and cannot attach the note to one it never mentions.
fn mentioned_names(nodes: &[AdvisoryNode], note: &str) -> Vec<String> {
    let text = note.to_ascii_lowercase();
    let mut found: Vec<(usize, String)> = Vec::new();
    for name in note_names(nodes) {
        let spoken = name.replace(['-', '_'], " ");
        let at = [name.as_str(), spoken.as_str()]
            .iter()
            .filter_map(|w| first_word_at(&text, w))
            .min();
        if let Some(i) = at {
            found.push((i, name));
        }
    }
    found.sort();
    found.into_iter().map(|(_, n)| n).collect()
}

/// Byte offset of the first whole-word `word` in `text` (both lowercase, `word`
ASCII), allowing
/// a plural "s" after it.
fn first_word_at(text: &str, word: &str) -> Option<usize> {
    if word.is_empty() {
        return None;
    }
    let bytes = text.as_bytes();
    let mut from = 0;
    while let Some(k) = text[from..].find(word) {
        let i = from + k;
        let rest = &text[i + word.len()..];
        let ends = |r: &str| r.bytes().next().map_or(true, |b|
!b.is_ascii_alphanumeric());
        let starts = i == 0 || !bytes[i - 1].is_ascii_alphanumeric();
        if starts && (ends(rest) || (rest.starts_with('s') && ends(&rest[1..]))) {
            return Some(i);
        }
        // `word` is ASCII, so the byte after a match start is a character boundary
        from = i + 1;
    }
    None
}

/// The note cut into clauses, in order: at sentence and clause punctuation, and at
"and", "but",
/// "so", "while" and "because" when both sides are clauses of their own (three words
or more
/// before, two or more after), so "the phone and the notebook are busy" stays whole.
fn note_clauses(note: &str) -> Vec<String> {
    const JOINS: [&str; 5] = [" and ", " but ", " so ", " while ", " because "];
```

```
    let mut out = Vec::new();
    for seg in note.split(|c: char| matches!(c, '.' | '!' | '?' | ';' | ':' | ',' |
'\n')) {
        let mut rest = seg.trim();
        loop {
            let low = rest.to_ascii_lowercase();
            // ASCII lowercasing keeps byte offsets, so a cut found in `low` is one
in `rest`
            let cut = JOINS
                .iter()
                .flat_map(|w| low.match_indices(w).map(|(i, m)| (i,
m.len())).collect::<Vec<_>>())
                .filter(|&(i, n)| {
                    rest[..i].split_whitespace().count() >= 3 && rest[i +
n..].split_whitespace().count() >= 2
                })
                .min();
            match cut {
                Some((i, n)) => {
                    out.push(rest[..i].trim().to_string());
                    rest = rest[i + n..].trim();
                }
                None => {
                    out.push(rest.to_string());
                    break;
                }
            }
        }
    }
    out.retain(|c| !c.is_empty());
    out
}

/// What the evidence for one device may hold at most, so six of them keep the
grammar small.
const EVIDENCE_MAX: usize = 160;

/// What the note says about each device it names, in the order it names them: the
clauses that
/// name the device, with the clauses after them that name no device ("Leave the
laptop alone, I'm
/// compiling on it"). Found by the node; the model only judges it.
fn device_evidence(nodes: &[AdvisoryNode], note: &str) -> Vec<(String, String)> {
    let named = mentioned_names(nodes, note);
    let mut said: Vec<Vec<String>> = vec![Vec::new(); named.len()];
    let mut current: Vec<usize> = Vec::new();
    for clause in note_clauses(note) {
        let low = clause.to_ascii_lowercase();
        let here: Vec<usize> = named
            .iter()
```

```
            .enumerate()
            .filter(|(_, n)| {
                let spoken = n.replace(['-', '_'], " ");
                first_word_at(&low, n).is_some() || first_word_at(&low,
&spoken).is_some()
            })
            .map(|(i, _)| i)
            .collect();
        if !here.is_empty() {
            current = here;
        }
        for &i in &current {
            said[i].push(clause.clone());
        }
    }
    named
        .into_iter()
        .zip(said)
        .map(|(n, s)| {
            let mut e = s.join(", ");
            if e.len() > EVIDENCE_MAX {
                let mut end = EVIDENCE_MAX;
                while !e.is_char_boundary(end) {
                    end -= 1;
                }
                e = format!("{}...", e[..end].trim_end());
            }
            (n, e)
        })
        .collect()
}

/// The advisory prompt: the operator's note and what each label means. The answer's
lines name
/// the devices and quote what the note says about each (the grammar writes them), so
the model
/// adds only the labels. Always within MAX_ADVISORY_PROMPT_BYTES: a note is
shortened to fit.
///
/// No temperatures, letters, example answers or device list to mix up. Llama 3.2 3B,
given
/// temperatures, picked a device with less headroom than another while saying it had
the most
/// (v13); given device letters, it answered with the letters of two other devices
(v14); given an
/// example answer, it copied its labels (v15); given every named device in one line,
it gave one
/// device's label to the next (v16).
fn build_advisory_prompt(directive: &str) -> String {
    let head = "An operator wrote this note about the devices in an AI mesh:\n\"";
```

```
    let tail = "\"\n\nFor each device the note names, decide what the note asks for
that device:\n\
                AVOID = keep AI jobs off it (battery low or flat, in use, in a call,
overheating, \
                about to be unplugged or to leave)\n\
                USE = run AI jobs on it (asked for, free or able to take the jobs,
charging or \
                plugged in)\n\
                ANY = the note asks neither for it\n\
                Each answer line gives a device and what the note says about it; end
the line with \
                AVOID, USE or ANY.";
    let room = MAX_ADVISORY_PROMPT_BYTES.saturating_sub(head.len() + tail.len());
    let mut note = String::new();
    for c in directive.chars() {
        if note.len() + c.len_utf8() > room {
            break;
        }
        note.push(c);
    }
    format!("{head}{note}{tail}")
}

/// `s` as a GBNF string literal: quotes and backslashes escaped, anything outside
printable ASCII
/// as \uXXXX or \UXXXXXXXX, so text from a note can go into the grammar safely.
fn gbnf_literal(s: &str) -> String {
    let mut out = String::from("\"");
    for c in s.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            ' '..='~' => out.push(c),
            c if (c as u32) <= 0xFFFF => out.push_str(&format!("\\u{:04X}", c as
u32)),
            c => out.push_str(&format!("\\U{:08X}", c as u32)),
        }
    }
    out.push('"');
    out
}

/// GBNF for the answer: one line per device the note names, in the order it names
them, all
/// written by the grammar except the label: `<name>: "<what the note says>" =>
<AVOID|USE|ANY>`.
/// llama.cpp enforces it while sampling, and the answer ends after the last label.
`None` when
/// the note names no device.
fn advisory_grammar(evidence: &[(String, String)]) -> Option<String> {
```

```
    if evidence.is_empty() {
        return None;
    }
    let lines: Vec<String> = evidence
        .iter()
        .map(|(n, e)| format!("{} (\"AVOID\" | \"USE\" | \"ANY\")",
gbnf_literal(&format!("{n}: \"{e}\" => ")))
        .collect();
    Some(format!("root ::= {}", lines.join(" \"\\n\" ")))
}

/// What the model read in the operator's note, by device name.
#[derive(Clone, Debug, PartialEq)]
struct NoteReading {
    /// Devices the note asks to keep AI jobs off.
    avoid: Vec<String>,
    /// Devices the note asks to run AI jobs on.
    prefer: Vec<String>,
    /// The model's one-line reason.
    reason: String,
}

/// Record one device's label; true for AVOID or USE.
fn add_label(r: &mut NoteReading, name: String, label: &str) -> bool {
    let list = match label.trim_end_matches([',', ';',
'.']).to_ascii_uppercase().as_str() {
        "AVOID" => &mut r.avoid,
        "USE" => &mut r.prefer,
        _ => return false,
    };
    if !list.contains(&name) {
        list.push(name);
    }
    true
}

/// Parse the answer: lines `phone: "the phone is nearly flat" => AVOID` (any case).
The reason is
/// what the note says about the devices it spares or asks for. Older answers and
runs without the
/// grammar, `laptop=ANY phone=AVOID - reason`, are read too. `None` when neither
form is there.
fn parse_reading(out: &str) -> Option<NoteReading> {
    let mut r = NoteReading { avoid: Vec::new(), prefer: Vec::new(), reason:
String::new() };
    let mut said = Vec::new();
    let mut any = false;
    for line in out.lines().map(str::trim) {
        let Some(at) = line.rfind("=>") else {
            continue;
```

```
        };
        let left = &line[..at];
        let Some(colon) = left.find(':') else {
            continue;
        };
        let Some(name) = note_name(&left[..colon]) else {
            continue;
        };
        any = true;
        let label = line[at + 2..].split_whitespace().next().unwrap_or("");
        if add_label(&mut r, name.clone(), label) {
            said.push(format!("{name}: {}", left[colon +
1..].trim().trim_matches('"')));
        }
    }
    if any {
        r.reason = truncate(&said.join("; "), 200);
        return Some(r);
    }
    let line = out.lines().map(str::trim).find(|l| l.contains('='))?;
    let (pairs, reason) = match line.find(" - ") {
        Some(i) => (&line[..i], line[i + 3..].trim()),
        None => (line, ""),
    };
    r.reason = truncate(reason, 200);
    for word in pairs.split_whitespace() {
        let Some((name, value)) = word.split_once('=') else {
            continue;
        };
        let Some(name) = note_name(name) else {
            continue;
        };
        any = true;
        add_label(&mut r, name, value);
    }
    any.then_some(r)
}

/// Where the note sends jobs: among the usable devices (below their limit, fresh
vitals) that the
/// note does not spare, the rules' pick (idle first, most headroom) of those it asks
to USE, or
/// of all of them when it asks for none. `None` when the note spares every usable
device: the
/// rules then decide alone.
fn reading_choice(nodes: &[AdvisoryNode], r: &NoteReading) -> Option<NodeId> {
    let named = |n: &AdvisoryNode, list: &[String]|
note_name(&n.name).is_some_and(|x| list.contains(&x));
    let open: Vec<AdvisoryNode> = nodes
        .iter()
```

```
        .filter(|n| n.vitals.as_ref().is_some_and(|v| !v.hot()) && !named(n,
&r.avoid))
        .cloned()
        .collect();
    let asked: Vec<AdvisoryNode> = open.iter().filter(|n| named(n,
&r.prefer)).cloned().collect();
    rules_choice(&asked).or_else(|| rules_choice(&open))
}

/// "spare phone; use laptop" / "names no device".
fn describe_reading(r: &NoteReading) -> String {
    let mut parts = Vec::new();
    if !r.avoid.is_empty() {
        parts.push(format!("spare {}", r.avoid.join(", ")));
    }
    if !r.prefer.is_empty() {
        parts.push(format!("use {}", r.prefer.join(", ")));
    }
    if parts.is_empty() {
        "names no device".into()
    } else {
        parts.join("; ")
    }
}

/// The reason a decision carries: the model's, or what it read when it gave none.
fn decision_reason(r: &NoteReading) -> String {
    if r.avoid.is_empty() && r.prefer.is_empty() {
        "the note names no device: most headroom".into()
    } else if r.reason.is_empty() {
        "as the operator's note asks".into()
    } else {
        r.reason.clone()
    }
}

/// "phone (17d9812b3663)", or the short id alone for a device without a usable name.
fn device_tag(nodes: &[AdvisoryNode], id: &NodeId) -> String {
    match nodes.iter().find(|n| &n.id == id).and_then(|n| note_name(&n.name)) {
        Some(n) => format!("{n} ({})", short_id(id)),
        None => short_id(id),
    }
}

/// Compare where the note's reading sends jobs with the rules' choice, for the log
and `/relay`.
fn reading_verdict(r: Option<&NoteReading>, target: Option<NodeId>, nodes: &
[AdvisoryNode]) -> String {
    let name = |id: &NodeId| device_tag(nodes, id);
    let rules = rules_choice(nodes);
```

```
    let Some(r) = r else {
        return match rules {
            Some(x) => format!("no reading of the note parsed; thermal rules prefer
{}", name(&x)),
            None => "no reading of the note parsed; no device is below its thermal
limit".into(),
        };
    };
    let read = describe_reading(r);
    match (target, rules) {
        (Some(t), Some(x)) if t == x => format!("llama reads the note: {read}; jobs
→ {}; thermal rules agree", name(&t)),
        (Some(t), Some(x)) => format!("llama reads the note: {read}; jobs → {};
thermal rules prefer {}", name(&t), name(&x)),
        (Some(t), None) => format!("llama reads the note: {read}; jobs → {}; thermal
rules find no device below its limit", name(&t)),
        (None, _) => format!("llama reads the note: {read}; that spares every usable
device, so the rules decide"),
    }
}

/// What the deterministic rules would choose among the same devices: the coolest
idle device
/// below its limit (`rank_targets`, the ranking offload uses).
fn rules_choice(nodes: &[AdvisoryNode]) -> Option<NodeId> {
    let any = SocketAddr::from(([0, 0, 0, 0], 0));
    let cands = nodes.iter().map(|n| (n.id, any, n.vitals.clone())).collect();
    rank_targets(cands, 1).first().map(|(id, _)| *id)
}

/// Context size passed to llama.cpp unless --llama-flags sets one. Prompts are
capped at
/// MAX_PROMPT_BYTES (at most ~1K tokens) plus the answer budget, so 4K always fits.
/// Leaving it unset is not safe: current llama.cpp then uses the model's full
trained context —
/// 128K tokens for Llama 3.x, about 14 GiB of KV cache for the 3B model and 4 GiB
for 1B —
/// and the kernel's out-of-memory killer ends the run while the model loads.
const LLM_CTX: &str = "4096";
/// Answer budget (tokens) unless --llama-flags sets one.
const LLM_PREDICT: &str = "128";
/// An advisory's answer budget: it quotes what the note says about each device it
names, up to
/// six devices of EVIDENCE_MAX bytes. Appended after the operator's flags, so it
always holds.
const ADVISORY_PREDICT: &str = "320";

/// Defaults for flags the operator did not set, so an explicit `-n 256` or `-c 8192`
in
/// --llama-flags is never followed by a contradicting duplicate.
```

```
fn default_llama_args(flags: &[String]) -> Vec<&'static str> {
    let has = |names: &[&str]| {
        flags
            .iter()
            .any(|f| names.iter().any(|n| f == n || f.starts_with(&format!("{n}="))))
    };
    let mut out = Vec::new();
    if !has(&["-n", "--predict", "--n-predict"]) {
        out.extend(["-n", LLM_PREDICT]);
    }
    if !has(&["-c", "--ctx-size"]) {
        out.extend(["-c", LLM_CTX]);
    }
    out
}

/// Everything after the model and prompt: defaults the operator did not set, the
operator's
/// flags, then what this run needs. llama.cpp lets a later flag override an earlier
one, so an
/// advisory's `--temp 0` (greedy: always the single most likely answer) beats a
sampling
/// temperature the operator set for ordinary jobs, which keep it.
fn llama_run_args(flags: &[String], grammar: Option<&str>, greedy: bool) ->
Vec<String> {
    let mut out: Vec<String> =
default_llama_args(flags).into_iter().map(String::from).collect();
    out.extend(flags.iter().cloned());
    if let Some(g) = grammar {
        out.push("--grammar".into());
        out.push(g.to_string());
    }
    if greedy {
        out.push("-n".into());
        out.push(ADVISORY_PREDICT.into());
        out.push("--temp".into());
        out.push("0".into());
    }
    out
}

/// Why llama.cpp ended unsuccessfully, for the log and `/relay`.
fn llm_exit_reason(status: std::process::ExitStatus, stderr: &[u8], model_path: &str)
-> String {
    #[cfg(unix)]
    {
        use std::os::unix::process::ExitStatusExt;
        if status.signal() == Some(9) {
            return format!(
                "killed by SIGKILL while running, almost always the kernel's out-of-
```

```
memory \
                 killer: close other programs, or use a smaller tier or quant (last
output: {})",
                truncate(&llm_failure_hint(stderr, model_path), 160)
            );
        }
    }
    format!("exited {status}: {}", llm_failure_hint(stderr, model_path))
}

/// Once the answer has started, this long without new output means the model is done
and is
/// waiting for input (llama.cpp's chat mode) rather than still generating. Generous
enough for a
/// thermally throttled phone, which still emits several tokens per second.
const LLM_IDLE_AFTER_ANSWER: Duration = Duration::from_secs(8);
/// Ceiling on captured stdout, so a runaway process cannot grow memory without
bound.
const LLM_STDOUT_CAP: usize = 64 * 1024;

/// Current llama.cpp ships `llama-cli` as an interactive chat that answers and then
waits for the
/// next message; the one-shot completion tool is `llama-completion`. When the binary
was left at
/// the default, prefer the one-shot tool if it is installed. An explicit path is
never changed.
fn prefer_completion_bin(bin: String) -> String {
    if bin == "llama-cli" && which_on_path("llama-completion").is_some() {
        return "llama-completion".into();
    }
    bin
}

/// What llama.cpp prints when it echoes the prompt: the formatted prompt with its
special tokens
/// (`<|...|>`) rendered as nothing.
fn llm_echo(formatted: &str) -> String {
    let mut out = String::with_capacity(formatted.len());
    let mut rest = formatted;
    while let Some(start) = rest.find("<|") {
        match rest[start..].find("|>") {
            Some(end) => {
                out.push_str(&rest[..start]);
                rest = &rest[start + end + 2..];
            }
            None => break,
        }
    }
    out.push_str(rest);
    out
```

```
}

/// Has the model started its answer, as opposed to still echoing the prompt or
loading?
fn answer_started(out: &[u8], echo: &str) -> bool {
    // Windows builds of llama.cpp may end lines with CRLF; the echo is matched on
LF.
    let lossy = String::from_utf8_lossy(out).replace("\r\n", "\n");
    let s: &str = &lossy;
    if !echo.is_empty() {
        if let Some(i) = s.find(echo) {
            return s[i + echo.len()..].chars().any(|c| !c.is_whitespace());
        }
        if echo.starts_with(s.trim_start()) {
            return false; // the echo is still being printed
        }
    }
    s.chars().any(|c| !c.is_whitespace())
}

/// Reduce raw stdout to the answer: drop the echoed prompt, llama.cpp's end-of-text
marker, and
/// the interactive `>` prompt (and "EOF by user") that chat mode prints after
answering.
fn clean_llm_output(raw: &str, echo: &str) -> String {
    let normalized = raw.replace("\r\n", "\n");
    let mut s = normalized.as_str();
    if let Some(i) = s.find("\n> EOF by user") {
        s = &s[..i];
    }
    if !echo.is_empty() {
        if let Some(i) = s.find(echo) {
            if i <= 64 {
                s = &s[i + echo.len()..];
            }
        }
    }
    let mut t = s.trim().to_string();
    if let Some(stripped) = t.strip_suffix("[end of text]") {
        t = stripped.trim_end().to_string();
    }
    loop {
        let trimmed = t.trim_end();
        match trimmed.rfind('\n') {
            Some(pos) if trimmed[pos + 1..].trim() == ">" => t =
trimmed[..pos].to_string(),
            None if trimmed.trim() == ">" => t.clear(),
            _ => break,
        }
    }
```

```
    t.trim().to_string()
}

/// Read whatever is available. Built on the core `AsyncRead` trait so it needs no
extra tokio
/// features.
async fn read_some<R: tokio::io::AsyncRead + Unpin>(r: &mut R, buf: &mut [u8]) ->
std::io::Result<usize> {
    std::future::poll_fn(|cx| {
        let mut rb = tokio::io::ReadBuf::new(&mut *buf);
        match tokio::io::AsyncRead::poll_read(std::pin::Pin::new(&mut *r), cx, &mut
rb) {
            std::task::Poll::Ready(Ok(())) =>
std::task::Poll::Ready(Ok(rb.filled().len())),
            std::task::Poll::Ready(Err(e)) => std::task::Poll::Ready(Err(e)),
            std::task::Poll::Pending => std::task::Poll::Pending,
        }
    })
    .await
}

enum LlmStop {
    /// stdout closed: the process finished on its own.
    Exited,
    /// Answer started, then nothing new for LLM_IDLE_AFTER_ANSWER: waiting for
input.
    Idle,
    Cap,
    Timeout,
    Read(String),
}

/// One model run, whichever runner serves it.
struct ModelCall<'a> {
    /// The prompt, already in the model's template.
    formatted: String,
    grammar: Option<&'a str>,
    /// An advisory: greedy decoding and ADVISORY_PREDICT tokens.
    greedy: bool,
    /// Answer budget in tokens. None keeps the operator's `-n` (else LLM_PREDICT).
    n_predict: Option<u32>,
    /// A chat client's sampling, over the operator's.
    temperature: Option<f32>,
    top_p: Option<f32>,
}

/// What a run produced. Token counts come from the kept-loaded model only.
struct ModelAnswer {
    text: String,
    prompt_tokens: Option<u64>,
```

```
    completion_tokens: Option<u64>,
    /// The answer stopped at its token budget rather than ending by itself.
    hit_limit: bool,
}

/// A bare prompt in the model's chat template: an /infer job, a peer's job, an
advisory.
async fn run_llm_with(
    st: &Shared,
    prompt: &str,
    budget: Duration,
    grammar: Option<&str>,
    greedy: bool,
) -> Result<String, String> {
    let call = ModelCall {
        formatted: st.cfg.prompt_format.wrap(prompt),
        grammar,
        greedy,
        n_predict: None,
        temperature: None,
        top_p: None,
    };
    run_model(st, &call, budget).await.map(|a| a.text)
}

/// Run on the kept-loaded model when this node has one, else with --llama-bin. A
server that
/// turns out to be gone mid-run hands this run to --llama-bin rather than losing it.
async fn run_model(st: &Shared, call: &ModelCall<'_>, budget: Duration) ->
Result<ModelAnswer, String> {
    let started = Instant::now();
    let out = match engine_ready(st, budget).await {
        EngineUse::Server { port, key } => {
            let left = budget.saturating_sub(started.elapsed());
            match server_complete(st, port, &key, call, left).await {
                Ok(a) => Ok(a),
                Err(ServerErr::Failed(e)) => Err(e),
                Err(ServerErr::Gone(e)) => {
                    engine_lost(st, &e).await;
                    warn!(
                        "🧠 the kept-loaded model stopped answering ({e}); this job
runs with {}",
                        st.cfg.llama_bin
                    );
                    run_llm_cli(st, call,
budget.saturating_sub(started.elapsed())).await
                }
            }
        }
        EngineUse::Cli => run_llm_cli(st, call,
```

```
budget.saturating_sub(started.elapsed())).await,
        EngineUse::Loading => Err("the model is still loading on this
device".into()),
    };
    *lock(&st.engine_used) = Instant::now();
    out
}

/// What a chat run adds to the per-job runner's flags: its answer budget and
sampling, after the
/// operator's, since llama.cpp lets a later flag win. Nothing for an /infer job (its
command line
/// is v18's) or an advisory (its own -n and --temp 0 stand).
fn call_overrides(call: &ModelCall<'_>) -> Vec<String> {
    let mut out = Vec::new();
    if call.greedy {
        return out;
    }
    if let Some(n) = call.n_predict {
        out.push("-n".to_string());
        out.push(n.to_string());
    }
    if let Some(t) = call.temperature {
        out.push("--temp".to_string());
        out.push(t.to_string());
    }
    if let Some(p) = call.top_p {
        out.push("--top-p".to_string());
        out.push(p.to_string());
    }
    out
}

/// Run llama.cpp with the prompt as a *completion* (`-no-cnv`), streaming its
output: the
/// per-job runner, for a node without the kept-loaded model or while it is
unavailable.
///
/// Waiting for the process to exit is not enough: llama.cpp builds whose `llama-cli`
is a chat
/// program (or that enter interactive mode anyway) answer and then sit waiting for
the next
/// message, so the node used to hit its full timeout and discard an answer it
already had. The
/// output is read as it arrives; once the answer has started and the stream goes
quiet, the
/// process is stopped and the answer kept. No shell is involved.
async fn run_llm_cli(st: &Shared, call: &ModelCall<'_>, budget: Duration) ->
Result<ModelAnswer, String> {
    use tokio::process::Command;
```

```
    let formatted = &call.formatted;
    let echo = llm_echo(formatted);
    let done = |text: String| ModelAnswer {
        text,
        prompt_tokens: None,
        completion_tokens: None,
        hit_limit: false,
    };

    let mut cmd = Command::new(&st.cfg.llama_bin);
    cmd.arg("-m").arg(&st.cfg.model_path).arg("-p").arg(formatted);
    cmd.args(llama_run_args(&st.cfg.llama_flags, call.grammar, call.greedy));
    cmd.args(call_overrides(call));
    cmd.stdin(Stdio::null())
        .stdout(Stdio::piped())
        // Captured, not discarded: it is what turns "exit status 1" into a useful
reason.
        .stderr(Stdio::piped())
        .kill_on_drop(true);

    let mut child = cmd.spawn().map_err(|e| {
        format!(
            "spawn {}: {e} — is it on PATH? (try --llama-bin llama-completion)",
            st.cfg.llama_bin
        )
    })?;
    let mut stdout = child.stdout.take().ok_or_else(|| "no stdout
pipe".to_string())?;
    let mut stderr = child.stderr.take().ok_or_else(|| "no stderr
pipe".to_string())?;
    // Drain stderr concurrently: llama.cpp logs heavily there, and a full pipe would
stall it.
    let err_task = tokio::spawn(async move {
        let mut kept = Vec::new();
        let mut chunk = [0u8; 4096];
        while let Ok(n) = read_some(&mut stderr, &mut chunk).await {
            if n == 0 {
                break;
            }
            if kept.len() < LLM_STDOUT_CAP {
                kept.extend_from_slice(&chunk[..n]);
            }
        }
        kept
    });

    let deadline = tokio::time::Instant::now() + budget;
    let mut out: Vec<u8> = Vec::new();
    let mut buf = [0u8; 4096];
```

```
    let stop = loop {
        let now = tokio::time::Instant::now();
        if now >= deadline {
            break LlmStop::Timeout;
        }
        let answering = answer_started(&out, &echo);
        let wait = if answering {
            (deadline - now).min(LLM_IDLE_AFTER_ANSWER)
        } else {
            deadline - now
        };
        match tokio::time::timeout(wait, read_some(&mut stdout, &mut buf)).await {
            Ok(Ok(0)) => break LlmStop::Exited,
            Ok(Ok(n)) => {
                out.extend_from_slice(&buf[..n]);
                if out.len() > LLM_STDOUT_CAP {
                    break LlmStop::Cap;
                }
            }
            Ok(Err(e)) => break LlmStop::Read(e.to_string()),
            Err(_) if answering => break LlmStop::Idle,
            Err(_) => break LlmStop::Timeout,
        }
    };

    let text = String::from_utf8_lossy(&out).into_owned();
    match stop {
        LlmStop::Exited => {
            let status = tokio::time::timeout(Duration::from_secs(5),
child.wait()).await;
            let stderr_bytes = err_task.await.unwrap_or_default();
            match status {
                Ok(Ok(s)) if !s.success() => {
                    return Err(llm_exit_reason(s, &stderr_bytes,
&st.cfg.model_path));
                }
                Ok(Ok(_)) => {}
                _ => {
                    let _ = child.kill().await;
                }
            }
            Ok(done(clean_llm_output(&text, &echo)))
        }
        LlmStop::Idle | LlmStop::Cap => {
            let _ = child.kill().await;
            err_task.abort();
            info!("🧠 {} kept running after its answer; stopped it and kept the
answer", st.cfg.llama_bin);
            let answer = clean_llm_output(&text, &echo);
            if answer.is_empty() {
```

```
                Err("model went idle without an answer".into())
            } else {
                Ok(done(answer))
            }
        }
        LlmStop::Timeout => {
            let _ = child.kill().await;
            err_task.abort();
            Err(format!("timed out after {:?}", budget))
        }
        LlmStop::Read(e) => {
            let _ = child.kill().await;
            err_task.abort();
            Err(format!("read: {e}"))
        }
    }
}

// ──────────────────────────────────── kept-loaded model (llama-server)
──────────────────────────────────
//
// A job used to start llama.cpp, read the whole model off flash, answer and exit:
seconds of
// loading for every answer. v19 keeps the model loaded in llama.cpp's own server,
started once on
// loopback (a free port, a random API key, one slot), asked over HTTP, and unloaded
after a quiet
// spell. The thermal gate, the cooldown and the one-run-at-a-time slot are
unchanged: they sit in
// front of whichever runner serves. A node whose llama.cpp has no server, or whose
server will not
// start, runs every job with --llama-bin, as before.

/// Loading takes seconds for 1B and can take minutes for 8B off a phone's flash.
const ENGINE_LOAD_TIMEOUT: Duration = Duration::from_secs(240);
/// After the server fails, jobs use --llama-bin for this long before it is tried
again.
const ENGINE_RETRY_AFTER: Duration = Duration::from_secs(300);
/// How often the idle unload looks.
const ENGINE_IDLE_CHECK: Duration = Duration::from_secs(30);
/// How much of the server's own output is kept, for the reason when it fails.
const ENGINE_LOG_TAIL: usize = 16 * 1024;
/// In the identity directory: the running server's pid and port, so one a crash
orphaned is found.
const ENGINE_PID_FILE: &str = "llama-server.pid";

/// The running server.
struct EngineProc {
    child: tokio::process::Child,
    port: u16,
```

```
    /// Asked on every request: the server also listens for other programs on this
device.
    key: String,
    log: Arc<Mutex<Vec<u8>>>,
    spawned: Instant,
    ready: bool,
}

#[derive(Default)]
struct Engine {
    proc: Option<EngineProc>,
    /// When it last failed, and why: jobs use --llama-bin until ENGINE_RETRY_AFTER
has passed.
    failed: Option<(Instant, String)>,
}

/// What /health shows of the kept-loaded model.
#[derive(Clone, Debug, Default)]
struct EngineView {
    /// "off", "unloaded", "loading", "ready", or "per-job" (none, or it failed: --
llama-bin serves)
    state: &'static str,
    port: u16,
    load_ms: u64,
    detail: String,
}

impl EngineView {
    fn at_start(enabled: bool) -> Self {
        EngineView {
            state: if enabled { "unloaded" } else { "off" },
            ..EngineView::default()
        }
    }
}

impl AppState {
    fn engine_note(&self, state: &'static str, port: u16, load_ms: u64, detail: &str)
{
        *lock(&self.engine_view) = EngineView {
            state,
            port,
            load_ms,
            detail: truncate(&sanitize(detail), 200),
        };
    }

    fn engine_json(&self) -> serde_json::Value {
        let v = lock(&self.engine_view).clone();
        serde_json::json!({
```

```
            "runner": if self.cfg.llama_server.is_some() { "llama-server" } else {
"llama-bin" },
            "state": v.state,
            "port": (v.port != 0).then_some(v.port),
            "load_ms": (v.load_ms != 0).then_some(v.load_ms),
            "idle_unload_secs": self.cfg.llama_idle.map(|d| d.as_secs()),
            "detail": v.detail,
        })
    }
}

/// Where a run goes.
enum EngineUse {
    Server { port: u16, key: String },
    Cli,
    Loading,
}

/// How a run on the server failed. `Gone`: the server itself is unavailable, and the
run can still
/// go to --llama-bin. `Failed`: this run failed (a refused grammar, a timeout, a bad
request).
enum ServerErr {
    Gone(String),
    Failed(String),
}

/// The client for the kept-loaded model: loopback, no proxy (a system proxy would
see the prompts),
/// and no overall timeout (each run carries its own).
fn build_engine_client() -> Result<reqwest::Client, BoxErr> {
    reqwest::Client::builder()
        .no_proxy()
        .connect_timeout(Duration::from_secs(3))
        .pool_idle_timeout(Duration::from_secs(30))
        .build()
        .map_err(Into::into)
}

/// The server binary. `auto`: llama-server beside --llama-bin (llama.cpp's build and
its release
/// archives put them in one directory), else on PATH. Anything else is a name or a
path.
fn server_binary(choice: &str, llama_bin: &str) -> Option<PathBuf> {
    if choice != "auto" {
        return which_on_path(choice);
    }
    let name = if cfg!(windows) { "llama-server.exe" } else { "llama-server" };
    if let Some(dir) = which_on_path(llama_bin).and_then(|p|
p.parent().map(Path::to_path_buf)) {
```

```
        let beside = dir.join(name);
        if beside.is_file() {
            return Some(beside);
        }
    }
    which_on_path("llama-server")
}

/// The value of the last of `names` in `flags` (`-t 4` or `--threads=4`): llama.cpp
lets a later
/// flag win. A flag with nothing after it is ignored.
fn flag_value(flags: &[String], names: &[&str]) -> Option<String> {
    let mut found = None;
    let mut i = 0;
    while i < flags.len() {
        let f = flags[i].as_str();
        if names.contains(&f) {
            if let Some(v) = flags.get(i + 1) {
                found = Some(v.clone());
                i += 2;
                continue;
            }
        } else if let Some((k, v)) = f.split_once('=') {
            if names.contains(&k) {
                found = Some(v.to_string());
            }
        }
        i += 1;
    }
    found
}

/// The operator's sampling settings in --llama-flags, as llama-server request
fields.
fn sampling_flags(flags: &[String]) -> Vec<(&'static str, String)> {
    let known: [(&[&str], &'static str); 7] = [
        (&["--temp", "--temperature"], "temperature"),
        (&["--top-p"], "top_p"),
        (&["--top-k"], "top_k"),
        (&["--min-p"], "min_p"),
        (&["--repeat-penalty"], "repeat_penalty"),
        (&["--repeat-last-n"], "repeat_last_n"),
        (&["-s", "--seed"], "seed"),
    ];
    known
        .iter()
        .filter_map(|(names, field)| flag_value(flags, names).map(|v| (*field, v)))
        .collect()
}
```

```
/// llama-server's command line: the model, loopback only, one slot, the context
size, and the
/// operator's resource flags (threads, GPU layers, batch sizes). Sampling goes with
each request
/// instead. llama-cli's own flags (-no-cnv, -n) are left out, and so are --mlock and
--no-mmap,
/// which recent servers replaced with --load-mode: a flag the server does not know
stops it from
/// starting.
fn server_args(model: &str, port: u16, flags: &[String]) -> Vec<String> {
    let mut out: Vec<String> = vec![
        "-m".into(),
        model.into(),
        "--host".into(),
        "127.0.0.1".into(),
        "--port".into(),
        port.to_string(),
        "-np".into(),
        "1".into(),
        "-c".into(),
        flag_value(flags, &["-c", "--ctx-size"]).unwrap_or_else(||
LLM_CTX.to_string()),
    ];
    let passed: [&[&str]; 4] = [
        &["-t", "--threads"],
        &["-ngl", "--n-gpu-layers", "--gpu-layers"],
        &["-b", "--batch-size"],
        &["-ub", "--ubatch-size"],
    ];
    for names in passed {
        if let Some(v) = flag_value(flags, names) {
            out.push(names[0].to_string());
            out.push(v);
        }
    }
    out
}

/// The body of one run on llama-server's own /completion endpoint: the templated
prompt (the node
/// applies the template, as with --llama-bin), the operator's sampling, the run's
own settings, and
/// streaming, so a run cut off by its deadline stops generating.
fn server_request(call: &ModelCall<'_>, flags: &[String]) -> serde_json::Value {
    let mut body = serde_json::Map::new();
    body.insert("prompt".into(), serde_json::Value::from(call.formatted.as_str()));
    for (field, raw) in sampling_flags(flags) {
        let v = if matches!(field, "top_k" | "repeat_last_n" | "seed") {
            raw.parse::<i64>().ok().map(serde_json::Value::from)
        } else {
```

```
            raw.parse::<f64>().ok().filter(|f|
f.is_finite()).map(serde_json::Value::from)
        };
        if let Some(v) = v {
            body.insert(field.to_string(), v);
        }
    }
    if let Some(t) = call.temperature {
        body.insert("temperature".into(), serde_json::Value::from(f64::from(t)));
    }
    if let Some(p) = call.top_p {
        body.insert("top_p".into(), serde_json::Value::from(f64::from(p)));
    }
    let n_predict: i64 = if call.greedy {
        body.insert("temperature".into(), serde_json::Value::from(0.0f64));
        ADVISORY_PREDICT.parse().unwrap_or(320)
    } else {
        call.n_predict
            .map(i64::from)
            .or_else(|| flag_value(flags, &["-n", "--predict", "--n-
predict"]).and_then(|v| v.parse().ok()))
            .unwrap_or_else(|| LLM_PREDICT.parse().unwrap_or(128))
    };
    body.insert("n_predict".into(), serde_json::Value::from(n_predict));
    if let Some(g) = call.grammar {
        body.insert("grammar".into(), serde_json::Value::from(g));
    }
    body.insert("cache_prompt".into(), serde_json::Value::from(true));
    body.insert("stream".into(), serde_json::Value::from(true));
    serde_json::Value::Object(body)
}

/// Splits a server-sent-events stream into lines, whatever the chunk boundaries.
#[derive(Default)]
struct SseLines {
    buf: Vec<u8>,
}

impl SseLines {
    /// The complete lines this chunk finishes, without their line ends.
    fn push(&mut self, chunk: &[u8]) -> Vec<String> {
        self.buf.extend_from_slice(chunk);
        let mut out = Vec::new();
        while let Some(i) = self.buf.iter().position(|&b| b == b'\n') {
            let line: Vec<u8> = self.buf.drain(..=i).collect();
            let text = String::from_utf8_lossy(&line[..line.len() - 1]).into_owned();
            out.push(text.trim_end_matches('\r').to_string());
        }
        out
    }
```

```
    /// Bytes still waiting for their line end.
    fn pending(&self) -> usize {
        self.buf.len()
    }
}

/// The payload of an SSE `data:` line.
fn sse_data(line: &str) -> Option<&str> {
    line.strip_prefix("data:").map(str::trim_start)
}

/// The server's reason for refusing a run, from its JSON error when it sent one.
fn server_error_text(status: u16, body: &str) -> String {
    let parsed = serde_json::from_str::<serde_json::Value>(body).ok();
    let msg = parsed
        .as_ref()
        .and_then(|v| {
            let e = v.get("error").unwrap_or(v);
            e.get("message")
                .and_then(|m| m.as_str())
                .or_else(|| e.as_str())
                .map(String::from)
        })
        .unwrap_or_else(|| body.to_string());
    format!("llama-server {status}: {}", truncate(&sanitize(&msg).replace('\n', " "),
300))
}

/// One run on the kept-loaded model, read as it is generated.
async fn server_complete(
    st: &Shared,
    port: u16,
    key: &str,
    call: &ModelCall<'_>,
    budget: Duration,
) -> Result<ModelAnswer, ServerErr> {
    let body = serde_json::to_vec(&server_request(call, &st.cfg.llama_flags))
        .map_err(|e| ServerErr::Failed(format!("request: {e}")))?;
    let url = format!("http://127.0.0.1:{port}/completion");
    let run = async {
        let sent = st
            .engine_client
            .post(&url)
            .header("authorization", format!("Bearer {key}"))
            .header("content-type", "application/json")
            .body(body)
            .send()
            .await;
        let mut resp = match sent {
```

```
            Ok(r) => r,
            Err(e) if e.is_connect() => return Err(ServerErr::Gone(format!("connect:
{e}"))),
            Err(e) => return Err(ServerErr::Failed(format!("llama-server: {e}"))),
        };
        let status = resp.status();
        if !status.is_success() {
            let text = resp
                .bytes()
                .await
                .map(|b| String::from_utf8_lossy(&b).into_owned())
                .unwrap_or_default();
            return Err(ServerErr::Failed(server_error_text(status.as_u16(), &text)));
        }
        let mut lines = SseLines::default();
        let mut out = ModelAnswer {
            text: String::new(),
            prompt_tokens: None,
            completion_tokens: None,
            hit_limit: false,
        };
        let mut finished = false;
        while !finished {
            let chunk = match resp.chunk().await {
                Ok(Some(c)) => c,
                Ok(None) => break,
                Err(e) => return Err(ServerErr::Failed(format!("llama-server stream:
{e}"))),
            };
            for line in lines.push(&chunk) {
                let Some(v) = sse_data(&line).and_then(|d| serde_json::from_str::
<serde_json::Value>(d).ok())
                else {
                    continue;
                };
                if let Some(err) = v.get("error") {
                    return Err(ServerErr::Failed(server_error_text(500,
&err.to_string())));
                }
                if let Some(c) = v.get("content").and_then(|c| c.as_str()) {
                    out.text.push_str(c);
                }
                if v.get("stop").and_then(|x| x.as_bool()) == Some(true) {
                    out.prompt_tokens = v.get("tokens_evaluated").and_then(|x|
x.as_u64());
                    out.completion_tokens = v.get("tokens_predicted").and_then(|x|
x.as_u64());
                    out.hit_limit = v.get("stopped_limit").and_then(|x| x.as_bool())
== Some(true)
                        || v.get("stop_type").and_then(|x| x.as_str()) ==
```

```
Some("limit");
                    finished = true;
                }
            }
            if out.text.len() > LLM_STDOUT_CAP || lines.pending() > LLM_STDOUT_CAP {
                break;
            }
        }
        if !finished && out.text.is_empty() {
            return Err(ServerErr::Gone("it closed the stream without an
answer".into()));
        }
        Ok::<ModelAnswer, ServerErr>(out)
    };
    match tokio::time::timeout(budget, run).await {
        Ok(r) => r,
        Err(_) => Err(ServerErr::Failed(format!("timed out after {budget:?}"))),
    }
}

/// A loopback port nothing listens on right now, for the server.
fn free_loopback_port() -> Option<u16> {
    std::net::TcpListener::bind((Ipv4Addr::LOCALHOST, 0))
        .ok()?
        .local_addr()
        .ok()
        .map(|a| a.port())
}

/// Keep the tail of the server's output, for the reason when it fails.
async fn drain_log<R: tokio::io::AsyncRead + Unpin>(mut r: R, log:
Arc<Mutex<Vec<u8>>>) {
    let mut chunk = [0u8; 4096];
    while let Ok(n) = read_some(&mut r, &mut chunk).await {
        if n == 0 {
            break;
        }
        let mut l = lock(&log);
        l.extend_from_slice(&chunk[..n]);
        let excess = l.len().saturating_sub(ENGINE_LOG_TAIL);
        if excess > 0 {
            l.drain(..excess);
        }
    }
}

/// The server's last words, as an operator can act on them.
fn engine_log_hint(log: &Mutex<Vec<u8>>, model_path: &str) -> String {
    llm_failure_hint(&lock(log), model_path)
}
```

```
fn engine_forget_pid(st: &AppState) {
    let _ = std::fs::remove_file(st.cfg.data_dir.join(ENGINE_PID_FILE));
}

/// Start the server on a free loopback port. It is not ready until its /health says
so.
fn engine_spawn(st: &AppState, bin: &Path) -> Result<EngineProc, String> {
    let port = free_loopback_port().ok_or("no free loopback port for llama-server")?;
    let mut raw = [0u8; 16];
    OsRng.fill_bytes(&mut raw);
    let key = hex(&raw);
    let mut cmd = tokio::process::Command::new(bin);
    cmd.args(server_args(&st.cfg.model_path, port, &st.cfg.llama_flags))
        // Both spellings: llama.cpp releases have read the API key from either.
        .env("LLAMA_API_KEY", &key)
        .env("LLAMA_ARG_API_KEY", &key)
        .stdin(Stdio::null())
        .stdout(Stdio::piped())
        .stderr(Stdio::piped())
        .kill_on_drop(true);
    #[cfg(windows)]
    cmd.creation_flags(0x0800_0000); // CREATE_NO_WINDOW
    let mut child = cmd.spawn().map_err(|e| format!("spawn {}: {e}",
bin.display()))?;
    let log = Arc::new(Mutex::new(Vec::new()));
    if let Some(out) = child.stdout.take() {
        tokio::spawn(drain_log(out, log.clone()));
    }
    if let Some(err) = child.stderr.take() {
        tokio::spawn(drain_log(err, log.clone()));
    }
    if let Some(pid) = child.id() {
        let _ = std::fs::write(st.cfg.data_dir.join(ENGINE_PID_FILE), format!("{pid}
{port}\n"));
    }
    Ok(EngineProc {
        child,
        port,
        key,
        log,
        spawned: Instant::now(),
        ready: false,
    })
}

/// Wait for the server's /health to answer 200 (it answers 503 while it loads the
model) until
/// `until`. Err(Some(reason)) when it died; Err(None) when it is still loading at
`until`.
```

```
async fn engine_wait_ready(st: &AppState, p: &mut EngineProc, until: Instant) ->
Result<(), Option<String>> {
    let url = format!("http://127.0.0.1:{}/health", p.port);
    loop {
        match p.child.try_wait() {
            Ok(Some(status)) => {
                return Err(Some(format!(
                    "llama-server exited ({status}): {}",
                    engine_log_hint(&p.log, &st.cfg.model_path)
                )));
            }
            Ok(None) => {}
            Err(e) => return Err(Some(format!("llama-server: {e}"))),
        }
        let ok = st
            .engine_client
            .get(&url)
            .header("authorization", format!("Bearer {}", p.key))
            .timeout(Duration::from_secs(2))
            .send()
            .await
            .is_ok_and(|r| r.status().is_success());
        if ok {
            return Ok(());
        }
        if Instant::now() >= until {
            return Err(None);
        }
        tokio::time::sleep(Duration::from_millis(250)).await;
    }
}

/// The kept-loaded model for a run that can wait `wait` for it, started if it is not
running.
/// `Cli` when this node has none (--llama-server off, no binary, a failure in the
last
/// ENGINE_RETRY_AFTER), `Loading` while it is still loading when `wait` runs out.
async fn engine_ready(st: &Shared, wait: Duration) -> EngineUse {
    let Some(choice) = st.cfg.llama_server.as_deref() else {
        return EngineUse::Cli;
    };
    let until = Instant::now() + wait;
    let Ok(mut eng) = tokio::time::timeout(wait, st.engine.lock()).await else {
        return EngineUse::Loading;
    };
    if eng.proc.is_none() {
        if eng.failed.as_ref().is_some_and(|(at, _)| at.elapsed() <
ENGINE_RETRY_AFTER) {
            return EngineUse::Cli;
        }
```

```
        // No model file: the per-job runner fails with the reason the operator
needs.
        if !Path::new(&st.cfg.model_path).is_file() {
            return EngineUse::Cli;
        }
        let spawned = match server_binary(choice, &st.cfg.llama_bin) {
            Some(bin) => engine_spawn(st, &bin),
            None => Err(format!(
                "no llama-server beside {} or on PATH (build it: cmake --build
llama.cpp/build --target llama-server)",
                st.cfg.llama_bin
            )),
        };
        match spawned {
            Ok(p) => {
                info!(
                    "🧠 loading {} into llama-server on 127.0.0.1:{}; it stays loaded
between jobs",
                    st.cfg.model_path, p.port
                );
                st.engine_note("loading", p.port, 0, "");
                eng.proc = Some(p);
            }
            Err(why) => {
                warn!("🧠 {why}: jobs load the model each time with {}",
st.cfg.llama_bin);
                eng.failed = Some((Instant::now(), why.clone()));
                st.engine_note("per-job", 0, 0, &why);
                return EngineUse::Cli;
            }
        }
    }
    enum Next {
        Use(u16, String),
        Wait,
        Fail(String),
    }
    let next = match eng.proc.as_mut() {
        None => return EngineUse::Cli,
        Some(p) if p.ready => match p.child.try_wait() {
            Ok(None) => Next::Use(p.port, p.key.clone()),
            Ok(Some(status)) => Next::Fail(format!(
                "llama-server exited ({status}): {}",
                engine_log_hint(&p.log, &st.cfg.model_path)
            )),
            Err(e) => Next::Fail(format!("llama-server: {e}")),
        },
        Some(p) => {
            let deadline = until.min(p.spawned + ENGINE_LOAD_TIMEOUT);
            match engine_wait_ready(st, p, deadline).await {
```

```
                Ok(()) => {
                    p.ready = true;
                    let ms = p.spawned.elapsed().as_millis() as u64;
                    info!(
                        "🧠 model loaded in {:.1} s; kept loaded in llama-server
(127.0.0.1:{})",
                        ms as f64 / 1000.0,
                        p.port
                    );
                    st.engine_note("ready", p.port, ms, "");
                    Next::Use(p.port, p.key.clone())
                }
                Err(None) if p.spawned.elapsed() < ENGINE_LOAD_TIMEOUT => Next::Wait,
                Err(reason) => Next::Fail(reason.unwrap_or_else(|| {
                    format!("llama-server did not finish loading in {} s",
ENGINE_LOAD_TIMEOUT.as_secs())
                })),
            }
        }
    };
    match next {
        Next::Use(port, key) => EngineUse::Server { port, key },
        Next::Wait => EngineUse::Loading,
        Next::Fail(why) => {
            warn!(
                "🧠 {why}: jobs load the model each time with {} for the next {}
min",
                st.cfg.llama_bin,
                ENGINE_RETRY_AFTER.as_secs() / 60
            );
            if let Some(mut dead) = eng.proc.take() {
                let _ = dead.child.kill().await;
            }
            engine_forget_pid(st);
            eng.failed = Some((Instant::now(), why.clone()));
            st.engine_note("per-job", 0, 0, &why);
            EngineUse::Cli
        }
    }
}

/// The server stopped answering mid-run: stop it, and serve with --llama-bin for a
while.
async fn engine_lost(st: &Shared, why: &str) {
    let mut eng = st.engine.lock().await;
    if let Some(mut p) = eng.proc.take() {
        let _ = p.child.kill().await;
    }
    engine_forget_pid(st);
    eng.failed = Some((Instant::now(), why.to_string()));
```

```
    st.engine_note("per-job", 0, 0, why);
}

/// At startup: stop a server a crashed run left behind, then load the model, so the
first job
/// does not wait for it.
async fn engine_start_up(st: Shared) {
    if st.cfg.llama_server.is_none() {
        return;
    }
    let dir = st.cfg.data_dir.clone();
    let _ = tokio::task::spawn_blocking(move || engine_reap_orphan(&dir)).await;
    if Path::new(&st.cfg.model_path).is_file() && !st.draining() {
        let _ = engine_ready(&st, ENGINE_LOAD_TIMEOUT).await;
    }
}

/// Unload the kept-loaded model after --llama-idle-secs without a job; the next job
loads it again.
async fn engine_idle_loop(st: Shared) {
    let Some(idle) = st.cfg.llama_idle else { return };
    if st.cfg.llama_server.is_none() {
        return;
    }
    let mut shutdown = st.shutdown.subscribe();
    loop {
        tokio::select! {
            _ = tokio::time::sleep(ENGINE_IDLE_CHECK) => {}
            _ = shutdown.recv() => return,
        }
        let quiet = lock(&st.engine_used).elapsed();
        if quiet < idle || st.llm_gate.available_permits() == 0 {
            continue;
        }
        let Ok(mut eng) = st.engine.try_lock() else { continue };
        // A job takes the slot before the model, so a free slot here means no run is
using it.
        if !eng.proc.as_ref().is_some_and(|p| p.ready) ||
st.llm_gate.available_permits() == 0 {
            continue;
        }
        if let Some(mut p) = eng.proc.take() {
            let _ = p.child.kill().await;
        }
        engine_forget_pid(&st);
        st.engine_note("unloaded", 0, 0, "idle; the next job loads it again");
        info!(
            "🧠 unloaded the model after {} min without a job; the next job loads it
again",
            quiet.as_secs() / 60
```

```
        );
    }
}

/// Stop the kept-loaded model with the node.
async fn engine_stop(st: &Shared) {
    let mut eng = st.engine.lock().await;
    if let Some(mut p) = eng.proc.take() {
        let _ = p.child.kill().await;
        info!("🧠 stopped llama-server");
    }
    engine_forget_pid(st);
}

/// A server left running by a node that crashed still holds its model's memory. Its
pid file names
/// it; it is stopped only if that pid is still a llama-server.
fn engine_reap_orphan(dir: &Path) {
    let path = dir.join(ENGINE_PID_FILE);
    let Ok(text) = std::fs::read_to_string(&path) else { return };
    let _ = std::fs::remove_file(&path);
    let Some(pid) = text.split_whitespace().next().and_then(|p| p.parse::<u32>
().ok()) else {
        return;
    };
    if pid == std::process::id() || !is_llama_server(pid) {
        return;
    }
    if stop_process(pid) {
        info!("🧠 stopped llama-server pid {pid}, left running by an earlier run of
this node");
    } else {
        warn!("🧠 llama-server pid {pid} from an earlier run is still running; stop
it to free its memory");
    }
}

#[cfg(any(target_os = "linux", target_os = "android"))]
fn is_llama_server(pid: u32) -> bool {
    std::fs::read(format!("/proc/{pid}/cmdline"))
        .is_ok_and(|c| String::from_utf8_lossy(&c).contains("llama-server"))
}

#[cfg(windows)]
fn is_llama_server(pid: u32) -> bool {
    use std::os::windows::process::CommandExt;
    std::process::Command::new("tasklist")
        .args(["/FI", format!("PID eq {pid}").as_str(), "/FO", "CSV", "/NH"])
        .creation_flags(0x0800_0000)
        .output()
```

```
        .is_ok_and(|o|
String::from_utf8_lossy(&o.stdout).to_ascii_lowercase().contains("llama-server"))
}

#[cfg(not(any(target_os = "linux", target_os = "android", windows)))]
fn is_llama_server(_pid: u32) -> bool {
    false
}

#[cfg(unix)]
fn stop_process(pid: u32) -> bool {
    // `kill` is not on every Termux PATH; Android always has its own.
    ["kill", "/system/bin/kill"].iter().any(|k| {
        std::process::Command::new(k)
            .arg(pid.to_string())
            .stdout(Stdio::null())
            .stderr(Stdio::null())
            .status()
            .is_ok_and(|s| s.success())
    })
}

#[cfg(windows)]
fn stop_process(pid: u32) -> bool {
    use std::os::windows::process::CommandExt;
    std::process::Command::new("taskkill")
        .args(["/PID", pid.to_string().as_str(), "/F"])
        .creation_flags(0x0800_0000)
        .stdout(Stdio::null())
        .stderr(Stdio::null())
        .status()
        .is_ok_and(|s| s.success())
}

#[cfg(not(any(unix, windows)))]
fn stop_process(_pid: u32) -> bool {
    false
}

/// Is the configured model file actually there, and is the runner on PATH?
///
/// Checked at startup and surfaced in `/health`, because the alternative is what
happened
/// before 3.3: a node that looks completely healthy, answers every endpoint, and
fails only
/// at the moment you ask it to think — with an error that names neither the file nor
the
/// binary. A node with no model is still a useful mesh node (it routes, gossips, and
offloads
/// inference to peers), so this warns rather than refusing to start.
```

```
fn model_ready(cfg: &Config) -> (bool, bool) {
    let model_ok = Path::new(&cfg.model_path).is_file();
    let bin_ok = which_on_path(&cfg.llama_bin).is_some();
    (model_ok, bin_ok)
}

/// Minimal `which`. An absolute or relative path is tested directly; a bare name is
looked up
/// across PATH.
fn which_on_path(bin: &str) -> Option<PathBuf> {
    let direct = Path::new(bin);
    if direct.components().count() > 1 || direct.is_absolute() {
        return direct.is_file().then(|| direct.to_path_buf());
    }
    let names = exe_candidates(bin, cfg!(windows));
    let path = std::env::var_os("PATH")?;
    std::env::split_paths(&path)
        .flat_map(|d| names.iter().map(move |n| d.join(n)))
        .find(|c| c.is_file())
}

/// File names a bare program name can have on disk: on Windows `llama-cli` is
`llama-cli.exe`.
fn exe_candidates(bin: &str, windows: bool) -> Vec<String> {
    let mut names = vec![bin.to_string()];
    if windows && Path::new(bin).extension().is_none() {
        names.push(format!("{bin}.exe"));
    }
    names
}

fn report_model_readiness(cfg: &Config) {
    let (model_ok, bin_ok) = model_ready(cfg);
    if model_ok && bin_ok {
        info!("🧠 model ready: {} via {}", cfg.model_path, cfg.llama_bin);
        return;
    }
    if !model_ok {
        let abs = std::fs::canonicalize(&cfg.model_path)
            .map(|p| p.display().to_string())
            .unwrap_or_else(|_| {
                std::env::current_dir()
                    .map(|d| d.join(&cfg.model_path).display().to_string())
                    .unwrap_or_else(|_| cfg.model_path.clone())
            });
        warn!(
            "⚠ no model file for tier {}: looked for {abs}. Inference will fail until
this \
             exists — pass --model-path (or --model-light/-standard/-mid/-heavy). The
node \
```

```
             still routes and can offload inference to peers.",
            cfg.tier.label()
        );
    }
    if !bin_ok {
        warn!(
            "⚠ llama binary {:?} is not on PATH. Try --llama-bin llama-completion, or
give \
             an absolute path to the binary in your llama.cpp build.",
            cfg.llama_bin
        );
    }
}

/// Turn llama.cpp's stderr into something an operator can act on.
///
/// Kept to one line and sanitized because this string travels: it goes into an
`InferOutcome`
/// and back over the mesh to whoever asked, so it must not carry control characters
or run
/// to thousands of lines of model-loading noise.
fn llm_failure_hint(stderr: &[u8], model_path: &str) -> String {
    let text = String::from_utf8_lossy(stderr);
    let flat = sanitize(&text).replace('\n', " ");
    let lower = flat.to_lowercase();

    // Recognise the failures that actually happen on a phone, in the order they
happen.
    if lower.contains("no such file") || lower.contains("failed to load model") {
        return format!(
            "model not found or unreadable at {model_path:?} — set --model-heavy/--
model-path              to the real file (absolute paths are safest)"
        );
    }
    if lower.contains("unknown argument") || lower.contains("invalid argument")
        || lower.contains("error while handling argument")
    {
        return format!(
            "{} rejected a flag — check --llama-flags, and note some builds
want              llama-completion rather than llama-cli: {}",
            "llama",
            truncate(flat.trim(), 240)
        );
    }
    if lower.contains("out of memory") || lower.contains("cannot allocate") {
        return "out of memory loading the model — use a smaller quant or a smaller
tier".into();
    }

    // Nothing recognised: hand back the tail, which is where the real error usually
```

```
is.
    let tail: String = flat.chars().rev().take(400).collect::<String>
().chars().rev().collect();
    if tail.trim().is_empty() {
        "no stderr output".into()
    } else {
        truncate(tail.trim(), 400)
    }
}

// ──────────────────────────────────── router ────────────────────────────────────

fn build_router(state: Shared) -> Router {
    Router::new()
        .route("/id", get(get_id))
        .route("/health", get(get_health))
        .route("/peers", get(get_peers))
        .route("/infer", post(post_infer))
        .route("/v1/models", get(get_models))
        .route(
            "/v1/chat/completions",
            post(post_chat_completions).layer(DefaultBodyLimit::max(CHAT_MAX_BODY)),
        )
        .route("/emcon", post(post_emcon))
        .route("/shutdown", post(post_shutdown))
        .route("/advise", post(post_advise))
        .route("/relay", get(get_relay).post(post_relay))
        .route("/events", get(get_events))
        .route("/ui", get(get_ui))
        .route("/", get(get_ui))
        .route("/hello", post(post_hello))
        .route("/msg", post(post_msg))
        .layer(DefaultBodyLimit::max(
            MAX_HTTP_BODY.max(HELLO1_LEN + 256).max(MAX_FRAME_LEN),
        ))
        // A panic in one handler becomes a 500 for that request instead of taking
the whole
        // node offline — which matters most for the parsers that touch
unauthenticated bytes.
        .layer(tower_http::catch_panic::CatchPanicLayer::new())
        .with_state(state)
}

// ──────────────────────────────────── client (outbound)
────────────────────────────────────


/// Load every bundle in `<data-dir>/bundles/`, re-verifying each signature.
///
/// Files here are either ones this node cached itself or ones an operator
provisioned out of
```

```
/// band. Both are re-verified: the filename is a convenience, never the authority. A
file
/// whose contents do not match its name is rejected, so dropping in a bundle under
someone
/// else's fingerprint achieves nothing.
fn load_bundles(dir: &Path) -> HashMap<NodeId, crypto::PeerBundle> {
    let mut out = HashMap::new();
    let entries = match std::fs::read_dir(dir) {
        Ok(e) => e,
        Err(_) => return out,
    };
    for entry in entries.flatten() {
        let path = entry.path();
        if path.extension().and_then(|e| e.to_str()) != Some("bundle") {
            continue;
        }
        let raw = match std::fs::read(&path) {
            Ok(r) => r,
            Err(e) => {
                warn!("could not read {}: {e}", path.display());
                continue;
            }
        };
        let Some(parsed) = crypto::parse_bundle(&raw) else {
            warn!("⚠ {} failed signature verification; ignoring", path.display());
            continue;
        };
        let claimed = path.file_stem().and_then(|s| s.to_str()).unwrap_or("");
        if !claimed.is_empty() && claimed != hex(&parsed.fpr) {
            warn!(
                "⚠ {} contains identity {} — filename does not match; ignoring",
                path.display(),
                short_id(&parsed.fpr)
            );
            continue;
        }
        out.insert(parsed.fpr, parsed);
    }
    out
}

/// Fetch and verify a peer's identity bundle from `GET /id`.
async fn fetch_bundle(
    client: &reqwest::Client,
    addr: SocketAddr,
) -> Result<(Vec<u8>, crypto::PeerBundle), BoxErr> {
    let url = format!("http://{addr}/id");
    let resp = client.get(&url).send().await?.error_for_status()?;
    // The bundle is a fixed size; reject an oversized advertised length before
downloading.
```

```
    if let Some(len) = resp.content_length() {
        if len > (BUNDLE_LEN as u64) + 64 {
            return Err(format!("peer {addr} /id content-length {len} too
large").into());
        }
    }
    let bytes = resp.bytes().await?;
    if bytes.len() != BUNDLE_LEN {
        return Err(format!("peer {addr} returned a {}-byte id (want {BUNDLE_LEN})",
bytes.len()).into());
    }
    let parsed = crypto::parse_bundle(&bytes)
        .ok_or_else(|| -> BoxErr { format!("peer {addr} id failed
verification").into() })?;
    Ok((bytes.to_vec(), parsed))
}

/// Perform the full handshake against `addr` and return a ready client session.
///
/// Uses a cached bundle when one exists. That is not just a saving: fetching `/id`
publishes
/// the responder's identity in cleartext, so doing it every round would broadcast
the mesh's
/// participant graph no matter how well `hello1` hides the initiator.
async fn dial(
    state: &Shared,
    client: &reqwest::Client,
    addr: SocketAddr,
) -> Result<ClientSession, BoxErr> {
    let cached = state
        .peers_by_addr(addr)
        .and_then(|id| state.cached_bundle(&id));

    if let Some(peer) = cached {
        let fpr = peer.fpr;
        match dial_with_bundle(state, client, addr, peer).await {
            Ok(sess) => return Ok(sess),
            Err(e) => {
                // One retry with a fresh fetch, because a cached bundle can be
legitimately
                // out of date — the peer may have rotated its ML-KEM key. Without
this the
                // cache would turn rotation into a permanent partition.
                debug!("cached bundle for {} did not work ({e}); refetching",
short_id(&fpr));
                state.forget_bundle(&fpr);
            }
        }
    }
```

```
    let (raw, parsed) = fetch_bundle(client, addr).await?;
    state.cache_bundle(&raw, &parsed);
    dial_with_bundle(state, client, addr, parsed).await
}

async fn dial_with_bundle(
    state: &Shared,
    client: &reqwest::Client,
    addr: SocketAddr,
    peer: crypto::PeerBundle,
) -> Result<ClientSession, BoxErr> {
    if peer.fpr == state.me.fpr() {
        return Err("refusing to dial self".into());
    }
    if state.is_revoked(&peer.fpr) {
        return Err(format!("peer {} is revoked", short_id(&peer.fpr)).into());
    }
    // v17 checked only revocation here, so a strict node still handshook with,
gossiped to and
    // offloaded jobs to any identity a peer introduced. Strict: no hello (it carries
this node's
    // identity) to anyone outside the list.
    if !state.trust_allows(&peer.fpr) {
        return Err(format!("peer {} at {addr} is not in this node's trust list",
short_id(&peer.fpr)).into());
    }
    let ch = client_build_hello(&state.me, &peer, state.bound_port, now_unix())
        .ok_or("failed to build hello")?;

    let resp = client
        .post(format!("http://{addr}/hello"))
        .body(ch.hello1.clone())
        .send()
        .await?
        .error_for_status()?;
    let hello2 = resp.bytes().await?;
    if hello2.len() != HELLO2_LEN {
        return Err(format!("peer {addr} returned a malformed hello2").into());
    }
    let peer_fpr = peer.fpr;
    let keys = client_finish_hello(ch, &hello2).ok_or("responder key confirmation
failed")?;
    // TOFU pins a responder once it has proved it holds the key; strict already
listed it.
    if !state.admit_peer(&peer_fpr) {
        return Err(format!("peer {} was not admitted by the trust store",
short_id(&peer_fpr)).into());
    }
    Ok(ClientSession::from_keys(peer_fpr, keys))
}
```

```
/// Send one request frame on a session and return the decrypted reply, if any.
async fn request(
    client: &reqwest::Client,
    addr: SocketAddr,
    sess: &mut ClientSession,
    msg: &Msg,
) -> Result<Option<Msg>, BoxErr> {
    request_inner(client, addr, sess, msg, None).await
}

/// As `request`, but with an explicit per-request timeout. Inference can
legitimately run for
/// minutes, which the client's short default would cut off.
async fn request_with_timeout(
    client: &reqwest::Client,
    addr: SocketAddr,
    sess: &mut ClientSession,
    msg: &Msg,
    timeout: Duration,
) -> Result<Option<Msg>, BoxErr> {
    request_inner(client, addr, sess, msg, Some(timeout)).await
}

async fn request_inner(
    client: &reqwest::Client,
    addr: SocketAddr,
    sess: &mut ClientSession,
    msg: &Msg,
    timeout: Option<Duration>,
) -> Result<Option<Msg>, BoxErr> {
    let frame = sess.seal(msg).ok_or("failed to seal frame")?;
    let mut req = client.post(format!("http://{addr}/msg")).body(frame);
    if let Some(t) = timeout {
        req = req.timeout(t);
    }
    let resp = req.send().await?;
    if resp.status() == StatusCode::NO_CONTENT {
        return Ok(None);
    }
    let resp = resp.error_for_status()?;
    let reply = resp.bytes().await?;
    if reply.is_empty() {
        return Ok(None);
    }
    Ok(sess.open(&reply))
}

// ──────────────────────────────────── distributed inference
──────────────────────────────────
```

```
//
// Expansion here is horizontal: every node runs a whole model and answers
independently, so
// adding nodes adds throughput and independent opinions. It does not add capacity
for a model
// larger than the biggest single node can hold — that would require exchanging
activations
// per token, which a lossy high-latency radio makes far more expensive than the
compute saved.
//
// Two things this buys that a single node cannot: offload (a hot or busy node
borrows a cool
// one) and cross-checking (several models answer the same prompt, and agreement is
evidence).

/// One peer's answer in a fan-out.
#[derive(Debug, Clone, Serialize)]
struct InferOutcome {
    node: String,
    tier: String,
    ok: bool,
    text: String,
    elapsed_ms: u64,
}

/// Normalise an answer enough to compare two models' phrasing without being fooled
by
/// whitespace or case. Deliberately crude: this is a similarity hint, not a semantic
judgement.
fn normalize_answer(s: &str) -> String {
    s.split_whitespace()
        .map(|w| w.trim_matches(|c: char| !c.is_alphanumeric()).to_lowercase())
        .filter(|w| !w.is_empty())
        .collect::<Vec<_>>()
        .join(" ")
}

/// Group identical normalised answers and report the largest cluster.
/// Returns (agreeing count, total successful answers, the agreed text).
fn consensus(outcomes: &[InferOutcome]) -> (usize, usize, String) {
    let good: Vec<&InferOutcome> = outcomes.iter().filter(|o| o.ok).collect();
    let mut buckets: HashMap<String, (usize, String)> = HashMap::new();
    for o in &good {
        let key = normalize_answer(&o.text);
        let e = buckets.entry(key).or_insert((0, o.text.clone()));
        e.0 += 1;
    }
    let best = buckets.values().max_by_key(|(n, _)| *n);
    match best {
        Some((n, text)) => (*n, good.len(), text.clone()),
```

```
        None => (0, 0, String::new()),
    }
}

/// Ask up to `fanout` peers the same prompt, in parallel, and optionally answer
locally too.
///
/// Each peer is reached over its own authenticated session, so a fan-out is N
independent
/// mutually-authenticated exchanges rather than a broadcast — there is no path here
for an
/// unauthenticated node to inject an answer.
async fn infer_distributed(
    state: &Shared,
    client: &reqwest::Client,
    prompt: &str,
    fanout: usize,
    include_local: bool,
    local_budget: Duration,
    remote_budget: Duration,
) -> Vec<InferOutcome> {
    let mut outcomes = Vec::new();

    if include_local {
        let started = Instant::now();
        let (ok, text) = match infer_locally(state, prompt, local_budget).await {
            Ok(t) => (true, t),
            Err(e) => (false, e),
        };
        outcomes.push(InferOutcome {
            node: hex(&state.me.fpr()),
            tier: state.cfg.tier.label().to_string(),
            ok,
            text,
            elapsed_ms: started.elapsed().as_millis() as u64,
        });
    }

    // Coolest idle peers first, from their piggybacked vitals; peers already at
their own
    // thermal limit are skipped, since they would refuse. Chosen up front so no lock
is held
    // across an await.
    let targets: Vec<(NodeId, SocketAddr)> =
state.ranked_targets(fanout.min(MAX_FANOUT));

    let mut tasks: tokio::task::JoinSet<InferOutcome> = tokio::task::JoinSet::new();
    for (peer_id, addr) in targets {
        let st = state.clone();
        let cl = client.clone();
```

```
        let p = prompt.to_string();
        tasks.spawn(async move {
            let started = Instant::now();
            match infer_from_peer(&st, &cl, peer_id, addr, &p, remote_budget).await {
                Ok(reply) => reply,
                Err(e) => InferOutcome {
                    node: hex(&peer_id),
                    tier: "unknown".into(),
                    ok: false,
                    text: e,
                    elapsed_ms: started.elapsed().as_millis() as u64,
                },
            }
        });
    }

    // A slow peer must not hold up the whole fan-out past the budget. This waits on
the
    // remote deadline only — the local generation already finished above.
    let collect = async {
        while let Some(res) = tasks.join_next().await {
            if let Ok(o) = res {
                outcomes.push(o);
            }
        }
    };
    if tokio::time::timeout(remote_budget + Duration::from_secs(5), collect)
        .await
        .is_err()
    {
        warn!("fan-out deadline reached; reporting the answers that arrived");
        tasks.shutdown().await;
    }
    outcomes
}

/// Run one prompt on one peer over an authenticated session, reusing a cached one if
live.
async fn infer_from_peer(
    state: &Shared,
    client: &reqwest::Client,
    peer_id: NodeId,
    addr: SocketAddr,
    prompt: &str,
    budget: Duration,
) -> Result<InferOutcome, String> {
    let req = Msg::InferRequest {
        id: OsRng.next_u64(),
        prompt: prompt.to_string(),
        deadline_ms: budget.as_millis() as u64,
```

```
        chat: None,
        max_tokens: None,
        temperature: None,
        top_p: None,
    };
    infer_msg_from_peer(state, client, peer_id, addr, &req, budget).await
}

/// Send one inference request (a prompt or a conversation) to a peer and wait for
its answer.
async fn infer_msg_from_peer(
    state: &Shared,
    client: &reqwest::Client,
    peer_id: NodeId,
    addr: SocketAddr,
    req: &Msg,
    budget: Duration,
) -> Result<InferOutcome, String> {
    if state.is_revoked(&peer_id) {
        return Err("peer is revoked".into());
    }
    let mut sess = match state.take_client_session(&peer_id) {
        Some(s) => s,
        None => dial(state, client, addr)
            .await
            .map_err(|e| format!("dial: {e}"))?,
    };
    // Inference legitimately takes much longer than gossip, so this request carries
its own
    // timeout rather than the client's short default.
    let reply = request_with_timeout(client, addr, &mut sess, req, budget +
Duration::from_secs(5))
        .await
        .map_err(|e| format!("request: {e}"))?;

    match reply {
        Some(Msg::InferReply {
            ok,
            text,
            tier,
            node,
            elapsed_ms,
            ..
        }) => {
            state.put_client_session(sess);
            Ok(InferOutcome {
                node,
                tier,
                ok,
                text,
```

```
                elapsed_ms,
            })
        }
        _ => Err("peer returned no inference reply".into()),
    }
}

/// Ports to probe for a node no longer at `failed`: the mesh's base window (`base` up to
/// `base + PORT_SWEEP`, where rotated nodes land) and the window just above `failed`.
fn sweep_ports(failed: u16, base: u16) -> Vec<u16> {
    let mut out = Vec::new();
    for start in [Some(base), failed.checked_add(1)].into_iter().flatten() {
        for off in 0..=PORT_SWEEP {
            if let Some(p) = start.checked_add(off) {
                if p != failed && !out.contains(&p) {
                    out.push(p);
                }
            }
        }
    }
    out
}

/// Apply a new sighting of a peer already in the table.
///
/// A direct sighting (a handshake, which proves the address) always wins and clears the failure
/// count. Hearsay (another node's peer list) only moves a peer that is failing where we have
/// it, and keeps the failure count: a stale list must not drag a working address back to the
/// port a node rotated off, and two lists that disagree must not keep a dead peer alive.
fn update_known_peer(p: &mut Peer, addr: SocketAddr, source: Option<NodeId>) {
    if source.is_none() {
        p.addr = addr;
        p.fails = 0;
        // Direct observation outranks hearsay: once we have spoken to a peer ourselves, it no
        // longer counts against whoever introduced it.
        p.learned_from = None;
    } else if p.fails > 0 {
        p.addr = addr;
    }
}

/// Find a node that rotated off `failed` by probing `sweep_ports` on the same host. A closed
```

```
/// port costs one TCP refusal; an open one costs a full mutually authenticated
handshake, so
/// another service, or another node, on a nearby port is never mistaken for the one
wanted.
async fn sweep_for_node(
    state: &Shared,
    client: &reqwest::Client,
    failed: SocketAddr,
    want: Option<NodeId>,
) -> Option<(NodeId, SocketAddr)> {
    for port in sweep_ports(failed.port(), state.cfg.port) {
        let addr = SocketAddr::new(failed.ip(), port);
        if !peer_ok(&addr, state.cfg.allow_public, state.cfg.allow_loopback) {
            return None;
        }
        let probe = tokio::time::timeout(
            Duration::from_millis(500),
            tokio::net::TcpStream::connect(addr),
        )
        .await;
        if !matches!(probe, Ok(Ok(_))) {
            continue;
        }
        match dial(state, client, addr).await {
            Ok(sess) => {
                let id = sess.peer_id;
                if id == state.me.fpr() {
                    continue;
                }
                // Any node met here was authenticated where it stands, so keep it
even when it
                // is not the one being looked for.
                state.remember_peer(id, addr);
                state.put_client_session(sess);
                if want.is_none_or(|w| w == id) {
                    return Some((id, addr));
                }
            }
            Err(e) => debug!("sweep {addr}: {e}"),
        }
    }
    None
}

// ──────────────────────────────────── gossip ────────────────────────────────────

/// Seed the peer table by dialing the configured seeds once at startup.
async fn bootstrap_seeds(state: &Shared, client: &reqwest::Client) {
    for addr in &state.cfg.seeds {
        if !peer_ok(addr, state.cfg.allow_public, state.cfg.allow_loopback) {
```

```
            warn!("skipping seed {addr}: address not permitted by policy");
            continue;
        }
        match dial(state, client, *addr).await {
            Ok(sess) => {
                let id = sess.peer_id;
                state.remember_peer(id, *addr);
                // Keep the freshly negotiated session so the first gossip round
reuses it
                // instead of immediately handshaking again.
                state.put_client_session(sess);
                info!("🌱 bootstrapped from seed {addr} ({})", short_id(&id));
            }
            Err(e) => {
                // The seed may have rotated off the port it was configured with.
                match sweep_for_node(state, client, *addr, None).await {
                    Some((id, found)) => info!(
                        "🌱 seed {addr} had moved; bootstrapped from {found} ({})",
                        short_id(&id)
                    ),
                    None => warn!("seed {addr} unreachable: {e}"),
                }
            }
        }
    }
}

/// One gossip round: pick a random known peer, exchange a ping and a peer list.
async fn gossip_round(state: &Shared, client: &reqwest::Client) {
    let candidate = {
        let peers = lock(&state.peers);
        if peers.is_empty() {
            None
        } else {
            let idx = rand_below(peers.len());
            peers.iter().nth(idx).map(|(id, p)| (*id, p.addr))
        }
    };
    let Some((peer_id, addr)) = candidate else {
        // No peers to try. Not a failure — there is nothing to be cut off from.
        state.note_link(false, false);
        return;
    };

    // Never gossip with a revoked identity, even if it is still in the peer table.
    if state.is_revoked(&peer_id) {
        lock(&state.peers).remove(&peer_id);
        lock(&state.client_sessions).remove(&peer_id);
        return;
    }
```

```
    let outcome = gossip_with(state, client, peer_id, addr).await;
    state.note_peer_result(&peer_id, outcome.is_ok());
    state.note_link(true, outcome.is_ok());
    let failed = outcome.is_err();
    if let Err(e) = outcome {
        debug!("gossip with {} failed: {e}", short_id(&peer_id));
    }
    if failed {
        let fails = lock(&state.peers).get(&peer_id).map(|p| p.fails);
        if fails.is_some_and(|f| REDISCOVER_AT_FAILS.contains(&f)) {
            if let Some((_, found)) = sweep_for_node(state, client, addr,
Some(peer_id)).await {
                info!("🔎 {} moved from {addr}; found it at {found}",
short_id(&peer_id));
            }
        }
    }
}

/// Exchange a ping and a peer list with `addr`, reusing a cached session when one is
live.
///
/// A full handshake (ML-KEM encap/decap plus an ML-DSA signature and verification)
costs far
/// more than the exchange itself, so we only pay it when there is no usable session:
on first
/// contact, after `CLIENT_SESSION_MAX_AGE`, after `MAX_FRAMES_PER_SESSION`, or when
the peer
/// has forgotten us. A session is only returned to the cache if the exchange
succeeded, so a
/// peer that restarted (and dropped our session server-side) is re-dialled on the
next round.
async fn gossip_with(
    state: &Shared,
    client: &reqwest::Client,
    peer_id: NodeId,
    addr: SocketAddr,
) -> Result<(), BoxErr> {
    let (mut sess, reused) = match state.take_client_session(&peer_id) {
        Some(s) => (s, true),
        None => (dial(state, client, addr).await?, false),
    };
    state.remember_peer(sess.peer_id, addr);

    let nonce = OsRng.next_u64();
    let ping_msg = state.gossip_ping(&peer_id, nonce);
    let ping = request(client, addr, &mut sess, &ping_msg).await;

    // A cached session the peer no longer recognises (restart, eviction, expiry)
```

```
shows up as
    // an error or a missing pong. Drop it and handshake once more, this round.
    let ping = match ping {
        Ok(Some(Msg::Pong {
            nonce: got,
            vitals,
            acks,
        })) if got == nonce => {
            state.note_pong(&peer_id, vitals, &acks);
            Ok(())
        }
        other => {
            if reused {
                debug!("cached session with {} is stale; re-dialling",
short_id(&peer_id));
                sess = dial(state, client, addr).await?;
                match request(client, addr, &mut sess, &ping_msg).await? {
                    Some(Msg::Pong {
                        nonce: got,
                        vitals,
                        acks,
                    }) if got == nonce => {
                        state.note_pong(&peer_id, vitals, &acks);
                        Ok(())
                    }
                    _ => Err::<(), BoxErr>("no valid pong after re-dial".into()),
                }
            } else {
                match other {
                    Err(e) => Err(e),
                    _ => Err("no valid pong".into()),
                }
            }
        }
    };
    ping?;

    if let Some(Msg::Peers { peers }) = request(client, addr, &mut sess,
&Msg::GetPeers).await? {
        ingest_peer_entries(state, peer_id, &peers);
    }

    state.put_client_session(sess);
    Ok(())
}

async fn gossip_loop(state: Shared, client: reqwest::Client) {
    let mut rx = state.shutdown.subscribe();
    if state.emcon() {
        warn!("🩸 starting under EMCON — no seed bootstrap, no gossip until it is
```

```
lifted");
    } else {
        bootstrap_seeds(&state, &client).await;
    }
    if state.set_ready() {
        let peers = lock(&state.peers).len();
        info!(
            "✅ ready: {} on :{} with {peers} peer(s), build {BUILD_TAG}",
            state.cfg.label, state.bound_port
        );
        state.event("ready", &format!("{} is up with {peers} peer(s)",
state.cfg.label), Some(&state.me.fpr()), None);
    }
    let base = Duration::from_secs(state.cfg.gossip_secs);
    // Gossip rounds in a row with an empty peer table; seeds are retried every few
of them, so
    // a node started before its seed (or cut off from every peer) rejoins on its
own.
    let mut lonely_rounds: u32 = 0;
    loop {
        // Re-jittered every iteration rather than once: a single random offset held
for the
        // lifetime of the process is still a fixed cadence, just a different one.
        let period = jittered(base);
        tokio::select! {
            _ = rx.recv() => {
                info!("gossip loop stopping");
                break;
            }
            _ = tokio::time::sleep(period) => {
                if state.emcon() || state.draining() {
                    // Still expire sessions — that is local bookkeeping, not an
emission.
                    lock(&state.server_sessions).retain(|_, s| !s.expired());
                    continue;
                }
                gossip_round(&state, &client).await;
                if lock(&state.peers).is_empty() && !state.cfg.seeds.is_empty() {
                    if lonely_rounds % RESEED_EVERY_ROUNDS == 0 {
                        bootstrap_seeds(&state, &client).await;
                    }
                    lonely_rounds = lonely_rounds.wrapping_add(1);
                } else {
                    lonely_rounds = 0;
                }
                retry_deferred_advisories(&state);
                lock(&state.server_sessions).retain(|_, s| !s.expired());
            }
        }
    }
```

```
}

// ──────────────────────────────────── radio simulator
──────────────────────────────────
//
// The UDP "radio" now carries the same authenticated frames as /msg, against the
same session
// table. A datagram that is not a valid frame on a known session is dropped silently
— forged
// packets no longer reach the LLM or the peer table.

/// Bind the mesh API, optionally walking up `rotate` ports when the first choice is
taken.
///
/// Rotation is safe here only because the identity directory is chosen from the
*requested*
/// port (or `--node-name`), never the bound one — see `identity_dir_name`. The node
keeps its
/// keys and its peers' trust; only the address it listens on moves. Peers learn the
new port
/// from this node's signed handshake as soon as it dials them, and a node that
cannot reach a
/// peer at its old port sweeps the nearby ports for it (`sweep_for_node`), so
rotation is on by
/// default. A second copy of the same identity never gets this far: see
`lock_identity`.
/// A port someone else has: in use, or on Windows inside a range the system reserved
(Hyper-V and
/// WSL exclude port ranges, and binding there fails with "access denied" instead of
"in use").
fn port_unavailable(e: &std::io::Error) -> bool {
    e.kind() == std::io::ErrorKind::AddrInUse
        || (cfg!(windows) && e.kind() == std::io::ErrorKind::PermissionDenied)
}

async fn bind_tcp(bind: IpAddr, port: u16, rotate: u16) -> Result<(TcpListener, u16),
BoxErr> {
    for off in 0..=rotate {
        let candidate = match port.checked_add(off) {
            Some(p) => p,
            None => break,
        };
        match TcpListener::bind(SocketAddr::new(bind, candidate)).await {
            Ok(l) => {
                let actual = l.local_addr()?.port();
                if off > 0 {
                    warn!(
                        "⚠ TCP {port} was taken; rotated to {actual}. Identity is
unchanged; \
                         peers learn :{actual} from the handshake, and seeds pointing
```

```
at \
                        :{port} find it by sweeping."
                    );
                }
                return Ok((l, actual));
            }
            Err(e) if port_unavailable(&e) => continue,
            Err(e) => return Err(format!("could not bind {bind}:{candidate}:
{e}").into()),
        }
    }
    let tried = if rotate > 0 {
        format!("ports {}..={}", port, port.saturating_add(rotate))
    } else {
        format!("TCP port {port}")
    };
    Err(format!(
        "{tried} already in use, by other programs or by other nodes (a second copy
of \
         this node would have been refused before binding). Free one (`ss -ltnp`
shows who \
         holds them), start with `--port <other>`, or raise `--port-rotate`. Identity
\
         comes from --node-name (or the requested port), so changing the port does
not \
         change this node's keys."
    )
    .into())
}

async fn bind_udp(bind: IpAddr, mut port: u16) -> std::io::Result<(UdpSocket, u16)> {
    for _ in 0..16 {
        match UdpSocket::bind(SocketAddr::new(bind, port)).await {
            Ok(s) => {
                let actual = s.local_addr()?.port();
                return Ok((s, actual));
            }
            Err(e) if port_unavailable(&e) => {
                warn!("UDP {port} in use, trying {}", port + 1);
                port = port.wrapping_add(1);
            }
            Err(e) => return Err(e),
        }
    }
    Err(std::io::Error::new(
        std::io::ErrorKind::AddrInUse,
        format!(
            "no free UDP port in the 16 starting at {port}; pass --udp-port <other> \
             or stop the process holding that range"
        ),
```

```
    ))
}

async fn radio_loop(state: Shared, socket: UdpSocket) {
    let mut rx = state.shutdown.subscribe();
    let inflight = Arc::new(Semaphore::new(RADIO_MAX_INFLIGHT));
    let mut buf = vec![0u8; MAX_UDP_FRAME];
    // Datagram handlers live in a JoinSet rather than being detached, so shutdown
can wait
    // for work that is genuinely in progress (a decrypted directive mid-inference,
say)
    // instead of having it disappear when the process exits.
    let mut tasks: tokio::task::JoinSet<()> = tokio::task::JoinSet::new();
    loop {
        tokio::select! {
            _ = rx.recv() => {
                info!("📻 radio simulator stopping ({} in flight)", tasks.len());
                break;
            }
            // Reap finished handlers so the set does not grow without bound.
            Some(_) = tasks.join_next(), if !tasks.is_empty() => {}
            res = socket.recv_from(&mut buf) => {
                let (n, from) = match res {
                    Ok(v) => v,
                    Err(e) => { debug!("radio recv error: {e}"); continue; }
                };
                // Simulated loss.
                if state.cfg.loss_rate > 0
                    && (rand_below(100) as u32) < state.cfg.loss_rate
                {
                    continue;
                }
                let frame = buf[..n].to_vec();
                let st = state.clone();
                let permit = match inflight.clone().try_acquire_owned() {
                    Ok(p) => p,
                    Err(_) => { debug!("radio overloaded, dropping datagram from
{from}"); continue; }
                };
                let latency = state.cfg.latency_ms;
                tasks.spawn(async move {
                    let _permit = permit;
                    if latency > 0 {
                        tokio::time::sleep(Duration::from_millis(latency)).await;
                    }
                    // Authenticate against the session table; forged frames decrypt
to nothing.
                    if let Some((msg, peer_id, sid, ip, pport)) =
st.open_server_frame(&frame) {
                        if st.is_revoked(&peer_id) {
```

```
                            warn!("⛔ dropping radio frame from revoked {}",
short_id(&peer_id));
                            lock(&st.server_sessions).remove(&sid);
                            return;
                        }
                        let _ = handle_msg(&st, peer_id, SocketAddr::new(ip, pport),
msg).await;
                    } else {
                        debug!("📻 dropped unauthenticated datagram from {from}");
                    }
                });
            }
        }
    }

    // Drain in-flight handlers within the grace period. Anything still running when
the
    // budget expires is aborted; because the LLM child is spawned with kill_on_drop,
that
    // also terminates an inference rather than orphaning it.
    let pending = tasks.len();
    if pending > 0 {
        debug!("📻 draining {pending} in-flight datagram handler(s)");
        let drained = tokio::time::timeout(state.cfg.shutdown_grace, async {
            while tasks.join_next().await.is_some() {}
        })
        .await;
        if drained.is_err() {
            warn!(
                "📻 {} handler(s) still running after {:?}; aborting them",
                tasks.len(),
                state.cfg.shutdown_grace
            );
            tasks.shutdown().await;
        }
    }
    info!("📻 radio simulator stopped");
}

// ──────────────────────────────────── main ────────────────────────────────────

fn build_client() -> Result<reqwest::Client, BoxErr> {
    reqwest::Client::builder()
        .connect_timeout(Duration::from_secs(5))
        .timeout(Duration::from_secs(20))
        .pool_idle_timeout(Duration::from_secs(30))
        .user_agent(concat!("harmony-node/", env!("CARGO_PKG_VERSION")))
        .build()
        .map_err(Into::into)
}
```

```
/// Proof that this process owns its identity; dropping it (or exiting, crash
included) releases
/// the identity. On Linux and Android it is a listening socket in the abstract
namespace, named
/// after the identity directory: the kernel allows one per name and frees it with
the process,
/// so it can never go stale, and it needs no file-locking support from the toolchain
(Rust's
/// `File::try_lock` reports "unsupported" on Android with some releases). Elsewhere
it is an
/// exclusive lock on `node.lock`.
struct IdentityLock {
    #[cfg(any(target_os = "linux", target_os = "android"))]
    _sock: Option<std::os::unix::net::UnixListener>,
    #[cfg(not(any(target_os = "linux", target_os = "android")))]
    _file: std::fs::File,
}

/// Claim `dir`'s identity for the life of the process.
///
/// Rotation is on by default, so without this a second copy of the same node (the
same keys)
/// would quietly take the next port and join the mesh as a duplicate.
fn lock_identity(dir: &Path) -> Result<IdentityLock, BoxErr> {
    std::fs::create_dir_all(dir)?;
    // Canonical, so ~/x and the same directory reached through a symlink are one
identity.
    let dir = std::fs::canonicalize(dir)?;
    let held = || -> BoxErr {
        let holder = std::fs::read_to_string(pid_file(&dir))
            .ok()
            .and_then(|p| p.trim().parse::<u32>().ok())
            .map(|p| format!(" (pid {p})"))
            .unwrap_or_default();
        format!(
            "this node identity ({}) is already running in another process{holder}.
Stop it \
             first (`bash ~/node.sh stop`, or `pkill -x rust_p2p_node`), or give this
instance \
             its own --node-name.",
            dir.display()
        )
        .into()
    };

    #[cfg(any(target_os = "linux", target_os = "android"))]
    {
        #[cfg(target_os = "android")]
        use std::os::android::net::SocketAddrExt;
```

```
        #[cfg(target_os = "linux")]
        use std::os::linux::net::SocketAddrExt;
        use std::os::unix::ffi::OsStrExt;
        use std::os::unix::net::{SocketAddr as UnixAddr, UnixListener};

        let tag = crypto::transcript32(&[b"harmony-identity-lock",
dir.as_os_str().as_bytes()]);
        let name = format!("harmony-node-{}", hex(&tag));
        let addr = UnixAddr::from_abstract_name(name.as_bytes())?;
        match UnixListener::bind_addr(&addr) {
            Ok(sock) => Ok(IdentityLock { _sock: Some(sock) }),
            Err(e) if e.kind() == std::io::ErrorKind::AddrInUse => Err(held()),
            Err(e) => {
                warn!("identity lock unavailable ({e}); a duplicate instance would
not be detected");
                Ok(IdentityLock { _sock: None })
            }
        }
    }

    #[cfg(not(any(target_os = "linux", target_os = "android")))]
    {
        let path = dir.join("node.lock");
        let f = std::fs::OpenOptions::new()
            .create(true)
            .write(true)
            .truncate(false)
            .open(&path)?;
        match f.try_lock() {
            Ok(()) => Ok(IdentityLock { _file: f }),
            Err(std::fs::TryLockError::WouldBlock) => Err(held()),
            Err(std::fs::TryLockError::Error(e)) => {
                warn!("identity lock unavailable ({e}); a duplicate instance would
not be detected");
                Ok(IdentityLock { _file: f })
            }
        }
    }
}

async fn run(cfg: Config) -> Result<(), BoxErr> {
    // One process per identity — before binding, so a duplicate fails instead of
rotating.
    let _identity_lock = lock_identity(&cfg.data_dir)?;
    let pid_path = pid_file(&cfg.data_dir);
    let _ = std::fs::write(&pid_path, format!("{}\n", std::process::id()));
    let me = Identity::load_or_create(&cfg.data_dir)?;
    info!("🛰  node identity {}", hex(&me.fpr()));
    report_model_readiness(&cfg);
```

```
    let mut trust = TrustStore::load(cfg.trust_mode, &cfg.data_dir, &cfg.trusted);
    trust.apply_revocations(&cfg.revoked);
    match cfg.trust_mode {
        TrustMode::Strict if cfg.trusted.is_empty() => {
            warn!("🔒 strict trust with an empty list (--trust / HARMONY_TRUST):
every peer will be refused")
        }
        TrustMode::Strict => info!("🔒 strict trust: {} listed identities, nobody
else", cfg.trusted.len()),
        TrustMode::Tofu => warn!(
            "⚠ TOFU trust: any device that reaches this port and completes a
handshake becomes a \
             trusted peer. Set HARMONY_TRUST_MODE=strict and HARMONY_TRUST=<ids>
(mesh_upgrade.sh does)"
        ),
    }
    if !cfg.advisors.is_empty() {
        info!("🧭 AI placement decisions adopted from {} advisor(s) besides this node
and --cmd-allow", cfg.advisors.len());
    }
    if cfg.admin_token.is_some() {
        warn!("🔑 HARMONY_ADMIN_TOKEN is set: it crosses the network in plain HTTP;
send it only through an SSH tunnel");
    }
    let loaded_revocations = trust.revoked.len();
    let (shutdown, _) = broadcast::channel(4);

    // Bind both listeners *before* building shared state, so the state can record
the port
    // we actually got. Peers are told that port in the handshake; advertising the
requested
    // one after rotation would leave them unable to reach us back.
    let (listener, bound_port) = bind_tcp(cfg.bind, cfg.port,
cfg.port_rotate).await?;
    let (udp, udp_port) = bind_udp(cfg.udp_bind, cfg.udp_port).await?;

    // Read before `cfg` is moved into the state below.
    let cfg_emcon = cfg.emcon;
    let loaded_bundles = load_bundles(&cfg.data_dir.join("bundles"));
    if !loaded_bundles.is_empty() {
        info!("🖨 {} peer bundle(s) loaded from cache", loaded_bundles.len());
    }
    let state: Shared = Arc::new(AppState {
        thermal: ThermalProvider,
        trust: Mutex::new(trust),
        peers: Mutex::new(HashMap::new()),
        server_sessions: Mutex::new(HashMap::new()),
        client_sessions: Mutex::new(HashMap::new()),
        hello_rate: Mutex::new(HashMap::new()),
        msg_rate: Mutex::new(HashMap::new()),
```

```
        seen_hellos: Mutex::new(HashMap::new()),
        handshake_gate: Semaphore::new(MAX_CONCURRENT_HANDSHAKES),
        emcon: AtomicBool::new(cfg_emcon),
        link: Mutex::new(LinkHealth::default()),
        bundles: Mutex::new(loaded_bundles),
        peer_vitals: Mutex::new(HashMap::new()),
        relay: Mutex::new(RelayState::default()),
        placement: Mutex::new(None),
        events: Mutex::new(EventLog::default()),
        phase: AtomicU8::new(PHASE_STARTING),
        stop: tokio::sync::Notify::new(),
        llm_gate: Semaphore::new(1),
        llm_last: Mutex::new(Instant::now() - LLM_COOLDOWN),
        shutdown: shutdown.clone(),
        engine: tokio::sync::Mutex::new(Engine::default()),
        engine_view: Mutex::new(EngineView::at_start(cfg.llama_server.is_some())),
        engine_used: Mutex::new(Instant::now()),
        engine_client: build_engine_client()?,
        bound_port,
        me,
        cfg,
    });
    state.purge_revoked();
    if loaded_revocations > 0 {
        info!("⛔ {loaded_revocations} revoked identity/identities in force");
    }

    info!("📻 radio simulator on udp://{}:{}", state.cfg.udp_bind, udp_port);
    let radio = tokio::spawn(radio_loop(state.clone(), udp));

    let client = build_client()?;
    let gossip = tokio::spawn(gossip_loop(state.clone(), client.clone()));
    // The kept-loaded model: stop one a crashed run left behind, load it now so the
first job does
    // not wait for it, and unload it when it sits idle.
    tokio::spawn(engine_start_up(state.clone()));
    tokio::spawn(engine_idle_loop(state.clone()));

    info!(
        "🚀 Harmony mesh API on http://{}:{}  (suite: ML-KEM-1024 + ML-DSA-87 + AES-
256-GCM + SHA-384)",
        state.cfg.bind, bound_port
    );
    let app = build_router(state.clone()).into_make_service_with_connect_info::
<SocketAddr>();

    // Phase 1: on the first signal, stop taking new jobs and say goodbye to peers;
then stop
    // accepting connections and let in-flight requests finish, but only for the
grace period.
```

```
    let signalled = Arc::new(tokio::sync::Notify::new());
    let drain = {
        let (st, cl, sig) = (state.clone(), client.clone(), signalled.clone());
        async move {
            stop_requested(&st).await;
            sig.notify_one();
            begin_draining(&st, &cl).await;
        }
    };
    let grace = state.cfg.shutdown_grace;
    let server = std::future::IntoFuture::into_future(axum::serve(listener,
app).with_graceful_shutdown(drain));
    tokio::select! {
        r = server => {
            if let Err(e) = r {
                error!("http server error: {e}");
            }
        }
        _ = async {
            signalled.notified().await;
            tokio::time::sleep(FAREWELL_BUDGET + grace).await;
        } => {
            warn!("requests still in flight after {grace:?}; closing them");
        }
    }
    info!("⏳ draining background work (grace {:?})", state.cfg.shutdown_grace);

    // Phase 2: tell the background loops to wind down, then wait for them. Each loop
drains
    // its own in-flight work inside the same budget, so this bound is on the whole
shutdown.
    let _ = shutdown.send(());
    let drained = tokio::time::timeout(state.cfg.shutdown_grace, async {
        let _ = gossip.await;
        let _ = radio.await;
    })
    .await;
    if drained.is_err() {
        warn!(
            "background tasks did not finish within {:?}; exiting anyway",
            state.cfg.shutdown_grace
        );
    }

    // Phase 3: drop cached key material explicitly rather than relying on process
exit.
    // `release` builds use panic=abort, so a panic skips this entirely — these
zeroize on
    // drop, which is a best-effort nicety, not a guarantee.
    lock(&state.client_sessions).clear();
```

```
    lock(&state.server_sessions).clear();
    // The model server goes with the node (after a crash, its pid file lets the next
start stop it).
    let _ = tokio::time::timeout(Duration::from_secs(5), engine_stop(&state)).await;
    let _ = std::fs::remove_file(&pid_path);

    info!("👋 shutdown complete");
    Ok(())
}

// ──────────────────────────────────── graceful shutdown
────────────────────────────────────

/// First shutdown signal or request: stop taking new jobs, tell peers, and arm a
second-signal
/// escape.
async fn begin_draining(st: &Shared, client: &reqwest::Client) {
    st.phase.store(PHASE_DRAINING, Ordering::SeqCst);
    info!("🛑 draining: new jobs refused, telling peers this node is leaving");
    st.event("leaving", "shutting down: new jobs refused, saying goodbye to peers",
Some(&st.me.fpr()), None);
    let again = Arc::clone(st);
    tokio::spawn(async move {
        stop_requested(&again).await;
        warn!("second shutdown signal or request: exiting now without finishing in-
flight work");
        std::process::exit(130);
    });
    if st.emcon() {
        info!("🔌 EMCON: leaving without a goodbye (no emissions)");
        return;
    }
    say_goodbye(st, client).await;
}

/// Tell every known peer this node is leaving, concurrently and within
FAREWELL_BUDGET.
async fn say_goodbye(st: &Shared, client: &reqwest::Client) {
    let peers: Vec<(NodeId, SocketAddr)> = lock(&st.peers).iter().map(|(id, p)| (*id,
p.addr)).collect();
    if peers.is_empty() {
        return;
    }
    let total = peers.len();
    let mut tasks = tokio::task::JoinSet::new();
    for (id, addr) in peers {
        let st = Arc::clone(st);
        let client = client.clone();
        tasks.spawn(async move {
            let mut sess = match st.take_client_session(&id) {
```

```
                Some(s) => s,
                None => dial(&st, &client, addr).await.ok()?,
            };
            request_with_timeout(&client, addr, &mut sess, &Msg::Bye,
FAREWELL_BUDGET).await.ok()?;
            Some(id)
        });
    }
    let mut told = 0usize;
    let _ = tokio::time::timeout(FAREWELL_BUDGET, async {
        while let Some(r) = tasks.join_next().await {
            if matches!(r, Ok(Some(_))) {
                told += 1;
            }
        }
    })
    .await;
    tasks.abort_all();
    info!("👋 told {told}/{total} peer(s) this node is leaving");
}

/// Where this identity's pid file lives; it names the holder in "already running"
errors.
fn pid_file(dir: &Path) -> PathBuf {
    dir.join("node.pid")
}

/// The first of SIGTERM or Ctrl+C, and POST /shutdown on the operator API.
async fn stop_requested(st: &AppState) {
    tokio::select! {
        _ = shutdown_signal() => {},
        _ = st.stop.notified() => info!("🛑 shutdown requested over the operator
API"),
    }
}

async fn shutdown_signal() {
    let ctrl_c = async {
        // A process that cannot register for Ctrl+C (no console, on Windows) must
not read that
        // as "Ctrl+C pressed", or it would shut down as soon as it started.
        if tokio::signal::ctrl_c().await.is_err() {
            std::future::pending::<()>().await;
        }
    };
    #[cfg(unix)]
    let terminate = async {
        if let Ok(mut s) =
            tokio::signal::unix::signal(tokio::signal::unix::SignalKind::terminate())
        {
```

```
            s.recv().await;
        }
    };
    #[cfg(not(unix))]
    let terminate = std::future::pending::<()>();

    tokio::select! {
        _ = ctrl_c => {},
        _ = terminate => {},
    }
    info!("🛑 shutdown signal received");
}

// ———————————————————————————— signed updates
————————————————————————————
//
// mesh_upgrade.sh has the laptop build the Windows notebook's node, and the notebook
installs it by
// itself. The laptop is trusted to build, not to release: the phone signs a manifest
of exactly the
// staged files (`--sign-update`), and the notebook checks it with the node it
already runs
// (`--verify-update DIR --update-signer <phone id>`) before installing anything. A
file changed,
// added or removed after signing, a BUILD that disagrees with the manifest, or a
manifest signed by
// anyone but the pinned signer is refused, and `issued` lets the notebook refuse a
rollback to an
// older signed build. What this cannot catch is a build host that was already
compromised when the
// phone signed; that would take reproducible builds.

const UPDATE_HEADER: &str = "harmony-update 1";
const UPDATE_MANIFEST: &str = "MANIFEST";
const UPDATE_SIG: &str = "MANIFEST.sig";
/// Signing context, so an update signature can never pass for a directive, a bundle
or a hello.
const CTX_UPDATE: &[u8] = b"harmony-v3/update";
/// What a staging directory may hold besides the signed files: the manifest, its
signature, and
/// DEPLOYED, which the notebook writes back on the laptop after installing.
const UPDATE_UNSIGNED: [&str; 3] = [UPDATE_MANIFEST, UPDATE_SIG, "DEPLOYED"];
/// A staged build is a handful of files (the node, its DLLs, node.ps1, BUILD,
mesh.env).
const UPDATE_MAX_FILES: usize = 64;

#[derive(Debug, Clone, PartialEq)]
struct UpdateFile {
    name: String,
    size: u64,
```

```
    /// SHA-384, lowercase hex.
    sha384: String,
}

#[derive(Debug, Clone, PartialEq)]
struct UpdateManifest {
    build: String,
    issued: u64,
    signer: NodeId,
    /// The signer's ML-DSA-87 verifying key; its fingerprint must be `signer`.
    vk: Vec<u8>,
    /// Sorted by name, no repeats.
    files: Vec<UpdateFile>,
}

/// A staged file's name: letters, digits, `.`, `_` and `-`, starting with a letter
or digit. No
/// paths, no hidden files, nothing a shell or Windows reads specially.
fn update_name_ok(n: &str) -> bool {
    !n.is_empty()
        && n.len() <= 64
        && n.as_bytes()[0].is_ascii_alphanumeric()
        && n.bytes().all(|b| b.is_ascii_alphanumeric() || matches!(b, b'.' | b'_' |
b'-'))
        && !n.contains("..")
}

/// A build tag as mesh_upgrade.sh writes it: `harmony-v` and a number.
fn update_build_ok(b: &str) -> bool {
    b.strip_prefix("harmony-v")
        .is_some_and(|d| !d.is_empty() && d.len() <= 6 && d.bytes().all(|c|
c.is_ascii_digit()))
}

fn is_lower_hex(s: &str, len: usize) -> bool {
    s.len() == len && s.bytes().all(|b| b.is_ascii_digit() ||
(b'a'..=b'f').contains(&b))
}

fn is_decimal(s: &str) -> bool {
    !s.is_empty() && s.len() <= 20 && s.bytes().all(|b| b.is_ascii_digit())
}

fn manifest_text(m: &UpdateManifest) -> String {
    use std::fmt::Write as _;
    let mut t = String::new();
    let _ = writeln!(t, "{UPDATE_HEADER}");
    let _ = writeln!(t, "build {}", m.build);
    let _ = writeln!(t, "issued {}", m.issued);
    let _ = writeln!(t, "signer {}", hex(&m.signer));
```

```
    let _ = writeln!(t, "vk {}", b64::encode(&m.vk));
    for f in &m.files {
        let _ = writeln!(t, "file {} {} {}", f.sha384, f.size, f.name);
    }
    t
}

/// Parse a manifest strictly: every line in its place and in canonical form, file
names sorted and
/// unique, BUILD among them, a newline at the end and nothing after it.
fn parse_manifest(text: &str) -> Result<UpdateManifest, String> {
    let mut lines = text.split('\n');
    if lines.next() != Some(UPDATE_HEADER) {
        return Err(format!("not a {UPDATE_HEADER} manifest"));
    }
    let mut field = |key: &str| -> Result<String, String> {
        lines
            .next()
            .and_then(|l| l.strip_prefix(key))
            .and_then(|v| v.strip_prefix(' '))
            .map(str::to_string)
            .ok_or_else(|| format!("manifest: expected a {key} line"))
    };
    let build = field("build")?;
    let issued_s = field("issued")?;
    let signer_s = field("signer")?;
    let vk_s = field("vk")?;
    if !update_build_ok(&build) {
        return Err(format!("manifest: {build:?} is not a build tag"));
    }
    if !is_decimal(&issued_s) {
        return Err("manifest: bad issued time".into());
    }
    let issued: u64 = issued_s.parse().map_err(|_| "manifest: bad issued
time".to_string())?;
    if !is_lower_hex(&signer_s, 64) {
        return Err("manifest: bad signer".into());
    }
    let signer = unhex32(&signer_s).ok_or("manifest: bad signer")?;
    let vk = b64::decode(&vk_s)
        .filter(|k| k.len() == SIG_PK_LEN)
        .ok_or("manifest: bad key")?;
    let rest: Vec<&str> = lines.collect();
    let Some((last, file_lines)) = rest.split_last() else {
        return Err("manifest: no files".into());
    };
    if !last.is_empty() {
        return Err("manifest: it must end with a newline".into());
    }
    let mut files: Vec<UpdateFile> = Vec::new();
```

```
    for line in file_lines {
        let mut p = line.splitn(4, ' ');
        let (Some("file"), Some(h), Some(sz), Some(name)) = (p.next(), p.next(),
p.next(), p.next()) else {
            return Err(format!("manifest: bad line {:?}", truncate(line, 80)));
        };
        if !is_lower_hex(h, 96) || !is_decimal(sz) || !update_name_ok(name) {
            return Err(format!("manifest: bad line {:?}", truncate(line, 80)));
        }
        let size: u64 = sz.parse().map_err(|_| "manifest: bad size".to_string())?;
        if files.last().is_some_and(|f| f.name.as_str() >= name) {
            return Err("manifest: file names out of order or repeated".into());
        }
        files.push(UpdateFile { name: name.to_string(), size, sha384: h.to_string()
});
    }
    if files.is_empty() || files.len() > UPDATE_MAX_FILES || !files.iter().any(|f|
f.name == "BUILD") {
        return Err("manifest: no BUILD, or no files, or too many".into());
    }
    Ok(UpdateManifest { build, issued, signer, vk, files })
}

/// The files of a staged build, sorted: every regular file except the unsigned ones.
A
/// subdirectory, a symlink or an odd name is an error: what is installed must be
exactly what was
/// signed, and nothing may ride along unsigned (a DLL next to the node would be
loaded by it).
fn staged_files(dir: &Path) -> Result<Vec<String>, String> {
    let rd = std::fs::read_dir(dir).map_err(|e| format!("cannot read {}: {e}",
dir.display()))?;
    let mut names = Vec::new();
    for entry in rd {
        let entry = entry.map_err(|e| format!("cannot read {}: {e}",
dir.display()))?;
        let name = entry
            .file_name()
            .into_string()
            .map_err(|n| format!("unexpected entry in the staged build: {n:?}"))?;
        if UPDATE_UNSIGNED.contains(&name.as_str()) {
            continue;
        }
        let is_file = entry.file_type().map(|t| t.is_file()).unwrap_or(false);
        if !is_file || !update_name_ok(&name) {
            return Err(format!("unexpected entry in the staged build: {name}"));
        }
        names.push(name);
    }
    names.sort();
```

```
    if names.len() > UPDATE_MAX_FILES {
        return Err(format!("{} files staged; at most {UPDATE_MAX_FILES}",
names.len()));
    }
    Ok(names)
}

fn hash_staged(dir: &Path, name: &str) -> Result<(u64, String), String> {
    crypto::sha384_file(&dir.join(name))
        .map(|(size, d)| (size, hex(&d)))
        .map_err(|e| format!("{name}: {e}"))
}

fn update_sig_input(manifest: &[u8]) -> Vec<u8> {
    let mut m = Vec::with_capacity(CTX_UPDATE.len() + manifest.len());
    m.extend_from_slice(CTX_UPDATE);
    m.extend_from_slice(manifest);
    m
}

/// `--sign-update DIR`: hash every staged file, sign the manifest with this node's
identity, and
/// write MANIFEST and MANIFEST.sig into DIR.
fn sign_update_dir(me: &Identity, dir: &Path, issued: u64) -> Result<UpdateManifest,
String> {
    let build = std::fs::read_to_string(dir.join("BUILD"))
        .map_err(|e| format!("no BUILD in {}: {e}", dir.display()))?
        .trim()
        .to_string();
    if !update_build_ok(&build) {
        return Err(format!("BUILD holds {build:?}, not a build tag"));
    }
    let mut files = Vec::new();
    for name in staged_files(dir)? {
        let (size, sha384) = hash_staged(dir, &name)?;
        files.push(UpdateFile { name, size, sha384 });
    }
    let m = UpdateManifest {
        build,
        issued,
        signer: me.fpr(),
        vk: me.vk_bytes().to_vec(),
        files,
    };
    let text = manifest_text(&m);
    // What is signed must read back as the same manifest, or every check would
refuse it.
    match parse_manifest(&text) {
        Ok(back) if back == m => {}
        _ => return Err("internal: the manifest does not read back".into()),
```

```
    }
    let sig = me.id.sign(&update_sig_input(text.as_bytes()));
    std::fs::write(dir.join(UPDATE_MANIFEST), text.as_bytes())
        .map_err(|e| format!("write {UPDATE_MANIFEST}: {e}"))?;
    std::fs::write(dir.join(UPDATE_SIG), format!("{}\n", b64::encode(&sig)))
        .map_err(|e| format!("write {UPDATE_SIG}: {e}"))?;
    Ok(m)
}

/// `--verify-update DIR --update-signer ID`: the staged build in DIR is exactly what
ID signed.
fn verify_update_dir(dir: &Path, signer: &NodeId) -> Result<UpdateManifest, String> {
    let raw = std::fs::read(dir.join(UPDATE_MANIFEST)).map_err(|e| format!("no
{UPDATE_MANIFEST}: {e}"))?;
    if raw.len() > 64 * 1024 {
        return Err("manifest too large".into());
    }
    let text = std::str::from_utf8(&raw).map_err(|_| "the manifest is not
text".to_string())?;
    let m = parse_manifest(text)?;
    if &m.signer != signer {
        return Err(format!(
            "signed by {}, not by the pinned signer {}",
            short_id(&m.signer),
            short_id(signer)
        ));
    }
    if crypto::fingerprint(&m.vk) != m.signer {
        return Err("the manifest's key is not its signer's".into());
    }
    let sig_text = std::fs::read_to_string(dir.join(UPDATE_SIG)).map_err(|e| format!
("no {UPDATE_SIG}: {e}"))?;
    let sig = b64::decode(sig_text.trim()).ok_or("the signature is not base64")?;
    if !crypto::verify(&m.vk, &update_sig_input(&raw), &sig) {
        return Err("bad signature: the manifest was changed, or this key did not sign
it".into());
    }
    let staged = staged_files(dir)?;
    let listed: Vec<&str> = m.files.iter().map(|f| f.name.as_str()).collect();
    if let Some(extra) = staged.iter().find(|n| !listed.contains(&n.as_str())) {
        return Err(format!("{extra} is not in the signed manifest"));
    }
    if let Some(missing) = listed.iter().find(|n| !staged.iter().any(|s| s.as_str()
== **n)) {
        return Err(format!("{missing} is in the signed manifest but not staged"));
    }
    for f in &m.files {
        let (size, sha384) = hash_staged(dir, &f.name)?;
        if size != f.size || sha384 != f.sha384 {
            return Err(format!("{} does not match the signed manifest", f.name));
```

```
        }
    }
    let build = std::fs::read_to_string(dir.join("BUILD")).unwrap_or_default();
    if build.trim() != m.build {
        return Err(format!("BUILD says {:?}, the manifest {:?}", build.trim(),
m.build));
    }
    Ok(m)
}

fn main() -> Result<(), BoxErr> {
    tracing_subscriber::fmt()
        .with_env_filter(
            tracing_subscriber::EnvFilter::try_from_default_env()
                .unwrap_or_else(|_| "info".into()),
        )
        .init();

    let cli = Cli::parse();

    // Fuzzing runs synchronously and exits non-zero on any finding, so it is usable
in CI
    // as well as by hand.
    #[cfg(feature = "fuzz")]
    if let Some(target) = cli.fuzz {
        let reports = fuzz::run(target, cli.fuzz_iters, cli.fuzz_seed,
cli.fuzz_out.clone());
        if !fuzz::print_summary(&reports, cli.fuzz_seed) {
            std::process::exit(1);
        }
        return Ok(());
    }

    // Maintenance modes run synchronously, without standing up the runtime.
    if cli.print_id || cli.rotate_kem {
        let base = cli.data_dir.clone().unwrap_or_else(default_data_base);
        // The directory the node itself uses, so --node-name is honoured here too.
        let dir = base.join(identity_dir_name(&cli.node_name, cli.port));
        let me = if cli.rotate_kem {
            // Never rewrite keys under a running node.
            let _lock = lock_identity(&dir)?;
            Identity::rotate_kem(&dir)?
        } else {
            Identity::load_or_create(&dir)?
        };
        println!("{}", hex(&me.fpr()));
        return Ok(());
    }
    if let Some(dir) = cli.sign_update.clone() {
        let base = cli.data_dir.clone().unwrap_or_else(default_data_base);
```

```
        let id_dir = base.join(identity_dir_name(&cli.node_name, cli.port));
        // Never sign with a key made up on the spot: a missing identity is an error
here.
        if !id_dir.join("identity.mldsa87.seed").exists() {
            println!("FAIL no node identity in {} to sign with", id_dir.display());
            std::process::exit(1);
        }
        let me = Identity::load_or_create(&id_dir)?;
        match sign_update_dir(&me, &dir, now_unix()) {
            Ok(m) => {
                println!("signed {} file(s) of {} as {}", m.files.len(), m.build,
hex(&me.fpr()));
                return Ok(());
            }
            Err(e) => {
                println!("FAIL {e}");
                std::process::exit(1);
            }
        }
    }
    if let Some(dir) = cli.verify_update.clone() {
        let Some(signer) = cli.update_signer.as_deref().and_then(unhex32) else {
            println!("FAIL --verify-update needs --update-signer <64 hex>");
            std::process::exit(2);
        };
        match verify_update_dir(&dir, &signer) {
            Ok(m) => {
                println!(
                    "OK build={} issued={} files={} signer={}",
                    m.build,
                    m.issued,
                    m.files.len(),
                    short_id(&m.signer)
                );
                return Ok(());
            }
            Err(e) => {
                println!("FAIL {e}");
                std::process::exit(1);
            }
        }
    }

    let cfg = Config::from_cli(cli)?;
    let runtime = tokio::runtime::Builder::new_multi_thread()
        .enable_all()
        .build()?;
    runtime.block_on(run(cfg))
}
```

```
// ──────────────────────────────────── fuzzing ────────────────────────────────────
//
// Built into the binary behind `--features fuzz` rather than as a `cargo-fuzz`
target,
// because this crate is a `[[bin]]` with no lib for libFuzzer to link against, and
// cargo-fuzz needs a nightly toolchain and the LLVM sanitizer runtime — neither of
which
// Termux provides. This runs on stable, on the device, against the real code paths.
//
// The point is not only "does it panic". A parser that survives garbage but
*accepts* a
// forged bundle or a forged hello is worse than one that crashes, because nothing
visible
// goes wrong. So every target asserts structural invariants on anything it accepts,
and
// any acceptance of a mutated input that is not byte-identical to a known-good seed
is
// reported as a forgery, which is a hard failure.

#[cfg(feature = "fuzz")]
mod fuzz {
    use super::*;
    use std::panic::{catch_unwind, AssertUnwindSafe};

    #[derive(Clone, Copy, Debug, PartialEq, Eq, ValueEnum)]
    pub enum Target {
        /// parse_frame — the first thing unauthenticated network bytes touch
        Frame,
        /// crypto::parse_bundle — signature verification on an identity from the
wire
        Bundle,
        /// server_verify_hello — decapsulation, AEAD and ML-DSA on an
unauthenticated body
        Hello,
        /// pad_payload / unpad_payload — the length prefix is attacker-influenced
        Pad,
        /// ReplayWindow — arithmetic on counters an attacker picks
        Window,
        All,
    }

    impl Target {
        fn each() -> &'static [Target] {
            &[Target::Frame, Target::Bundle, Target::Hello, Target::Pad,
Target::Window]
        }
        /// Whether accepting a mutated input is by itself a failure.
        ///
        /// `parse_frame` is a pure structural parser — authentication happens later,
```

```
in
        /// `aead_open` — so it is *supposed* to accept almost any well-sized buffer,
and
        /// counting that as a forgery would bury the real signal under a hundred
thousand
        /// false positives. `parse_bundle` and `server_verify_hello` both verify a
signature,
        /// so for those an acceptance that is not a known-good seed is exactly the
bug worth
        /// looking for.
        fn authenticating(self) -> bool {
            matches!(self, Target::Bundle | Target::Hello)
        }

        fn label(self) -> &'static str {
            match self {
                Target::Frame => "frame",
                Target::Bundle => "bundle",
                Target::Hello => "hello",
                Target::Pad => "pad",
                Target::Window => "window",
                Target::All => "all",
            }
        }
    }

    /// xorshift64*. Deterministic and seedable so a finding can be reproduced
exactly; not
    /// cryptographic, and does not need to be — it is driving mutations, not keys.
    struct Rng(u64);

    impl Rng {
        fn new(seed: u64) -> Self {
            Self(if seed == 0 { 0x9E3779B97F4A7C15 } else { seed })
        }
        fn next(&mut self) -> u64 {
            let mut x = self.0;
            x ^= x >> 12;
            x ^= x << 25;
            x ^= x >> 27;
            self.0 = x;
            x.wrapping_mul(0x2545F4914F6CDD1D)
        }
        fn below(&mut self, n: usize) -> usize {
            if n <= 1 { 0 } else { (self.next() % n as u64) as usize }
        }
        fn byte(&mut self) -> u8 {
            (self.next() >> 24) as u8
        }
        fn fill(&mut self, out: &mut [u8]) {
```

```
            for b in out.iter_mut() {
                *b = self.byte();
            }
        }
    }

    /// Mutation strategies, weighted by how often each has historically found parser
bugs:
    /// small edits near structure (length fields, offsets) beat uniform randomness,
which
    /// mostly gets rejected by the first length check and exercises nothing.
    fn mutate(rng: &mut Rng, corpus: &[Vec<u8>]) -> Vec<u8> {
        let base = &corpus[rng.below(corpus.len())];
        let mut v = base.clone();
        match rng.below(10) {
            0 => {
                // Entirely random, random length. Cheap, and covers the "not even
close" case.
                let n = rng.below(MAX_FRAME_LEN + 64);
                v = vec![0u8; n];
                rng.fill(&mut v);
            }
            1 => {
                // Bit flips — the classic, and what catches off-by-one bounds
checks.
                for _ in 0..=rng.below(8) {
                    if v.is_empty() { break; }
                    let i = rng.below(v.len());
                    v[i] ^= 1u8 << rng.below(8);
                }
            }
            2 => {
                for _ in 0..=rng.below(4) {
                    if v.is_empty() { break; }
                    let i = rng.below(v.len());
                    v[i] = rng.byte();
                }
            }
            3 => {
                // Truncation. Every `&buf[a..b]` is a potential panic here.
                if !v.is_empty() { v.truncate(rng.below(v.len())); }
            }
            4 => {
                let extra = rng.below(256);
                let mut tail = vec![0u8; extra];
                rng.fill(&mut tail);
                v.extend_from_slice(&tail);
            }
            5 => {
                // Splice two corpus entries: produces structurally plausible
```

```
nonsense.
                let other = &corpus[rng.below(corpus.len())];
                if !v.is_empty() && !other.is_empty() {
                    let cut = rng.below(v.len());
                    let from = rng.below(other.len());
                    v.truncate(cut);
                    v.extend_from_slice(&other[from..]);
                }
            }
            6 => {
                // Interesting byte values at interesting places.
                const EDGE: [u8; 6] = [0x00, 0x01, 0x7f, 0x80, 0xfe, 0xff];
                for _ in 0..=rng.below(16) {
                    if v.is_empty() { break; }
                    let i = rng.below(v.len());
                    v[i] = EDGE[rng.below(EDGE.len())];
                }
            }
            7 => {
                // Overwrite the leading bytes — where every length prefix and
version byte
                // in this protocol lives.
                let n = (1 + rng.below(4)).min(v.len());
                for b in v.iter_mut().take(n) { *b = rng.byte(); }
            }
            8 => {
                // Maximal length prefixes specifically: unpad_payload reads a u16
here.
                if v.len() >= 2 { v[0] = 0xff; v[1] = 0xff; }
            }
            _ => {
                // Duplicate a chunk, which shifts every later offset.
                if !v.is_empty() {
                    let a = rng.below(v.len());
                    let b = (a + 1 + rng.below(v.len() - a)).min(v.len());
                    let chunk = v[a..b].to_vec();
                    v.extend_from_slice(&chunk);
                }
            }
        }
        v
    }

    #[derive(Default)]
    pub struct Report {
        pub target: &'static str,
        pub cases: u64,
        /// Inputs the parser accepted. Expected to be ~0 for mutated input.
        pub accepted: u64,
        pub panics: u64,
```

```
        /// Accepted an input that was not a known-good seed — i.e. a forgery.
        pub forgeries: u64,
        /// Accepted an input whose structure violates what the type promises.
        pub broken_invariants: u64,
        pub elapsed: Duration,
    }

    impl Report {
        fn failed(&self) -> bool {
            self.panics > 0 || self.forgeries > 0 || self.broken_invariants > 0
        }
    }

    struct Ctx {
        out: PathBuf,
        saved: usize,
    }

    impl Ctx {
        /// Persist a finding so it can be replayed later. Capped, because a
systematic bug
        /// would otherwise write a hundred thousand near-identical files.
        fn save(&mut self, target: &str, kind: &str, input: &[u8]) {
            if self.saved >= 32 {
                return;
            }
            if std::fs::create_dir_all(&self.out).is_err() {
                return;
            }
            let digest = crypto::transcript32(&[input]);
            let path = self
                .out
                .join(format!("{target}-{kind}-{}.bin", hex(&digest[..8])));
            if std::fs::write(&path, input).is_ok() {
                self.saved += 1;
                eprintln!("    saved {}", path.display());
            }
        }
    }

    /// Run `body` with panics caught and the panic hook silenced, so one crash is a
data
    /// point rather than the end of the run.
    fn guarded<F: FnOnce() -> Outcome>(body: F) -> Option<Outcome> {
        catch_unwind(AssertUnwindSafe(body)).ok()
    }

    enum Outcome {
        Rejected,
        Accepted,
```

```
        BrokenInvariant(&'static str),
    }

    fn is_seed(input: &[u8], corpus: &[Vec<u8>]) -> bool {
        corpus.iter().any(|c| c.as_slice() == input)
    }

    // ── corpora ──

    fn frame_corpus() -> Vec<Vec<u8>> {
        let key = [0x5au8; 32];
        let sid = [0x11u8; 8];
        let mut out = Vec::new();
        for msg in [Msg::Ping { nonce: 1, vitals: None, carry: None, acks: Vec::new()
}, Msg::GetPeers] {
            if let Some(f) = seal_msg(&key, &sid, 0, &msg) {
                out.push(f);
            }
        }
        // A frame at every bucket boundary, so truncation lands on real edges.
        for b in PAD_BUCKETS {
            out.push(vec![0x42u8; b + FRAME_HDR_LEN + TAG_LEN]);
        }
        out.push(vec![0u8; FRAME_HDR_LEN + TAG_LEN]);
        out
    }

    fn bundle_corpus() -> Vec<Vec<u8>> {
        (0..3).map(|_| Identity::ephemeral().bundle.clone()).collect()
    }

    fn hello_corpus(responder: &crypto::PeerBundle) -> Vec<Vec<u8>> {
        let mut out = Vec::new();
        for port in [3005u16, 65535] {
            let me = Identity::ephemeral();
            if let Some(ch) = client_build_hello(&me, responder, port, now_unix()) {
                out.push(ch.hello1);
            }
        }
        out.push(vec![PROTO_VER; HELLO1_LEN]);
        out
    }

    // ── targets ──

    fn check_frame(input: &[u8]) -> Outcome {
        match parse_frame(input) {
            None => Outcome::Rejected,
            Some((sid, ctr, hdr, ct)) => {
                if input.len() > MAX_FRAME_LEN {
```

```
                    return Outcome::BrokenInvariant("accepted a frame over
MAX_FRAME_LEN");
                }
                if hdr.len() != FRAME_HDR_LEN || hdr != &input[..FRAME_HDR_LEN] {
                    return Outcome::BrokenInvariant("header is not the first 16
bytes");
                }
                if ct.len() != input.len() - FRAME_HDR_LEN || ct.len() < TAG_LEN {
                    return Outcome::BrokenInvariant("ciphertext length
inconsistent");
                }
                if sid != input[..8] {
                    return Outcome::BrokenInvariant("session id does not match the
header");
                }
                let mut c = [0u8; 8];
                c.copy_from_slice(&input[8..16]);
                if ctr != u64::from_be_bytes(c) {
                    return Outcome::BrokenInvariant("counter does not match the
header");
                }
                Outcome::Accepted
            }
        }
    }

    fn check_bundle(input: &[u8]) -> Outcome {
        match crypto::parse_bundle(input) {
            None => Outcome::Rejected,
            Some(b) => {
                if input.len() != BUNDLE_LEN {
                    return Outcome::BrokenInvariant("accepted a bundle of the wrong
length");
                }
                if b.vk.len() != SIG_PK_LEN || b.ek.len() != KEM_EK_LEN {
                    return Outcome::BrokenInvariant("bundle field lengths wrong");
                }
                if b.fpr != crypto::fingerprint(&b.vk) {
                    return Outcome::BrokenInvariant("fingerprint does not match the
key");
                }
                if b.vk != input[..SIG_PK_LEN] || b.ek !=
input[SIG_PK_LEN..SIG_PK_LEN + KEM_EK_LEN] {
                    return Outcome::BrokenInvariant("parsed fields do not match the
input");
                }
                Outcome::Accepted
            }
        }
    }
```

```
    fn check_hello(me: &Identity, input: &[u8]) -> Outcome {
        match server_verify_hello(me, input, now_unix()) {
            Err(_) => Outcome::Rejected,
            Ok(v) => {
                if input.len() != HELLO1_LEN {
                    return Outcome::BrokenInvariant("accepted a hello of the wrong
length");
                }
                if v.init_fpr == [0u8; 32] {
                    return Outcome::BrokenInvariant("accepted an all-zero initiator
identity");
                }
                Outcome::Accepted
            }
        }
    }

    /// Padding is checked as a round trip rather than against a corpus: the property
that
    /// matters is that anything padded comes back byte-identical, and that garbage
never
    /// panics or escapes its buffer.
    fn check_pad(rng: &mut Rng) -> Outcome {
        let n = rng.below(5000);
        let mut payload = vec![0u8; n];
        rng.fill(&mut payload);

        if let Some(padded) = pad_payload(&payload, &PAD_BUCKETS) {
            if padded.len() < payload.len() + PAD_PREFIX {
                return Outcome::BrokenInvariant("padded output smaller than its
input");
            }
            match unpad_payload(&padded) {
                Some(back) if back == payload.as_slice() => {}
                _ => return Outcome::BrokenInvariant("pad/unpad round trip lost
data"),
            }
        }

        // Now garbage: unpad must never panic and never hand back out-of-bounds
data.
        let m = rng.below(2048);
        let mut junk = vec![0u8; m];
        rng.fill(&mut junk);
        if let Some(out) = unpad_payload(&junk) {
            if out.len() + PAD_PREFIX > junk.len() {
                return Outcome::BrokenInvariant("unpad returned more than it was
given");
            }
```

```
        }
        Outcome::Rejected
    }

    /// The replay window is pure counter arithmetic on values an attacker chooses,
including
    /// u64::MAX and 0. A wrong shift here silently disables replay protection.
    fn check_window(rng: &mut Rng) -> Outcome {
        let mut w = ReplayWindow::default();
        for _ in 0..32 {
            let ctr = match rng.below(4) {
                0 => rng.next(),
                1 => u64::MAX - rng.below(4) as u64,
                2 => rng.below(128) as u64,
                _ => rng.next() % 1024,
            };
            if w.check(ctr) {
                w.update(ctr);
                // Immediately replaying what was just accepted must always be
refused.
                if w.check(ctr) {
                    return Outcome::BrokenInvariant("accepted an immediate replay");
                }
            }
        }
        Outcome::Rejected
    }

    pub fn run(target: Target, iters: u64, seed: u64, out: PathBuf) -> Vec<Report> {
        let targets: Vec<Target> = if target == Target::All {
            Target::each().to_vec()
        } else {
            vec![target]
        };

        // Silence the panic hook: a systematic crash would otherwise print a
backtrace per
        // iteration and bury the summary.
        let prior = std::panic::take_hook();
        std::panic::set_hook(Box::new(|_| {}));

        let mut ctx = Ctx { out, saved: 0 };
        let mut reports = Vec::new();

        for t in targets {
            eprintln!("==> fuzzing {} ({iters} cases)", t.label());
            let mut rng = Rng::new(seed ^ (t as
u64).wrapping_mul(0x9E3779B97F4A7C15));
            let mut rep = Report { target: t.label(), ..Default::default() };
            let started = Instant::now();
```

```
            let responder_id = Identity::ephemeral();
            let responder = crypto::parse_bundle(&responder_id.bundle)
                .expect("a freshly built bundle must verify");
            let corpus = match t {
                Target::Frame => frame_corpus(),
                Target::Bundle => bundle_corpus(),
                Target::Hello => hello_corpus(&responder),
                _ => vec![vec![0u8; 1]],
            };

            for _ in 0..iters {
                rep.cases += 1;
                let input = mutate(&mut rng, &corpus);
                let outcome = match t {
                    Target::Frame => guarded(|| check_frame(&input)),
                    Target::Bundle => guarded(|| check_bundle(&input)),
                    Target::Hello => guarded(|| check_hello(&responder_id, &input)),
                    Target::Pad => guarded(|| check_pad(&mut rng)),
                    Target::Window => guarded(|| check_window(&mut rng)),
                    Target::All => unreachable!("expanded above"),
                };
                match outcome {
                    None => {
                        rep.panics += 1;
                        eprintln!("  !! PANIC on a {}-byte input", input.len());
                        ctx.save(t.label(), "panic", &input);
                    }
                    Some(Outcome::BrokenInvariant(why)) => {
                        rep.broken_invariants += 1;
                        eprintln!("  !! INVARIANT: {why}");
                        ctx.save(t.label(), "invariant", &input);
                    }
                    Some(Outcome::Accepted) => {
                        rep.accepted += 1;
                        if t.authenticating() && !is_seed(&input, &corpus) {
                            rep.forgeries += 1;
                            eprintln!(
                                "  !! FORGERY: {} accepted a mutated {}-byte input",
                                t.label(),
                                input.len()
                            );
                            ctx.save(t.label(), "forgery", &input);
                        }
                    }
                    Some(Outcome::Rejected) => {}
                }
            }

            rep.elapsed = started.elapsed();
```

```
            reports.push(rep);
        }

        std::panic::set_hook(prior);
        reports
    }

    pub fn print_summary(reports: &[Report], seed: u64) -> bool {
        println!();
        println!("{:<9} {:>9} {:>9} {:>7} {:>9} {:>10} {:>9}",
                 "target", "cases", "accepted", "panics", "forgeries", "invariants",
"cases/s");
        let mut ok = true;
        for r in reports {
            let secs = r.elapsed.as_secs_f64().max(0.000_001);
            println!(
                "{:<9} {:>9} {:>9} {:>7} {:>9} {:>10} {:>9.0}",
                r.target, r.cases, r.accepted, r.panics, r.forgeries,
                r.broken_invariants, r.cases as f64 / secs
            );
            ok &= !r.failed();
        }
        println!();
        println!(
            "`accepted` is only a failure for the signature-verifying parsers
(bundle, hello);\n\
             parse_frame is structural by design and authenticates nothing."
        );
        println!();
        if ok {
            println!("no panics, no forgeries, no invariant violations.");
            println!("reproduce with --fuzz-seed {seed}");
        } else {
            println!("FINDINGS ABOVE. Reproduce the exact run with --fuzz-seed
{seed}");
        }
        ok
    }
}

// ──────────────────────────────────── tests ────────────────────────────────────

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn handshake_roundtrip_yields_matching_keys() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
```

```
        let server_bundle = crypto::parse_bundle(&server.bundle).expect("server
bundle verifies");
        let ch = client_build_hello(&client, &server_bundle, 3005,
now_unix()).unwrap();
        let hello1 = ch.hello1.clone();

        let verified = server_verify_hello(&server, &hello1, now_unix()).unwrap();
        assert_eq!(verified.init_fpr, client.fpr());
        assert_eq!(verified.port, 3005);

        let hs = server_finish_hello(&hello1, &verified).unwrap();
        let client_keys = client_finish_hello(ch, &hs.hello2).unwrap();

        // Directions must line up: client-send == server-recv and vice versa.
        assert_eq!(&client_keys.k_i2r[..], &hs.keys.k_i2r[..]);
        assert_eq!(&client_keys.k_r2i[..], &hs.keys.k_r2i[..]);
        assert_eq!(client_keys.session_id, hs.keys.session_id);
    }

    /// A hello1 that opens correctly but carries a signature over different content.
    ///
    /// Needed because tampering with the visible bytes now yields `Undecryptable`
(the AEAD
    /// catches it first), so the signature check can only be exercised from inside a
    /// well-formed, correctly-encrypted identity block.
    fn hello_with_bad_signature(client: &Identity, server: &Identity) -> Vec<u8> {
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let (ct_s, ss_s) = crypto::kem_encapsulate(&bundle.ek).unwrap();
        let eph = crypto::KemKeypair::generate();
        let h0 = hello_h0(&bundle.fpr, PROTO_VER, &ct_s, eph.ek_bytes());
        let k0 = crypto::derive_pre_identity_key(&h0, &ss_s).unwrap();
        let now = now_unix();
        // Signed over port 9999 while the block claims 3005 — a valid signature for
a
        // statement the sender is not actually making.
        let sig = client.id.sign(&hello_sig_input(
            &bundle.fpr,
            PROTO_VER,
            &ct_s,
            eph.ek_bytes(),
            now,
            9999,
        ));
        let mut inner = Vec::with_capacity(INNER_LEN);
        inner.extend_from_slice(&now.to_be_bytes());
        inner.extend_from_slice(&3005u16.to_be_bytes());
        inner.extend_from_slice(client.vk_bytes());
        inner.extend_from_slice(&sig);
        assert_eq!(inner.len(), INNER_LEN);
```

```
        let mut hello1 = Vec::with_capacity(HELLO1_LEN);
        hello1.push(PROTO_VER);
        hello1.extend_from_slice(&ct_s);
        hello1.extend_from_slice(eph.ek_bytes());
        let aad = hello1.clone();
        hello1.extend_from_slice(&crypto::aead_seal(&k0, 0, &aad, &inner).unwrap());
        assert_eq!(hello1.len(), HELLO1_LEN);
        hello1
    }

    #[test]
    fn forged_signature_is_rejected() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let hello1 = hello_with_bad_signature(&client, &server);
        assert!(matches!(
            server_verify_hello(&server, &hello1, now_unix()),
            Err(HelloError::BadSignature)
        ));
    }

    #[test]
    fn tampered_hello_header_is_rejected() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
        let mut hello1 = ch.hello1;
        // The KEM ciphertext is authenticated as associated data, so altering it
breaks the
        // tag rather than silently changing the derived key.
        hello1[1] ^= 0xff;
        assert!(matches!(
            server_verify_hello(&server, &hello1, now_unix()),
            Err(HelloError::Undecryptable)
        ));
    }

    #[test]
    fn hello_bound_to_wrong_responder_fails() {
        let server = Identity::ephemeral();
        let other = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
        // The wrong responder cannot even read the identity block, let alone accept
it. This
        // is stronger than v2, where it could read the identity and then reject the
signature.
```

```
        assert!(matches!(
            server_verify_hello(&other, &ch.hello1, now_unix()),
            Err(HelloError::Undecryptable)
        ));
    }

    /// The whole point of protocol 3: nothing on the wire names the initiator.
    #[test]
    fn hello1_does_not_expose_the_initiator_identity() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();

        let vk = client.vk_bytes();
        assert!(
            !contains_subslice(&ch.hello1, vk),
            "initiator verifying key appears in cleartext in hello1"
        );
        assert!(
            !contains_subslice(&ch.hello1, &client.fpr()),
            "initiator fingerprint appears in cleartext in hello1"
        );
        // The responder's fingerprint must not be on the wire either — it is mixed
into the
        // transcript, never transmitted, so a scanner cannot tell who a hello is
addressed to.
        assert!(!contains_subslice(&ch.hello1, &server.fpr()));
    }

    fn contains_subslice(haystack: &[u8], needle: &[u8]) -> bool {
        if needle.is_empty() || needle.len() > haystack.len() {
            return false;
        }
        haystack.windows(needle.len()).any(|w| w == needle)
    }

    #[test]
    fn stale_hello_is_rejected() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix() -
10_000).unwrap();
        assert!(matches!(
            server_verify_hello(&server, &ch.hello1, now_unix()),
            Err(HelloError::Stale)
        ));
    }
```

```
    #[test]
    fn frame_roundtrip_and_replay() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
        let verified = server_verify_hello(&server, &ch.hello1, now_unix()).unwrap();
        let hs = server_finish_hello(&ch.hello1, &verified).unwrap();
        let mut csess = ClientSession::from_keys(server.fpr(),
client_finish_hello(ch, &hs.hello2).unwrap());

        let frame = csess.seal(&Msg::Ping { nonce: 7, vitals: None, carry: None,
acks: Vec::new() }).unwrap();

        let mut ssess = ServerSession {
            peer_id: client.fpr(),
            peer_ip: "127.0.0.1".parse().unwrap(),
            peer_port: 3005,
            k_recv: hs.keys.k_i2r.clone(),
            k_send: hs.keys.k_r2i.clone(),
            recv_win: ReplayWindow::default(),
            send_ctr: 0,
            sid: hs.keys.session_id,
            created: Instant::now(),
            last_used: Instant::now(),
        };

        // First delivery authenticates and updates the window.
        let (sid, ctr, hdr, ct) = parse_frame(&frame).unwrap();
        assert_eq!(sid, ssess.sid);
        assert!(ssess.recv_win.check(ctr));
        let pt = crypto::aead_open(&ssess.k_recv, ctr, hdr, ct).unwrap();
        ssess.recv_win.update(ctr);
        let msg = open_msg(&pt).unwrap();
        assert!(matches!(msg, Msg::Ping { nonce: 7, .. }));

        // Replaying the same frame is refused by the window.
        assert!(!ssess.recv_win.check(ctr));
    }

    #[test]
    fn tampered_frame_fails_to_open() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
        let verified = server_verify_hello(&server, &ch.hello1, now_unix()).unwrap();
        let hs = server_finish_hello(&ch.hello1, &verified).unwrap();
        let mut csess = ClientSession::from_keys(server.fpr(),
client_finish_hello(ch, &hs.hello2).unwrap());
```

```
        let mut frame = csess.seal(&Msg::Ping { nonce: 1, vitals: None, carry: None,
acks: Vec::new() }).unwrap();
        let last = frame.len() - 1;
        frame[last] ^= 0x01;
        let (_sid, ctr, hdr, ct) = parse_frame(&frame).unwrap();
        assert!(crypto::aead_open(&hs.keys.k_i2r, ctr, hdr, ct).is_none());
    }

    #[test]
    fn peer_policy_blocks_loopback_and_metadata_by_default() {
        let lo: SocketAddr = "127.0.0.1:3005".parse().unwrap();
        let meta: SocketAddr = "169.254.169.254:80".parse().unwrap();
        let lan: SocketAddr = "192.168.1.20:3005".parse().unwrap();
        let pub_: SocketAddr = "8.8.8.8:3005".parse().unwrap();
        assert!(!peer_ok(&lo, false, false));
        assert!(peer_ok(&lo, false, true)); // dev loopback opt-in
        assert!(!peer_ok(&meta, false, false)); // link-local blocked
        assert!(!peer_ok(&meta, true, false)); // even with allow_public
        assert!(peer_ok(&lan, false, false));
        assert!(!peer_ok(&pub_, false, false));
        assert!(peer_ok(&pub_, true, false));
    }

    #[test]
    fn replay_window_accepts_reordered_within_range() {
        let mut w = ReplayWindow::default();
        for c in [10u64, 12, 11, 9] {
            assert!(w.check(c), "counter {c} should be fresh");
            w.update(c);
        }
        // 9..=12 are now all seen.
        for c in [9u64, 10, 11, 12] {
            assert!(!w.check(c));
        }
        // Far-future still accepted.
        assert!(w.check(1000));
    }

    fn tmp_dir(tag: &str) -> PathBuf {
        let p = std::env::temp_dir().join(format!(
            "harmony-test-{tag}-{}-{}",
            std::process::id(),
            OsRng.next_u64()
        ));
        std::fs::create_dir_all(&p).unwrap();
        p
    }

    #[test]
    fn revocation_beats_pinning_and_persists() {
```

```
        let dir = tmp_dir("revoke");
        let good = [7u8; 32];
        let bad = [9u8; 32];

        let mut ts = TrustStore::load(TrustMode::Tofu, &dir, &[good, bad]);
        assert!(ts.admit(&good));
        assert!(ts.admit(&bad), "pinned before revocation");

        ts.apply_revocations(&[bad]);
        assert!(ts.is_revoked(&bad));
        assert!(!ts.admit(&bad), "revocation must override an existing pin");
        assert!(ts.admit(&good), "revoking one peer must not affect others");

        // A restart re-reads revoked_peers.txt, so the revocation survives.
        let mut reloaded = TrustStore::load(TrustMode::Tofu, &dir, &[bad]);
        assert!(reloaded.is_revoked(&bad));
        assert!(!reloaded.admit(&bad), "revocation must survive a restart");

        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
    fn strict_mode_refuses_unknown_identities() {
        let dir = tmp_dir("strict");
        let known = [1u8; 32];
        let unknown = [2u8; 32];
        let mut ts = TrustStore::load(TrustMode::Strict, &dir, &[known]);
        assert!(ts.admit(&known));
        assert!(!ts.admit(&unknown));
        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
    fn kem_rotation_keeps_identity_and_changes_encapsulation_key() {
        let dir = tmp_dir("rotate");
        let before = Identity::load_or_create(&dir).unwrap();
        let fpr_before = before.fpr();
        let ek_before = crypto::parse_bundle(&before.bundle).unwrap().ek;
        drop(before);

        let after = Identity::rotate_kem(&dir).unwrap();
        let parsed = crypto::parse_bundle(&after.bundle).expect("rotated bundle still
verifies");

        assert_eq!(after.fpr(), fpr_before, "identity must survive KEM rotation");
        assert_eq!(parsed.fpr, fpr_before, "bundle must carry the same fingerprint");
        assert_ne!(parsed.ek, ek_before, "encapsulation key must actually change");

        // The rotated key must still complete a handshake.
        let client = Identity::ephemeral();
```

```
        let ch = client_build_hello(&client, &parsed, 3005, now_unix()).unwrap();
        let verified = server_verify_hello(&after, &ch.hello1, now_unix()).unwrap();
        let hs = server_finish_hello(&ch.hello1, &verified).unwrap();
        let keys = client_finish_hello(ch, &hs.hello2).unwrap();
        assert_eq!(&keys.k_i2r[..], &hs.keys.k_i2r[..]);

        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
    fn client_session_expires_after_frame_budget() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
        let verified = server_verify_hello(&server, &ch.hello1, now_unix()).unwrap();
        let hs = server_finish_hello(&ch.hello1, &verified).unwrap();
        let mut sess =
            ClientSession::from_keys(server.fpr(), client_finish_hello(ch,
&hs.hello2).unwrap());

        assert!(!sess.expired(), "a fresh session is usable");
        sess.send_ctr = MAX_FRAMES_PER_SESSION;
        assert!(sess.expired(), "a session must retire at its frame budget");
    }

    #[test]
    fn identity_dir_is_independent_of_the_bound_port() {
        // Without --node-name, identity follows the *requested* port.
        assert_eq!(identity_dir_name(&None, 3005), "node-3005");

        // With --node-name, the port is irrelevant — this is what makes rotating the
        // listening port safe, since the keys are found the same way either way.
        let named = Some("gateway-alpha".to_string());
        assert_eq!(identity_dir_name(&named, 3005), "gateway-alpha");
        assert_eq!(
            identity_dir_name(&named, 3006),
            identity_dir_name(&named, 3005),
            "a node's identity must not move when its port does"
        );
    }

    #[test]
    fn node_name_cannot_escape_the_data_directory() {
        // Path separators and traversal must not survive into a directory name.
        for hostile in ["../../etc/passwd", "a/b", "..", "./x", "with space"] {
            let got = identity_dir_name(&Some(hostile.to_string()), 3005);
            assert!(!got.contains('/'), "{hostile:?} produced {got:?}");
            assert!(!got.contains(".."), "{hostile:?} produced {got:?}");
        }
```

```
        // An empty or fully-stripped name falls back rather than yielding "".
        assert_eq!(identity_dir_name(&Some(String::new()), 3005), "node-3005");
        assert_eq!(identity_dir_name(&Some("///".to_string()), 3005), "___");
    }

    fn outcome(node: &str, ok: bool, text: &str) -> InferOutcome {
        InferOutcome {
            node: node.into(),
            tier: "light-1b".into(),
            ok,
            text: text.into(),
            elapsed_ms: 1,
        }
    }

    #[test]
    fn consensus_finds_the_majority_answer() {
        let outcomes = vec![
            outcome("a", true, "Route via node 7."),
            outcome("b", true, "route via node 7"),
            outcome("c", true, "Hold position."),
            outcome("d", false, "thermal limit"),
        ];
        let (agree, answered, text) = consensus(&outcomes);
        assert_eq!(answered, 3, "failed answers must not count toward the total");
        assert_eq!(agree, 2, "formatting differences must not split a cluster");
        assert!(text.to_lowercase().contains("route via node 7"));
    }

    #[test]
    fn consensus_reports_no_majority_when_all_disagree() {
        let outcomes = vec![
            outcome("a", true, "left"),
            outcome("b", true, "right"),
            outcome("c", false, "busy"),
        ];
        let (agree, answered, _) = consensus(&outcomes);
        assert_eq!(answered, 2);
        assert_eq!(agree, 1, "a one-vote plurality is not agreement");
    }

    #[test]
    fn consensus_handles_no_successful_answers() {
        let outcomes = vec![outcome("a", false, "busy"), outcome("b", false,
"thermal")];
        assert_eq!(consensus(&outcomes), (0, 0, String::new()));
    }

    #[test]
    fn normalize_answer_ignores_case_space_and_edge_punctuation() {
```

```
        assert_eq!(
            normalize_answer("  Route via  NODE 7!! "),
            normalize_answer("route via node 7")
        );
        assert_ne!(normalize_answer("go left"), normalize_answer("go right"));
        // Only leading/trailing punctuation is stripped — internal punctuation is
part of the
        // token, so these stay distinct rather than being silently merged.
        assert_ne!(normalize_answer("node-7"), normalize_answer("node7"));
    }

    #[test]
    fn llama3_template_wraps_the_prompt_with_role_markers() {
        let out = PromptFormat::Llama3.wrap("Say OK.");
        assert!(out.starts_with("<|begin_of_text|>"));
        assert!(out.contains("<|start_header_id|>user<|end_header_id|>"));
        assert!(out.contains("Say OK.<|eot_id|>"));
        // Must end handing the turn to the assistant, or the model continues the
user turn —
        // which is what produces degenerate repetition on an instruct model.
        assert!(out.trim_end().ends_with("
<|start_header_id|>assistant<|end_header_id|>"));
    }

    #[test]
    fn raw_format_passes_the_prompt_through_untouched() {
        assert_eq!(PromptFormat::Raw.wrap("hello world"), "hello world");
        assert!(PromptFormat::Chatml.wrap("hi").contains("<|im_start|>assistant"));
    }

    #[test]
    fn sanitize_strips_control_characters_but_keeps_text() {
        // ANSI escapes and carriage returns can forge log lines and corrupt a
terminal
        // reading them; newline and tab are legitimate structure and must survive.
        assert_eq!(sanitize("ok\u{1b}[31mRED"), "ok[31mRED");
        assert_eq!(sanitize("a\rb"), "ab");
        assert_eq!(sanitize("line1\nline2\tcol"), "line1\nline2\tcol");
        assert_eq!(sanitize("plain"), "plain");
        assert_eq!(sanitize("null\0byte"), "nullbyte");
    }

    #[test]
    fn peer_provenance_limits_what_one_introducer_can_add() {
        // A trusted peer that turns hostile must not be able to fill the table. The
cap is
        // per introducer, so honest peers keep their own budget.
        let hostile = [1u8; 32];
        let honest = [2u8; 32];
        let mut table: Vec<Peer> = Vec::new();
```

```
        let mut try_add = |src: NodeId, table: &mut Vec<Peer>| -> bool {
            let introduced = table.iter().filter(|p| p.learned_from ==
Some(src)).count();
            if introduced >= MAX_PEERS_PER_SOURCE {
                return false;
            }
            table.push(Peer {
                addr: "192.168.1.1:3005".parse().unwrap(),
                fails: 0,
                learned_from: Some(src),
            });
            true
        };

        for _ in 0..MAX_PEERS_PER_SOURCE {
            assert!(try_add(hostile, &mut table));
        }
        assert!(!try_add(hostile, &mut table), "introducer must hit its cap");
        assert!(
            try_add(honest, &mut table),
            "one peer's cap must not consume another's"
        );
    }

    #[test]
    fn directly_observed_peers_are_not_charged_to_an_introducer() {
        // Once we handshake with a peer ourselves we no longer depend on hearsay, so
it
        // should stop counting against whoever first mentioned it.
        let mut p = Peer {
            addr: "192.168.1.1:3005".parse().unwrap(),
            fails: 3,
            learned_from: Some([9u8; 32]),
        };
        // Mirrors the `and_modify` branch for a direct observation (source = None).
        p.fails = 0;
        p.learned_from = None;
        assert_eq!(p.learned_from, None);
        assert_eq!(p.fails, 0);
    }

    #[test]
    fn capacity_failures_are_worth_offloading_but_bad_requests_are_not() {
        // These are about this device right now, so another node may well succeed.
        assert!(is_capacity_failure("thermal 71.2C at or above limit 65.0C"));
        assert!(is_capacity_failure("inference already in progress"));
        assert!(is_capacity_failure("rate limited"));
        // These would fail identically on every node — relaying them just wastes
peers.
```

```
        assert!(!is_capacity_failure("prompt exceeds 1024 bytes"));
        assert!(!is_capacity_failure("spawn llama-cli: No such file or directory"));
        assert!(!is_capacity_failure("this node does not serve inference to you"));
    }

    #[test]
    fn auto_tier_never_selects_heavy() {
        // Heavy is an explicit operator choice: on a phone it competes with the OS
for RAM
        // and throttles under sustained load, so detection must never opt in on its
own.
        assert_ne!(ModelTier::Auto.resolve(), ModelTier::Heavy);
        // Auto must never talk a device into a 6B model either — both big tiers are
        // explicit choices, because the cost is heat and memory pressure, not just
speed.
        assert_ne!(ModelTier::Auto.resolve(), ModelTier::Mid);
        assert_eq!(ModelTier::Heavy.resolve(), ModelTier::Heavy);
    }

    #[test]
    fn explicit_model_tier_overrides_hardware_detection() {
        // An operator's explicit choice must win, so a node can be pinned to a tier
        // regardless of what the device reports.
        assert_eq!(ModelTier::Light.resolve(), ModelTier::Light);
        assert_eq!(ModelTier::Standard.resolve(), ModelTier::Standard);
        assert_eq!(ModelTier::Mid.resolve(), ModelTier::Mid);
        // Auto must resolve to something runnable, never stay Auto.
        assert_ne!(ModelTier::Auto.resolve(), ModelTier::Auto);
    }

    /// A flat 60s timeout silently killed every large-model generation. The default
has to
    /// follow the tier, and bigger tiers must never get less time than smaller ones.
    #[test]
    fn timeout_default_scales_with_tier() {
        let light = ModelTier::Light.default_timeout_secs();
        let standard = ModelTier::Standard.default_timeout_secs();
        let mid = ModelTier::Mid.default_timeout_secs();
        let heavy = ModelTier::Heavy.default_timeout_secs();
        assert!(light < standard, "{light} !< {standard}");
        assert!(standard < mid, "{standard} !< {mid}");
        assert!(mid < heavy, "{mid} !< {heavy}");
        // An 8B model on a phone CPU needs minutes, not the old flat minute.
        assert!(heavy >= 300, "heavy tier gets only {heavy}s");
        // Auto resolves before this is read, but it must still be a usable number.
        assert!(ModelTier::Auto.default_timeout_secs() >= 60);
    }

    /// The operator waiting at a terminal and a peer spending this node's CPU are
different
```

```
    /// threat models, so they must not share a ceiling.
    #[test]
    fn local_inference_ceiling_exceeds_the_remote_one() {
        assert!(
            MAX_LOCAL_INFER > MAX_REMOTE_INFER,
            "a peer can demand as much CPU as the operator can"
        );
        // A peer must never be able to hold this node past the remote cap, whatever
the
        // tier's generous local default says.
        for t in [ModelTier::Light, ModelTier::Standard, ModelTier::Mid,
ModelTier::Heavy] {
            let asked = Duration::from_secs(t.default_timeout_secs());
            assert!(asked.min(MAX_REMOTE_INFER) <= MAX_REMOTE_INFER);
        }
    }

    /// "exited exit status: 1" was the only thing inference ever said when it
failed. Each
    /// of these is a failure that really happens on a phone, and each must come back
naming
    /// what to change.
    #[test]
    fn llm_failures_are_translated_into_something_actionable() {
        let missing = llm_failure_hint(
            b"llama_model_load: error loading model: No such file or directory\n",
            "models/llama-3.1-8b.gguf",
        );
        assert!(missing.contains("models/llama-3.1-8b.gguf"), "{missing}");
        assert!(missing.to_lowercase().contains("model not found"), "{missing}");

        let badflag = llm_failure_hint(b"error: unknown argument: -no-cnv\n",
"m.gguf");
        assert!(badflag.contains("llama-completion"), "{badflag}");

        let oom = llm_failure_hint(b"ggml: cannot allocate 4823 MB\n", "m.gguf");
        assert!(oom.to_lowercase().contains("out of memory"), "{oom}");

        // Anything unrecognised must still carry the tail, not be swallowed.
        let other = llm_failure_hint(b"something specific and weird happened",
"m.gguf");
        assert!(other.contains("weird"), "{other}");

        // Silence must say so rather than returning an empty string.
        assert_eq!(llm_failure_hint(b"", "m.gguf"), "no stderr output");
    }

    /// The hint crosses the mesh inside an InferOutcome, so it must stay one
sanitized line.
    #[test]
```

```
    fn llm_failure_hint_is_single_line_and_bounded() {
        let noisy = format!("line one\nline two\x1b[31m red\n{}", "x".repeat(5000));
        let hint = llm_failure_hint(noisy.as_bytes(), "m.gguf");
        assert!(!hint.contains('\n'), "hint spans lines");
        assert!(!hint.contains('\u{1b}'), "hint carries control characters");
        assert!(hint.chars().count() <= 512, "hint is {} chars",
hint.chars().count());
    }

    #[test]
    fn which_on_path_distinguishes_a_path_from_a_bare_name() {
        // An explicit path that does not exist must not be "found" via PATH lookup.
        assert!(which_on_path("/definitely/not/here/llama-cli").is_none());
        assert!(which_on_path("./also-not-here").is_none());
        // A bare name that does not exist anywhere on PATH.
        assert!(which_on_path("harmony-no-such-binary-xyzzy").is_none());
        // Something that does exist on every unix.
        assert!(which_on_path("sh").is_some(), "sh not found on PATH");
    }

    #[test]
    fn fingerprint_is_stable_and_bundle_verifies() {
        let id = Identity::ephemeral();
        let parsed = crypto::parse_bundle(&id.bundle).unwrap();
        assert_eq!(parsed.fpr, id.fpr());
        assert_eq!(parsed.vk, id.vk_bytes());
    }

    // ── traffic-analysis resistance ──

    #[test]
    fn padding_rounds_up_to_a_bucket_and_roundtrips() {
        for len in [0usize, 1, 100, 253, 254, 255, 1021, 1022, 4000] {
            let pt = vec![0xabu8; len];
            let padded = pad_payload(&pt, &PAD_BUCKETS).unwrap();
            assert!(
                PAD_BUCKETS.contains(&padded.len()),
                "len {len} padded to {} which is not a bucket",
                padded.len()
            );
            assert_eq!(unpad_payload(&padded).unwrap(), &pt[..]);
        }
    }

    #[test]
    fn payload_past_the_ladder_steps_in_whole_buckets() {
        let largest = PAD_BUCKETS[PAD_BUCKETS.len() - 1];
        let pt = vec![7u8; largest + 10];
        let padded = pad_payload(&pt, &PAD_BUCKETS).unwrap();
        assert_eq!(padded.len() % largest, 0);
```

```
        assert_eq!(unpad_payload(&padded).unwrap(), &pt[..]);
    }

    /// Messages of very different plaintext length must produce frames of identical
length,
    /// or the ciphertext size still identifies the message type.
    #[test]
    fn different_messages_seal_to_the_same_frame_size() {
        let server = Identity::ephemeral();
        let client = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
        let verified = server_verify_hello(&server, &ch.hello1, now_unix()).unwrap();
        let hs = server_finish_hello(&ch.hello1, &verified).unwrap();
        let mut csess =
            ClientSession::from_keys(server.fpr(), client_finish_hello(ch,
&hs.hello2).unwrap());

        let ping = csess.seal(&Msg::Ping { nonce: 1, vitals: None, carry: None, acks:
Vec::new() }).unwrap();
        let getpeers = csess.seal(&Msg::GetPeers).unwrap();
        assert_eq!(
            ping.len(),
            getpeers.len(),
            "frame length still distinguishes message kinds"
        );
        assert_eq!(ping.len(), PAD_BUCKETS[0] + FRAME_HDR_LEN + TAG_LEN);
    }

    #[test]
    fn oversized_frames_are_rejected_before_decryption() {
        let frame = vec![0u8; MAX_FRAME_LEN + 1];
        assert!(parse_frame(&frame).is_none());
    }

    #[test]
    fn jitter_stays_within_band_and_varies() {
        let base = Duration::from_secs(5);
        let lo = 2_500u128;
        let hi = 7_500u128;
        let mut seen = std::collections::HashSet::new();
        for _ in 0..200 {
            let d = jittered(base).as_millis();
            assert!((lo..=hi).contains(&d), "jittered to {d}ms, outside the band");
            seen.insert(d);
        }
        // A "jitter" that always returns the same value is not jitter.
        assert!(seen.len() > 50, "only {} distinct intervals", seen.len());
    }
```

```
    #[test]
    fn jitter_never_returns_zero() {
        // A zero period would turn the gossip loop into a busy spin.
        for base in [Duration::from_millis(1), Duration::from_secs(1)] {
            for _ in 0..100 {
                assert!(jittered(base) >= Duration::from_millis(1));
            }
        }
    }

    #[test]
    fn link_health_reports_isolation_only_after_repeated_blackouts() {
        let mut l = LinkHealth::default();
        // No peers: nothing to be cut off from, whatever happens.
        assert_eq!(l.state(0), LinkState::Healthy);

        l.record(true, false);
        assert_eq!(l.state(3), LinkState::Degraded);
        for _ in 1..DEGRADED_AFTER_FAILS {
            l.record(true, false);
        }
        assert_eq!(l.state(3), LinkState::Isolated);
        assert_eq!(l.blackouts, DEGRADED_AFTER_FAILS);

        // One success clears it — a recovered link is not a degraded one.
        l.record(true, true);
        assert_eq!(l.state(3), LinkState::Healthy);
        assert_eq!(l.blackouts, 0);
        assert!(l.last_contact.is_some());
    }

    #[test]
    fn rounds_with_no_peer_to_try_are_not_counted_as_failures() {
        let mut l = LinkHealth::default();
        for _ in 0..10 {
            l.record(false, false);
        }
        assert_eq!(l.blackouts, 0);
        assert_eq!(l.failed, 0);
        assert_eq!(l.rounds, 10);
        assert_eq!(l.state(3), LinkState::Healthy);
    }

    // ── provisioned / cached bundles ──

    #[test]
    fn provisioned_bundle_loads_and_is_reverified() {
        let dir = tmp_dir("bundles-ok");
        let id = Identity::ephemeral();
        std::fs::write(dir.join(format!("{}.bundle", hex(&id.fpr()))),
```

```
&id.bundle).unwrap();

        let loaded = load_bundles(&dir);
        assert_eq!(loaded.len(), 1);
        let b = loaded.get(&id.fpr()).unwrap();
        assert_eq!(b.vk, id.vk_bytes());
        std::fs::remove_dir_all(&dir).ok();
    }

    /// A bundle filed under someone else's fingerprint must not be trusted as that
identity,
    /// or dropping a file into the directory would be an identity-substitution
attack.
    #[test]
    fn provisioned_bundle_with_mismatched_filename_is_ignored() {
        let dir = tmp_dir("bundles-liar");
        let real = Identity::ephemeral();
        let victim = Identity::ephemeral();
        std::fs::write(
            dir.join(format!("{}.bundle", hex(&victim.fpr()))),
            &real.bundle,
        )
        .unwrap();

        assert!(load_bundles(&dir).is_empty());
        std::fs::remove_dir_all(&dir).ok();
    }

    #[test]
    fn corrupt_bundle_file_is_ignored_not_fatal() {
        let dir = tmp_dir("bundles-corrupt");
        let id = Identity::ephemeral();
        let mut raw = id.bundle.clone();
        raw[0] ^= 0xff; // breaks the self-signature
        std::fs::write(dir.join(format!("{}.bundle", hex(&id.fpr()))),
&raw).unwrap();
        std::fs::write(dir.join("not-a-bundle.txt"), b"ignore me").unwrap();

        assert!(load_bundles(&dir).is_empty());
        std::fs::remove_dir_all(&dir).ok();
    }

    #[test]
    fn missing_bundle_dir_is_empty_not_an_error() {
        let dir = std::env::temp_dir().join("harmony-test-no-such-dir-xyzzy");
        assert!(load_bundles(&dir).is_empty());
    }

    #[test]
    fn hello1_size_is_fixed() {
```

```
        // A variable-length hello would leak the identity size and break the length
check
        // that rejects malformed hellos before any crypto runs.
        let server = Identity::ephemeral();
        let bundle = crypto::parse_bundle(&server.bundle).unwrap();
        for _ in 0..5 {
            let client = Identity::ephemeral();
            let ch = client_build_hello(&client, &bundle, 3005, now_unix()).unwrap();
            assert_eq!(ch.hello1.len(), HELLO1_LEN);
        }
    }

    // ── piggyback propagation ──

    #[test]
    fn base64_roundtrips_every_length() {
        for len in 0..40usize {
            let data: Vec<u8> = (0..len).map(|i| (i * 37 + 11) as u8).collect();
            let enc = b64::encode(&data);
            assert_eq!(enc.len(), len.div_ceil(3) * 4);
            assert_eq!(b64::decode(&enc).as_deref(), Some(data.as_slice()));
        }
        assert_eq!(b64::encode(b"Man"), "TWFu");
        assert_eq!(b64::encode(b"Ma"), "TWE=");
        assert_eq!(b64::encode(b"M"), "TQ==");
        assert!(b64::decode("TWF").is_none());
        assert!(b64::decode("T=Fu").is_none());
        assert!(b64::decode("TW!u").is_none());
        assert!(b64::decode("TQ==TWFu").is_none(), "padding only at the end");
    }

    #[test]
    fn relayed_directive_signs_and_verifies() {
        let me = Identity::ephemeral();
        let now = now_unix();
        let r = sign_relayed(&me, "route heavy work away from node 7", now);
        assert_eq!(verify_relayed(&r, now), Ok(me.fpr()));
        let mut hop = r.clone();
        hop.hops = 3;
        assert_eq!(verify_relayed(&hop, now), Ok(me.fpr()), "hops is not signed");
    }

    #[test]
    fn relayed_directive_rejects_tampering_and_replay_windows() {
        let me = Identity::ephemeral();
        let other = Identity::ephemeral();
        let now = now_unix();
        let r = sign_relayed(&me, "advise", now);

        let mut body = r.clone();
```

```
        body.body = "advise otherwise".into();
        assert_eq!(verify_relayed(&body, now), Err("bad signature"));

        let mut forged = r.clone();
        forged.origin = hex(&other.fpr());
        assert_eq!(verify_relayed(&forged, now), Err("key does not match origin"));

        let mut swapped = r.clone();
        swapped.vk = b64::encode(other.vk_bytes());
        assert_eq!(verify_relayed(&swapped, now), Err("key does not match origin"));

        let mut far = r.clone();
        far.hops = RELAY_MAX_HOPS + 1;
        assert_eq!(verify_relayed(&far, now), Err("too many hops"));

        assert_eq!(verify_relayed(&r, now + RELAY_MAX_AGE.as_secs() + 1),
Err("expired"));
        let future = sign_relayed(&me, "advise", now + MAX_CLOCK_SKEW_SECS + 60);
        assert_eq!(verify_relayed(&future, now), Err("issued in the future"));

        let mut ctl = r.clone();
        ctl.body = "a\u{1b}[2J".into();
        assert_eq!(verify_relayed(&ctl, now), Err("control characters"));

        let mut bad_id = r.clone();
        bad_id.id = "not-an-id".into();
        assert_eq!(verify_relayed(&bad_id, now), Err("bad id"));
    }

    #[test]
    fn largest_relayed_directive_fits_one_frame() {
        let me = Identity::ephemeral();
        // Worst case for JSON size: every byte of a maximum-length body needs
escaping.
        let body = "\"".repeat(MAX_PROMPT_BYTES);
        let r = sign_relayed(&me, &body, now_unix());
        assert!(verify_relayed(&r, now_unix()).is_ok());
        let ping = Msg::Ping {
            nonce: u64::MAX,
            vitals: Some(Vitals {
                thermal_c: -12.5,
                max_temp: 65.0,
                busy: true,
                tier: "standard-3b".into(),
                label: "x".repeat(24),
            }),
            carry: Some(r),
            acks: vec!["f".repeat(32); RELAY_MAX_ACKS],
        };
        let json = serde_json::to_vec(&ping).unwrap();
```

```
        let largest = PAD_BUCKETS[PAD_BUCKETS.len() - 1];
        assert!(PAD_PREFIX + json.len() <= largest, "{} bytes", json.len());
    }

    #[test]
    fn plain_ping_is_unchanged_on_the_wire() {
        let ping = Msg::Ping { nonce: 9, vitals: None, carry: None, acks: Vec::new()
};
        assert_eq!(serde_json::to_string(&ping).unwrap(), r#"
{"t":"ping","nonce":9}"#);
        let old: Msg = serde_json::from_str(r#"{"t":"pong","nonce":9}"#).unwrap();
        assert!(matches!(old, Msg::Pong { nonce: 9, vitals: None, ref acks } if
acks.is_empty()));
        let new = Msg::Pong {
            nonce: 1,
            vitals: Some(Vitals {
                thermal_c: 40.0,
                max_temp: 65.0,
                busy: false,
                tier: "light-1b".into(),
                label: String::new(),
            }),
            acks: vec!["ab".repeat(16)],
        };
        let back: Msg =
serde_json::from_slice(&serde_json::to_vec(&new).unwrap()).unwrap();
        assert!(matches!(back, Msg::Pong { nonce: 1, vitals: Some(_), ref acks } if
acks.len() == 1));
    }

    #[test]
    fn offload_ranks_coolest_idle_peers_first_and_skips_hot_ones() {
        let addr: SocketAddr = "10.0.0.1:3005".parse().unwrap();
        let v = |t: f32, busy: bool| {
            Some(Vitals { thermal_c: t, max_temp: 65.0, busy, tier: "light-
1b".into(), label: String::new() })
        };
        let cands = vec![
            ([1u8; 32], addr, v(60.0, false)), // idle, 5 °C headroom
            ([2u8; 32], addr, v(70.0, false)), // over its limit: skipped
            ([3u8; 32], addr, None),           // no fresh vitals: after the known
ones
            ([4u8; 32], addr, v(35.0, false)), // idle, 30 °C headroom: first
            ([5u8; 32], addr, v(30.0, true)),  // coolest, but busy: after the idle
ones
        ];
        let order: Vec<u8> = rank_targets(cands.clone(), 8).iter().map(|(id, _)|
id[0]).collect();
        assert_eq!(order, vec![4, 1, 5, 3]);
        assert_eq!(rank_targets(cands, 1).len(), 1);
```

```
        let at_limit = vec![([9u8; 32], addr, v(65.0, false))];
        assert!(rank_targets(at_limit, 8).is_empty(), "at the limit counts as hot");

        let mut odd = v(40.0, false).unwrap();
        assert!(odd.plausible());
        odd.thermal_c = f32::NAN;
        assert!(!odd.plausible());
        odd.thermal_c = -999.0;
        assert!(!odd.plausible());
    }

    // ── llama.cpp output handling ──

    #[test]
    fn llm_echo_drops_special_tokens() {
        let f = PromptFormat::Llama3.wrap("What is 2+2?");
        assert_eq!(llm_echo(&f), "user\n\nWhat is 2+2?assistant\n\n");
        assert_eq!(llm_echo(&PromptFormat::Raw.wrap("hi")), "hi");
        assert_eq!(llm_echo("a <|unterminated"), "a <|unterminated");
    }

    #[test]
    fn clean_output_keeps_only_the_answer() {
        let f = PromptFormat::Llama3.wrap("What is the capital of France?");
        let echo = llm_echo(&f);
        // one-shot tool: prompt echo + answer + end marker
        let raw = format!("{echo}Paris. [end of text]\n");
        assert_eq!(clean_llm_output(&raw, &echo), "Paris.");
        // chat mode: answer, then the interactive prompt it waits at
        assert_eq!(clean_llm_output("Paris\n\n> ", &echo), "Paris");
        assert_eq!(clean_llm_output(&format!("{echo}Paris\n> EOF by user\n"), &echo),
"Paris");
        // a '>' inside the answer is not a prompt
        assert_eq!(clean_llm_output("use a > b to compare", &echo), "use a > b to
compare");
        // no echo present: trimmed as-is
        assert_eq!(clean_llm_output("  Paris  ", &echo), "Paris");
    }

    #[test]
    fn idle_watchdog_arms_only_after_the_answer_starts() {
        let echo = llm_echo(&PromptFormat::Llama3.wrap("Q?"));
        assert!(!answer_started(b"", &echo));
        assert!(!answer_started(b"user\n\nQ", &echo), "half-printed echo is not an
answer");
        assert!(!answer_started(format!("{echo}  \n").as_bytes(), &echo));
        assert!(answer_started(format!("{echo}Paris").as_bytes(), &echo));
        assert!(answer_started(b"Paris", &echo), "no echo at all: any text counts");
    }
```

```
    #[test]
    fn explicit_llama_bin_is_never_swapped() {
        assert_eq!(prefer_completion_bin("/opt/llama/bin/llama-cli".into()),
"/opt/llama/bin/llama-cli");
        assert_eq!(prefer_completion_bin("llama-server".into()), "llama-server");
    }

    // ── advisory ──

    fn advisory_fixture() -> Vec<AdvisoryNode> {
        let v = |t: f32, busy: bool, tier: &str| {
            Some(Vitals { thermal_c: t, max_temp: 65.0, busy, tier: tier.into(),
label: String::new() })
        };
        vec![
            AdvisoryNode { id: [1u8; 32], vitals: v(58.0, false, "standard-3b"), me:
true, name: "laptop".into() },
            AdvisoryNode { id: [2u8; 32], vitals: v(41.0, false, "light-1b"), me:
false, name: "phone".into() },
            AdvisoryNode { id: [3u8; 32], vitals: v(70.0, false, "light-1b"), me:
false, name: "tablet".into() },
            AdvisoryNode { id: [4u8; 32], vitals: None, me: false, name:
"desktop".into() },
        ]
    }

    #[test]
    fn the_node_finds_the_devices_a_note_names_in_the_order_it_names_them() {
        let nodes = mesh3();
        let named = |note: &str| mentioned_names(&nodes, note);
        let s = |v: &[&str]| v.iter().map(|x| x.to_string()).collect::<Vec<_>>();
        assert_eq!(named("I'm unplugging the phone. Save its battery: run AI jobs on
the laptop instead."), s(&["phone", "laptop"]));
        assert_eq!(named("Leave the laptop alone, I'm compiling on it. The notebook
can take the jobs."), s(&["laptop", "notebook"]));
        assert_eq!(named("Suggest which device should take heavy inference next."),
s(&[]));
        assert_eq!(named("The Phone's battery is at 8%."), s(&["phone"]), "any case,
possessive");
        assert_eq!(named("Both phones are hot; notebooks and laptops are fine."), s(&
["phone", "notebook", "laptop"]), "plurals");
        assert_eq!(named("Use the smartphone or the notebookish thing."), s(&[]),
"whole words only");
        assert_eq!(named("phone"), s(&["phone"]), "at the very start and end");
        let mut room = mesh3();
        room[2].name = "living-room".into();
        assert_eq!(mentioned_names(&room, "Keep jobs off the Living Room PC."), s(&
["living-room"]));
        assert_eq!(first_word_at("café phone", "phone"), Some(6), "after a multi-byte
character");
```

```
    }

    #[test]
    fn the_node_quotes_what_the_note_says_about_each_device() {
        let nodes = mesh3();
        let ev = |note: &str| device_evidence(&nodes, note);
        let p = |v: &[(&str, &str)]| v.iter().map(|(a, b)| (a.to_string(),
b.to_string())).collect::<Vec<_>>();
        assert_eq!(ev("Suggest which device should take heavy inference next."), p(&
[]));
        assert_eq!(
            ev("I'm unplugging the phone and heading out for two hours. Save its
battery: run AI jobs on the laptop instead."),
            p(&[("phone", "I'm unplugging the phone, heading out for two hours, Save
its battery"), ("laptop", "run AI jobs on the laptop instead")])
        );
        assert_eq!(ev("The phone's battery is at 8%. Keep AI jobs off it."), p(&
[("phone", "The phone's battery is at 8%, Keep AI jobs off it")]));
        assert_eq!(
            ev("I'm in a video call on the laptop for the next hour, so don't slow it
down."),
            p(&[("laptop", "I'm in a video call on the laptop for the next hour, so
don't slow it down")])
        );
        // the two v16 misses
        assert_eq!(
            ev("The notebook is overheating in the sun and the phone is nearly
flat."),
            p(&[("notebook", "The notebook is overheating in the sun"), ("phone",
"the phone is nearly flat")])
        );
        assert_eq!(
            ev("Leave the laptop alone, I'm compiling on it. The notebook can take
the jobs."),
            p(&[("laptop", "Leave the laptop alone, I'm compiling on it"),
("notebook", "The notebook can take the jobs")])
        );
        // "and" between two nouns does not cut the clause
        assert_eq!(
            ev("The phone and the notebook are both busy; the laptop is free."),
            p(&[("phone", "The phone and the notebook are both busy"), ("notebook",
"The phone and the notebook are both busy"), ("laptop", "the laptop is free")])
        );
        assert_eq!(ev("Plug the phone in and give it the jobs."), p(&[("phone", "Plug
the phone in, give it the jobs")]));
        // long evidence is capped on a character boundary
        let long = format!("The phone {}", "é".repeat(200));
        let e = ev(&long);
        assert!(e[0].1.len() <= EVIDENCE_MAX + 3 && e[0].1.ends_with("..."), "{}",
e[0].1);
```

```
    }

    #[test]
    fn advisory_prompt_has_the_note_and_the_labels_but_no_devices_or_numbers() {
        let note = "I'm unplugging the phone and heading out. Save its battery: run
AI jobs on the laptop.";
        let p = build_advisory_prompt(note);
        assert!(p.contains(&format!("\"{note}\"\n")), "{p}");
        assert!(p.contains("AVOID = keep AI jobs off it (battery low or flat") &&
p.contains("USE = run AI jobs on it"));
        assert!(!p.contains("=AVOID") && !p.contains("=> AVOID"), "no example answer
to copy:\n{p}");
        assert!(!p.contains("47.5") && !p.contains("headroom"), "no numbers:\n{p}");
        let big = build_advisory_prompt(&"x".repeat(MAX_PROMPT_BYTES));
        assert!(big.len() <= MAX_ADVISORY_PROMPT_BYTES &&
big.contains(&"x".repeat(MAX_PROMPT_BYTES)), "a full note fits");
    }

    #[test]
    fn device_names_are_plain_words_or_nothing() {
        assert_eq!(note_name("  Phone "), Some("phone".into()));
        assert_eq!(note_name("jetson-2"), Some("jetson-2".into()));
        assert_eq!(note_name("living room"), Some("living-room".into()));
        assert_eq!(note_name("say \"hi\""), None, "quotes would break the grammar");
        assert_eq!(note_name("café"), None);
        assert_eq!(note_name("-x"), None);
        assert_eq!(note_name(""), None);
        assert_eq!(note_name(&"q".repeat(25)), None);
    }

    #[test]
    fn llama_gets_a_bounded_context_unless_the_operator_sets_one() {
        let f = |s: &str| s.split_whitespace().map(String::from).collect::<Vec<_>>();
        assert_eq!(default_llama_args(&f("-no-cnv --temp 0.7")), ["-n", "128", "-c",
"4096"]);
        assert_eq!(default_llama_args(&f("-no-cnv -n 256")), ["-c", "4096"]);
        assert_eq!(default_llama_args(&f("--ctx-size 8192")), ["-n", "128"]);
        assert_eq!(default_llama_args(&f("--ctx-size=8192 --predict=64")), Vec::
<&str>::new());
    }

    #[cfg(unix)]
    #[test]
    fn sigkill_is_reported_as_out_of_memory() {
        use std::os::unix::process::ExitStatusExt;
        let stderr = b"llama_completion: load the model and apply lora adapter, if
any";
        let killed = llm_exit_reason(std::process::ExitStatus::from_raw(9), stderr,
"m.gguf");
        assert!(killed.contains("out-of-memory"), "{killed}");
```

```
        let failed = llm_exit_reason(std::process::ExitStatus::from_raw(1 << 8), b"",
"m.gguf");
        assert!(failed.starts_with("exited") && !failed.contains("out-of-memory"), "
{failed}");
        // and an OOM kill is never mistaken for a rejected grammar
        assert!(!grammar_rejected(&killed));
        assert!(grammar_rejected("exited exit status: 1: failed to parse grammar"));
        assert!(grammar_rejected("exited exit status: 1: llama rejected a flag — ...
--grammar"));
        assert!(!grammar_rejected("timed out after 120s"));
    }

    #[test]
    fn advisory_grammar_writes_each_line_but_the_label() {
        let ev = vec![
            ("notebook".to_string(), "The notebook is overheating in the
sun".to_string()),
            ("phone".to_string(), "the phone is \"nearly\" flat".to_string()),
        ];
        assert_eq!(
            advisory_grammar(&ev).unwrap(),
            "root ::= \"notebook: \\\"The notebook is overheating in the sun\\\" =>
\" (\"AVOID\" | \"USE\" | \"ANY\") \"\\n\" \"phone: \\\"the phone is \\\"nearly\\\"
flat\\\" => \" (\"AVOID\" | \"USE\" | \"ANY\")"
        );
        assert_eq!(advisory_grammar(&[]), None, "a note that names no device is not
sent to the model");
        assert_eq!(gbnf_literal("café \\ 😀"), "\"caf\\u00E9 \\\\ \\U0001F600\"");
    }

    #[test]
    fn the_per_device_answer_parses_and_its_reason_quotes_the_note() {
        let out = "notebook: \"The notebook is overheating in the sun\" =>
AVOID\nphone: \"the phone is nearly flat\" => avoid\n";
        let r = parse_reading(out).unwrap();
        assert_eq!(r.avoid, vec!["notebook".to_string(), "phone".to_string()]);
        assert!(r.prefer.is_empty());
        assert_eq!(r.reason, "notebook: The notebook is overheating in the sun;
phone: the phone is nearly flat");
        let r = parse_reading("laptop: \"Leave the laptop alone\" => AVOID\nnotebook:
\"The notebook can take the jobs\" => USE").unwrap();
        assert_eq!((r.avoid.clone(), r.prefer.clone()), (vec!["laptop".to_string()],
vec!["notebook".to_string()]));
        let none = parse_reading("phone: \"the phone is fine\" => ANY").unwrap();
        assert!(none.avoid.is_empty() && none.prefer.is_empty() &&
none.reason.is_empty());
    }

    #[test]
    fn the_reading_of_a_note_parses_in_any_case_and_keeps_the_reason() {
```

```
        let r = parse_reading("laptop=USE phone=AVOID notebook=ANY - phone battery
low, laptop requested").unwrap();
        assert_eq!((r.avoid.clone(), r.prefer.clone()), (vec!["phone".to_string()],
vec!["laptop".to_string()]));
        assert_eq!(r.reason, "phone battery low, laptop requested");
        let none = parse_reading("laptop=ANY phone=ANY notebook=ANY - no device
named").unwrap();
        assert!(none.avoid.is_empty() && none.prefer.is_empty());
        let two = parse_reading("Answer: laptop=avoid Phone=Avoid, notebook=use -
café closed").unwrap();
        assert_eq!(two.avoid, vec!["laptop".to_string(), "phone".to_string()]);
        assert_eq!(two.prefer, vec!["notebook".to_string()]);
        assert_eq!(two.reason, "café closed", "non-ASCII reasons survive");
        let bare = parse_reading("phone=AVOID").unwrap();
        assert_eq!((bare.avoid.len(), bare.reason.as_str()), (1, ""));
        assert_eq!(parse_reading("AVOID B PREFER A - the v14 format"), None);
        assert_eq!(parse_reading("I need more context"), None);
    }

    /// The three-device mesh as it stood when Llama 3.2 3B picked the notebook "for
the most
    /// headroom": laptop 47.5 C (17.5 headroom), phone 28.4 C (36.6), notebook 36.8
C (28.2).
    fn mesh3() -> Vec<AdvisoryNode> {
        let v = |t: f32, tier: &str| Some(Vitals { thermal_c: t, max_temp: 65.0,
busy: false, tier: tier.into(), label: String::new() });
        vec![
            AdvisoryNode { id: [1u8; 32], vitals: v(47.5, "standard-3b"), me: true,
name: "laptop".into() },
            AdvisoryNode { id: [2u8; 32], vitals: v(28.4, "standard-3b"), me: false,
name: "phone".into() },
            AdvisoryNode { id: [3u8; 32], vitals: v(36.8, "light-1b"), me: false,
name: "notebook".into() },
        ]
    }

    fn read(avoid: &[&str], prefer: &[&str]) -> NoteReading {
        NoteReading {
            avoid: avoid.iter().map(|s| s.to_string()).collect(),
            prefer: prefer.iter().map(|s| s.to_string()).collect(),
            reason: String::new(),
        }
    }

    #[test]
    fn the_node_turns_a_reading_into_a_device_with_the_rules_arithmetic() {
        let nodes = mesh3();
        let (laptop, phone, notebook) = (Some([1u8; 32]), Some([2u8; 32]), Some([3u8;
32]));
        // a note that names no device: the most headroom, which is the phone, not
```

```
the notebook
        assert_eq!(reading_choice(&nodes, &read(&[], &[])), phone);
        // spare the phone: the most headroom of the rest
        assert_eq!(reading_choice(&nodes, &read(&["phone"], &[])), notebook);
        // spare the phone, use the laptop: the laptop, though it has less headroom
        assert_eq!(reading_choice(&nodes, &read(&["phone"], &["laptop"])), laptop);
        // two asked for: the rules' pick of those two
        assert_eq!(reading_choice(&nodes, &read(&[], &["laptop", "notebook"])),
notebook);
        // the limit outranks the note: a hot device asked for is not chosen
        let mut hot = nodes.clone();
        hot[0].vitals.as_mut().unwrap().thermal_c = 70.0;
        assert_eq!(reading_choice(&hot, &read(&["phone"], &["laptop"])), notebook);
        // so does missing data
        let mut blind = nodes.clone();
        blind[0].vitals = None;
        assert_eq!(reading_choice(&blind, &read(&[], &["laptop"])), phone);
        // sparing every usable device leaves the decision to the rules
        assert_eq!(reading_choice(&nodes, &read(&["laptop", "phone", "notebook"], &
[])), None);
        // names match whatever case the labels have
        let mut caps = nodes.clone();
        caps[1].name = "Phone".into();
        assert_eq!(reading_choice(&caps, &read(&["phone"], &[])), notebook);
    }

    #[test]
    fn the_verdict_compares_the_readings_target_with_the_rules() {
        let nodes = mesh3();
        let r = read(&[], &[]);
        let v = reading_verdict(Some(&r), reading_choice(&nodes, &r), &nodes);
        assert!(v.contains("names no device") && v.contains("thermal rules agree"), "
{v}");
        let r = read(&["phone"], &["laptop"]);
        let v = reading_verdict(Some(&r), reading_choice(&nodes, &r), &nodes);
        assert!(v.contains("spare phone; use laptop") && v.contains("jobs → laptop
(") && v.contains("thermal rules prefer phone ("), "{v}");
        let r = read(&["laptop", "phone", "notebook"], &[]);
        assert!(reading_verdict(Some(&r), None, &nodes).contains("the rules
decide"));
        assert!(reading_verdict(None, None, &nodes).contains("no reading of the note
parsed"));
        assert_eq!(decision_reason(&read(&[], &[])), "the note names no device: most
headroom");
        assert_eq!(decision_reason(&read(&["phone"], &[])), "as the operator's note
asks");
    }

    #[test]
    fn relay_and_the_dry_run_refuse_the_same_bodies() {
```

```
        let code = |b: &str| directive_problem(b).map(|p| p.0);
        assert_eq!(code("  \n"), Some(StatusCode::BAD_REQUEST));
        assert_eq!(code(&"x".repeat(MAX_PROMPT_BYTES + 1)),
Some(StatusCode::PAYLOAD_TOO_LARGE));
        assert_eq!(code("spare the phone\u{1b}[2J"), Some(StatusCode::BAD_REQUEST),
"no terminal escapes");
        assert_eq!(code(&"é".repeat(MAX_PROMPT_BYTES / 2)), None, "the limit is
bytes, and this is exactly it");
        assert_eq!(code("The phone's battery is low.\nUse the laptop."), None);
    }

    // ── port rotation ──

    #[test]
    fn sweep_ports_covers_both_rotation_windows() {
        let p = sweep_ports(3006, 3005);
        assert_eq!(p.first(), Some(&3005), "base window first");
        assert!(!p.contains(&3006), "never re-probes the failed port");
        assert!(p.contains(&3015) && p.contains(&3017));
        assert_eq!(p.len(), p.iter().collect::<HashSet<_>>().len(), "no duplicates");
        let seed = sweep_ports(3005, 3005);
        assert_eq!(seed.first(), Some(&3006));
        assert!(!seed.contains(&3005));
        assert!(!sweep_ports(u16::MAX, 3005).contains(&u16::MAX), "no overflow at the
top");
    }

    #[test]
    fn hearsay_never_drags_a_reachable_peer_back() {
        let old: SocketAddr = "192.168.1.110:3005".parse().unwrap();
        let new: SocketAddr = "192.168.1.110:3006".parse().unwrap();
        let mut p = Peer { addr: new, fails: 0, learned_from: None };
        // a stale peer list still says :3005 — ignored while :3006 answers
        update_known_peer(&mut p, old, Some([9u8; 32]));
        assert_eq!(p.addr, new);
        // once :3006 fails, hearsay may move it, but the failure count stands
        p.fails = 2;
        update_known_peer(&mut p, old, Some([9u8; 32]));
        assert_eq!((p.addr, p.fails), (old, 2));
        // a handshake proves the address and clears the count
        update_known_peer(&mut p, new, None);
        assert_eq!((p.addr, p.fails, p.learned_from), (new, 0, None));
    }

    #[test]
    fn identity_lock_refuses_a_second_copy() {
        let dir = std::env::temp_dir().join(format!("harmony-lock-test-{}",
std::process::id()));
        let other = dir.join("other-node");
        let first = lock_identity(&dir).expect("first lock");
```

```
        match lock_identity(&dir) {
            Ok(_) => panic!("a second copy of the same identity must be refused"),
            Err(e) => assert!(e.to_string().contains("already running"), "{e}"),
        }
        let o = lock_identity(&other).expect("a different identity is not blocked");
        drop(first);
        assert!(lock_identity(&dir).is_ok(), "released when the first copy exits");
        drop(o);
        let _ = std::fs::remove_dir_all(&dir);
    }

    // ── AI placement, device labels, dashboard safety ──

    #[test]
    fn six_named_devices_keep_the_grammar_small() {
        // six devices, every name as long as it can be, all named, each with capped
evidence
        let mut many = advisory_fixture();
        for i in 0..2u8 {
            let mut n = many[1].clone();
            n.id = [10 + i; 32];
            many.push(n);
        }
        for (i, n) in many.iter_mut().enumerate() {
            n.name = format!("{i}{}", "n".repeat(23));
        }
        let names = note_names(&many);
        let note: String = names.iter().map(|n| format!("The {n} is {}. ", "busy
".repeat(40))).collect();
        let ev = device_evidence(&many, &note);
        assert_eq!(ev.len(), 6);
        assert!(ev.iter().all(|(_, e)| e.len() <= EVIDENCE_MAX + 3));
        assert!(advisory_grammar(&ev).unwrap().len() < 1600);
    }

    #[test]
    fn advisories_decode_greedily_and_jobs_keep_the_operators_sampling() {
        let f = |s: &str| s.split_whitespace().map(String::from).collect::<Vec<_>>();
        let flags = f("-no-cnv --temp 0.7 --top-p 0.9");
        let g = "root ::= \"phone: \\\"the phone is hot\\\"\" => \" (\"AVOID\" |
\"USE\" | \"ANY\")";
        // defaults first, then the operator's flags, the grammar, and the advisory's
answer
        // budget and --temp 0 last, so they win
        let adv = llama_run_args(&flags, Some(g), true);
        assert_eq!(&adv[..4], ["-n", "128", "-c", "4096"]);
        assert_eq!(&adv[adv.len() - 6..], ["--grammar", g, "-n", ADVISORY_PREDICT, "-
-temp", "0"]);
        // an advisory retried without its grammar is still greedy
        let retry = llama_run_args(&flags, None, true);
```

```
        assert_eq!(&retry[retry.len() - 4..], ["-n", ADVISORY_PREDICT, "--temp",
"0"]);
        // ordinary jobs: exactly the operator's sampling, nothing appended
        let job = llama_run_args(&flags, None, false);
        assert_eq!(job, ["-n", "128", "-c", "4096", "-no-cnv", "--temp", "0.7", "--
top-p", "0.9"]);
        assert_eq!(LlmRun::Job.prompt_cap(), MAX_PROMPT_BYTES);
        assert!(LlmRun::Advisory.prompt_cap() > MAX_PROMPT_BYTES &&
LlmRun::Advisory.greedy() && !LlmRun::Job.greedy());
    }

    #[test]
    fn deferred_advisories_wait_for_a_free_llama_before_spending_a_retry() {
        let rested = LLM_COOLDOWN + Duration::from_secs(1);
        assert!(advisory_retry_due(40.0, 65.0, true, rested));
        assert!(!advisory_retry_due(40.0, 65.0, false, rested), "busy: an attempt now
would fail at once");
        assert!(!advisory_retry_due(40.0, 65.0, true, Duration::from_millis(100)),
"inside the cooldown");
        assert!(!advisory_retry_due(65.0, 65.0, true, rested), "at the limit");
    }

    // ── Windows ──

    #[test]
    fn crlf_output_from_windows_builds_still_drops_the_echoed_prompt() {
        let echo = "Read the note.\nDevices:\nA (laptop): model standard-3b\n";
        let raw = "Read the note.\r\nDevices:\r\nA (laptop): model standard-
3b\r\nlaptop=USE - asked for\r\n[end of text]\r\n";
        assert_eq!(clean_llm_output(raw, echo), "laptop=USE - asked for");
        assert!(answer_started(raw.as_bytes(), echo));
        assert!(!answer_started(b"Read the note.\r\nDev", echo), "still echoing");
        // LF output is unchanged
        let lf = "Read the note.\nDevices:\nA (laptop): model standard-
3b\nlaptop=AVOID - x\n";
        assert_eq!(clean_llm_output(lf, echo), "laptop=AVOID - x");
    }

    #[test]
    fn a_bare_program_name_also_matches_its_exe_on_windows() {
        assert_eq!(exe_candidates("llama-completion", true), ["llama-completion",
"llama-completion.exe"]);
        assert_eq!(exe_candidates("llama-completion.exe", true), ["llama-
completion.exe"]);
        assert_eq!(exe_candidates("llama-completion", false), ["llama-completion"]);
    }

    #[test]
    fn windows_thermal_samples_are_degrees_or_nothing() {
        assert_eq!(thermal_sample("45.85\r"), Some(45.85));
```

```
        assert_eq!(thermal_sample(" 27.85 "), Some(27.85));
        assert_eq!(thermal_sample("none"), None);
        assert_eq!(thermal_sample(""), None);
        assert_eq!(thermal_sample("-273.15"), None, "a zero-Kelvin zone is no
reading");
        assert_eq!(thermal_sample("500"), None);
    }

    #[test]
    fn placement_decision_round_trips_and_rejects_malformed_bodies() {
        let b = PlacementBody {
            answers: "ab".repeat(16),
            target: "cd".repeat(32),
            label: "laptop".into(),
            reason: "phone battery low".into(),
            rules: Some("ef".repeat(32)),
            ttl_s: PLACEMENT_TTL_SECS,
        };
        let body = placement_body(&b);
        assert!(body.starts_with(PLACEMENT_PREFIX));
        assert!(body.len() < MAX_PROMPT_BYTES);
        assert_eq!(parse_placement(&body), Some(b.clone()));
        // an ordinary directive is not a decision
        assert_eq!(parse_placement("Which node should take heavy inference next?"),
None);
        let bad = |f: &dyn Fn(&mut PlacementBody)| {
            let mut x = b.clone();
            f(&mut x);
            parse_placement(&placement_body(&x))
        };
        assert_eq!(bad(&|x| x.target = "zz".into()), None);
        assert_eq!(bad(&|x| x.answers = "short".into()), None);
        assert_eq!(bad(&|x| x.ttl_s = 5), None);
        assert_eq!(bad(&|x| x.reason = "r".repeat(301)), None);
        assert_eq!(bad(&|x| x.rules = Some("nothex".into())), None);
        // a signed decision verifies like any relayed message and fits a frame
        let me = Identity::ephemeral();
        let r = sign_relayed(&me, &body, now_unix());
        assert!(verify_relayed(&r, now_unix()).is_ok());
        let p = Placement::from_body(&r.id, me.fpr(), r.issued, &b).unwrap();
        assert_eq!(p.expires, r.issued + PLACEMENT_TTL_SECS);
        assert_eq!(p.target, [0xcd; 32]);
    }

    #[test]
    fn rules_veto_hot_busy_and_silent_devices() {
        let v = |t: f32, busy: bool| Vitals { thermal_c: t, max_temp: 65.0, busy,
tier: "standard-3b".into(), label: "laptop".into() };
        assert_eq!(placement_veto(Some(&v(48.0, false))), None);
        assert!(placement_veto(Some(&v(80.0, false))).unwrap().contains("80 °C, at or
```

```
over its 65 °C limit"));
        assert!(placement_veto(Some(&v(65.0, false))).is_some(), "at the limit counts
as over");
        assert!(placement_veto(Some(&v(40.0, true))).unwrap().contains("busy"));
        assert!(placement_veto(None).unwrap().contains("no fresh vitals"));
    }

    #[test]
    fn vitals_label_is_optional_on_the_wire() {
        let v = Vitals { thermal_c: 40.0, max_temp: 65.0, busy: false, tier: "light-
1b".into(), label: String::new() };
        assert!(!serde_json::to_string(&v).unwrap().contains("label"), "older nodes
see the old shape");
        let old: Vitals = serde_json::from_str(r#"
{"thermal_c":40.0,"max_temp":65.0,"busy":false,"tier":"x"}"#).unwrap();
        assert_eq!(old.label, "");
        let named = Vitals { label: "phone".into(), ..v };
        assert!
(serde_json::to_string(&named).unwrap().contains(r#""label":"phone""#));
    }

    #[test]
    fn device_label_prefers_the_flag_and_cleans_it() {
        assert_eq!(device_label(Some("  jetson-2  ")), "jetson-2");
        assert_eq!(device_label(Some(&"q".repeat(40))).len(), 24);
        assert_eq!(device_label(Some("bad\u{1b}[31mname")), "bad[31mname");
        assert!(!device_label(Some("   ")).is_empty(), "blank falls back to a
guess");
        assert!(!device_label(None).is_empty());
    }

    #[test]
    fn event_log_is_capped_and_resumable() {
        let mut log = EventLog::default();
        for i in 0..(EVENT_LOG_MAX + 5) {
            log.push("job", &format!("e{i}"), Some(&[1u8; 32]), None);
        }
        assert_eq!(log.items.len(), EVENT_LOG_MAX);
        assert_eq!(log.next, (EVENT_LOG_MAX + 5) as u64);
        let tail = log.since(log.next - 2);
        assert_eq!(tail.len(), 2);
        assert_eq!(tail[1].text, format!("e{}", EVENT_LOG_MAX + 4));
        assert_eq!(tail[0].from.as_deref(), Some("01".repeat(32).as_str()));
        log.push("x", "a\u{1b}[2Jb", None, None);
        assert_eq!(log.since(log.next - 1)[0].text, "a[2Jb", "control characters
stripped");
    }

    #[test]
    fn loopback_admin_refuses_foreign_pages() {
```

```
        let h = |pairs: &[(&'static str, &str)]| {
            let mut m = HeaderMap::new();
            for (k, v) in pairs {
                m.insert(*k, v.parse().unwrap());
            }
            m
        };
        assert!(loopback_request_ok(&h(&[])), "curl: no Host check needed beyond
loopback");
        assert!(loopback_request_ok(&h(&[("host", "127.0.0.1:3005")])));
        assert!(loopback_request_ok(&h(&[("host", "localhost:8080"), ("origin",
"http://localhost:8080")])));
        assert!(loopback_request_ok(&h(&[("host", "[::1]:3005"), ("origin",
"http://[::1]:3005")])));
        assert!(!loopback_request_ok(&h(&[("host", "evil.example:3005")])), "DNS
rebinding");
        assert!(!loopback_request_ok(&h(&[("host", "127.0.0.1:3005"), ("origin",
"https://evil.example")])));
        assert!(!loopback_request_ok(&h(&[("host", "127.0.0.1:3005"), ("origin",
"null")])));
        assert!(!loopback_request_ok(&h(&[("host", "127.0.0.1.evil.example")])));
        assert!(!loopback_request_ok(&h(&[("host", "[::1].evil.example")])));
        assert!(json_content(&h(&[("content-type", "application/json; charset=utf-
8")])));
        assert!(!json_content(&h(&[("content-type", "text/plain")])));
        assert!(!json_content(&h(&[])));
    }

    #[test]
    fn dashboard_is_self_contained() {
        assert!(UI_HTML.starts_with("<!doctype html>"));
        for banned in ["http://", "https://", "src=\"//", "@import"] {
            assert!(!UI_HTML.contains(banned), "dashboard must not load {banned}");
        }
        for route in ["/health", "/peers", "/events", "/relay", "/infer"] {
            assert!(UI_HTML.contains(route), "dashboard uses {route}");
        }
    }

    // ── graceful startup and shutdown ──

    #[test]
    fn goodbye_is_a_small_tagged_message() {
        assert_eq!(serde_json::to_string(&Msg::Bye).unwrap(), r#"{"t":"bye"}"#);
        assert!(matches!(serde_json::from_str::<Msg>(r#"{"t":"bye"}"#).unwrap(),
Msg::Bye));
    }

    #[test]
    fn shutting_down_is_a_capacity_failure_so_callers_offload() {
```

```
        assert!(is_capacity_failure("shutting down"));
        assert!(!is_capacity_failure("prompt exceeds 1024 bytes"));
    }

    #[test]
    fn already_running_names_the_holder() {
        let dir = std::env::temp_dir().join(format!("harmony-pid-test-{}",
std::process::id()));
        let first = lock_identity(&dir).expect("first lock");
        std::fs::write(pid_file(&dir), "4242\n").unwrap();
        let err = match lock_identity(&dir) {
            Ok(_) => panic!("second copy must be refused"),
            Err(e) => e.to_string(),
        };
        assert!(err.contains("already running") && err.contains("(pid 4242)"), "
{err}");
        drop(first);
        let _ = std::fs::remove_dir_all(&dir);
    }

    // ── v18 hardening ──

    #[test]
    fn strict_trust_ignores_pins_left_by_tofu() {
        let dir = tmp_dir("strict-pins");
        let listed = [1u8; 32];
        let stray = [2u8; 32];
        // TOFU pins whoever completes a handshake, and reads its pins back after a
restart
        let mut tofu = TrustStore::load(TrustMode::Tofu, &dir, &[]);
        assert!(tofu.allows(&stray), "TOFU lets a newcomer handshake");
        assert!(tofu.admit(&stray));
        assert!(TrustStore::load(TrustMode::Tofu, &dir, &
[]).pinned.contains(&stray));
        // strict trusts the list, whatever the pin file holds, and never writes to
it
        let mut strict = TrustStore::load(TrustMode::Strict, &dir, &[listed]);
        assert!(strict.allows(&listed) && strict.admit(&listed));
        assert!(!strict.allows(&stray) && !strict.admit(&stray), "a TOFU pin must not
survive the switch");
        assert!(!strict.allows(&[3u8; 32]));
        let pins = std::fs::read_to_string(dir.join("trusted_peers.txt")).unwrap();
        assert!(!pins.contains(&hex(&listed)));
        // revocation still beats the list
        strict.apply_revocations(&[listed]);
        assert!(!strict.allows(&listed) && !strict.admit(&listed));
        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
```

```
    fn placement_signers_need_an_explicit_grant() {
        let me = [1u8; 32];
        let laptop = [2u8; 32];
        let stranger = [3u8; 32];
        let listed: HashSet<NodeId> = [laptop].into_iter().collect();
        let none = HashSet::new();
        assert!(placement_signer_ok(&me, &me, &none, &none), "its own decisions");
        assert!(placement_signer_ok(&laptop, &me, &listed, &none), "an --advisor");
        assert!(placement_signer_ok(&laptop, &me, &none, &listed), "a --cmd-allow
identity");
        // v17 adopted this one whenever this node had asked
        assert!(!placement_signer_ok(&stranger, &me, &listed, &listed));
    }

    #[test]
    fn a_decision_lasts_ten_minutes_at_most() {
        let b = PlacementBody {
            answers: "ab".repeat(16),
            target: "cd".repeat(32),
            label: "laptop".into(),
            reason: "spare the phone".into(),
            rules: None,
            ttl_s: 3600,
        };
        assert!(parse_placement(&placement_body(&b)).is_some(), "3600 s still parses
(wire compatibility)");
        let p = Placement::from_body("d", [1u8; 32], 1_000, &b).unwrap();
        assert_eq!(p.expires, 1_000 + PLACEMENT_TTL_SECS);
    }

    #[test]
    fn mesh_endpoints_refuse_public_addresses() {
        let ip = |s: &str| s.parse::<IpAddr>().unwrap();
        for a in ["127.0.0.1", "::1", "192.168.1.110", "10.0.0.5", "172.16.3.4",
"100.64.1.2", "fd00::1", "::ffff:192.168.1.5"] {
            assert!(inbound_ok(ip(a), false), "{a} must be allowed");
        }
        for a in ["8.8.8.8", "1.1.1.1", "169.254.169.254", "2001:db8::1", "0.0.0.0",
"fe80::1", "224.0.0.1"] {
            assert!(!inbound_ok(ip(a), false), "{a} must be refused");
        }
        assert!(inbound_ok(ip("8.8.8.8"), true), "--allow-public");
        assert!(!inbound_ok(ip("169.254.169.254"), true), "link-local metadata stays
refused");
    }

    fn staged_build(tag: &str) -> PathBuf {
        let dir = tmp_dir(tag);
        std::fs::write(dir.join("rust_p2p_node.exe"), b"MZ the node").unwrap();
        std::fs::write(dir.join("libwinpthread-1.dll"), b"MZ a runtime").unwrap();
```

```
        std::fs::write(dir.join("node.ps1"), b"# node.ps1 - start, stop").unwrap();
        std::fs::write(dir.join("mesh.env"),
b"HARMONY_TRUST_MODE=strict\n").unwrap();
        std::fs::write(dir.join("BUILD"), b"harmony-v18\n").unwrap();
        dir
    }

    #[test]
    fn a_signed_update_verifies_and_any_change_is_refused() {
        let phone = Identity::ephemeral();
        let dir = staged_build("upd");
        let m = sign_update_dir(&phone, &dir, 1_700_000_000).unwrap();
        assert_eq!((m.build.as_str(), m.files.len()), ("harmony-v18", 5));
        let ok = verify_update_dir(&dir, &phone.fpr()).unwrap();
        assert_eq!((ok.build.as_str(), ok.issued), ("harmony-v18", 1_700_000_000));
        // DEPLOYED, which the notebook writes back on the laptop, is not part of the
build
        std::fs::write(dir.join("DEPLOYED"), b"harmony-v17\n").unwrap();
        assert!(verify_update_dir(&dir, &phone.fpr()).is_ok());

        // pinned to another signer
        let other = Identity::ephemeral();
        assert!(verify_update_dir(&dir, &other.fpr()).unwrap_err().contains("pinned
signer"));

        // a changed file
        let exe = dir.join("rust_p2p_node.exe");
        std::fs::write(&exe, b"MZ the node, patched").unwrap();
        assert!(verify_update_dir(&dir,
&phone.fpr()).unwrap_err().contains("rust_p2p_node.exe"));
        std::fs::write(&exe, b"MZ the node").unwrap();
        assert!(verify_update_dir(&dir, &phone.fpr()).is_ok());

        // an added file: a DLL planted next to the node would be loaded by it
        std::fs::write(dir.join("version.dll"), b"MZ planted").unwrap();
        assert!(verify_update_dir(&dir,
&phone.fpr()).unwrap_err().contains("version.dll"));
        std::fs::remove_file(dir.join("version.dll")).unwrap();

        // a removed file
        let ps1 = std::fs::read(dir.join("node.ps1")).unwrap();
        std::fs::remove_file(dir.join("node.ps1")).unwrap();
        assert!(verify_update_dir(&dir,
&phone.fpr()).unwrap_err().contains("node.ps1"));
        std::fs::write(dir.join("node.ps1"), &ps1).unwrap();

        // an edited manifest: a later issue date, to pass an old build off as newer
        let mpath = dir.join(UPDATE_MANIFEST);
        let text = std::fs::read_to_string(&mpath).unwrap();
        std::fs::write(&mpath, text.replace("issued 1700000000", "issued
```

```
1800000000")).unwrap();
        assert!(verify_update_dir(&dir,
&phone.fpr()).unwrap_err().contains("signature"));
        // the same manifest re-signed by someone else, with that key's id in it
        std::fs::write(&mpath, &text).unwrap();
        let forged = sign_update_dir(&other, &dir, 1_700_000_000).unwrap();
        assert_eq!(forged.signer, other.fpr());
        assert!(verify_update_dir(&dir, &phone.fpr()).unwrap_err().contains("pinned
signer"));
        sign_update_dir(&phone, &dir, 1_700_000_000).unwrap();
        assert!(verify_update_dir(&dir, &phone.fpr()).is_ok());
        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
    fn update_manifests_read_strictly() {
        let phone = Identity::ephemeral();
        let dir = staged_build("upd-parse");
        sign_update_dir(&phone, &dir, 1_700_000_000).unwrap();
        let text = std::fs::read_to_string(dir.join(UPDATE_MANIFEST)).unwrap();
        let m = parse_manifest(&text).unwrap();
        assert_eq!(manifest_text(&m), text, "the format round-trips");
        for bad in ["../x", ".hidden", "a/b", "a\\b", "", "C:x", "x y", "con..txt"] {
            assert!(!update_name_ok(bad), "{bad:?}");
        }
        for good in ["rust_p2p_node.exe", "libgcc_s_seh-1.dll", "node.ps1", "BUILD",
"mesh.env"] {
            assert!(update_name_ok(good), "{good:?}");
        }
        assert!(parse_manifest(&text.replace(UPDATE_HEADER, "harmony-update
2")).is_err());
        assert!(parse_manifest(&text.replacen("file ", "file  ", 1)).is_err());
        assert!(parse_manifest(text.trim_end()).is_err(), "no final newline");
        assert!(parse_manifest(&format!("{text}file x 1 zzz\n")).is_err());
        // swapping two file lines puts them out of order
        let mut lines: Vec<&str> = text.lines().collect();
        let n = lines.len();
        lines.swap(n - 1, n - 2);
        assert!(parse_manifest(&(lines.join("\n") + "\n")).is_err());
        // a subdirectory in a staged build is refused when signing
        std::fs::create_dir(dir.join("sub")).unwrap();
        assert!(sign_update_dir(&phone, &dir, 1).unwrap_err().contains("sub"));
        let _ = std::fs::remove_dir_all(&dir);
    }

    // ── v19: the model kept loaded, and the OpenAI API ──

    fn turn(role: &str, content: &str) -> ChatTurn {
        ChatTurn { role: role.into(), content: content.into() }
    }
```

```
    #[test]
    fn one_user_turn_is_exactly_the_job_template() {
        for f in [PromptFormat::Llama3, PromptFormat::Chatml, PromptFormat::Raw] {
            assert_eq!(f.wrap_chat(&[turn("user", "hi there")]), f.wrap("hi there"),
"{f:?}");
        }
    }

    #[test]
    fn conversations_keep_their_roles_in_order() {
        let c = [turn("system", "Be brief."), turn("user", "Hi"), turn("assistant",
"Hello."), turn("user", "Why?")];
        assert_eq!(
            PromptFormat::Llama3.wrap_chat(&c),
            "<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\nBe brief.
<|eot_id|>\
             <|start_header_id|>user<|end_header_id|>\n\nHi<|eot_id|>\
             <|start_header_id|>assistant<|end_header_id|>\n\nHello.<|eot_id|>\
             <|start_header_id|>user<|end_header_id|>\n\nWhy?<|eot_id|>\
             <|start_header_id|>assistant<|end_header_id|>\n\n"
        );
        assert_eq!(
            PromptFormat::Chatml.wrap_chat(&c[1..3]),
            "<|im_start|>user\nHi<|im_end|>\n<|im_start|>assistant\nHello.
<|im_end|>\n<|im_start|>assistant\n"
        );
        assert_eq!(
            PromptFormat::Raw.wrap_chat(&c[1..]),
            "User: Hi\n\nAssistant: Hello.\n\nUser: Why?\n\nAssistant:"
        );
    }

    #[test]
    fn long_conversations_lose_their_oldest_turns_first() {
        let big = "x".repeat(3000);
        let c = vec![turn("system", "rules"), turn("user", &big), turn("assistant",
&big), turn("user", &big)];
        let (kept, dropped) = fit_conversation(c, CHAT_MAX_PROMPT_BYTES).unwrap();
        // the oldest user turn goes, and the assistant turn it leaves at the front
goes with it
        assert_eq!(dropped, 2);
        assert_eq!(kept.iter().map(|t| t.role.as_str()).collect::<Vec<_>>(),
["system", "user"]);
        assert!(chat_bytes(&kept) <= CHAT_MAX_PROMPT_BYTES);
        let small = vec![turn("user", "a"), turn("assistant", "b"), turn("user",
"c")];
        assert_eq!(fit_conversation(small.clone(), CHAT_MAX_PROMPT_BYTES).unwrap(),
(small, 0));
        // a last message over the cap on its own is refused, not cut
```

```
        let huge = vec![turn("user", &"y".repeat(CHAT_MAX_PROMPT_BYTES))];
        assert!(fit_conversation(huge, CHAT_MAX_PROMPT_BYTES).is_err());
    }

    #[test]
    fn openai_requests_become_conversations() {
        let req: ChatRequest = serde_json::from_str(
            r#"{
                "model": "gpt-4o", "stream": true, "max_tokens": 5000, "temperature":
3,
                "messages": [
                    {"role": "developer", "content": "Answer in French."},
                    {"role": "user", "content": [{"type": "text", "text": "Bonjour"},
{"type": "image_url", "image_url": {"url": "x"}}]},
                    {"role": "assistant", "content": null, "tool_calls": []},
                    {"role": "tool", "content": "42", "tool_call_id": "a"},
                    {"role": "user", "content": "Ça va ?"}
                ],
                "presence_penalty": 0.5
            }"#,
        )
        .unwrap();
        assert_eq!(req.stream, Some(true));
        let job = chat_job(req).unwrap();
        assert_eq!(
            job.turns,
            vec![turn("system", "Answer in French."), turn("user", "Bonjour"),
turn("user", "Ça va ?")]
        );
        assert_eq!(job.max_tokens, Some(CHAT_MAX_TOKENS));
        assert_eq!(job.temperature, Some(2.0));
        let refuse = |body: &str| chat_job(serde_json::from_str::<ChatRequest>
(body).unwrap()).unwrap_err();
        assert!(refuse(r#"{"messages":
[{"role":"system","content":"x"}]}"#).contains("user message"));
        assert!(refuse(r#"{"n":2,"messages":
[{"role":"user","content":"x"}]}"#).contains("n > 1"));
        assert!(refuse(r#"{"messages":
[{"role":"wizard","content":"x"}]}"#).contains("wizard"));
    }

    #[test]
    fn answers_have_the_openai_shapes() {
        let a = ChatAnswer {
            text: "Hi \"there\"\nfriend".into(),
            prompt_tokens: Some(12),
            completion_tokens: None,
            finish: "stop",
        };
        let v = chat_completion_json("chatcmpl-1", 7, "harmony", &a,
```

```
serde_json::json!({ "device": "phone" }));
        assert_eq!(v["object"], "chat.completion");
        assert_eq!(v["choices"][0]["message"]["role"], "assistant");
        assert_eq!(v["choices"][0]["message"]["content"], a.text.as_str());
        assert_eq!(v["choices"][0]["finish_reason"], "stop");
        assert_eq!(v["usage"]["prompt_tokens"], 12);
        assert_eq!(v["usage"]["completion_tokens"], approx_tokens(&a.text));
        assert_eq!(v["usage"]["total_tokens"], 12 + approx_tokens(&a.text));
        let sse = chat_stream_body("chatcmpl-1", 7, "harmony", &a);
        let events: Vec<&str> = sse.split("\n\n").filter(|e|
!e.is_empty()).collect();
        assert_eq!(events.len(), 4);
        assert!(events.iter().all(|e| e.starts_with("data: ")));
        assert_eq!(events[3], "data: [DONE]");
        let chunks: Vec<serde_json::Value> =
            events[..3].iter().map(|e|
serde_json::from_str(&e[6..]).unwrap()).collect();
        assert!(chunks.iter().all(|c| c["object"] == "chat.completion.chunk" &&
c["id"] == "chatcmpl-1"));
        assert_eq!(chunks[0]["choices"][0]["delta"]["role"], "assistant");
        assert_eq!(chunks[1]["choices"][0]["delta"]["content"], a.text.as_str());
        assert!(chunks[1]["choices"][0]["finish_reason"].is_null());
        assert_eq!(chunks[2]["choices"][0]["finish_reason"], "stop");
    }

    #[test]
    fn mesh_requests_stay_readable_by_older_nodes() {
        // A prompt request is byte for byte what v18 sent, and v18's parses here.
        let plain = Msg::InferRequest {
            id: 1,
            prompt: "p".into(),
            deadline_ms: 5,
            chat: None,
            max_tokens: None,
            temperature: None,
            top_p: None,
        };
        let wire = r#"{"t":"infer_request","id":1,"prompt":"p","deadline_ms":5}"#;
        assert_eq!(serde_json::to_string(&plain).unwrap(), wire);
        assert!(matches!(serde_json::from_str::<Msg>(wire).unwrap(),
Msg::InferRequest { chat: None, .. }));
        // A conversation's prompt is its last user message, which is what a v18 node
answers: it
        // ignores the fields it does not know.
        #[derive(Deserialize)]
        #[serde(tag = "t", rename_all = "snake_case")]
        enum V18 {
            InferRequest { prompt: String, deadline_ms: u64 },
        }
        let job = ChatJob {
```

```
            turns: vec![turn("system", "s"), turn("user", "first"), turn("assistant",
"a"), turn("user", "second")],
            max_tokens: Some(64),
            temperature: None,
            top_p: None,
        };
        let msg = chat_request(&job, Duration::from_secs(60)).unwrap();
        let V18::InferRequest { prompt, deadline_ms } =
            serde_json::from_slice::<V18>
(&serde_json::to_vec(&msg).unwrap()).unwrap();
        assert_eq!((prompt.as_str(), deadline_ms), ("second", 60_000));
        assert!(matches!(&msg, Msg::InferRequest { chat: Some(t), max_tokens:
Some(64), .. } if t.len() == 4));
    }

    #[test]
    fn every_mesh_message_fits_one_frame() {
        // Quotes double when JSON escapes them: this conversation fits the byte cap
but not a
        // frame, so the request drops its oldest turns until it does.
        let q = "\"".repeat(2500);
        let job = ChatJob {
            turns: vec![turn("user", &q), turn("assistant", &q), turn("user", &q)],
            max_tokens: None,
            temperature: None,
            top_p: None,
        };
        assert!(chat_bytes(&job.turns) <= CHAT_MAX_PROMPT_BYTES);
        let msg = chat_request(&job, Duration::from_secs(60)).expect("still
sendable");
        assert!(fits_one_frame(&msg));
        assert!(matches!(&msg, Msg::InferRequest { chat: Some(t), .. } if t.len() ==
1));
        // A long reply is cut to fit as well.
        let mut reply = Msg::InferReply {
            id: 1,
            ok: true,
            text: "\u{1F600}\"".repeat(4000),
            tier: "t".into(),
            node: "n".into(),
            elapsed_ms: 1,
        };
        assert!(!fits_one_frame(&reply));
        fit_reply(&mut reply);
        assert!(fits_one_frame(&reply));
        let Msg::InferReply { text, .. } = &reply else { unreachable!() };
        assert!(!text.is_empty());
    }

    #[test]
```

```
    fn the_server_gets_resource_flags_and_each_request_its_sampling() {
        let flags: Vec<String> =
            "-no-cnv --repeat-penalty 1.15 --temp 0.7 --top-p 0.9 -t 4 -c 8192 --
mlock -n 256 --top-k=40"
                .split_whitespace()
                .map(String::from)
                .collect();
        let joined = server_args("m.gguf", 4242, &flags).join(" ");
        assert!(joined.starts_with("-m m.gguf --host 127.0.0.1 --port 4242 -np 1 -c
8192"), "{joined}");
        assert!(joined.ends_with("-c 8192 -t 4"), "{joined}");
        for cli_only in ["-no-cnv", "--temp", "--top-p", "--repeat-penalty", "--top-
k", "-n ", "--mlock"] {
            assert!(!joined.contains(cli_only), "{cli_only} must not reach llama-
server: {joined}");
        }
        assert!(server_args("m.gguf", 1, &[]).join(" ").ends_with(&format!("-c
{LLM_CTX}")));

        let job = ModelCall { formatted: "P".into(), grammar: None, greedy: false,
n_predict: None, temperature: None, top_p: None };
        let r = server_request(&job, &flags);
        assert_eq!(r["prompt"], "P");
        assert_eq!(r["temperature"], 0.7);
        assert_eq!(r["top_p"], 0.9);
        assert_eq!(r["repeat_penalty"], 1.15);
        assert_eq!(r["top_k"], 40);
        assert_eq!(r["n_predict"], 256);
        assert_eq!(r["stream"], true);
        assert_eq!(r["cache_prompt"], true);
        assert!(r.get("grammar").is_none());
        let chat = ModelCall { n_predict: Some(64), temperature: Some(0.2), top_p:
Some(0.5), ..job };
        let r = server_request(&chat, &flags);
        assert_eq!(r["n_predict"], 64);
        assert_eq!(r["temperature"].as_f64().map(|t| (t * 10.0).round()), Some(2.0));
        let advisory = ModelCall {
            formatted: "A".into(),
            grammar: Some("root ::= \"x\""),
            greedy: true,
            n_predict: None,
            temperature: None,
            top_p: None,
        };
        let r = server_request(&advisory, &flags);
        assert_eq!(r["temperature"], 0.0);
        assert_eq!(r["n_predict"].as_i64(), ADVISORY_PREDICT.parse::<i64>().ok());
        assert_eq!(r["grammar"], "root ::= \"x\"");
        let bare = server_request(&advisory, &[]);
        assert!(bare.get("top_p").is_none());
```

```
        assert_eq!(bare["n_predict"].as_i64(), ADVISORY_PREDICT.parse::<i64>().ok());
        let plain = ModelCall { greedy: false, grammar: None, ..advisory };
        assert_eq!(server_request(&plain, &[])["n_predict"].as_i64(),
LLM_PREDICT.parse::<i64>().ok());
    }

    #[test]
    fn the_per_job_runner_gets_chat_settings_after_the_operators() {
        let plain = ModelCall { formatted: String::new(), grammar: None, greedy:
false, n_predict: None, temperature: None, top_p: None };
        assert!(call_overrides(&plain).is_empty(), "an /infer job keeps v18's command
line exactly");
        let chat = ModelCall { n_predict: Some(300), temperature: Some(0.5), top_p:
Some(0.25), ..plain };
        assert_eq!(call_overrides(&chat), ["-n", "300", "--temp", "0.5", "--top-p",
"0.25"]);
        let greedy = ModelCall { greedy: true, ..chat };
        assert!(call_overrides(&greedy).is_empty(), "an advisory's own -n and --temp
0 stand");
    }

    #[test]
    fn flags_follow_llama_cpp_precedence() {
        let f: Vec<String> = ["-t", "2", "--threads=6", "-c", "2048", "-
t"].iter().map(|s| s.to_string()).collect();
        assert_eq!(flag_value(&f, &["-t", "--threads"]).as_deref(), Some("6"), "the
last given wins; a dangling one is ignored");
        assert_eq!(flag_value(&f, &["-c", "--ctx-size"]).as_deref(), Some("2048"));
        assert_eq!(flag_value(&f, &["-ngl"]), None);
    }

    #[test]
    fn server_sent_events_are_read_across_chunks() {
        let mut l = SseLines::default();
        assert!(l.push(b"data: {\"content\":\"Hel").is_empty());
        assert_eq!(
            l.push(b"lo\"}\n\ndata: {\"stop\":true}\r\n"),
            ["data: {\"content\":\"Hello\"}", "", "data: {\"stop\":true}"]
        );
        assert_eq!(l.pending(), 0);
        assert_eq!(sse_data("data: {\"a\":1}"), Some("{\"a\":1}"));
        assert_eq!(sse_data("data:{}"), Some("{}"));
        assert_eq!(sse_data(": keep-alive"), None);
        // a character split between two chunks is not mangled
        let e = "data: é\n".as_bytes();
        let mut l = SseLines::default();
        assert!(l.push(&e[..7]).is_empty());
        assert_eq!(l.push(&e[7..]), ["data: é"]);
    }
```

```
    #[test]
    fn server_errors_say_what_the_server_said() {
        let grammar = r#"{"error":{"code":400,"message":"failed to parse
grammar","type":"invalid_request_error"}}"#;
        assert_eq!(server_error_text(400, grammar), "llama-server 400: failed to
parse grammar");
        assert!(grammar_rejected(&server_error_text(400, grammar)), "the advisory
retries without it");
        assert_eq!(server_error_text(500, "plain\nfailure"), "llama-server 500: plain
failure");
        assert_eq!(server_error_text(503, r#"{"message":"Loading model"}"#), "llama-
server 503: Loading model");
    }

    #[test]
    fn a_loading_model_is_a_capacity_failure() {
        assert!(is_capacity_failure("the model is still loading on this device"));
        assert!(!is_capacity_failure("conversation exceeds 8192 bytes"));
        assert!(request_fault("conversation exceeds 8192 bytes"));
        assert!(!request_fault("thermal 70.0C at or above limit 65.0C"));
    }

    #[test]
    fn the_server_is_found_beside_llama_bin() {
        let dir = std::env::temp_dir().join(format!("harmony-engine-bin-{}",
std::process::id()));
        std::fs::create_dir_all(&dir).unwrap();
        let exe = if cfg!(windows) { ".exe" } else { "" };
        let bin = dir.join(format!("llama-completion{exe}"));
        let server = dir.join(format!("llama-server{exe}"));
        std::fs::write(&bin, b"").unwrap();
        assert_eq!(server_binary("auto", bin.to_str().unwrap()),
which_on_path("llama-server"), "none beside it: PATH decides");
        std::fs::write(&server, b"").unwrap();
        assert_eq!(server_binary("auto", bin.to_str().unwrap()),
Some(server.clone()));
        assert_eq!(server_binary(server.to_str().unwrap(), "elsewhere"),
Some(server.clone()));
        assert_eq!(server_choice(" off "), None);
        assert_eq!(server_choice("auto").as_deref(), Some("auto"));
        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
    fn an_orphan_is_stopped_only_if_it_is_a_llama_server() {
        let dir = std::env::temp_dir().join(format!("harmony-engine-pid-{}",
std::process::id()));
        std::fs::create_dir_all(&dir).unwrap();
        // this test process is no llama-server: it is left alone, and the stale file
is cleared
```

```
        std::fs::write(dir.join(ENGINE_PID_FILE), format!("{} 4242\n",
std::process::id())).unwrap();
        engine_reap_orphan(&dir);
        assert!(!dir.join(ENGINE_PID_FILE).exists());
        std::fs::write(dir.join(ENGINE_PID_FILE), "not a pid\n").unwrap();
        engine_reap_orphan(&dir);
        assert!(!dir.join(ENGINE_PID_FILE).exists());
        let _ = std::fs::remove_dir_all(&dir);
    }

    #[test]
    fn truncating_bytes_keeps_whole_characters() {
        assert_eq!(truncate_bytes("héllo", 2), "h");
        assert_eq!(truncate_bytes("héllo", 3), "hé");
        assert_eq!(truncate_bytes("abc", 10), "abc");
        assert_eq!(chat_tokens(None), CHAT_DEFAULT_TOKENS);
        assert_eq!(chat_tokens(Some(0)), 1);
        assert_eq!(chat_tokens(Some(1_000_000)), CHAT_MAX_TOKENS);
    }

    #[test]
    fn the_dashboard_shows_how_the_model_runs() {
        assert!(UI_HTML.contains("function runnerText"));
        assert!(UI_HTML.contains("engine: h.engine"));
    }
}
```